

Базы данных

Лекция 00 – Административа

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

Оглавление

План учебной дисциплины.....	3
Место учебной дисциплины.....	3
Содержание учебной дисциплины.....	8
Литература основная.....	10
Литература дополнительная.....	11
Важные замечания относительно литературы.....	12
Основная вычислительная среда.....	14
Приложения для работы с ER-моделями и прочим, что имеет отношение к БД.....	15
Лабораторные работы.....	16
Замечания по лабораторным работам.....	18
Состав документации, которую следует представить для сдачи лабораторной работы.....	18
Общие требования к оформлению ПЗ.....	22
Варианты тем лабораторных работ.....	24
Низкоуровневые операции логического уровня в таблицах.....	26

План учебной дисциплины

Лекций:	4+5+4+3 (32)
Лабораторных:	5+4+4+3 (32)
Самостоятельная работа:	44
Зачет	

Место учебной дисциплины.

Учебная дисциплина «Базы данных» предназначена для формирования расширенных знаний в области представления и обработки данных в рамках обучения студентов методикам и способам создания и проектирования программного обеспечения.

Цель преподавания учебной дисциплины: подготовка специалистов, владеющих теоретическими и практическими знаниями в области проектирования реляционных баз данных, использования их в решении различных производственных задач, а также их настройки и обслуживания.

Задачи изучения учебной дисциплины:

- формирование знаний о способах представления, хранения и обработки данных в системах баз данных;
- формирование знаний о методологиях проектирования баз данных и информационных систем, формирование практических навыков проектирования **реляционных баз данных**;
- формирование знаний и навыков практического использования языка **SQL** в создании, обслуживании и работе с базами данных;
- формирование знаний и навыков создания пользовательских приложений для работы с базами данных.

В результате изучения учебной дисциплины «Базы данных» формируются следующие компетенции:

академические:

- уметь применять базовые научно-теоретические знания для решения теоретических и практических задач;
- владеть системным и сравнительным анализом;
- уметь работать самостоятельно;
- владеть междисциплинарным подходом при решении проблем;
- иметь навыки, связанные с использованием технических устройств, управлением информацией и работой с компьютером;
- уметь учиться, повышать свою квалификацию в течение всей жизни;
- владеть основными методами, способами и средствами получения, хранения, переработки информации с использованием компьютерной техники;

социально-личностные:

- уметь работать в команде;

профессиональные:

- определять цели проектирования объектов профессиональной деятельности, критерии эффективности проектных решений, ограничений;
- реализовывать системный анализ объекта проектирования и предметной области, их взаимосвязей;
- разрабатывать требования и спецификации объектов профессиональной деятельности на основе анализа запросов пользователей, моделей предметной области и возможностей технических средств;
- выбирать средства вычислительной техники (ВТ), средства программирования с целью их применения для эффективной реализации аппаратно-программных комплексов;

- проектировать математическое, лингвистическое, информационное и программное обеспечение вычислительных систем (ВС) и автоматизированных систем на основе современных методов, средств и технологий проектирования, в том числе с использованием систем автоматизированного проектирования;

- работать с нормативно-технической документацией;

- систематизировать результаты и составлять отчеты по выполненной работе, обеспечивать контроль качества выполнения работ.

В результате изучения учебной дисциплины студент должен:

знать:

- термины и компоненты систем баз данных;

- модели представления данных;

- методологию проектирования баз данных;

- языки для работы с базами данных;

- программные интерфейсы для работы с базами данных.

уметь:

- применять модели данных для представления предметной области;

- создавать эффективные структуры баз данных для решения производственных задач;

- работать с базами данных на языке SQL;

- создавать прикладные программы, взаимодействующие с базами данных.

владеть:

- инструментальными средствами управления базами данных;

- инструментальными средствами разработки прикладного программного обеспечения нацеленного на работу с базами данных.

иметь представление:

- об организации и функционировании систем управления базами данных (СУБД);

- о тенденциях развития и перспективных направлениях использования баз данных.

Перечень учебных дисциплин, усвоение которых необходимо для изучения данной учебной дисциплины (программа).

Конструирование программ и языки программирования

Раздел 1. Язык программирования C++, в частности, темы:

- объекты и массивы объектов в C++ (**в БД нет ни объектов, ни методов**);
- потоки ввода-вывода в C++ (???)
- файловая система в C++ (???)
- дополнительные приемы конструирования программ.

На самом деле потребуется:

- уверенное владение клавиатурным вводом с терминала;
- Язык программирования C (или C++ в части императивного и структурного программирования);
- забыть ОО парадигму и вспомнить теорию множеств;
- прочитать внимательно документы:
 - ГОСТ 19.105 Общие требования к программным документам;
 - ГОСТ 19.106 Требования к программным документам, выполненным печатным способом;
 - ГОСТ 19.404 Пояснительная записка. Требования к содержанию и оформлению;
 - ГОСТ 19.701 Схемы алгоритмов, программ, данных и систем.

Содержание учебной дисциплины

№ тем	Наименование разделов, тем	Содержание
1	Введение в системы баз данных	Системы баз данных и информационные системы. Компоненты систем баз данных и их виды. Архитектура ANSI-SPARC. Функции и структура СУБД. Архитектура информационных систем.
2	Проектирование баз данных	Жизненный цикл информационных систем. Общая классификация моделей данных. Методология проектирования баз данных.
3	Концептуальные модели данных	Семантическое моделирование данных. Виды концептуальных моделей данных. ER-модель. ER-диаграмма.
4	Реляционная модель данных	Дореляционные логические модели данных. Структура реляционных данных. Целостность реляционных данных. Преобразование ER-диаграммы в реляционную схему данных. Реляционная алгебра и реляционное исчисление. Нормализация реляционных данных.
5	Языки баз данных	Язык SQL: идентификаторы, функции, типы данных, создание схемы данных, создание таблиц и индексов, выборка и модификация данных, представления, транзакции, управление доступом к данным. Язык QBE в сравнении с SQL.

№ тем	Наименование разделов, тем	Содержание
6	Физические модели данных	Файловые структуры данных и методы доступа: неупорядоченные, упорядоченные и хешированные файлы, индексы.
7	Разработка приложений	Виды приложений для работы с базами данных. Внедрение SQL-операторов в прикладные программы. Технологии и программные интерфейсы для доступа к базам данных.
8	Перспективы развития баз данных	Постреляционные и нереляционные СУБД. Хранилища данных. Интерактивная аналитическая обработка данных.
8	Локальные БД	Berkeley DB

Литература основная

1. Коннолли, Т. Базы данных: проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг, А. Страчан. – К.; М.; СПб.: ИД «Вильямс», 2003. – 1440 с.
2. **Дейт, К. Дж. SQL и реляционная теория. Как грамотно писать код на SQL / К. Дж. Дейт. - СПб.: Символ-Плюс, 2010. – 480 с.**
3. **Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – К.; М.; СПб.: ИД «Вильямс», 2006. – 1328 с.**
4. Грабер, М. SQL для простых смертных / М. Грабер. – М., «Лори», 2014. – 383 с.
5. Грабер, М. SQL / М. Грабер. – М., «Лори», 2003. – 664 с.
6. Гарсиа-Молина, Г. Системы баз данных. Полный курс / Г. Гарсиа-Молина, Д. Ульман, Д. Уидом. - К.; М.; СПб.: ИД «Вильямс», 2003. – 1088 с.
7. Yadava H. The Berkeley DB Book. Apress – 2007
8. Васильев А.Ю. Работа с PostgreSQL: настройка и масштабирование. – 2017.
9. Моргунов Е. П. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. — СПб.: БХВ-Петербург, 2018. — 336 с.: ил.
10. Лузанов П. и др. PostgreSQL для начинающих. – 2017.
11. Калабухов, Е. В. Базы данных, знаний и экспертные системы : лаб. практикум для студ. спец. I-40 02 01 «Вычислительные машины, системы и сети» всех форм обуч. / Е. В. Калабухов. – Минск : БГУИР, 2008. – 32 с. : ил.

Литература дополнительная

Агальцов, В.П. Базы данных. В 2-х т.Т. 1. Локальные базы данных: Учебник / В.П. Агальцов. - М.: ИД ФОРУМ, НИЦ ИНФРА-М, 2013. - 352 с.

Агальцов, В.П. Базы данных. В 2-х т. Т. 2. Распределенные и удаленные базы данных: Учебник / В.П. Агальцов. – М.: ИД ФОРУМ, НИЦ ИНФРА-М, 2013. - 272 с.

Голицына, О.Л. Базы данных: Учебное пособие / О.Л. Голицына, Н.В. Максимов, И.И. Попов. - М.: Форум, 2012. - 400 с.

Кириллов, В.В. Введение в реляционные базы данных / В.В. Кириллов, Г.Ю. Громов. - СПб.: БХВ-Петербург, 2012. - 464 с.

Карпова, И.П. Базы данных: Учебное пособие / И.П. Карпова. - СПб.: Питер, 2013. - 240 с.

Пирогов, В.Ю. Информационные системы и базы данных: организация и проектирование: Учебное пособие / В.Ю. Пирогов – СПб.: БХВ-Петербург, 2009. - 528 с.

Редько, В.Н. Базы данных и информационные системы / В.Н. Редько, И.А. Бассараб - М.: Знание, 2011. – 602 с.

Постолит, А. Visual Studio .NET: разработка приложений баз данных / А. Постолит. – СПб: БХВ, 2009. - 544 с.

Важные замечания относительно литературы

Представленный выше перечень взят из официальной программы, именно поэтому он здесь. Его наличие не означает, что все эти книги следует читать. Многие из них, что касается разработки реляционных баз данных, содержат информацию крайне сомнительного характера.

UML

Во многих из них можно встретить предложение использовать UML в качестве средства, якобы позволяющего эффективно проектировать объект автоматизации на всех уровнях.

Однако, следует заметить, что UML в своей основе является языком [графического описания] объектных моделей и он не в состоянии описывать системы хранения и манипулирования данными, в том числе и представленными в виде таблиц (реляционные системы).

Отличия в процессе проектирования появляются в процессе первичного рассмотрения задачи и серьезно углубляются на уровне модели «Сущность-связь».

ООП

Объектный подход и соответствующий язык можно использовать при проектировании систем, которые естественным образом могут быть представлены/описаны в рамках ОО модели, которая строится на принципах – класс, объект, абстракция данных, инкапсуляция, наследование, полиморфизм подтипов.

При проектировании реляционных (табличных) систем он не годится – здесь нет ни классов, ни объектов, ни методов, ни наследования, ни диаграмм использования, только теория множеств в чистом виде, сортировка и поиск.

Вера в то, что ООП – это серебряная пуля или «одна идеальная парадигма» – это ловушка.

Диаграммы использования

Особо следует отметить вред вариантов использования (use cases) – это метод, который навязывает частное решение, которое ограничивает схему данных и впоследствии это решение оказывается невозможно ни модифицировать, ни расширить.

При разработке баз данных следует всячески избегать подходов с вариантами использования на уровне пользователя – сегодня данные в организации принято использовать одним способом, завтра другим, а база данных должна соответствовать любому из вариантов, поскольку данные и связи между ними могут оставаться теми же.

Поэтому в первую очередь следует проектировать модель данных без принятия во внимание того, как пользователи используют эти данные сегодня, поскольку практика показывает, что после создания базы данных варианты использования данных меняются кардинально – старые методы перестают быть актуальными и появляются новые.

СУБД всегда определяет полное множество вариантов использования данных (операций) в рамках конкретной схемы данных – пользователю остается лишь **выбрать и реализовать** те, которые ему подходят.

Как произносится правильно:

Термин	Транскрипция 1	Транскрипция 1
SQL	[ˈɛsˈkjuˈɛl]	[ˈsiːkwəl];
PostgreSQL	[ˈpoʊstˈɡrɛsˈsiːkwəl]	[ˈpoʊstˈɡrɛsˈkjuˈɛl]

Основная вычислительная среда

ОС: Linux, MS Windows

СУБД: PostgreSQL, Berkeley DB

PostgreSQL – продвинутая система управления объектно-реляционными базами данных

Пакет postgresql-server содержит программы, необходимые для создания и запуска сервера PostgreSQL, который позволяет создавать и поддерживать базы данных PostgreSQL.

Базовый пакет postgresql содержит клиентские программы, необходимые для доступа к серверу СУБД PostgreSQL, а также документацию в формате HTML для всей системы. Эти клиентские программы могут располагаться на той же машине, что и сервер PostgreSQL, или на удаленной машине, который обращается к серверу PostgreSQL через сетевое соединение.

Berkeley DB – The Berkeley DB database library for C

База данных Berkeley (Berkeley DB) – это программный инструментарий, обеспечивающий **встроенную поддержку** баз данных как для традиционных, так и для клиент-серверных приложений. Berkeley DB включает в себя B+tree, расширенное линейное хеширование, методы доступа к записям фиксированной и переменной длины, транзакции, блокировку, ведение журнала, кэширование в разделяемой памяти и восстановление базы данных. Berkeley DB поддерживает API C, C++ и Perl. Он используется многими приложениями, включая Python и Perl.

Приложения для работы с ER-моделями и прочим, что имеет отношение к БД

В отличие от ООП, где есть UML и куча софта для рисования диаграмм, программных средств для разработки модели «Сущность-связь», годной для преобразования в реляционную, за исключением входящих в состав проприетарных СУБД, практически нет.

Тем не менее, программные средства, предназначенные для разработки реляционной модели, присутствуют почти в каждой из СУБД. Поэтому проектирование баз данных с самого начала чаще всего ведется именно на этом уровне – реляционном.

Однако, программа курса требует и включает в качестве лабораторной работы разработку модели «сущность-связь». Поэтому нужен инструмент для вычерчивания такого рода диаграмм. В качестве такого инструмента можно использовать любые схеморисовалки, позволяющие оформить схему в соответствии с требованиями ГОСТ 19 и практиками полиграфии русского языка в отношении надписей на блоках. Например, visio в MS Office или draw в Libre/Openoffice.

Есть также и другие схемочертилки.

Что касается реляционных диаграмм, то несмотря на большое количество рисовалок, сопровождающих различные СУБД, подходящих для выполнения лабораторных работ практически нет.

Дело в том, что отрисовка схем ими выполняется после того, как будут созданы сами таблицы и ограничения, в основном для фиксации существующих схем данных, что в корне противоречит самой сути использования реляционных диаграмм, особенно в рамках лабораторных работ.

Лабораторные работы

Студент выбирает тему и последовательно ее развивает в рамках лабораторных работ. Все лабораторные должны быть правильно оформлены.

№ 1 «Создание ER-диаграммы»

Выполняется концептуальное проектирование БД с использованием ER-модели представления данных (модели «сущность-связь») с учетом семантических ограничений заданной предметной области и представить модель в виде ER-диаграммы.

№ 2 «Создание реляционной схемы данных»

Выполняется логическое проектирование БД. Спроектированная в ЛР № 1 ER-диаграмма преобразуется в реляционную схему данных и оформляется в графическом виде.

№ 3 «Реализация базы данных»

Установка и настройка PostgreSQL. Изучение консольного клиента `psql`.

Реализация схемы базы данных по ранее построенной реляционной схеме данных (см. лабораторную работу № 2). Требуется сформировать SQL-запросы для создания таблиц базы данных и выполнить их в СУБД, а также заполнить таблицы данными с помощью оператора `INSERT`.

№ 4 «Реализация SQL-запросов на простую выборку данных»

В лабораторной работе выполняется создание простых запросов на выборку данных на языке SQL с использованием предложений `SELECT`, `FROM (JOINS)`, `WHERE` и `ORDER BY` оператора `SELECT`. В работе также требуется рассмотреть использование скалярных функций.

№ 5 Выборка данных с использованием подзапросов, агрегатных функций, группировки и операций над множествами

Используются предложение `GROUP BY` оператора `SELECT`, операции над множествами (`UNION`, `INTERSECT`, `MINUS`).

Лабораторная работа № 6 (на автомат)

Освоение низкоуровневых механизмов взаимодействия с базами данных.

Часть 1. Создание локальной базы данных класса ACID с помощью библиотеки Berkeley DB.

В лабораторной работе выполняется создание совместно используемой несколькими клиентами базы данных с помощью библиотеки Berkeley DB.

БД содержит несколько таблиц, связанных отношениями 1:M, N:M, N:1.

Часть 2. Создание интерактивного приложения для работы с локальной БД, поддерживающего основные операции – вставка/удаление записей, просмотр таблиц, получение/изменение записей, агрегатные операции.

Замечания по лабораторным работам

1) Лабораторные выполняются индивидуально.

2) Студент выбирает одну из тем, представленных ниже, и все пять лабораторных работ выполняет в ее рамках.

3) Структура лабораторной работы должна быть максимально близка к реальности.

В подавляющем случае никакому хозяину/руководителю предприятия, которое вы выбрали для автоматизации в рамках лабораторных, никакая база данных не нужна – ему удобнее вести вообще все делопроизводство шариковой ручкой в журналах.

Причем в двух – один для себя, другой для проверяющих. Поэтому каждый из вас должен стоять на позициях работников из проверяющих органов, например, следователя или налогового инспектора, если ваша модель имеет дело с деньгами. Это значит, например, что если у вас в столовой есть сотрудники и должности, то следователь, расследующий отравление гречневой кашей, должен иметь возможность получить из вашей БД информацию о всех, кто работал на должности повара в период с и по, а также что он там такого мог сварить. Ну и далее в этом ключе.

Состав документации, которую следует представить для сдачи лабораторной работы

1) Пояснительная записка в формате PDF для каждой из лабораторных работ. На титульном листе обязательно должно присутствовать название выбранной темы и название лабораторной работы. Номер темы необязателен. Требования к оформлению текстовой части см. ниже.

2) SQL-скрипты для лабораторных №№ 3, 4, 5. Состав скриптов см. ниже.

Пояснительная записка и скрипты должны располагаться в каталоге labN, где N – номер лабораторной. Каталог архивируется архиватором tar и *может быть* сжат компрессором, который tar умеет использовать.

lab1

Структура описания ПЗ следующая:

1. Описание ER-модели «Название темы»

1.1. Сущности

Описание всех сущностей и их атрибутов. Все сущности и их атрибуты должны быть поименованы на русском языке. Для сущностей приводятся их абстрактные типы без какой либо привязки к типам языков программирования («деньги», «дата», ...)

Количество сущностей — не менее семи.

1.2. Связи

Здесь описание всех связей между сущностями.

Количество связей многие ко многим — не менее трех.

2. Схема ER-модели.

lab2

Структура описания ПЗ следующая:

1. Описание реляционной модели «Название темы»

1.1. Перечень всех таблиц и каким сущностям они соответствуют. Перечень всех полей и каким атрибутам они соответствуют. Типы не указываем. Перечень ограничений. Перечень атрибутов, требующих сортировки при визуализации.

1.2. Связи

Здесь описание всех связей между сущностями.

2. Схема ER-модели из лабораторной №1.

3. Схема реляционной модели.

lab3

Структура описания ПЗ следующая:

1. Схема реляционной модели из лабораторной №2.
2. Описывается скрипт создания, описываются ограничения. Обосновываются выбранные типы данных для полей.
3. Описывается скрипт заполнения.

В составе архива с отчетом по лабораторной два скрипта — скрипт создания БД (`create.sql`) и скрипт ее заполнения (`insert.sql`). Скрипты создания и заполнения должны иметь комментарии для получения представления, что они делают.

Сами скрипты копировать в ПЗ не нужно.

lab4

Структура описания ПЗ следующая:

1. Схема реляционной модели из лабораторной №2.
2. Краткое описание каждого из скриптов получения данных из базы (будет использоваться в качестве комментария в скрипте с запросами).

В этой лабораторной три скрипта и файл с результатами:

`create.sql` из lab3;

`insert.sql` из lab3;

`select.sql` с запросами.

`results.txt`

lab5

Структура описания ПЗ следующая:

1. Схема реляционной модели из лабораторной №2.

2. Коротко описывается каждый скрипт получения данных из базы (будет использоваться в качестве комментария в скрипте с запросами).

В этой лабораторной четыре скрипта и файл с результатами:

create.sql из lab3;

insert.sql из lab3;

select.sql с запросами из lab4;

results.txt

select5.sql с запросами;

results5.txt.

Схемы выполняются на отдельной странице в ориентации «Ландшафт» с центрированной подрисуночной подписью в формате ГОСТ 7-32 (Рисунок 2 – Схема ER-модели «Название темы»).

Выход запросов не должен быть пустым, поэтому БД должна быть пополнена данными.

В ряде случаев может быть изменена и ее структура. В таком случае все изменения добавления в схему БД и в скрипты подробно описываются в ПЗ соответствующей лабораторной работы.

Лабораторные 3, 4 и 5 выполняются в консольном клиенте psql. Устанавливается режим протоколирования с выводом в файл. Этот файл редактируется, в частности, размечается по отдельным запросам комментариями, и из него формируется файл с результатами.

Все должно быть читабельно и аккуратно.

Скрипты следует форматировать отступами, чтобы была видна их структура.

Общие требования к оформлению ПЗ

Формат страницы текстовой части А4. Все листы кроме первого нумеруются внизу по центру.

Поля:

левое — 3 см;

правое — 1 см;

верхнее — 2 см;

нижнее — 2 см.

Абзац начинается с абзацного отступа размером в 5 букв 'н' (как учили в школе).

Межстрочный интервал одинарный.

Шрифт абзаца пропорциональный типа serif (Times New Roman, PT Astra Serif, Liberation Serif, ...), кегль 12-14 pt.

Шрифт имен таблиц, полей и прочих идентификаторов по тексту – моноширинный с отличием единицы от буквы 'l' и с плотностью, равной плотности основного шрифта абзаца (Hack, Fira Code, gost, Intel One Mono).

Переносы в тексте включены, в заголовках отключены.

Перечисления

Ненумерованные перечисления (перечни, списки) ничем не обозначаются, либо используется дефис, после которого идет неразрывный пробел.

Именованные перечисления оформляются, как ненумерованные без дефиса, имя при этом выделяется гарнитурой или полужирным.

Нумерованные перечисления нумеруются цифрой со скобкой или малой буквой русского алфавита, после которой идет неразрывный пробел.

Оформление схем

Толщина линий блоков и линий – 0,25..0,3 мм.

Надписи в блоках шрифт типа sans (Arial, PT Sans, PT Astra Sans, Liberation Sans, ...) тем же размером, что и текст абзаца.

Концы линий связи помечаются цифрами или буквами.

Варианты тем лабораторных работ

- 1) Автозаправка.
- 2) Автосалон.
- 3) Автостоянка.
- 4) Автошкола.
- 5) Аэропорт.
- 6) Банк.
- 7) Больница.
- 8) Военкомат.
- 9) Гостиница.
- 10) Грузоперевозки.
- 11) Детский сад.
- 12) Железнодорожный вокзал (автовокзал, такси и т.п.).
- 13) Завод.
- 14) Зоопарк.
- 15) Кафе (бар).
- 16) Кинотеатр.
- 17) Локальная компьютерная сеть (кабельное телевидение).
- 18) Магазин (музыкальный, продовольственный и т.п.).
- 19) Налоговая инспекция.
- 20) Общежитие.
- 21) Оператор связи.
- 22) Организация концертов.
- 23) Поликлиника.
- 24) Прокат видеодисков.

- 25) Ресторан.
- 26) СТО.
- 27) Столовая.
- 28) Студия звукозаписи (киностудия).
- 29) Туристическое агентство.
- 30) Футбольный клуб.
- 31) Школа.
- 32) Бассейн.

Низкоуровневые операции логического уровня в таблицах

TABLE LIST:LST (ID, NAME, DATE, ...)

TABLE ITEM:ITM (ID, LIST_ID, PART_NO, NAME, PROVIDER, QUANT, ...)

PRIMKEY LST:BY_ID (ID) # LST:ID

PRIMKEY ITM:BY_ID (ID) # ITM:ID

INDEX LST:BY_NAME (NAME)

INDEX LST:BY_DATE (DATE)

INDEX ITM:BY_NAME (NAME)

INDEX ITM:BY_NAME_INLIST (LIST_ID, NAME)

INDEX ITM:BY_PNO_INLIST (LIST_ID, PART_NO) # кандидат в PrimKey (?поставщик?)

Простейшие сериальные операции

SET(TABLE) – установка курсора в «сыром» порядке записей

NEXT(TABLE) – просмотр таблицы в направлении с начала к концу

PREV(TABLE) – просмотр таблицы в направлении с конца к началу

INSERT(RECORD, TABLE) – вставка в таблицу без обновления индексов

Прямой доступ по первичному ключу

GET(RECORD, TABLE, KEY) – получить строку (запись) из таблицы по значению перв. ключа

PUT(RECORD, TABLE, KEY) – обновить строку (запись) в таблице по значению перв. ключа

Добавление/удаление

ADD(RECORD, TABLE) – добавить строку (запись) в таблицу и обновить все индексы

DEL(RECORD, TABLE, KEY) – удалить строку (запись) из таблицы и обновить все индексы

Прямой доступ по ключу

GET(RECORD, TABLE, KEY, KEY_VALUE) – получить строку по значению ключа

PUT(RECORD, TABLE, KEY, KEY_VALUE) – обновить строку по значению ключа

ADD(RECORD, TABLE) – добавить строку (запись) в таблицу и обновить все индексы

DEL(RECORD, TABLE, KEY) – удалить строку (запись) из таблицы и обновить все индексы

Сериальные операции в порядке индексов

SET(TABLE, KEY, KEY_VALUE) – установить курсор доступа в порядке индекса

NEXT(TABLE, KEY) – просмотр таблицы в направлении с начала к концу

PREV(TABLE, KEY) – просмотр таблицы в направлении с конца к началу

Базы данных

Лекция 01 – Введение

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2022.09.01

Оглавление

Введение в системы баз данных.....	3
На основе файловых систем.....	3
Системы баз данных.....	5
Пользователи.....	5
Прикладные программы.....	8
База данных.....	9
Система управления базами данных.....	10
Трехуровневая архитектура ANSI-SPARC.....	11
Схема базы данных (резюме).....	19
Независимость от данных.....	21
Система управления базами данных.....	22
Функции СУБД.....	22
ACID.....	25
Модель «Сущность-связь».....	27
ER-диаграмма.....	28
Символы ERD-сущностей.....	29
Символы ERD-связей.....	31
Символы ERD-атрибутов.....	32
Символы физических ER-диаграмм.....	34
Поля.....	34
Ключи.....	35
Типы.....	36
Нотация ER-диаграмм.....	37
Установка и первичная настройка postgres.....	39

Введение в системы баз данных

Базы данных (БД) — неотъемлемая часть современной жизни (наиболее значимые сферы применения — коммуникационные системы, транспорт, финансовые системы, наука).

Дальнейшее расширение применения — практически везде, где есть необходимость хранения и поиска данных.

На основе файловых систем

Начало истории развития БД может характеризоваться следующими этапами:

1) начальное накопление данных — книги

Это бумажные картотеки (карточки, папки) обычно индексируемые по какому-либо признаку для ускорения поиска (например, картотека библиотеки) — ручные операции, *эффективен только поиск по индексу* (по фамилии авторов, названию книги) и *практически невозможен по сложным критериям* (найти среднее число книг написанных авторами с фамилией начинающейся на «А» и т.п.).

Рост требований по поиску разнообразной информации (усложнение запросов, рост числа данных, требование быстрого ответа на запрос), а также рост мощностей и доступности вычислительной техники (особенно появление магнитных носителей (ленты, диски)) привело к появлению файловых систем.

2) Файловая система — набор системных программных средств, которые выполняют некоторые операции для пользователей (ввод данных, операции с файлами, генерация фиксированного набора специализированных отчетов), каждая программа определяет жестко свои собственные данные (структура и методы доступа) и управляет ими, все данные децентрализованы и хранятся в местах их обработки (например, по отделам предприятия).

Данные в этих системах хранятся как наборы записей (record) в файлах, каждая запись содержит поля (field) хранящих определенные характеристики.

Ограничения файловых систем:

1) разделение и изоляция данных — данные для обработки должны выбираться из нескольких файлов (сложность одновременной обработки данных из нескольких файлов);

2) дублирование данных — (накопление файлов с данными по одному объекту в различных отделах) — неэкономное использование ресурсов (дисковое пространство) и риск нарушения целостности данных (ошибки, если данные в разных отделах различаются, требуются проверки данных);

3) зависимость от данных — (физическая структура и способы доступа различны для разных приложений) сложно изменять файлы (добавить новое поле), для преобразования данных нужны специальные программы-конверторы и изменение приложений;

4) несовместимость форматов файлов — (если приложения создаются с использованием различных языков программирования (COBOL, C, PASCAL)) необходима выработка общего формата хранения (затраты времени);

5) фиксированные запросы (рост количества приложений) — число запросов $\approx$ числу приложений, рост запросов — рост числа приложений (документирование и поддержка функциональности системы — усложнение сопровождения, снижение мер безопасности по защите).

все ограничения файловых систем — следствие двух фундаментальных причин:

1) определение данных содержится внутри приложений, а не отдельно от них;

2) кроме приложений нет других инструментов для доступа к данным и их обработки.

Системы баз данных

Система баз данных (СБД) – компьютеризированная система для хранения информации в БД.

Компоненты СБД:

- пользователи;
- прикладные программы;
- базы данных;
- система управления базами данных.



Рисунок 1 – Компоненты системы баз данных

Пользователи

1) Пользователи делятся на 4 группы:

- 1) администраторы;
- 2) разработчики баз данных;
- 3) прикладные программисты;
- 4) пользователи.

Администраторы

- администраторы данных;
- администраторы баз данных.

Администраторы данных — отвечают за *управление данными*, включая планирование БД, разработку и сопровождение стандартов, бизнес правил (описывают основные характеристики данных с точки зрения организации) и деловых процедур, а также концептуальное и логическое проектирование БД;

Администраторы баз данных — отвечают за физическую реализацию БД, включая физическое проектирование, обеспечение безопасности и целостности данных, а также обеспечение максимальной производительности приложений и пользователей (более технический характер по сравнению с администратором данных).

Разработчики баз данных

- разработчики логической базы данных;
- разработчики физической базы данных.

разработчики логической базы данных занимаются идентификацией данных, связей между данными и устанавливают ограничения, накладываемые на хранимые данные — (ответ на вопрос **ЧТО?**);

разработчики физической базы данных по готовой логической модели создают физическую реализацию (формирование таблиц, выбор структур хранения, методов доступа, мер защиты) — (ответ на вопрос **КАК?**).

Прикладные программисты

Создают приложения предоставляющие пользователям необходимые функциональные возможности (действия над базой данных).

Пользователи (клиенты БД)

- наивные пользователи — осуществляют доступ к БД через прикладные программы;
- опытные пользователи — могут осуществлять доступ к БД с использованием языков запросов или создавать собственные прикладные программы.

Прикладные программы

- пользователи;
- **прикладные программы;**
- базы данных;
- система управления базами данных.

Прикладные программы обеспечивают удобный для пользователей доступ к БД.

Реализуются с использованием языков программирования 3-го (процедурные языки — C, COBOL, Fortran, Ada, Pascal) или 4-го поколения (SQL, QBE). 4-е поколение (“4GL”) — непроцедурные языки, возможно генерирование прикладного приложения по параметрам, заданных пользователем.

Языки программирования делятся на:

- языки представления информации (языки запросов или генераторы форм и отчетов);
- специализированные языки (электронных таблиц и БД);
- генераторы приложений для определения, вставки, обновления или извлечения сведений из БД;
- языки очень высокого уровня для генерации кода приложений.

База данных

- пользователи;
- прикладные программы;
- **базы данных;**
- система управления базами данных.

База данных (БД) — совокупность логически связанных данных, хранящихся в компьютеризованной системе и отражающих некоторую предметную область человеческой деятельности.

БД — единое, большое хранилище данных (набор интегрированных записей с самоописанием), содержащее данные с минимальной долей избыточности, к которым может обращаться большое число пользователей.

Описание данных называется системным каталогом или словарем данных, а сами элементы описания — метаданные (данные о данных) обеспечивают независимость между программами и данными.

Система управления базами данных

- пользователи;
- прикладные программы;
- базы данных;
- **система управления базами данных.**

Система управления базами данных (СУБД) — программное обеспечение, с помощью которого пользователи могут определять, создавать и поддерживать БД, а также осуществлять к ней контролируемый доступ.

Главные преимущества СУБД — преодоление ограничений файловых систем (в основном из-за обеспечения централизованного управления данными).

Трехуровневая архитектура ANSI-SPARC

Основная цель СУБД — предоставление пользователям данных в абстрактном представлении, т.е. сокрытие от пользователей особенностей хранения и управления данными.

Традиционный подход к построению систем ранее был сосредоточен только на определении данных из двух разных представлений:

- представление пользователя (внешняя схема)
- представление компьютера (внутренняя схема).

С точки зрения пользователя, определение данных находится в контексте отчетов и экранов, предназначенных для помощи людям в выполнении их конкретной работы.

Требуемая структура данных из представления пользователя меняется в зависимости от бизнес-среды и индивидуальных предпочтений этого пользователя.

С точки зрения компьютера, данные определяются в терминах файловых структур для хранения и поиска.

Требуемая структура данных для компьютерного хранения зависит от конкретной используемой компьютерной технологии и необходимости эффективной обработки данных.

Для удовлетворения конкретных потребностей бизнеса на протяжении многих лет эти два традиционных представления данных определялись аналитиками для каждого приложения.

Как правило, внутреннюю схему, определенную для исходного приложения, нельзя сразу использовать для последующих приложений, что в результате приводило к созданию избыточных и часто противоречивых определений одних и тех же данных. Данные определялись компоновкой физических записей и последовательно обрабатывались в ранних информационных системах.

Признание этой проблемы привело исследовательскую группу ANSI/X3/SPARC по системам управления базами данных к выводу, что в идеальной среде управления данными необходимо третье представление данных — «концептуальная схема».

Концептуальная схема

Концептуальная схема представляет собой единое интегрированное определение данных в рамках предприятия, которое не привязано к какому-либо отдельному применению данных и не зависит от того, как данные физически хранятся или к ним осуществляется доступ.

Целью этой концептуальной схемы является предоставление последовательного определения значений и взаимосвязей данных, которые можно использовать для интеграции, совместного использования и управления целостностью данных.

Отчет ANSI/SPARC был задуман как *основа* для компьютерных систем, *способных обмениваться информацией*. Все поставщики баз данных приняли терминологию трех схем, но реализовали ее несовместимыми способами. В течение следующих двадцати лет различные группы пытались определить стандарты для концептуальной схемы и ее сопоставления с базами данных и языками программирования. К сожалению, ни у одного из поставщиков не было сильного стимула делать свои форматы совместимыми с форматами конкурентов. Было подготовлено несколько отчетов, но не стандартов.

По мере развития практики администрирования данных и появления новых графических методов термин «схема» уступил место термину «модель».

Концептуальная модель представляет собой представление данных, которое согласовывается между конечными пользователями и администраторами баз данных, охватывающее те объекты, о которых важно хранить данные, значение данных и взаимосвязь данных друг с другом.

Цель трехуровневой архитектуры состоит в том, чтобы отделить представления пользователей друг от друга, что достигается отделением пользовательского представления базы данных от ее физического представления:

- трехуровневая архитектура допускает *независимые настраиваемые* представления пользователей. Это значит, что каждый пользователь должен иметь возможность доступа к одним и тем же данным, но иметь различное настраиваемое представление о них. Представления должны быть независимыми – изменения в одном представлении не должны влиять на другие.

- трехуровневая архитектура скрывает информацию о физическом хранилище от пользователей, т.е. пользователям не нужно иметь дело с информацией о физическом хранилище БД.

- администратор базы данных (администратор данных) должен иметь возможность изменять структуры хранения базы данных, не затрагивая представления пользователей.

- на внутреннюю структуру базы данных не должны влиять изменения физических аспектов хранилища, например, переход на новый диск. или подлежащую файловую систему.

Архитектура ANSI-SPARC имеет три уровня:

- 1) **Внешний**, реализующий представления пользователя и описывающие различные внешние представления данных. **Для данной базы данных может быть много внешних схем.**

- 2) **Концептуальный**, реализующий способ описания того, какие данные хранятся во всей базе данных и как эти данные взаимосвязаны.

Концептуальная схема описывает все элементы данных и отношения между ними вместе с ограничениями целостности. **Для каждой базы данных существует только одна концептуальная схема.**

- 3) **Внутренний**, определяющий как база данных физически представлена в компьютерной системе. Внутренняя схема на самом нижнем уровне содержит определения хранимых записей, методов представления, полей данных и индексов. **Для каждой базы данных существует только одна внутренняя схема.**

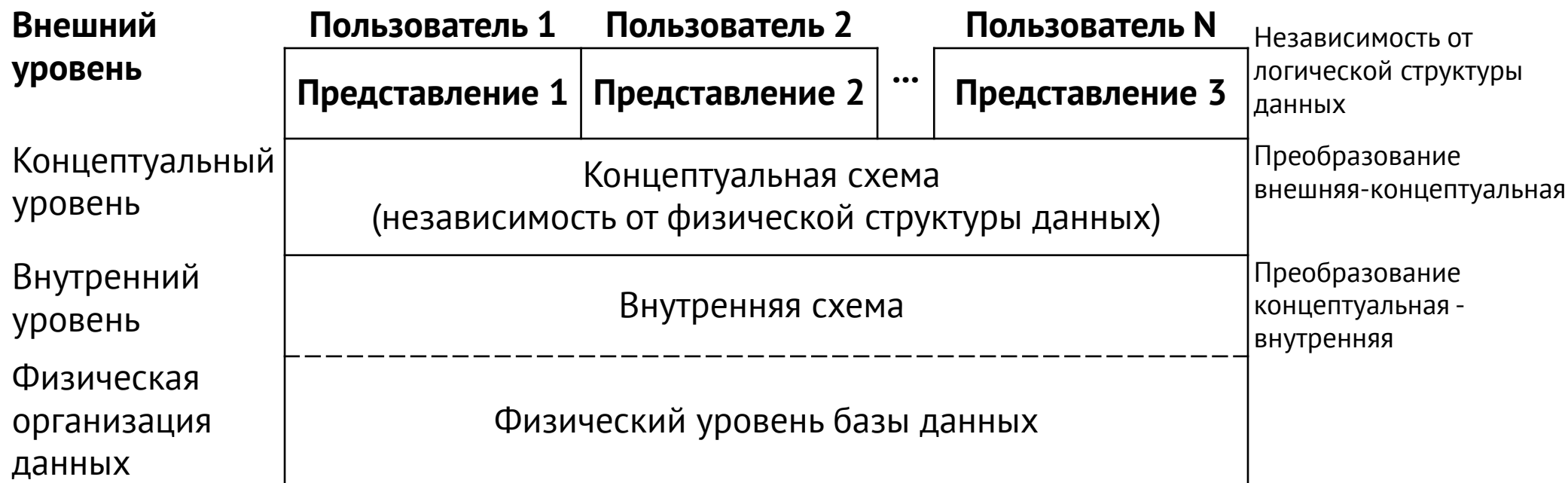


Рисунок 2 – Трехуровневая архитектура ANSI-SPARC.

Внешний уровень

Внешний уровень – представление БД с точки зрения пользователей.

Для каждой группы пользователей описываются только необходимая им часть БД.

Т.е. каждый пользователь имеет свое представление о «реальном мире», и не видит лишние с его точки зрения данные.

Разные представления (view) могут по-разному отображать одни и те же данные (например, дату).

Часть данных при этом может не храниться в БД (так называемые производные и вычисляемые данные), а получаться по мере надобности (например, возраст сотрудников), что не потребует лишних обновлений БД и уменьшит ее объем.

Внешний уровень	Пользователь 1	Пользователь 2	...	Пользователь N	Независимость от логической структуры данных
	Представление 1	Представление 2		Представление 3	
Концептуальный уровень	Концептуальная схема (независимость от физической структуры данных)				Преобразование внешняя-концептуальная
Внутренний уровень	Внутренняя схема				Преобразование концептуальная - внутренняя
Физическая организация данных	Физический уровень базы данных				

Концептуальный уровень

Концептуальный уровень — это обобщающее представление БД. Уровень описывает, какие данные хранятся в БД, а также связи между ними.

Является промежуточным уровнем в архитектуре и содержит логическую структуру всей БД с точки зрения администратора базы данных (DBA), а именно:

- все сущности;
- их атрибуты;
- связи;
- накладываемые на данные ограничения;
- семантическая информация о данных;
- информация о мерах безопасности и поддержки целостности данных.

Все внешние представления получаются из данных этого уровня, но этот уровень не содержит никаких сведений о методах хранения данных (например, может быть информация о длине полей

их типе, названии, но нет данных об объеме в байтах и т.п.).

Внешний уровень	Пользователь 1	Пользователь 2	...	Пользователь N	Независимость от логической структуры данных
	Представление 1	Представление 2		Представление 3	
Концептуальный уровень	Концептуальная схема (независимость от физической структуры данных)				Преобразование внешняя-концептуальная
Внутренний уровень	Внутренняя схема				Преобразование концептуальная - внутренняя
Физическая организация данных	Физический уровень базы данных				

Внутренний уровень

Внутренний уровень – это физическое представление базы данных в компьютере.

Уровень описывает, как информация хранится в БД, в частности, описывает физическую реализацию. Уровень предназначен для достижения оптимальной производительности и обеспечения экономного распределения дискового пространства.

На этом уровне содержится описание структур данных и описание организации файлов базы данных, а также осуществляется взаимодействие СУБД с методами доступа операционной системы для работы с данными.

Уровень содержит информацию о распределении дискового пространства, описание подробностей хранимых записей (реальная длина – байты), сведения о размещении записей, сжатии и шифровании данных.



Физический уровень

Ниже внутреннего уровня находится физический уровень, который контролируется операционной системой, в том числе и под руководством СУБД, причем разделение функций ОС и СУБД варьируется от системы к системе.

Физический уровень использует только известные операционной системе элементы.

Схема базы данных (резюме)

Общее описание базы данных называется схемой базы данных.

Для каждого уровня архитектуры ANSI-SPARC существуют свои схемы.

На внешнем уровне имеется несколько внешних схем или подсхем, которые обеспечивают различные представления данных для разных пользователей.

Для концептуального и внешнего уровня имеются соответственно *концептуальная* и *внешняя* схемы, которые для каждой БД существуют в единственном экземпляре.

Концептуальная схема описывает все элементы данных, связи между ними, ограничения целостности и т.п.

Внутренняя схема описывает определения хранимых записей, методов представления, описания полей данных, индексов и т.п.

СУБД отвечает за установление соответствия между этими схемами (проверка их непротиворечивости — например, чтобы можно было каждую внешнюю схему вывести на основе концептуальной схемы (на основе данных внутренней схемы), проверить соответствие имен и т.п.).

Концептуальная схема связана с внутренней посредством концептуально-внутреннего отображения (для поиска фактической записи на физическом устройстве хранения, которая соответствует логической записи в концептуальной схеме с учетом ограничений).

Каждая внешняя схема связана с концептуальной схемой с помощью внешне-концептуального отображения (отображение имен пользовательского представления на соответствующую часть концептуальной схемы). Например, пользователь хочет получить данные из БД, при этом его запрос преобразуется к виду принятому на концептуальном уровне (отображение внешний-концептуальный, например, изменение имен полей для получения логического описания), затем логическое описание отображается на внутренний уровень (отображение концептуальный-внутренний) для доступа к реальным данным, затем производится обратный процесс.

Следует различать описание БД и саму БД (описание БД – схема базы данных, создается в процессе проектирования (редко), а сама БД (детализация) – информация содержащаяся в таблицах может меняться часто).

Совокупность данных, хранящихся в БД на определенный момент времени называется *состоянием* БД (состояний может быть различное множество).

Развитие

На концепции трех схем основана усовершенствованная методология информационного моделирования IDEF1X.

Еще одна – Zachman Framework (1987). В этой структуре модель с тремя схемами превратилась в пирог из шести слоев.

Структура Захмана

Контекстуальный перечень (перечень важных сущностей)
Контекстуальная модель (сущности и связи)
Логическая модель (атрибуты и идентификаторы)
Физическая модель (ключи, требования к хранению)
Язык определения данных (листинг конфигурации)
Экземпляры данных (хранилище операций)

Независимость от данных

Независимость от данных — основополагающий принцип построения СУБД. В соответствии с этим принципом, в системе должны поддерживаться отдельные представления данных для пользователя («логическое представление») и для системных механизмов среды хранения БД («физическое представление»). Такое разделение избавляет пользователя от необходимости знания принятого способа хранения БД и позволяет динамически в процессе эксплуатации системы оптимизировать способ хранения БД для обеспечения более высокой производительности системы и (или) более рационального использования ресурсов памяти.

Различают два типа независимости от данных:

логическая независимость от данных — защищенность внешних схем от изменений, вносимых в концептуальную схему — добавление и удаление новых сущностей, атрибутов и связей должны осуществляться без изменений уже существующих внешних схем или изменения прикладных программ;

физическая независимость от данных — защищенность концептуальной схемы от изменений, вносимых во внутреннюю схему (изменения в структурах хранения, модификация индексов и т.п. должны осуществляться без изменения концептуальной и внешней схем, пользователи могут заметить изменения только по изменению производительности).

Система управления базами данных

СУБД (DBMS) — программное обеспечение, с помощью которого пользователи могут определять, создавать и поддерживать БД, а также осуществлять к ней контролируемый доступ.

Фактически это прослойка между БД и пользователем (прикладной программой) для скрывания особенностей хранения и управления данными (абстрагирование).

Функции СУБД

Для СУБД характерно наличие следующих функций и служб (сервисов) (первые восемь — Кодд, 1982):

1) СУБД должна предоставлять пользователям возможность сохранять, изменять и обновлять данные в БД. Это основная функция СУБД.

Способ реализации этой функции должен скрывать от пользователя детали физической реализации системы (абстрагирование).

2) СУБД должна иметь доступный конечным пользователям каталог, в котором хранится описание элементов данных — *системный каталог* — хранилище информации, описывающей данные БД (метаданные):

- имена, типы и размеры элементов данных;
- имена связей;
- накладываемые на данные ограничения поддержки целостности;
- имена санкционированных пользователей;
- описание схем архитектуры БД;
- статистические данные (счетчики событий);

Зачем нужен системный каталог:

- централизованный контроль доступа к данным;
- определение смысла данных для понимания их предназначения (семантика);
- упрощается определение владельца данных или прав доступа к ним;
- облегчается протоколирование изменений БД;
- последствия изменений могут быть определены до их внесения в БД (усиление безопасности и целостности данных);

3) СУБД должна иметь механизм, который гарантирует выполнение либо всех операций обновления данной транзакции, либо ни одной из них.

Транзакция – набор действий выполняемых пользователем (прикладной программой) с целью доступа или изменения БД (при этом в БД вносится сразу несколько изменений).

Если во время внесения изменений происходит сбой, например, вносимые данные вызывают нарушение целостности данных, то все изменения должны быть отменены (возвращение в непротиворечивое состояние БД).

4) СУБД должна иметь механизм, который гарантирует корректное обновление БД при параллельном выполнении операций обновления многими пользователями (реализуется в рамках поддержки транзакций).

5) СУБД должна предоставлять средства восстановления БД на случай ее повреждения или разрушения (реализуется в рамках поддержки транзакций).

6) СУБД должна иметь механизм, гарантирующий возможность доступа к БД только санкционированных пользователей. Соккрытие части ненужных для пользователя данных и защита от любого несанкционированного доступа.

7) СУБД должна обладать способностью к интеграции с коммуникационным программным обеспечением.

8) СУБД должна обладать инструментами контроля за тем, чтобы данные и их изменения соответствовали заданным правилам. Это называется *целостностью базы данных*, выражающееся в корректности и непротиворечивости хранимых данных.

Целостность является одним из типов защиты БД и выражается в виде ограничений или правил сохранения непротиворечивости данных при их изменении — если ограничения нарушаются, изменения в БД не вносятся.

9) СУБД должна обладать инструментами поддержки независимости программ от фактической структуры БД.

Эта независимость достигается за счет реализации механизма поддержки представлений, например, в рамках трехуровневой концепции.

Достичь полной логической независимости от данных очень сложно, поскольку СУБД легко адаптируется к добавлению нового объекта (атрибута, связи и т.п.), но не к их удалению.

Некоторые СУБД запрещают вносить изменения в существующие компоненты концептуальной схемы.

10) СУБД должна предоставлять некоторый набор различных вспомогательных служб, предназначенных, в том числе, для эффективного администрирования БД – импорт данных, резервное сохранение, реорганизация индексов, сборка мусора, перераспределение памяти.

ACID

Atomicity – атомарность.

Consistency – согласованность, целостность.

Isolation – изолированность.

Durability – стойкость.

Атомарность гарантирует, что никакая транзакция не будет зафиксирована в системе частично.

Будут либо выполнены все ее подоперации, либо не выполнено ни одной. Поскольку на практике невозможно одновременно и атомарно выполнить всю последовательность операций внутри транзакции, вводится понятие «отката» (rollback) – если транзакцию не удастся полностью завершить, результаты всех ее до сих пор произведенных действий будут отменены и система вернется во «внешне исходное» состояние – со стороны будет казаться, что транзакции и не было. Возможно, счетчики, индексы и другие внутренние структуры могут измениться, но, если СУБД разработана без ошибок, это не повлияет на внешнее ее поведение.

Согласованность является более широким понятием. Например, в банковской системе может существовать требование равенства суммы, списываемой с одного счета, сумме, зачисляемой на другой. Это бизнес-правило и оно не может быть гарантировано только проверками целостности, его должны соблюсти программисты при написании кода транзакций. Если какая-либо транзакция произведет списание, но не произведет зачисления, то система останется в некорректном состоянии и свойство согласованности будет нарушено.

В ходе выполнения транзакции согласованность не требуется. В вышеприведенном примере списание и зачисление будут, скорее всего, двумя разными подоперациями в рамках одной транзакции и между их выполнением внутри транзакции будет видно несогласованное состояние системы.

При выполнении требования изоляции никаким другим транзакциям эта несогласованность не будет видна. Атомарность гарантирует, что транзакция либо будет полностью завершена, либо ни одна из операций транзакции не будет выполнена. Тем самым эта промежуточная несогласованность является скрытой.

Изолированность – во время выполнения транзакции параллельные транзакции не должны оказывать влияния на ее результат. Изолированность – требование дорогое, поэтому во многих реальных БД существуют режимы, не полностью изолирующие транзакцию.

Стойкость – независимо от проблем на нижних уровнях (к примеру, обесточивание системы или сбои в оборудовании) изменения, сделанные успешно завершенной транзакцией, должны остаться сохраненными после возвращения системы в работу. Другими словами, если пользователь получил подтверждение от системы, что транзакция выполнена, он может быть уверен, что сделанные им изменения не будут отменены из-за какого-либо сбоя.

Модель «Сущность-связь»

Модель сущность-связь (или ER-модель — Entity-Relationship model) описывает взаимосвязанные вещи, представляющие интерес в конкретной области знаний.

Базовая модель ER состоит из типов сущностей (которые классифицируют важные вещи) и определяет отношения, которые могут существовать между сущностями (экземплярами этих типов сущностей).

В разработке программного обеспечения модель ER обычно формируется для представления вещей, о которых необходимо помнить при выполнении бизнес-процессов. В этом качестве ER-модель становится абстрактной моделью данных, которая определяет структуру данных или информации, которая может быть реализована в базе данных, обычно реляционной базе данных.

В процессе проектирования баз данных схема, созданная на основе ER-модели, преобразуется в конкретную схему базы данных на основе выбранной модели данных — реляционной, объектной, иерархической, сетевой, ...

ER-модель представляет собой формальную конструкцию, которая сама по себе не предписывает никаких графических средств её визуализации и может быть просто словесным описанием в виде текста.

В качестве стандартной графической нотации, с помощью которой можно визуализировать ER-модель, используется *диаграмма «сущность-связь»* (Entity-Relationship diagram, ERD, ER-диаграмма).

Понятия «ER-модель» и «ER-диаграмма» обычно не различают, однако для визуализации ER-моделей могут быть использованы и другие графические нотации, либо вместо графического представления может использоваться текстовое описание.

Модель была разработана Питером Ченом для проектирования базы данных и опубликована в статье 1976 года с вариантами согласно данной идеи, существовавшими ранее. Он же предложил и самую популярную графическую нотацию для ER-модели.

ER-диаграмма

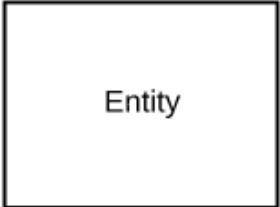
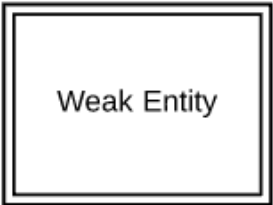
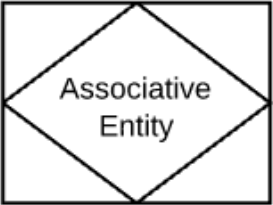
Схема «сущность-связь» (также ERD или ER-диаграмма) — это разновидность блок-схемы, где показано, как разные «сущности» (люди, объекты, концепции и так далее) связаны между собой внутри системы.

ER-диаграммы чаще всего применяются для проектирования и отладки реляционных баз данных в сфере образования, исследования и разработки программного обеспечения и информационных систем для бизнеса.

ER-диаграммы (или ER-модели) полагаются на стандартный набор символов, включая прямоугольники, ромбы, овалы и соединительные линии, для отображения сущностей, их атрибутов и связей. Эти диаграммы устроены по тому же принципу, что и грамматические структуры — сущности выполняют роль существительных, а связи — глаголов.

Символы ERD-сущностей

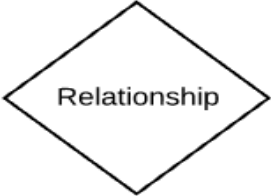
Под понятием «сущности» подразумеваются объекты или понятия, несущие важную информацию. С точки зрения грамматики, они, как правило, обозначаются существительными, например, «товар», «клиент», «заведение» или «промоакция». Ниже представлены три наиболее распространенных типа сущностей, используемых в ER-диаграммах.

Символ	Название	Описание
	Независимая (сильная) сущность	Не зависит от других сущностей и часто называется «родительской», так как у нее в подчинении обычно находятся слабые сущности. Независимые сущности сопровождаются первичным ключом, который позволяет идентифицировать каждый экземпляр сущности.
	Зависимая (слабая) сущность	Сущность, которая зависит от сущности другого типа. Не сопровождается первичным ключом и не имеет значения в схеме без своей родительской сущности.
	Ассоциативная сущность	Соединяет экземпляры сущностей разных типов. Также содержит атрибуты, характерные для связей между этими сущностями.

Символы ERD-связей

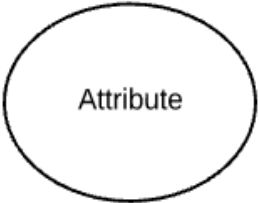
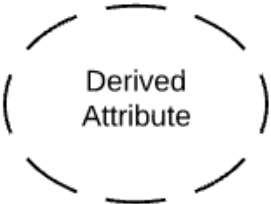
Связи используются в схемах «сущность-связь» для обозначения взаимодействия между **двумя сущностями**.

Грамматически связи, как правило, выражаются глаголами, например, «назначить», «закрепить», «отследить», и несут полезную информацию, которую невозможно получить, опираясь только на типы сущностей.

Символ	Название	Описание
	Связь	Отношение между сущностями
	Слабая связь	Связь между зависимой сущностью и ее «хозяином».

Символы ERD-атрибутов

ERD-атрибуты характеризуют сущности, позволяя пользователям лучше разобраться в устройстве базы данных. Атрибуты содержат информацию о сущностях, выделенных в концептуальной ER-диаграмме.

Символ	Название	Описание
	Атрибут	Характеризует сущность, а также отношения между двумя или более элементами (даты зачисления/увольнения)
	Многозначный атрибут	Атрибут, которому может быть присвоено несколько значений (дни работы заведения).
	Производный атрибут	Атрибут, чье значение можно вычислить, опираясь на значения связанных с ним атрибутов.

Символы физических ER-диаграмм

Физическая модель данных — самый детальный уровень ER-схем, где представлен процесс добавления информации в базу данных. Физические модели «сущность-связь» отображают всю структуру таблицы, включая названия столбцов, типы данных, ограничения столбцов, первичные и внешние ключи, а также отношения между таблицами.

Как показано ниже, таблицы представляют собой еще один способ отображения сущностей. Вот ключевые составляющие таблиц «сущность-связь»:

Поля

Поля — это участки таблицы, где задаются атрибуты сущностей. Под атрибутами обычно подразумеваются столбцы базы данных, которая моделируется по принципу «сущность-связь».

Bank	
	InterestRate
	LoanAmount

В примере InterestRate («процентная ставка») и LoanAmount («сумма займа») оба являются атрибутами сущности и оба определяются, как поля.

Ключи

Ключи — один из способов категоризации атрибутов.

ER-диаграммы помогают пользователям моделировать базы данных посредством таблиц, которые обеспечивают им упорядоченность, эффективность и высокую скорость работы.

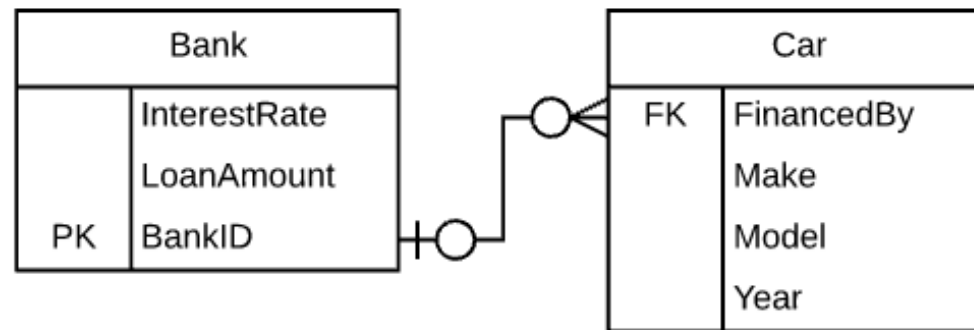
Ключи применяются с целью максимально эффективно связать между собой разные таблицы в базе данных.

Первичные ключи

Первичный ключ — это атрибут или сочетание атрибутов, идентифицирующих один конкретный экземпляр сущности.

Внешние ключи (foreign keys)

Внешний ключ создается каждый раз, когда атрибут привязывается к сущности посредством единичной или множественной связи.



Типы

Под типом подразумевается тип данных в соответствующем поле таблицы.

Однако это также может быть и тип сущности, то есть описание ее составляющих.

Например, у сущности «книга» могут быть следующие типы:

- автор;
- название;
- дата публикации;
- издательство.

Нотация ER-диаграмм

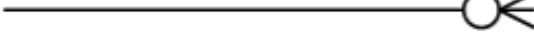
Нотаций много (Бахмана, ОМТ, IDEF, UML), однако наиболее интуитивной является нотация Crow's Foot («вороньи лапки»).

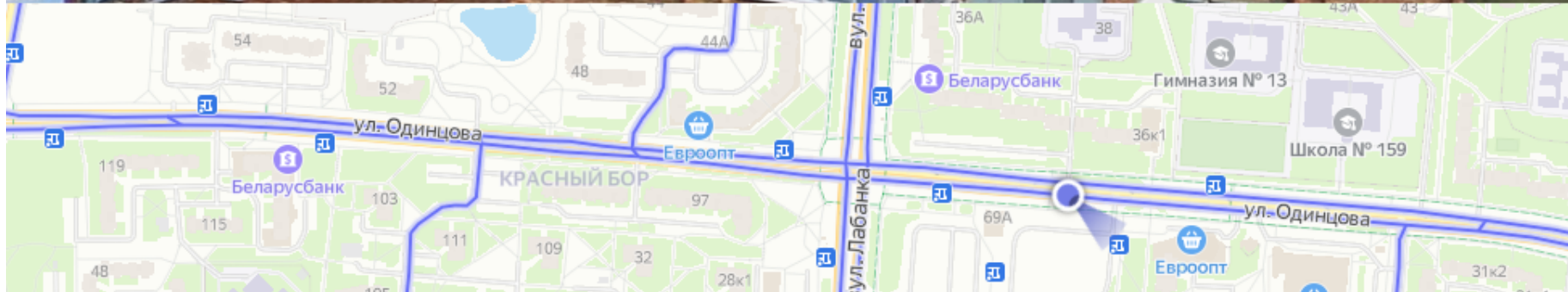
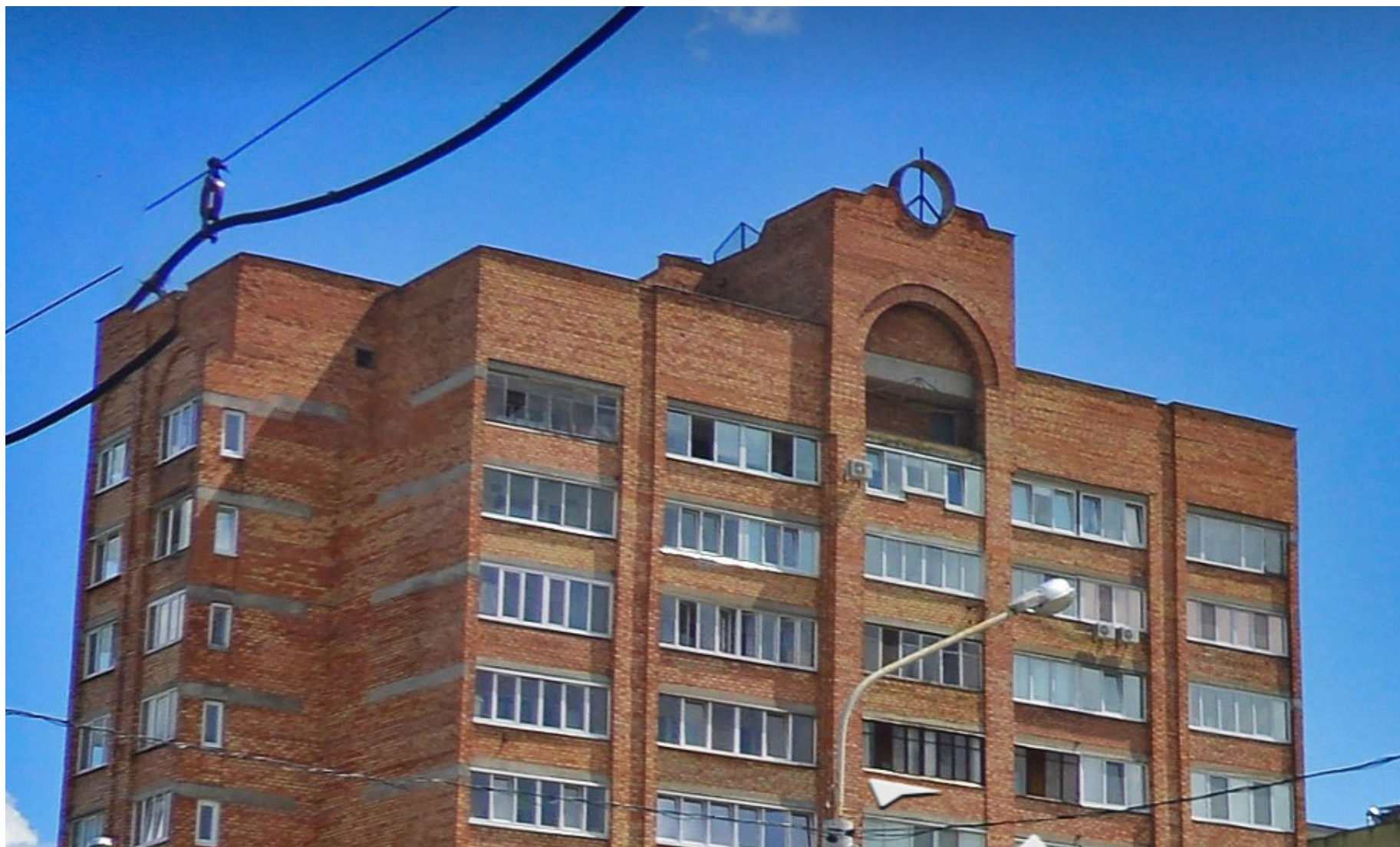
Кардинальность и ординальность

Под *кардинальностью* подразумевается максимальное число связей, которое может быть установлено между экземплярами разных сущностей.

Ординальность, в свою очередь, указывает минимальное количество связей между экземплярами двух сущностей.

Кардинальность и ординальность отображаются на соединительных линиях согласно выбранному формату нотации.

	One
	Many
	One (and only one)
	Zero or one
	One or many
	Zero or many



Установка и первичная настройка postgres

PostgreSQL — это продвинутая система управления объектно-реляционными базами данных. Базовый пакет postgresql Сервер PostgreSQL можно найти в подпакете postgresql-server.

Пакет	Описание
postgresql	содержит клиентские программы, необходимые для доступа к серверу СУБД PostgreSQL, а также документацию в формате HTML для всей системы. Эти клиентские программы могут располагаться на том же компьютере, что и сервер PostgreSQL, или на удаленном компьютере, который обращается к серверу PostgreSQL через сетевое соединение.
postgresql-docs	psql – консольный клиент. HTML-документация иногда бывает отдельным пакетом
postgresql-server	ПО сервера.

Установка на локальной системе

```
# dnf postgresql postgresql-docs postgresql-server
```

В процессе установки будет создан пользователь postgres, который является хозяином каталога /usr/local/pgsql/data, в котором будет располагаться кластер базы данных

Первичная настройка

```
$ su postgres -  
$ /usr/local/pgsql/bin/initdb -D /usr/local/pgsql/data  
$ /usr/local/pgsql/bin/pg_ctl -D /usr/local/pgsql/data -l logfile start  
$ /usr/local/pgsql/bin/createdb test  
$ /usr/local/pgsql/bin/psql test
```

pg_ctl – утилита для управления сервером.

Запуск сервера в системе с systemd

```
$ sudo systemctl enable postgresql  
$ sudo systemctl start postgresql
```

Проверка работы

```
$ sudo -u postgres psql -c "SELECT version();" 
```

Создание пользователя и базы данных

```
$ sudo -i -u postgres  
$ psql
```

```
$ sudo -u postgres createuser -D -A -P user  
$ sudo -u postgres createdb -O user user_db
```


Базы данных

Лекция 02 – Основные концепции. Терминология

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2022

2023.09.08

Оглавление

Основная терминология.....	3
Индексирование.....	19
Статические хеш-индексы.....	23
Статическое хеширование.....	23
Расширяемое хеширование.....	24
Индексы на основе В-дерева.....	25

Основная терминология

Существует много вариантов терминов, используемых при описании данных.

Широко признанным авторитетом в области баз данных, не связанным ни с какой конкретной фирмой-производителем ЭВМ, является CODASYL (COncference on DAta SYstems Languages – Ассоциация по языкам для систем обработки данных).

Байт (Byte)

Байт – наименьшая адресуемая группа битов (обычно состоит из 8 бит[ов]).

Элемент данных (Data Element)

Элемент данных – наименьшая единица поименованных данных. Может состоять из любого количества битов или байтов. Часто элемент данных называют *полем*.

Элементом данных может быть некоторая величина, например, в счет-фактуре.

Агрегат данных

Агрегат данных – поименованная совокупность элементов данных внутри *записи*, рассматриваемая как единое целое.

Например, агрегат данных **ДАТА** может состоять из элементов данных **МЕСЯЦ, ДЕНЬ, ГОД**.

Существуют два типа агрегатов данных – векторы и повторяющиеся группы.

Вектор – одномерная упорядоченная совокупность элементов данных (например, **ДАТА**).

Повторяющаяся группа – совокупность данных, которые встречаются несколько раз в экземпляре записи, например прием и выдача вкладов в записи счета сберкасс.

В повторяющуюся группу могут входить отдельные элементы данных, векторы, агрегаты данных или другие повторяющиеся группы.

Допускается вложенность агрегатов.

Запись (Record)

Запись — поименованная совокупность элементов или агрегатов данных.

Запись — агрегатный тип данных, инкапсулирующий набор значений различных типов без их сокрытия.

При чтении из базы данных может быть прочитана логическая запись как целиком, так и частично — в ряде случаев логической записью базы данных является структура данных, в которую входит несколько групп элементов данных (*сегментов*), не все из которых имеет смысл читать одновременно.

Верхний предел для числа возможных экземпляров записи конкретного типа зависит, в основном, от наличия необходимого оборудования для хранения и доступа.

Верхний предел для числа повторяющихся групп внутри записи обычно ограничен небольшим числом.

Сегмент (Segment)

Существует мнение, что нет необходимости различать агрегат данных и запись, поскольку и то и другое является совокупностью элементов данных. В терминологии, используемой фирмой IBM, а также в других источниках и агрегат данных, и запись называют сегментом.

Сегмент состоит из одного или нескольких элементов данных (обычно нескольких) и является основным квантом данных, передаваемым прикладной программе или получаемым от нее под управлением программного обеспечения, работающего с базой данных.

Физический уровень

- уровень блоков;
- уровень файлов.

Таблица

Записи *одинаковой структуры* обычно организуются в двумерные таблицы и представлены в ней строками. Элементы данных записей организованы в виде столбцов. Таблицы организованы в *базы данных*. Ниже БД, содержащая 2 таблицы, связанные через поле DEPT_NO.

DEPARTMENT – Отделы

DEPT_NO	DEPT_NAME	BUDGET
D1	Marketing	5 000 000.00
D2	Development	25 000 000.00
D3	Research	10 000 000.00

EMPLOYEES – Служащие

EMP_NO	EMP_NAME	DEPT_NO	SALARY
E1	Ivanov	D1	40 000.00
E2	Smirnov	D2	45 000.00
E3	Kuznetsov	D1	41 000.00
E4	Popov	D2	43 000.00
E5	Vasiljev	D3	55 000.00
E6	Petrov	D2	45 000.00

В разное время и в различных контекстах термин **запись** может означать экземпляр записи или тип записи, логическую запись или физическую запись, хранимую запись или виртуальную запись, а возможно, и еще что-нибудь.

В связи с этим в формальной реляционной модели термин запись не используется — вместо него применяется термин *кортеж* (tuple). Также в реляционной модели не используется термин *поле*, вместо него используется термин *атрибут*.

Кортеж

Термин кортеж в отношении к базам данных приблизительно соответствует понятию строки в таблице (так же, как термин отношение приблизительно соответствует понятию таблицы).

Соответственно, кортеж состоит из атрибутов.

Кортежем вообще называется набор взаимосвязанных величин.

Кортеж, содержащий две величины, называется *парой* (pair).

Кортеж, содержащий N величин, называется N-кортежем.

Таблица состоит из набора кортежей, каждый из которых содержит элементы данных одинакового типа. Она, таким образом, представляет собой двумерную матрицу элементов данных.

Элементы данных обычно обрабатываются группами.

В различных системах программного обеспечения эти группы называются по-разному (сегмент, кортеж).

Общепринятым является термин запись.

Отношение

Отношение — это формальное название таблицы.

Например, можно сказать, что база данных отделов и служащих, представленная выше, содержит два отношения.

В настоящее время в неформальном контексте термины отношение и таблица принято считать синонимами. На практике в подобном контексте термин таблица используется гораздо чаще, чем термин отношение.

Файл

Файл — поименованная совокупность всех экземпляров логических записей заданного типа.

В простом файле в каждой логической записи содержится одинаковое число элементов данных (рисунок 1).

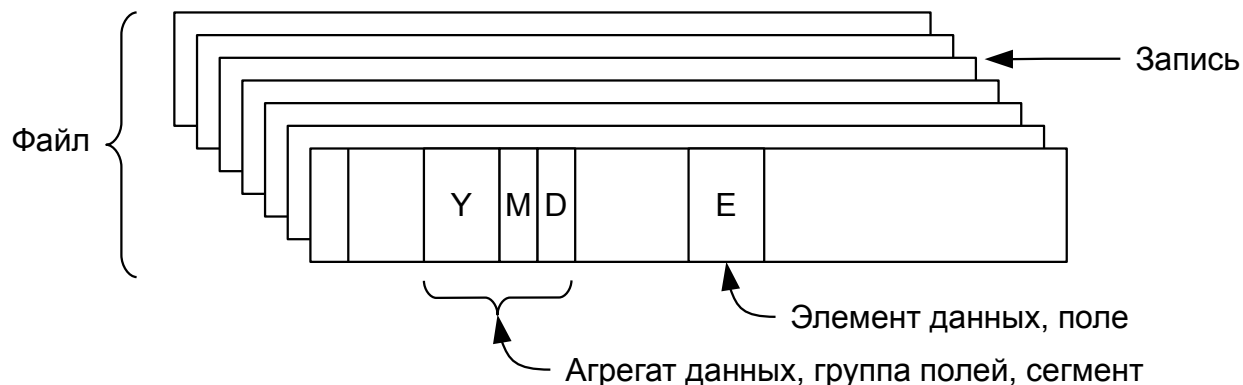


Рисунок 1 — Термины, используемые при описании данных с точки зрения прикладного программиста

В более сложном файле из-за наличия повторяющихся групп записи могут состоять из различного числа элементов данных.

База данных

База данных — совокупность экземпляров различных типов записей и отношений между записями, агрегатами данных, элементами данных.

Пример – счет-фактура

Некоторая величина в счет-фактуре является *элементом данных*.

Агрегатом данных может быть строка в счет-фактуре. Если она повторяется несколько раз, она будет *повторяющейся группой*.

Строка может рассматриваться, как отдельно адресуемая группа данных, в этом случае она будет *сегментом*.

Весь счет-фактура может рассматриваться, как *логическая запись*.

Вся совокупность записей счетов-фактур обычно хранится в одной или нескольких *таблицах*.

Физическая структура хранения логических записей на устройстве хранения зависит от СУБД.

Виртуальные данные

Слово *виртуальный*, относящееся к техническим средствам или данным, указывает на то, что некоторый элемент в запросе представляется прикладному программисту существующим, тогда как фактически в представленном виде он отсутствует.

Программист может обращаться за виртуальными данными, которые им предполагаются существующими, но не существуют фактически, по крайней мере в данной форме. Каждый раз, когда программист обращается за этими данными, они генерируются определенным образом, что возможно обеспечивает более компактную и удобную форму их хранения.

Прозрачные средства и данные

Виртуальное только представляется существующим, прозрачное представляется несуществующим, но на самом деле существует. Многие данные и механизмы, используемые при их хранении и передаче, могут быть скрыты от программиста (view).

База данных

Базу данных можно определить как совокупность взаимосвязанных хранящихся вместе данных при наличии такой минимальной избыточности, которая допускает их использование оптимальным образом для одного или нескольких приложений.

**Данные хранятся таким образом,
чтобы они были независимы от программ,
использующих эти данные.**

Для добавления новых или модификации существующих данных, а также для поиска данных в базе данных применяется общие методы (способы).

Данные структурируются таким образом, чтобы была обеспечена возможность дальнейшего наращивания приложений.

Говорят, что система содержит совокупность баз данных, если эти базы данных структурно полностью самостоятельны.

База данных может разрабатываться для:

- пакетной обработки данных;
- обработки в реальном времени (жесткие ограничения на время обработки запроса);
- оперативной обработки (в этом случае обработка каждой транзакции завершается к определенному моменту времени, но при этом на время обработки не накладывается жестких ограничений, существующих в системах реального времени).

Во многих базах данных предусмотрена совокупность вышеуказанных методов обработки, причем все три вида обработки могут исполняться параллельно.

Избыточность

База данных представляет возможность в значительной степени избавиться от избыточности.

Базу данных иногда определяют как *неизбыточную совокупность элементов данных*, однако в действительности избыточность в определенной степени допускают во многих базах данных с целью уменьшения времени доступа к данным и/или упрощения способов адресации.

Иногда некоторые записи дублируются для того, чтобы обеспечить возможность быстрого восстановления данных при их случайной потере.

Чтобы база данных была неизбыточной и удовлетворяла другим требованиям, приходится идти на компромисс. В этом случае говорят об *управляемой*, или *минимальной*, избыточности или о том, что хорошо разработанная база данных свободна от *излишней* избыточности.

Неуправляемая избыточность имеет несколько недостатков:

- 1) хранение нескольких копий данных приводит к дополнительным затратам;
- 2) что особенно серьезно, приходится выполнять многократные операции обновления для нескольких избыточных копий.

Избыточность поэтому обходится значительно дороже в тех случаях, когда при обработке файлов обновляется большое количество информации или, что еще хуже, часто вводятся новые элементы данных или уничтожаются старые.

- 3) вследствие того что различные копии данных могут соответствовать различным стадиям обновления, информация, выдаваемая системой, может быть противоречивой.

Если не использовать базы данных, то при обработке большого количества информации появится так много избыточных данных, что фактически станет невозможным сохранять их все на одном и том же уровне обновления.

Непрерывное расширение

Одной из наиболее важных характеристик большинства баз данных является их постоянное изменение и расширение.

По мере добавления новых типов данных или при появлении новой бизнес-логики должна быть обеспечена возможность быстрого изменения структуры базы данных.

Реорганизация базы данных должна осуществляться по возможности без перезаписи прикладных программ и в целом вызывать минимальное количество преобразований.

Простота изменения базы данных может оказать большое влияние на развитие приложений для баз данных, используемых в управлении производством.

Требования к обработке обычно изменяются непредсказуемым образом. При этом если возникает необходимость модификации выбранных структур данных, то приходится соответственно перезаписывать и отлаживать прикладные программы.

Чем большее количество прикладных программ имеется в наличии на установке, тем более дорогой становится эта процедура.

Поэтому одним из важных свойств базы данных является независимость данных и использующих их прикладных программ друг от друга в том смысле, что изменение одних не приводит к изменению других.

В действительности же полностью независимыми данные бывают так же редко, как и полностью избыточными.

Независимость данных определяется с различных точек зрения. Сведения, которыми должен располагать программист для доступа к данным, различны для различных баз данных.

Тем не менее независимость данных — это одна из основных причин использования систем управления базами данных.

Установление многосторонних связей

В том случае, когда один набор элементов данных используется для многих приложений, между элементами этого набора устанавливается множество различных взаимосвязей, необходимых для соответствующих прикладных программ.

Организация базы данных в значительной степени зависит от реализации связей между элементами данных и записями.

В базе данных, используемой многими приложениями, должны быть установлены многочисленные промежуточные взаимосвязи между данными. В этом случае при хранении и использовании данных контролировать их правильность, обеспечивать их защиту и секретность труднее, чем при хранении данных в простых, несвязанных файлах. Что касается обеспечения секретности данных и восстановления их после сбоев, то этот вопрос является очень важным при конструировании баз данных.

!!! В подавляющем количестве систем средства управления базами данных обеспечивают возможность пользователи использовать данные таким способом, который не был предусмотрен разработчиками системы. Это значит, что

**пользователи могут обращаться к БД с запросами,
которые заранее в ней не предусматривались.**

Наличие такой возможности означает организацию данных в системе, при которой доступ к ним осуществляется по различным путям, причем одни и те же данные могут использоваться для ответа на различные запросы.

При организации данных в виде ориентированных на конкретное приложение схем, в которых доступ к данным производится одним и тем же определенным способом, описанная выше возможность отсутствует. Вся существенная информация об объектах должна сохраняться одновременно и полностью, а не только та ее часть, которая необходима для конкретного варианта использования.

Идентификатор объекта

Программисту или администратору базы данных необходимо иметь возможность обращаться к записи или к кортежу, связанному с данным объектом.

Для этого необходимо иметь возможность идентифицировать запись или группу записей и располагать средствами их обнаружения в таблице.

С этой целью один из элементов данных обычно определяется в качестве идентификатора объекта.

Идентификатор объекта должен быть уникальным – никакой другой объект не может иметь то же значение данного элемента данных.

Идентификатором объекта **СЛУЖАЩИЙ** *может быть* **НОМЕР-СЛУЖАЩЕГО**.

Идентификатором объекта **СЧЕТ** *может быть* **НОМЕР-СЧЕТА**.

Иногда требуется более одного элемента данных для идентификации записи.

Например, для того чтобы идентифицировать банковскую операцию для записи **СЧЕТ**, необходимы идентификаторы **НОМЕР-СЧЕТА** и **НОМЕР-ОПЕРАЦИИ**.

Для идентификации записи о рейсах самолетов необходимы **НОМЕР-РЕЙСА** и **ДАТА**. Одного номера рейса недостаточно, так как вылет с одним и тем же номером рейса может происходить каждый день.

Первичный ключ

Идентификатор объекта рассматривается как ключ записи или группы записей. Такой ключ называется *первичным ключом*. (Primary Key)

Существуют также и вторичные ключи, которые не идентифицируют объект (единственным образом) (Secondary Key).

В тех случаях, когда в качестве ключа используются несколько элементов данных (полей, агрегатов), их обычно указывают как соединенные символом «плюс», например **НОМЕР-РЕЙСА + ДАТА**. Такое сочетание называется сцепленным ключом.

Иногда для формирования первичного ключа необходимы три, четыре и более элементов данных. Запись о кассовом сборе, например, в приведенной ниже группе идентифицируется комбинацией элементов **КИНОФИЛЬМ, КИНОТЕАТР, ДАТА**.

КИНОФИЛЬМ	КИНОТЕАТР	ДАТА	КАССОВЫЙ-СБОР
------------------	------------------	-------------	----------------------

Первичным ключом поэтому является **КИНОФИЛЬМ+КИНОТЕАТР+ДАТА**.

Первичный ключ — это такой элемент данных или такая совокупность элементов данных, которая единственным образом идентифицирует одну запись или группу записей.

Первичный ключ имеет большое значение, так как он используется для определения местоположения записи с помощью индексов или других методов адресации.

Избыточные значения

Значения атрибутов не обязательно должны запоминаться вместе с ключами таким способом, как это показано ниже.

Дело_№	Фамилия Имя Отчество	Пол	Звание	Рожд	Отдел	Должность
58231	Сидоров Иван Петрович	муж	06	1971.03.12	002	Начальник станции
12874	Смирнова Анна Ивановна	жен	08	1988.10.11	014	Оператор
31774	Бутусова Ульяна Алексеевна	жен	11	1999.08.18	002	Инженер-программист

Звание	
ID	Name
04	Полковник
05	Подполковник
...	
17	Кадет
18	Вольноопределяющийся

Должность	
08	Начальник станции
09	Инженер-программист
10	Оператор
...	
57	Вахтер
58	Повар

Почти всегда наблюдается существенная избыточность значений атрибутов в тех случаях, когда они хранятся так, как показано в атрибуте **Должность**.

Для того чтобы устранить избыточность, значения атрибутов могут храниться отдельно и снабжаться указателями на них со стороны ключей (Звание).

Вторичные ключи (Внешние ключи)

Можно использовать ключ, который идентифицирует не уникальную запись или кортеж, а все записи или группы, имеющие определенное свойство. Такой ключ называется *вторичным*.

Значение атрибута **ЦВЕТ**

Как вторичный ключ может быть использован атрибут **Отдел**. Этот ключ может быть использован для идентификации тех служащих, которые числятся в определенном отделе.

Иногда таблица имеет много вторичных ключей, которые используются для поиска записей с данными характеристиками.

Связь вторичного ключа с элементами данных или с группами, к которым он относится, может быть реализована различными способами. Один из таких способов – использование вторичного индекса. Вторичный индекс использует вторичный ключ как вход, а на выходе предоставляет первичный ключ, в результате чего может быть идентифицирована нужная запись или группа записей.

Элементарная форма вторичного индекса – инвертированный список.

Звание	Дело_№
05	13875
05	76014
06	58231
06	32960
08	12874

Должность	Дело_№
08	58231
08	32960
09	31774
09	11749
09	55813

Инвертированный список содержит все значения вторичного ключа и хранит вместе с каждым его значением соответствующие идентификаторы записи.

Инвертированные файлы

Существуют два основных способа, с помощью которых данные могут быть организованы и использованы.

Первый способ определяется тем, что каждый кортеж содержит значения атрибутов данного объекта или N:1 ссылки на справочные таблицы, содержащие атрибуты (LUT– LookUp Table).

Второй способ является инверсией первого. С помощью этого способа могут быть получены идентификаторы объектов, связанных с данным атрибутом.

Первый способ хранения данных полезен для ответа на вопрос: *каковы свойства данного объекта?*

Второй – для ответа на вопрос: *какие объекты имеют данное свойство?*

Полностью инвертированный файл – это такой файл, который хранит идентификаторы объектов, связанные с конкретным значением каждого атрибута.

Частично инвертированный файл является более простым и хранит идентификаторы объектов, связанные со значениями некоторых (но не всех) атрибутов.

Предикаты

предикат – это простое условие (терм) или логическая комбинация простых условий;

простое условие – это операция сравнения двух выражений;

выражение состоит из операций с константами и именами полей;

константа – это значение predetermined типа, например целое число или строка.

Запросы

Форма	Тип запроса	Пример
$A(E) = ?$	Обычный запрос атрибута. Каково значение атрибута A объекта E ?	Заработок торгового агента № 271 за последний месяц
$A(?) \text{ Rel } V$ Rel: $\{=, \neq, <, >, \geq, \leq\}$	Какие объекты имеют заданное значение атрибута. Запрос в инвертированный файл: Какой объект E имеет значение атрибута A , равное (неравное, меньше, больше, ...) V ?	Кто из торговых агентов заработал больше 2000 долл. за последний месяц?
$?(E) \text{ Rel } V$	Перечислить все атрибуты, имеющие заданный набор значений для данного объекта. Запрос менее простой: $?(E)=V$ — Какой атрибут или какие атрибуты объекта E имеют значение V ?	За какие месяцы заработки торгового агента No 271 превысили 2000 долл?
$?(E)=?$	Запрос на получение всей информации о данном объекте. Запрос на значения всех атрибутов объекта E .	Сообщить всю хранимую информацию о торговом агенте No 271
$A(?)=?$	Перечислить значения данного атрибута для каждого объекта. Запрос на значения атрибута A для всех объектов.	Перечислить заработки за последний месяц каждого торгового агента
$?(?) \text{ Rel } V$	Перечислить все атрибуты объектов, имеющие данное значение. Сложный запрос на все атрибуты всех объектов, имеющие значение V .	Для каждого торгового агента определить месяц, когда его заработок превышал 2000 долларов.

Индексирование

При выполнении запроса к таблице пользователя часто интересуют только некоторые записи в ней, например записи, имеющие определенное значение в некотором поле.

Индекс – это файл, помогающий движку базы данных быстро найти такие записи, не просматривая всю таблицу.

Наиболее распространенными способами реализации индексов являются:

- статическое хеширование;
- расширяемое хеширование;
- В-деревья.

Эффективность выполнения некоторых запросов может значительно повысить подходящая организация таблиц. Представим телефонный справочник – по сути, большая таблица, записи которой содержат имена, адреса и номера телефонов абонентов.

Фамилия Имя Отчество	Адрес	Тел. номер
----------------------	-------	------------

Эта таблица отсортирована сначала по фамилиям, а затем по именам.

Предположим, что нам нужно узнать номер телефона конкретного человека.

Существенно ускорить поиск помогает тот факт, что записи отсортированы по имени.

Например, можно выполнить бинарный поиск, который в худшем случае потребует просмотреть $\log_2 N$ записей, где N – общее число записей в справочнике. Это очень быстро. Например, если $N = 1\,000\,000$, то $\log_2 N < 20$, то есть чтобы найти нужного человека в справочнике, содержащем номера миллиона человек, никогда не придется просматривать больше 20 записей.

Телефонный справочник отлично приспособлен для поиска абонента по имени, но не подходит для быстрого поиска, например по номеру телефона или по адресу.

Единственный способ получить эту информацию из телефонной книги – просмотреть каждую запись в ней, что потребует просмотреть в среднем $N/2$ записей.

Для эффективного поиска абонентов по номеру телефона нужен справочник, отсортированный по номерам телефонов (такие справочники еще называют «*обратными телефонными справочниками*»). Однако такой справочник удобен, только если известен номер телефона. Если необходимо найти в таком справочнике номер телефона конкретного абонента, то снова придется просмотреть каждую запись.

Этот пример наглядно иллюстрирует важное ограничение организации таблиц – таблицу можно организовать (упорядочить) только каким-то одним способом. Если необходимо, чтобы поиск выполнялся быстро и по номеру телефона и по имени абонента, потребуются две отдельные копии телефонной книги, каждая со своей организацией. А если понадобится возможность быстро находить номер телефона по известному адресу, потребуется третий экземпляр телефонного справочника, отсортированный по адресу. Это справедливо и в отношении таблиц в базе данных.

Чтобы иметь возможность эффективно находить в таблице записи с определенным значением некоторого поля, нужна версия таблицы, организованная по этому полю.

Механизмы баз данных это удовлетворяют эту потребность, поддерживая *индексы*.

Таблица может иметь один или несколько индексов, каждый из которых определен для отдельного поля. Каждый индекс действует подобно версии таблицы, организованной по соответствующему полю.

Индекс — это файл с индексными записями. Файл индекса имеет одну индексную запись для каждой записи в соответствующей таблице. Каждая индексная запись имеет два поля, которые хранят идентификатор соответствующей записи в таблице (значение первичного ключа) и значение индексируемого поля в этой записи.

ID	Name	Address	Phone
----	------	---------	-------

Name	ID
------	----

Address	ID
---------	----

Phone	ID
-------	----

На поля в индексной записи можно условно ссылаться по именам `record_id` и `field_value`. Движок БД организует записи в файле индекса по полю `field_value`.

Можно считать для простоты, что записи в индексе по полю `field_val` отсортированы. В этом случае для того, чтобы найти запись по значению `field_val`, нужно выполнить бинарный поиск и извлечь из поля `record_id` . первичный ключ искомой записи.

Аналогично можно найти *все записи* со значением `field_val`. Бинарный поиск выдаст `record_id` первой записи с атрибутом `field_val`. Поскольку в индексе все записи отсортированы по `field_val`, остается только последовательно читать следующие записи в индексе, пока получаемый `field_val` будет удовлетворять поставленному условию.

18, 20

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	3	5	6	9	9	10	13	18	18	18	18	32	37	?
b0							b1				b2				e0
b0							b1				b2	b4	b3		e0

Насколько эффективно такое применение индексов?

В отсутствие индексов лучшее, что можно предпринять при обработке любого запроса, – выполнить последовательный поиск в таблице.

Если атрибут в таблице равномерно распределен и имеет вид «А = константа», справедливо правило – полезность индекса для поля А в таблице пропорциональна количеству уникальных значений в этом поле.

Согласно этому правилу, индекс наиболее полезен, когда индексируемое поле является первичным ключом таблицы, потому что каждая запись имеет свое уникальное значение ключа.

И наоборот, индекс будет бесполезен, если число уникальных значений в поле А меньше количества записей в блоке чтения.

Статические хеш-индексы

Статическое хеширование – это, пожалуй, самый простой способ реализации индексов. Это не самая эффективная стратегия, но достаточно простая и понятная для реализации.

Статическое хеширование

Статический хеш-индекс использует фиксированное число N ячеек, пронумерованных от 0 до $N-1$. Индекс также использует хеш-функцию, отображающую значения в ячейки. По результатам хеширования поля `field_val` каждая индексная запись помещается в свою ячейку.

Статический хеш-индекс действует следующим образом:

- чтобы сохранить индексную запись, ее нужно поместить в ячейку, вычисленную хеш-функцией;
- чтобы найти индексную запись, нужно вычислить хеш-функцию ключа поиска и просмотреть соответствующую ячейку;
- чтобы удалить индексную запись, нужно сначала найти ее (как указано выше), а затем удалить из ячейки.

Стоимость поиска с помощью хеш-индекса обратно пропорциональна количеству ячеек. Если индекс содержит B блоков и N ячеек, то на каждую ячейку будет приходиться около B/N блоков, поэтому для поиска в ячейке потребуются обратиться к B/N блоков.

$N = 17, B = 8$ ячейка полностью в одном блоке -> гарантированно одно чтение

$N = 17, B = 20$ ячейка располагается в двух блоках -> может быть два чтения

Расширяемое хеширование

Стоимость поиска при использовании индексов на основе статического хеширования обратно пропорциональна количеству ячеек – чем больше ячеек используется, тем меньше блоков в каждой из них.

Наиболее оптимально, когда каждую ячейку приходится ровно один блок.

Если бы размер индекса никогда не изменялся, то рассчитать это идеальное количество ячеек легко. Но на практике индексы растут по мере добавления новых записей в базу данных.

Если исходить из текущего размера индекса, то впоследствии, при его увеличении, каждая ячейка будет содержать несколько блоков индекса.

Если выбрать большее количество ячеек, ориентируясь на потребности в будущем, то пустые и почти пустые в данный момент ячейки будут напрасно расходовать значительный объем дискового пространства до тех пор, пока индекс не вырастет и их не заполнит.

Эту проблему решает стратегия, известная как *расширяемое хеширование*.

Суть этой стратегии заключается в использовании достаточно большого количества ячеек, чтобы гарантировать, что каждая ячейка никогда не будет содержать более одного блока.

Но теперь несколько ячеек могут располагаться в одном блоке, что позволяет большому количеству ячеек совместно использовать меньшее количество блоков и тем самым не допустить напрасного расходования дискового пространства.

Совместное использование блоков ячейками обеспечивается с помощью двух файлов – файла ячеек и каталога ячеек.

Файл ячеек содержит блоки индекса, а каталог ячеек отображает ячейки в блоки.

Каталог можно рассматривать как массив целых чисел, по одному для каждой ячейки. Пусть это массив `Dir`. Тогда если индексная запись хешируется в ячейку `b`, то запись будет сохранена в блоке `Dir[b]` файла ячеек.

Допустим, что в блоке размещаются 3 ячейки, всего ячеек 8, хеш-функция $h(x)=x \bmod 8$, в индексе семь записей с идентификаторами 1, 2, 4, 5, 7, 8, 12.

Каталог ячеек: [0 1 2 1 0 1 2 1]

Файл ячеек: [(4, r4) (8, r8) (12, r12)] [(1, r1) (5, r5) (7, r7)] [(2, r2)]

Надо заметить, что этот подход не работает, когда имеется слишком много записей с одинаковым значением `field_val`. Поскольку эти записи всегда будут хешироваться в одну и ту же ячейку, стратегия хеширования никак не сможет распределить их по нескольким ячейкам. В этом случае ячейка будет занимать столько блоков, сколько потребуется для хранения этих записей.

Индексы на основе В-дерева

Предыдущие две стратегии индексирования были основаны на хешировании.

Индексы на основе В-дерева — подход на основе сортировки. Основная идея заключается в сортировке индексных записей по значениям `field_val`.

Индексный файл – это последовательность индексных записей с полями `field_val` и `record_id`. При работе с индексным файлом для нас важно иметь возможность как можно быстрее находить идентификаторы записей (`record_id`) по значениям индексированного поля (`field_val`).

Кроме поиска поддерживается вставка и удаление.

Базы данных

Лекция 03.1 – Основные концепции. Индексирование

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2022

2022.09.24

Оглавление

Индексирование.....	3
Статические хеш-индексы.....	7
Статическое хеширование.....	7
Расширяемое хеширование.....	8
Индексы на основе В-дерева.....	9

Индексирование

При выполнении запроса к таблице пользователя часто интересуют только некоторые записи в ней, например записи, имеющие определенное значение в некотором поле.

Индекс – это файл, помогающий движку базы данных быстро найти такие записи, не просматривая всю таблицу.

Наиболее распространенными способами реализации индексов являются:

- статическое хеширование;
- расширяемое хеширование;
- В-деревья.

Эффективность выполнения некоторых запросов может значительно повысить подходящая организация таблиц. Представим телефонный справочник – по сути, большая таблица, записи которой содержат имена, адреса и номера телефонов абонентов.

Фамилия Имя Отчество	Адрес	Тел. номер
----------------------	-------	------------

Эта таблица отсортирована сначала по фамилиям, а затем по именам.

Предположим, что нам нужно узнать номер телефона конкретного человека.

Существенно ускорить поиск помогает тот факт, что записи отсортированы по имени.

Например, можно выполнить бинарный поиск, который в худшем случае потребует просмотреть $\log_2 N$ записей, где N – общее число записей в справочнике. Это очень быстро. Например, если $N = 1\,000\,000$, то $\log_2 N < 20$, то есть чтобы найти нужного человека в справочнике, содержащем номера миллиона человек, никогда не придется просматривать больше 20 записей.

Телефонный справочник отлично приспособлен для поиска абонента по имени, но не подходит для быстрого поиска, например по номеру телефона или по адресу.

Единственный способ получить эту информацию из телефонной книги – просмотреть каждую запись в ней, что потребует просмотреть в среднем $N/2$ записей.

Для эффективного поиска абонентов по номеру телефона нужен справочник, отсортированный по номерам телефонов (такие справочники еще называют «*обратными телефонными справочниками*»). Однако такой справочник удобен, только если известен номер телефона. Если вы решите найти в таком справочнике номер телефона конкретного абонента, то вам снова придется просмотреть каждую запись.

Этот пример наглядно иллюстрирует важное ограничение организации таблиц – таблицу можно организовать (упорядочить) только каким-то одним способом. Если необходимо, чтобы поиск выполнялся быстро и по номеру телефона и по имени абонента, потребуются две отдельные копии телефонной книги, каждая со своей организацией. А если понадобится возможность быстро находить номер телефона по известному адресу, потребуется третий экземпляр телефонного справочника, отсортированный по адресу. Это справедливо и в отношении таблиц в базе данных.

Чтобы иметь возможность эффективно находить в таблице записи с определенным значением некоторого поля, нужна версия таблицы, организованная по этому полю.

Механизмы баз данных это удовлетворяют эту потребность, поддерживая *индексы*.

Таблица может иметь один или несколько индексов, каждый из которых определен для отдельного поля. Каждый индекс действует подобно версии таблицы, организованной по соответствующему полю.

Индекс — это файл с индексными записями. Файл индекса имеет одну индексную запись для каждой записи в соответствующей таблице. Каждая индексная запись имеет два поля, которые хранят идентификатор соответствующей записи в таблице (значение первичного ключа) и значение индексируемого поля в этой записи.

ID	Name	Address	Phone
----	------	---------	-------

Name	ID
------	----

Address	ID
---------	----

Phone	ID
-------	----

На поля в индексной записи можно условно ссылаться по именам `record_id` и `field_value`. Движок БД организует записи в файле индекса по полю `field_value`.

Можно считать для простоты, что записи в индексе по полю `field_val` отсортированы. В этом случае для того, чтобы найти запись по значению `field_val`, нужно выполнить бинарный поиск и извлечь из поля `record_id` . первичный ключ искомой записи.

Аналогично можно найти все записи со значением `field_val`. Бинарный поиск выдаст `record_id` первой записи с атрибутом `field_val`. Поскольку в индексе все записи отсортированы по `field_val`, остается только последовательно читать следующие записи в индексе, пока получаемый `field_val` будет удовлетворять поставленному условию.

18, 20

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	3	3	5	6	9	9	10	13	18	18	18	18	32	37	?
b0							b1				b2				e0
b0							b1				b2	b4	b3		e0

Насколько эффективно такое применение индексов?

В отсутствие индексов лучшее, что можно предпринять при обработке любого запроса, – выполнить последовательный поиск в таблице.

Если атрибут в таблице равномерно распределен и имеет вид « $A = \text{константа}$ », справедливо правило – полезность индекса для поля A в таблице пропорциональна количеству уникальных значений в этом поле.

Согласно этому правилу, индекс наиболее полезен, когда индексируемое поле является первичным ключом таблицы, потому что каждая запись имеет свое уникальное значение ключа.

И наоборот, индекс будет бесполезен, если число уникальных значений в поле A меньше количества записей в блоке чтения.

Статические хеш-индексы

Статическое хеширование – это, пожалуй, самый простой способ реализации индексов. Это не самая эффективная стратегия, но достаточно простая и понятная для реализации.

Статическое хеширование

Статический хеш-индекс использует фиксированное число N ячеек, пронумерованных от 0 до $N-1$. Индекс также использует хеш-функцию, отображающую значения в ячейки. По результатам хеширования поля `field_val` каждая индексная запись помещается в свою ячейку.

Статический хеш-индекс действует следующим образом:

- чтобы сохранить индексную запись, ее нужно поместить в ячейку, вычисленную хеш-функцией;
- чтобы найти индексную запись, нужно вычислить хеш-функцию ключа поиска и просмотреть соответствующую ячейку;
- чтобы удалить индексную запись, нужно сначала найти ее (как указано выше), а затем удалить из ячейки.

Стоимость поиска с помощью хеш-индекса обратно пропорциональна количеству ячеек. Если индекс содержит B блоков и N ячеек, то на каждую ячейку будет приходиться около B/N блоков, поэтому для поиска в ячейке потребуются обратиться к B/N блоков.

$N = 17, B = 8$ ячейка полностью в одном блоке -> гарантированно одно чтение

$N = 17, B = 20$ ячейка располагается в двух блоках -> может быть два чтения

Расширяемое хеширование

Стоимость поиска при использовании индексов на основе статического хеширования обратно пропорциональна количеству ячеек – чем больше ячеек используется, тем меньше блоков в каждой из них.

Наиболее оптимально, когда каждую ячейку приходится ровно один блок.

Если бы размер индекса никогда не изменялся, то рассчитать это идеальное количество ячеек легко. Но на практике индексы растут по мере добавления новых записей в базу данных.

Если исходить из текущего размера индекса, то впоследствии, при его увеличении, каждая ячейка будет содержать несколько блоков индекса.

Если выбрать большее количество ячеек, ориентируясь на потребности в будущем, то пустые и почти пустые в данный момент ячейки будут напрасно расходовать значительный объем дискового пространства до тех пор, пока индекс не вырастет и их не заполнит.

Эту проблему решает стратегия, известная как *расширяемое хеширование*.

Суть этой стратегии заключается в использовании достаточно большого количества ячеек, чтобы гарантировать, что каждая ячейка никогда не будет содержать более одного блока.

Но теперь несколько ячеек могут располагаться в одном блоке, что позволяет большому количеству ячеек совместно использовать меньшее количество блоков и тем самым не допустить напрасного расходования дискового пространства.

Совместное использование блоков ячейками обеспечивается с помощью двух файлов – файла ячеек и каталога ячеек.

Файл ячеек содержит блоки индекса, а каталог ячеек отображает ячейки в блоки.

Каталог можно рассматривать как массив целых чисел, по одному для каждой ячейки. Пусть это массив `Dir`. Тогда если индексная запись хешируется в ячейку `b`, то запись будет сохранена в блоке `Dir[b]` файла ячеек.

Допустим, что в блоке размещаются 3 ячейки, всего ячеек 8, хеш-функция $h(x)=x \bmod 8$, в индексе семь записей с идентификаторами 1, 2, 4, 5, 7, 8, 12.

Каталог ячеек: [0 1 2 1 0 1 2 1]

Файл ячеек: [(4, r4) (8, r8) (12, r12)] [(1, r1) (5, r5) (7, r7)] [(2, r2)]

Надо заметить, что этот подход не работает, когда имеется слишком много записей с одинаковым значением `field_val`. Поскольку эти записи всегда будут хешироваться в одну и ту же ячейку, стратегия хеширования никак не сможет распределить их по нескольким ячейкам. В этом случае ячейка будет занимать столько блоков, сколько потребуется для хранения этих записей.

Индексы на основе В-дерева

Предыдущие две стратегии индексирования были основаны на хешировании.

Индексы на основе В-дерева — подход на основе сортировки. Основная идея заключается в сортировке индексных записей по значениям `field_val`.

Индексный файл – это последовательность индексных записей с полями `field_val` и `record_id`. При работе с индексным файлом для нас важно иметь возможность как можно быстрее находить идентификаторы записей (`record_id`) по значениям индексированного поля (`field_val`).

Кроме поиска поддерживается вставка и удаление.

Базы данных

Лекция 03.2 – Архитектура СУБД

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2022

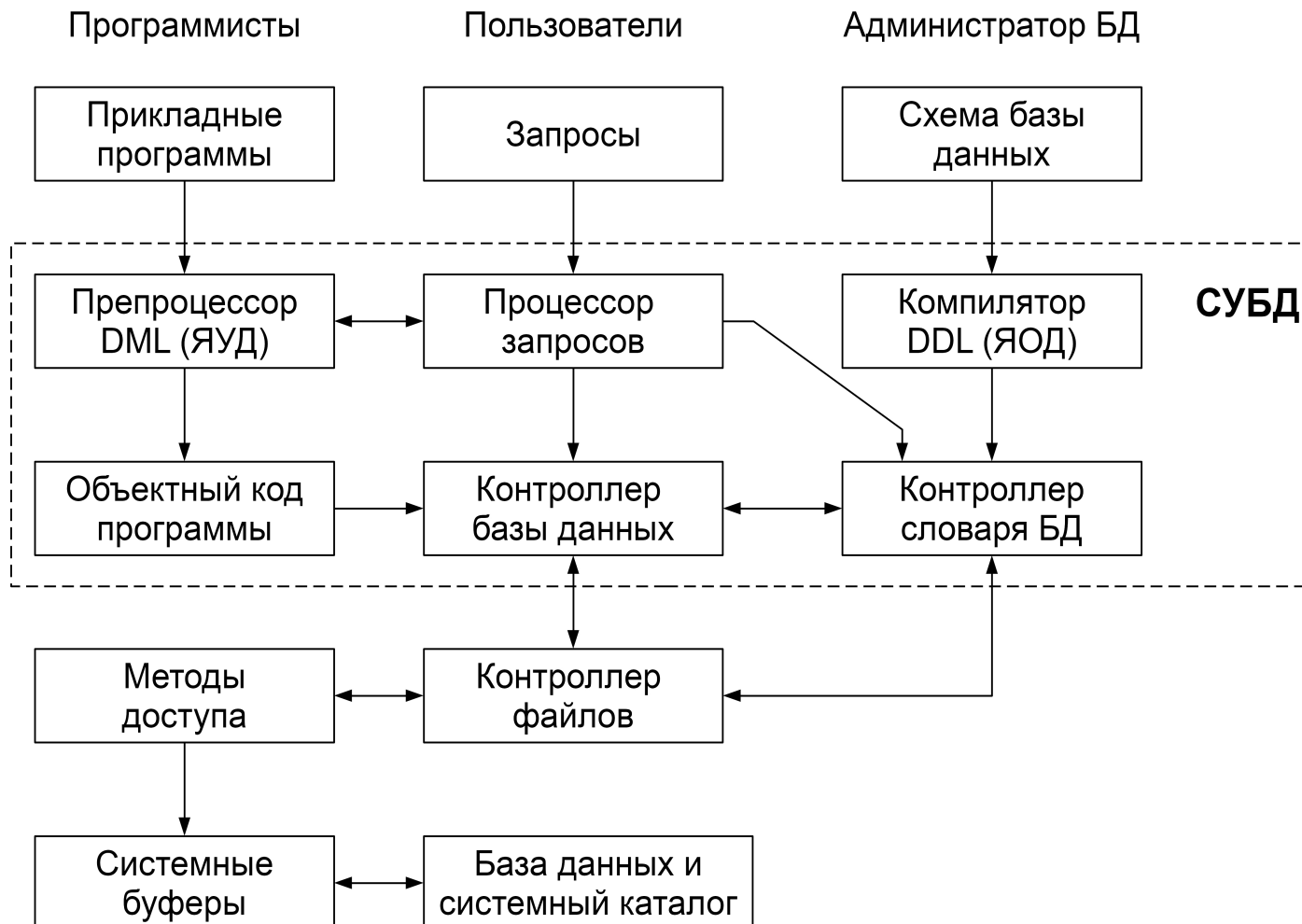
2022.09.24

Оглавление

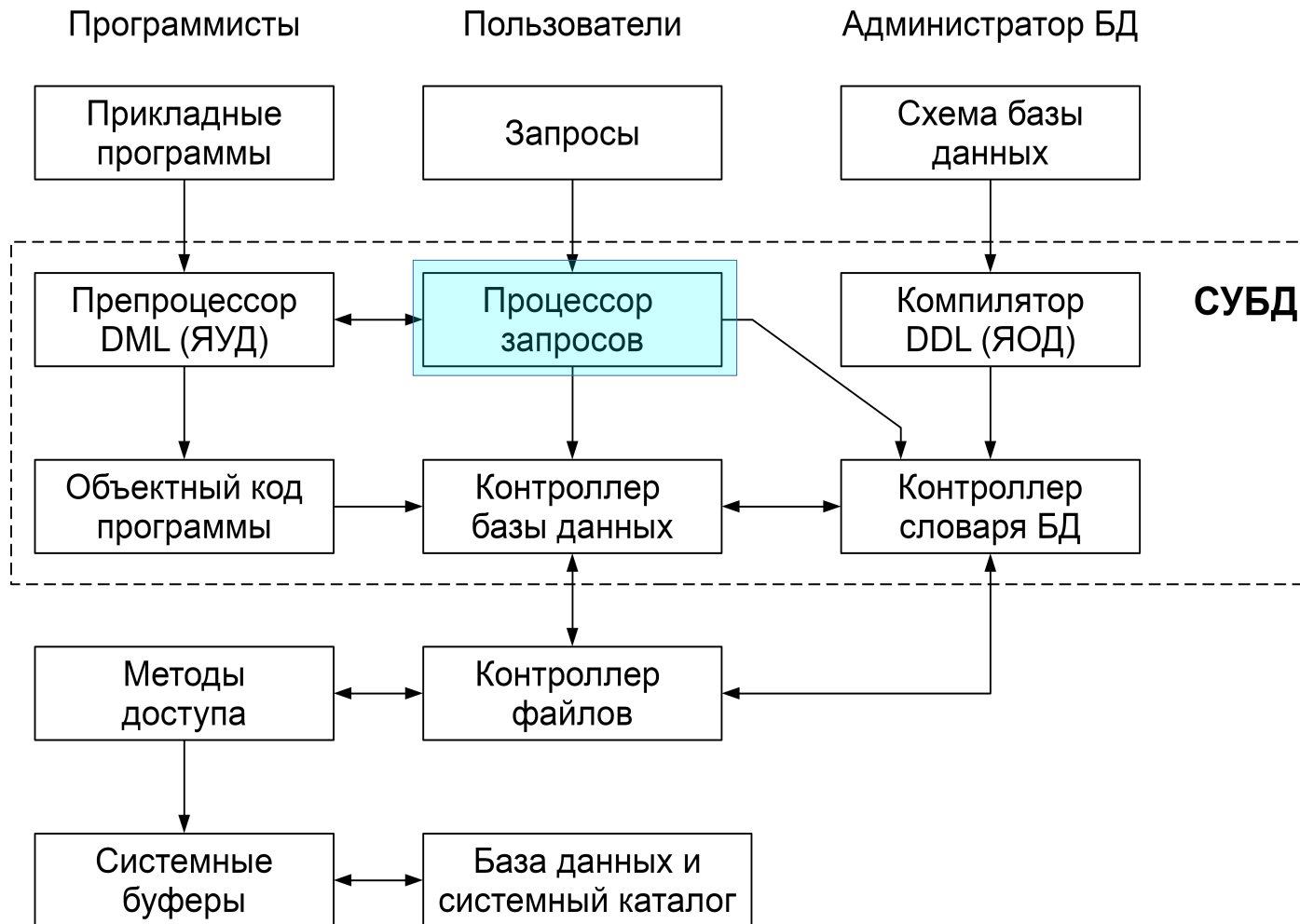
Компоненты СУБД.....	3
Администратор базы данных.....	10
Архитектура многопользовательских СУБД.....	15

Компоненты СУБД

СУБД состоит из программных компонентов (модулей), каждый модуль выполняет одну специфическую операцию (часть операций может поддерживаться операционной системой, но только базовые и контролируемые СУБД (как надстройкой над ОС)).



Процессор запросов – основной компонент СУБД – преобразует запросы в последовательность низкоуровневых инструкций для контроллера базы данных;



Контроллер базы данных – принимает запросы, проверяет внешние и концептуальные схемы для определения тех концептуальных записей, которые необходимы для выполнения запроса.

В запрос входят обычно следующие основные компоненты:

контроль прав доступа;

процессор команд;

средства контроля целостности;

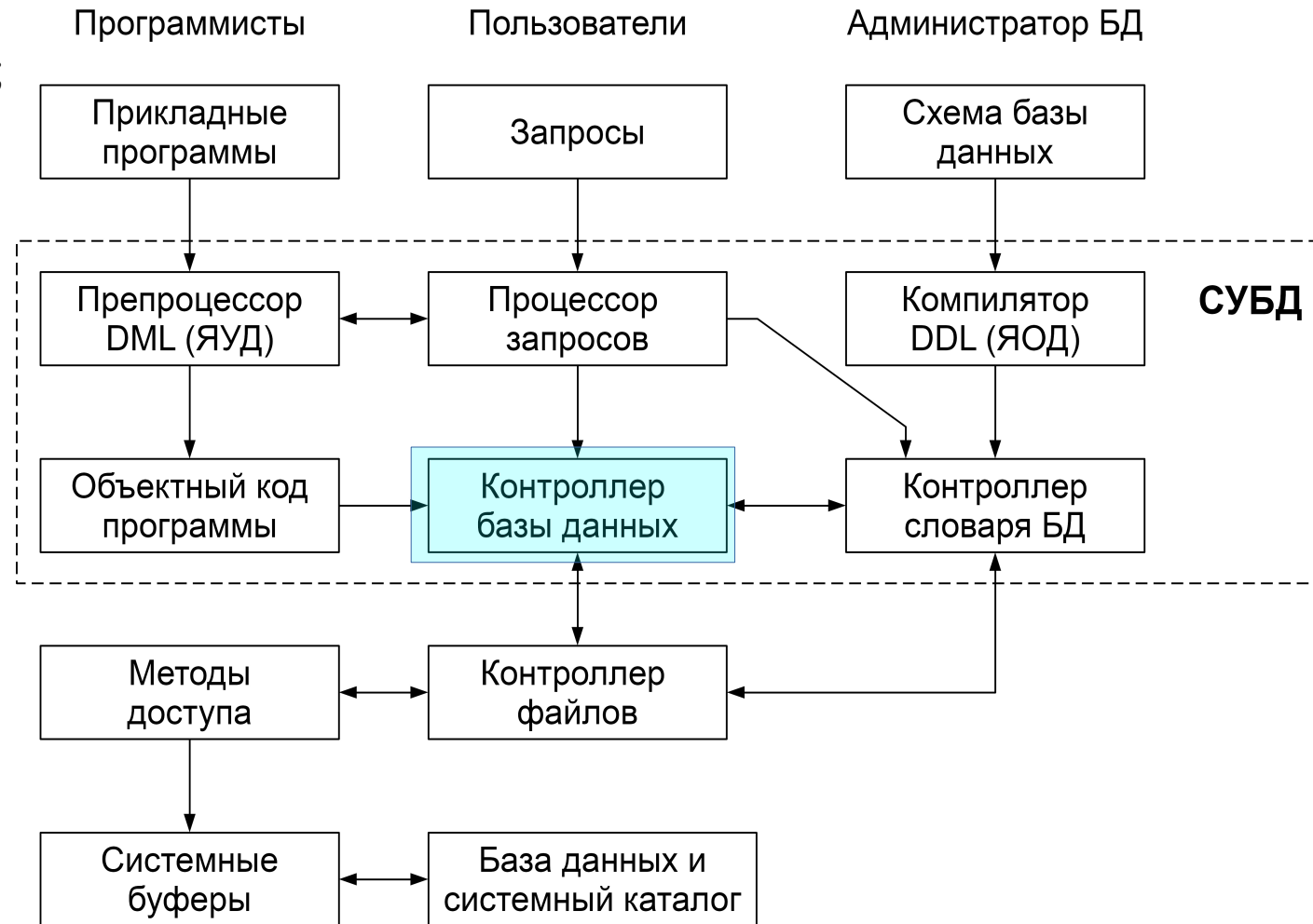
оптимизатор запросов;

контроллер транзакций;

планировщик;

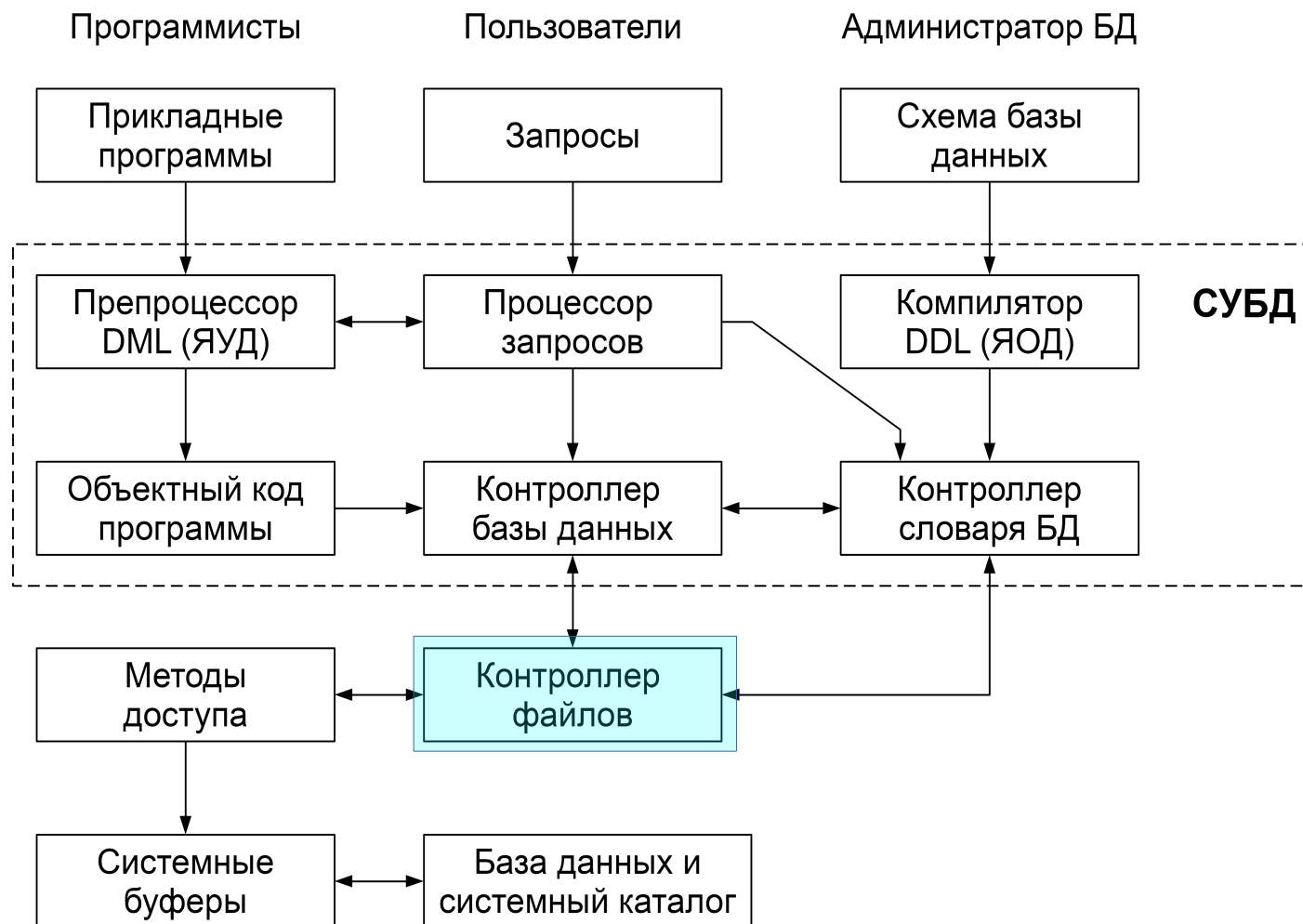
контроллер восстановления;

контроллер буферов (память-диск);



Контроллер файлов – манипулирует файлами БД.

Создает и поддерживает список структур и индексов, определенных во внутренней схеме, но не управляет вводом-выводом данных непосредственно, а лишь передает запросы соответствующим методам доступа, которые обмениваются данными между системными буферами и диском;



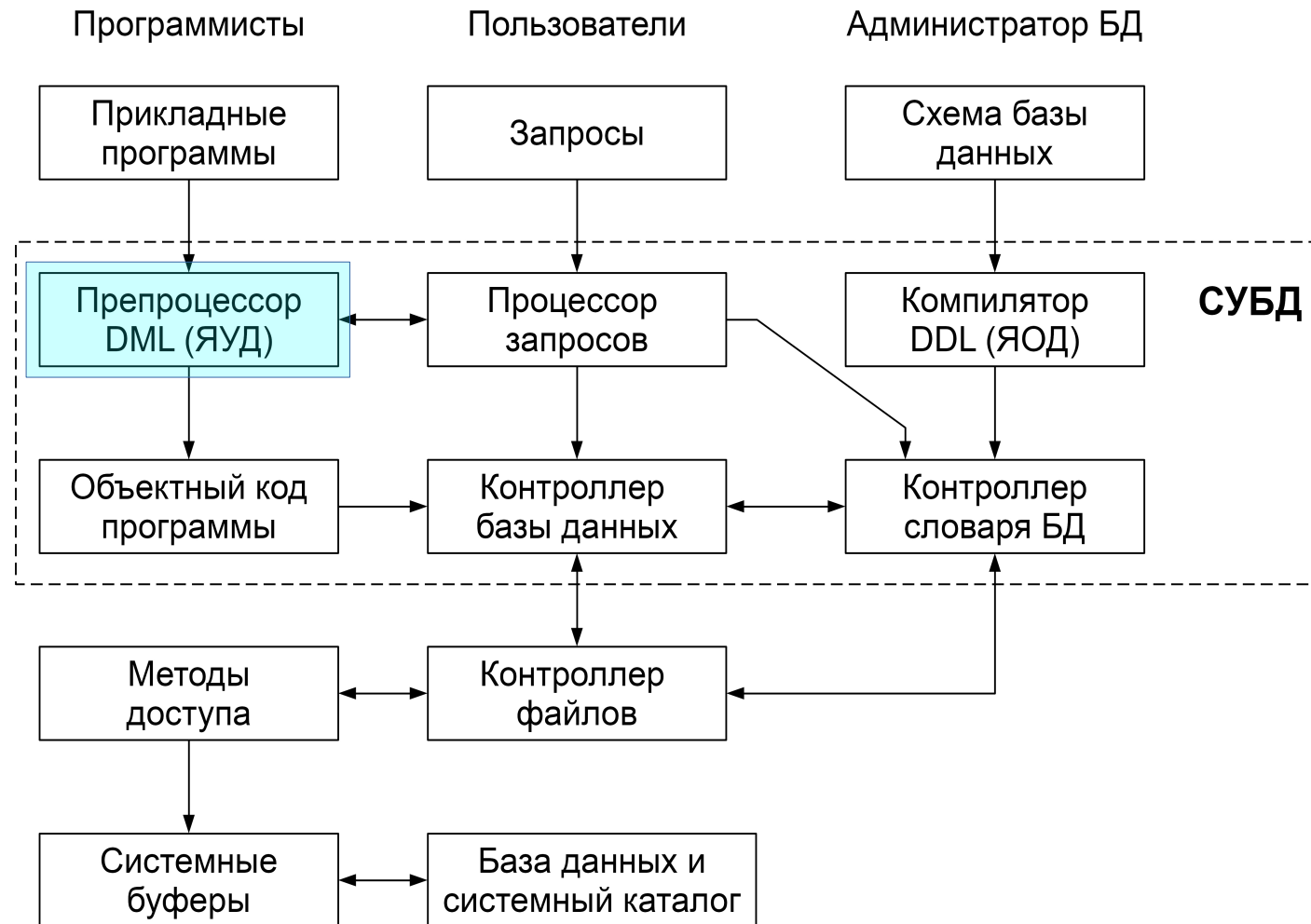
Преппроцессор языка DML – взаимодействует с процессором запросов и преобразует внедренные в прикладные программы DML-операторы в вызовы стандартных функций базового языка.

DML (Data Manipulation Language) – язык, содержащий набор операторов для поддержки основных операций манипулирования содержащимися в базе данными:

- вставка;
- модификация;
- извлечение данных
- удаление данных.

DML может быть двух типов:
процедурный – указывает, как можно получить результат оператора DML. Обычно оперирует данными по одной записи;

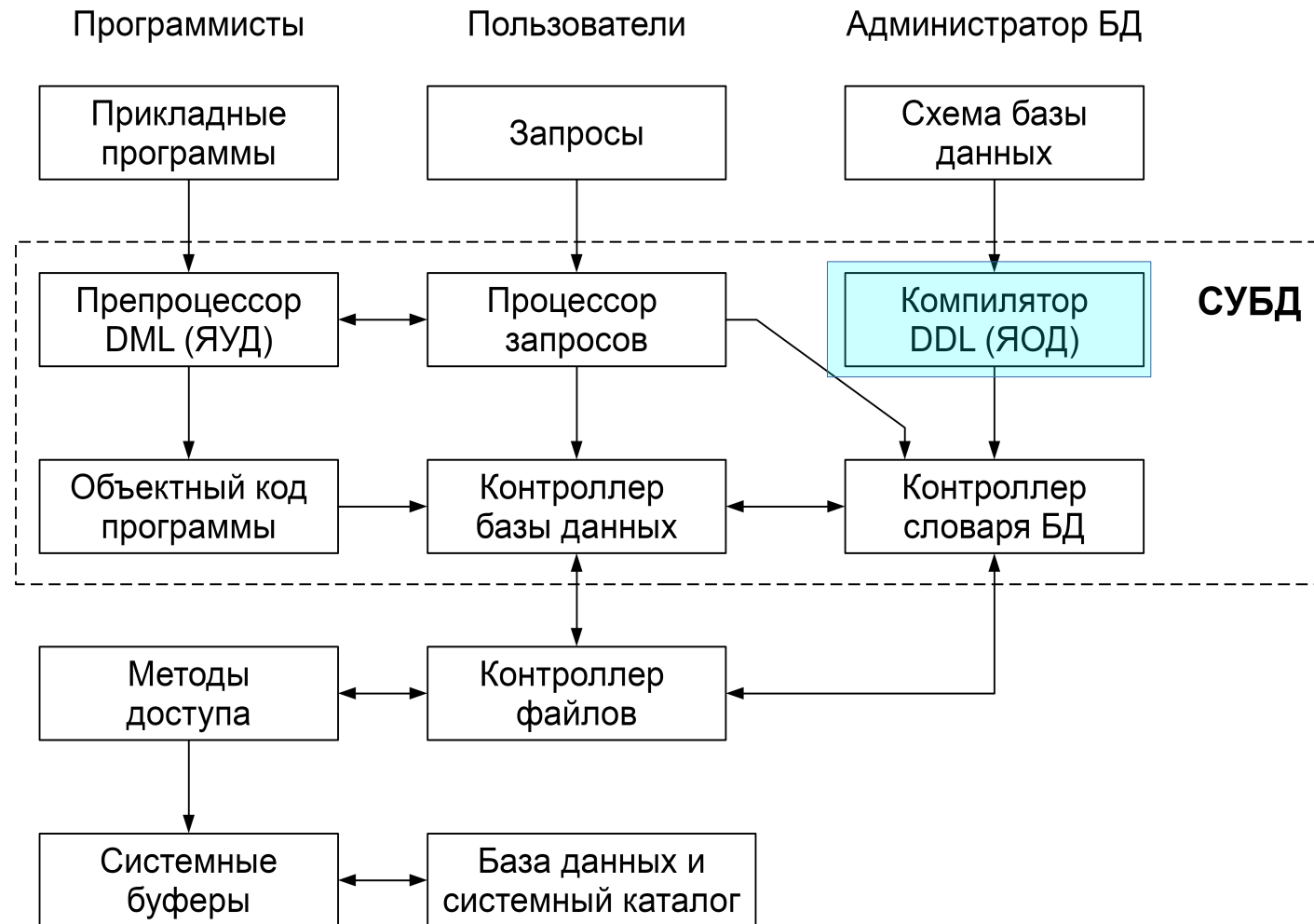
непроцедурный – описывает, какой результат будет получен. Обычно оперирует наборами записей (SQL).



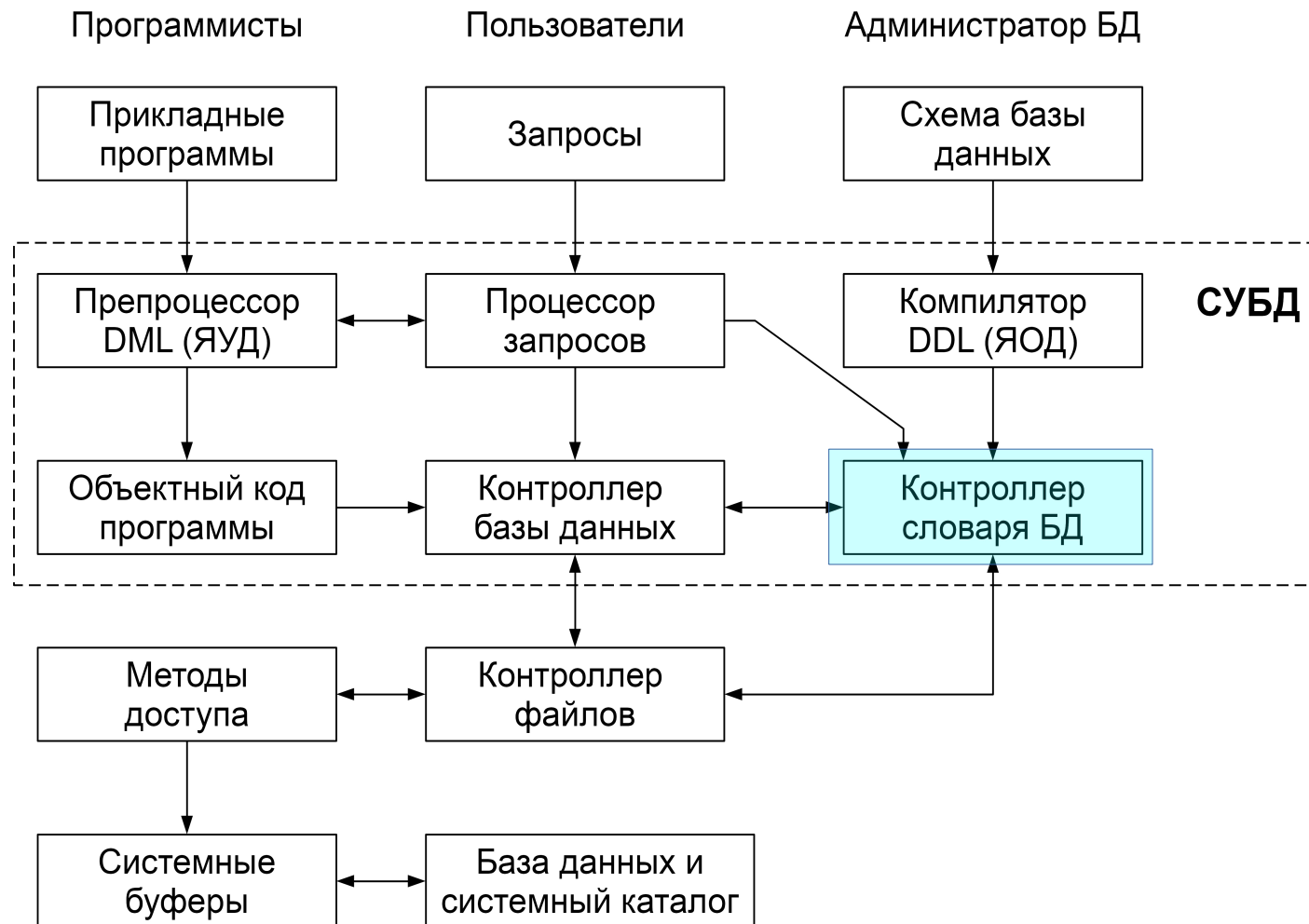
Компилятор языка DDL – преобразует DDL-команды в набор таблиц, содержащих метаданные, которые сохраняются в системном каталоге и частично в заголовках файлов с данными.

DDL (Data Definition Language) - описательный язык, который позволяет описывать сущности (объекты), необходимые для работы некоторого приложения, а также связи, имеющиеся между различными сущностями.

Теоретически, в архитектуре ANSI-SPARC, можно выделить языки DDL для каждой схемы, но на практике есть один, позволяющий задавать спецификации как минимум для внешней и концептуальной схем.



Контроллер словаря – управляет доступом к системному каталогу и обеспечивает работу с ним.
Почти все компоненты СУБД имеют доступ к системному каталогу.



Администратор базы данных

Администратор данных — человек, отвечающий за стратегию и политику принятия решений, связанных с данными предприятия.

Администратор базы данных (АБД) — это человек, обеспечивающий необходимую техническую поддержку для реализации принятых решений.

АБД отвечает за общее управление системой на техническом уровне. Функции АБД включают:

Определение концептуальной схемы

Администратор данных решает, **какая** именно информация должна храниться в базе данных, т.е. указывает те типы сущностей, в которых заинтересовано предприятие, а также определяет, какую информацию об этих сущностях необходимо вводить в базу данных. Этот процесс обычно называют логическим (или концептуальным) проектированием базы данных.

После того как содержимое базы данных на абстрактном уровне будет определено администратором данных, АБД создает соответствующую концептуальную схему, используя концептуальный язык определения данных.

Объектная (откомпилированная) форма этой схемы будет использоваться в СУБД для получения ответов на запросы, связанные с доступом к данным.

Исходная (не откомпилированная) форма будет играть роль справочного документа для пользователей системы.

Иногда концептуальную схему создает АД, а логическим проектированием занимается АБД.

Определение внутренней схемы

Администратор базы данных должен также решить, **как** данные будут представлены в хранимой базе данных. Этот процесс обычно называют физическим проектированием базы данных.

После завершения физического проектирования АБД создает соответствующие структуры хранения, используя внутренний язык определения данных, т.е. описывает *внутреннюю схему*.

Также он определяет соответствующее отображение между внутренней и концептуальной схемами.

Как и концептуальная схема, внутренняя схема и соответствующее отображение будут существовать в исходной и объектной формах.

Для определения всех трех схем используются соответствующие языки определения данных. Часто языки определения данных на концептуальном и/или внутреннем уровне могут включать в себя средства определения отображений, тем не менее, две функции – создание схемы и определение отображений – должны быть четко разделены.

Итак, проектирование осуществляется в определенной последовательности — вначале необходимо установить, какие данные требуются, а затем решить, как следует представить их в памяти.

Физическое проектирование выполняется после логического.

Взаимодействие с пользователями

В обязанности АБД входит также взаимодействие с пользователями для предоставления им необходимых данных и подготовки требуемых внешних схем или оказания помощи пользователям в их подготовке с применением прикладного внешнего языка определения данных.

Также функции АБД включают:

- консультации по разработке приложений;
- обеспечение требуемого технического обучения;
- предоставление помощи в выявлении и устранении возникающих проблем;
- прочие виды профессионального обслуживания.

Определение требований защиты и обеспечения целостности данных

Требования защиты и поддержки целостности данных рассматриваются как часть определения концептуальной схемы.

Концептуальный язык определения данных обычно включает в себя средства описания этих требований.

Определение процедур резервного копирования и восстановления

Предприятие, хранящее свои данные в СУБД полностью зависимо от бесперебойного функционирования этой СУБД.

В случае повреждения какой-либо части базы данных вследствие ошибки человека, отказа оборудования или сбоя программ операционной системы очень необходимо иметь возможность восстановить утраченные данные с минимальной задержкой и с наименьшим воздействием на остальную часть системы.

В идеале, данные, которые не были повреждены, совсем не должны затрагиваться в процессе восстановления.

АБД должен разработать и реализовать, во-первых, подходящую *схему восстановления*, включающую периодическую выгрузку базы данных на устройство резервного копирования (получение дампа), а во-вторых, процедуры загрузки базы данных из последнего созданного дампа в случае необходимости.

Требование *быстрого восстановления* поврежденных данных является одной из тех причин, по которым желательно организовать хранение данных не в каком-либо одном месте, а распределить их по нескольким отдельным базам данных.

Каждая из таких баз данных будет представлять собой оптимальный объект выгрузки или перезагрузки.

Существуют системы, хранящие десятки триллионов байт данных и имеет место тенденция к их росту. Такие системы очень больших баз данных (Very Large DataBase – VLDB) требуют тщательного и продуманного администрирования, особенно если необходимо обеспечить для пользователей постоянный доступ к базе данных.

Управление производительностью и реагирование на изменяющиеся требования

АБД отвечает за такую организацию системы, при которой можно поддерживать производительность, оптимальную для всего предприятия в целом, а также за корректировку работы системы (настройку) в соответствии с изменяющимися требованиями.

Например, может возникнуть необходимость в периодической реорганизации хранимой базы данных для обеспечения того, чтобы производительность системы всегда поддерживалась на приемлемом уровне.

Изменения на уровне физического хранения данных (внутреннем уровне) должны сопровождаться соответствующими изменениями в определении его отображения на концептуальный уровень, чтобы по возможности сохранять концептуальную схему неизменной.

Архитектура многопользовательских СУБД

Различают следующие архитектуры (по мере развития):

Телеобработка

В данном случае используется один компьютер, соединенный с терминалами пользователей.

Терминал – оконечное устройство, предназначенное только для ввода и отображения информации. Пользователь через терминал обращается к пользовательскому приложению, а то в свою очередь – к СУБД, затем результат идет в обратном порядке к пользователю. Основной пользовательский интерфейс на мейнфреймах.

Файловый сервер

Обработка данных распределена в сети. Это обычно локальная вычислительная сеть (ЛВС).

Пользовательские приложения и сама СУБД размещены и функционируют на рабочих станциях и обращаются к файловому серверу только при необходимости доступа к данным, которые располагаются в файлах.

Недостатки:

- большой объем сетевого трафика;
- необходимость в наличии СУБД на каждой рабочей станции;
- управление параллельностью, восстановлением и целостностью данных усложняется из-за доступа к данным от нескольких экземпляров СУБД.

Технология «клиент/сервер»

Единая двухуровневая система, состоящая из:

- Процесс-клиент (толстый или интеллектуальный клиент). Выполняет запросы на доступ к некоторым ресурсам. Внешний компонент, или внешний интерфейс.
- Процесс-сервер, предоставляющий некоторые ресурсы. Внутренний компонент, или машина базы данных.

Процессы совсем необязательно должны быть размещены на одном компьютере.

Обычно сервер располагается на одном узле ЛВС, а клиенты – на других узлах.

Сервер – это сама СУБД.

Он поддерживает все основные функции СУБД:

- определение данных;
- манипулирование данными;
- защиту данных
- поддержание целостности данных и т.д.

В частности, он предоставляет полную поддержку внешнего, концептуального и внутреннего уровней. Поэтому сервер в этом контексте – это просто другое название для СУБД.

Сервер принимает и обрабатывает запросы к базе данных, включая проверку полномочий клиента, обеспечение требований целостности, поддержку системного каталога, поддержку параллельного доступа, выполнение запроса и обновление данных, а затем передает полученные результаты обработки обратно клиенту.

Клиенты — это различные приложения, которые выполняются с помощью СУБД.

Таковыми являются как приложения, написанные пользователями, так и встроенные приложения, предоставляемые поставщиками СУБД или некоторыми сторонними поставщиками программного обеспечения.

В самом сервере разница между встроенными приложениями и приложениями, написанными пользователем, не обнаруживается, поскольку все эти приложения используют один и тот же интерфейс сервера, а именно интерфейс внешнего уровня.

Некоторые специальные "служебные" приложения могут оказаться исключениями. Они иногда работают непосредственно на внутреннем уровне системы. Подобные утилиты правильнее относить к встроенным компонентам СУБД, а не к приложениям в обычном смысле.

Клиент управляет пользовательским интерфейсом и логикой приложения. Обычно это рабочая станция с пользовательским приложением, которое:

- принимает от пользователя запрос;
- проверяет синтаксис;
- генерирует запрос к базе данных;
- передает сообщение серверу;
- ожидает поступление ответа;
- форматирует полученные данные для визуализации.

Приложения, написанные пользователями

В основном, это обычные прикладные программы, чаще всего написанные либо на языке программирования общего назначения, таком как С++ или Java (COBOL), либо на одном из специализированных языков четвертого поколения.

Приложения, предоставляемые поставщиками

Их часто называют *инструментальными средствами*.

В целом, назначение таких средств — упрощать процесс создания и выполнения других приложений, т.е. приложений, которые разрабатываются специально для решения некоторой конкретной задачи.

Часто эти приложения могут выглядеть вовсе не так, как приложения в общепринятом смысле.

Назначение инструментальных средств также состоит в предоставлении пользователям, особенно конечным, возможности создавать приложения без написания традиционных программ.

Например, одно из предоставляемых поставщиком СУБД инструментальных средств может быть генератором отчетов, с помощью которого конечный пользователь сможет получить отформатированный отчет, выполнив обычный запрос к системе.

Каждый такой запрос является по существу небольшим специальным приложением, написанным на языке очень высокого уровня (с конкретным назначением), а именно — на языке формирования отчетов.

Отдельно поставляемые инструментальные средства делятся на несколько самостоятельных классов:

- процессоры языков запросов;
- генераторы отчетов;
- графические бизнес-подсистемы;
- электронные таблицы;
- процессоры команд на естественных языках;
- статистические пакеты;

- средства управления копированием или средства извлечения данных;
- генераторы приложений, включая процессоры языков четвертого поколения;
- другие средства разработки приложений, включая CASE-инструменты (CASE, или Computer-Aided Software Engineering – автоматизированное проектирование и создание программ);
- инструментальные средства интеллектуального анализа данных и визуализации.

Преимущества технологии «клиент/сервер»

- более широкий доступ к существующим базам данных;
- повышение производительности системы, по сравнению с телеобработкой и/или ФС;
- снижение стоимости аппаратного обеспечения (серверная машина – более мощная, чем клиентские);
- снижение сетевого трафика (по сети идут только необходимые данные);
- повышается уровень непротиворечивости данных (проверка целостности данных выполняется централизованно на сервере, а не у клиентов).

Расширение двухуровневой технологии «клиент/сервер» трехуровневая структура.

- тонкий (неинтеллектуальный) клиент, управляющий только пользовательским интерфейсом;
- средний уровень (клиент), управляющий логикой пользовательского приложения;
- сервер базы данных.

Базы данных

Лекция 03 – Архитектура СУБД

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

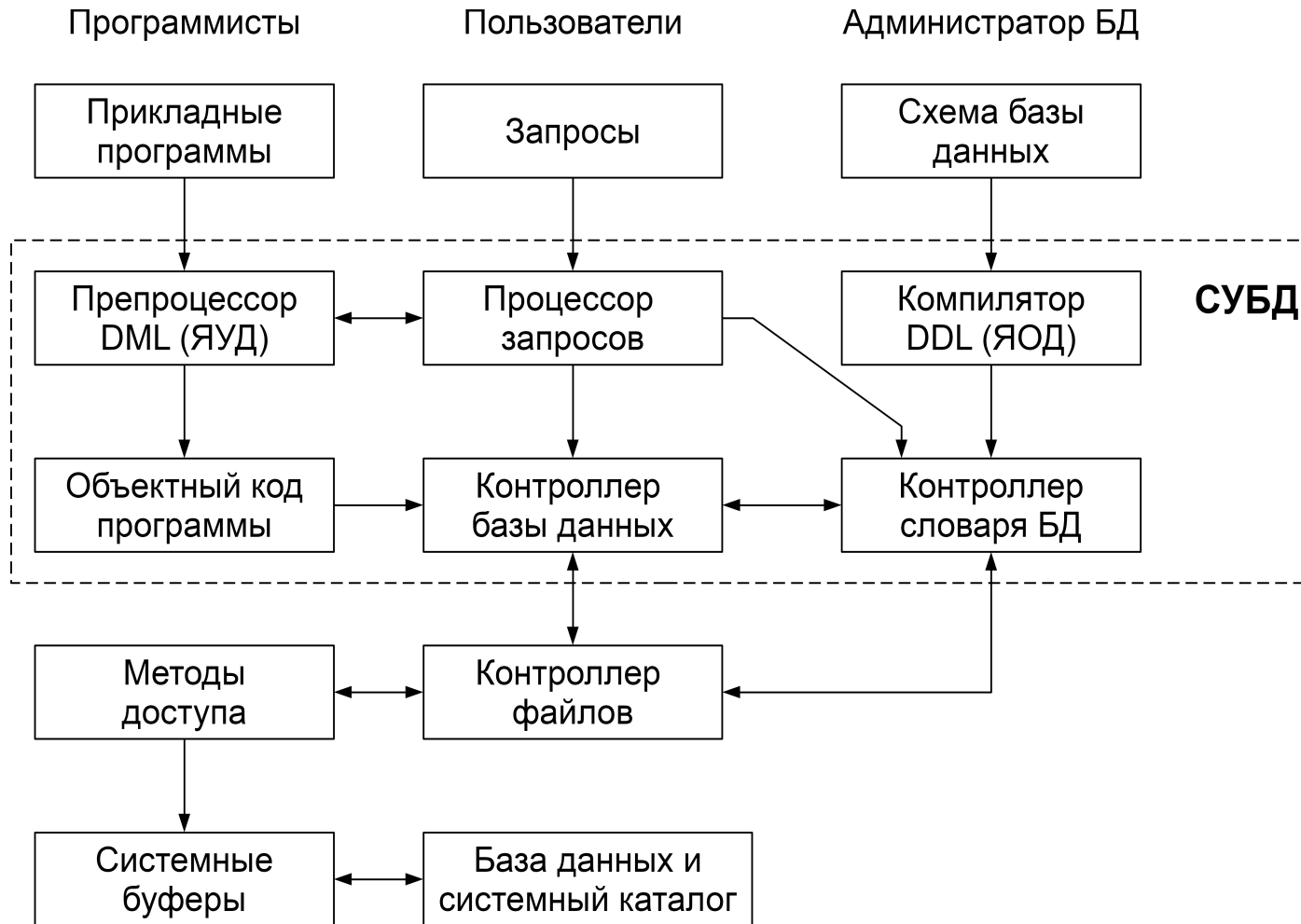
2023.09.15

Оглавление

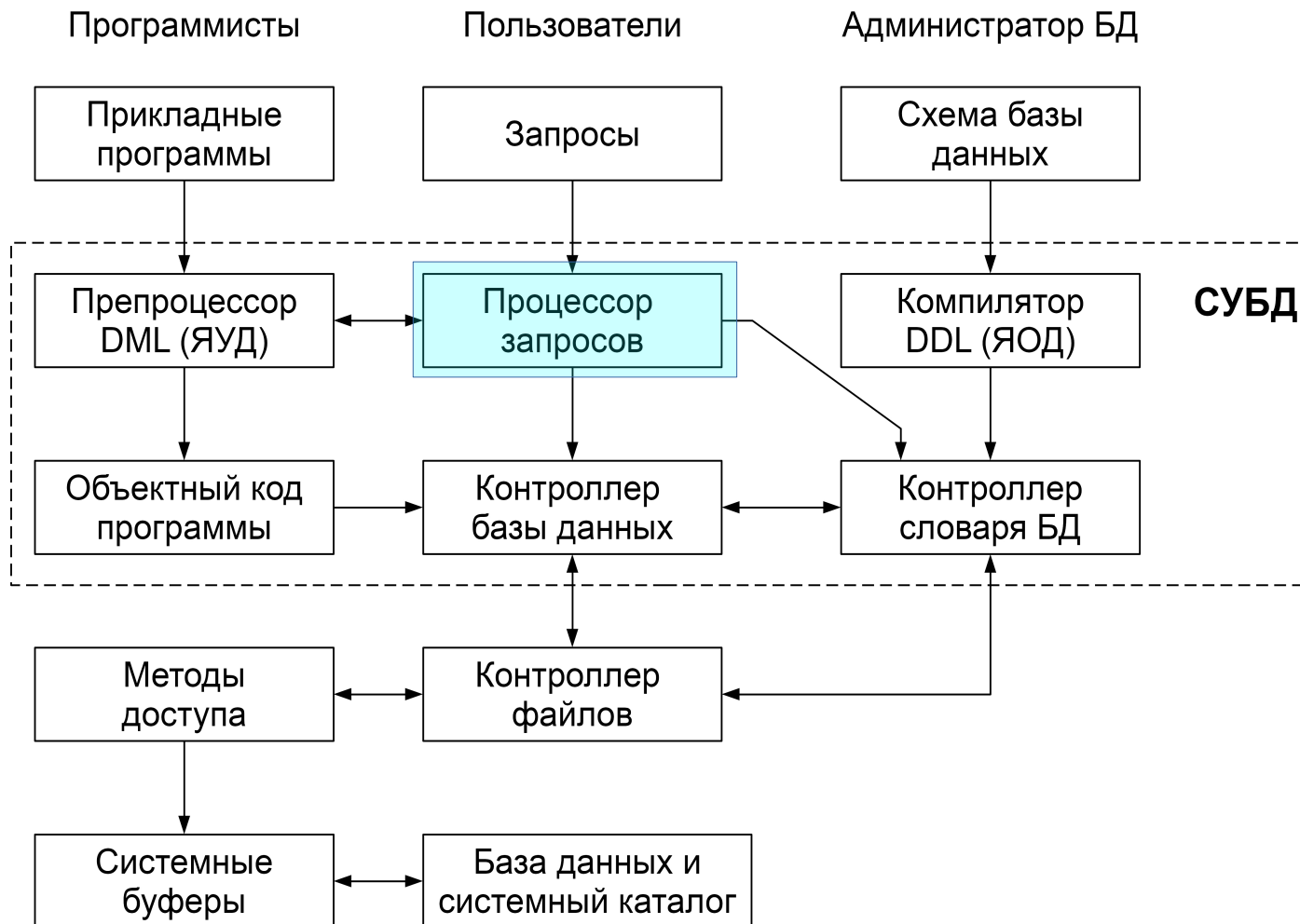
Компоненты СУБД.....	3
Администратор базы данных.....	10
Архитектура многопользовательских СУБД.....	15
Низкоуровневые операции логического уровня в таблицах.....	21

Компоненты СУБД

СУБД состоит из программных компонентов (модулей), каждый модуль выполняет одну специфическую операцию (часть операций может поддерживаться операционной системой, но только базовые и контролируемые СУБД (как надстройкой над ОС)).



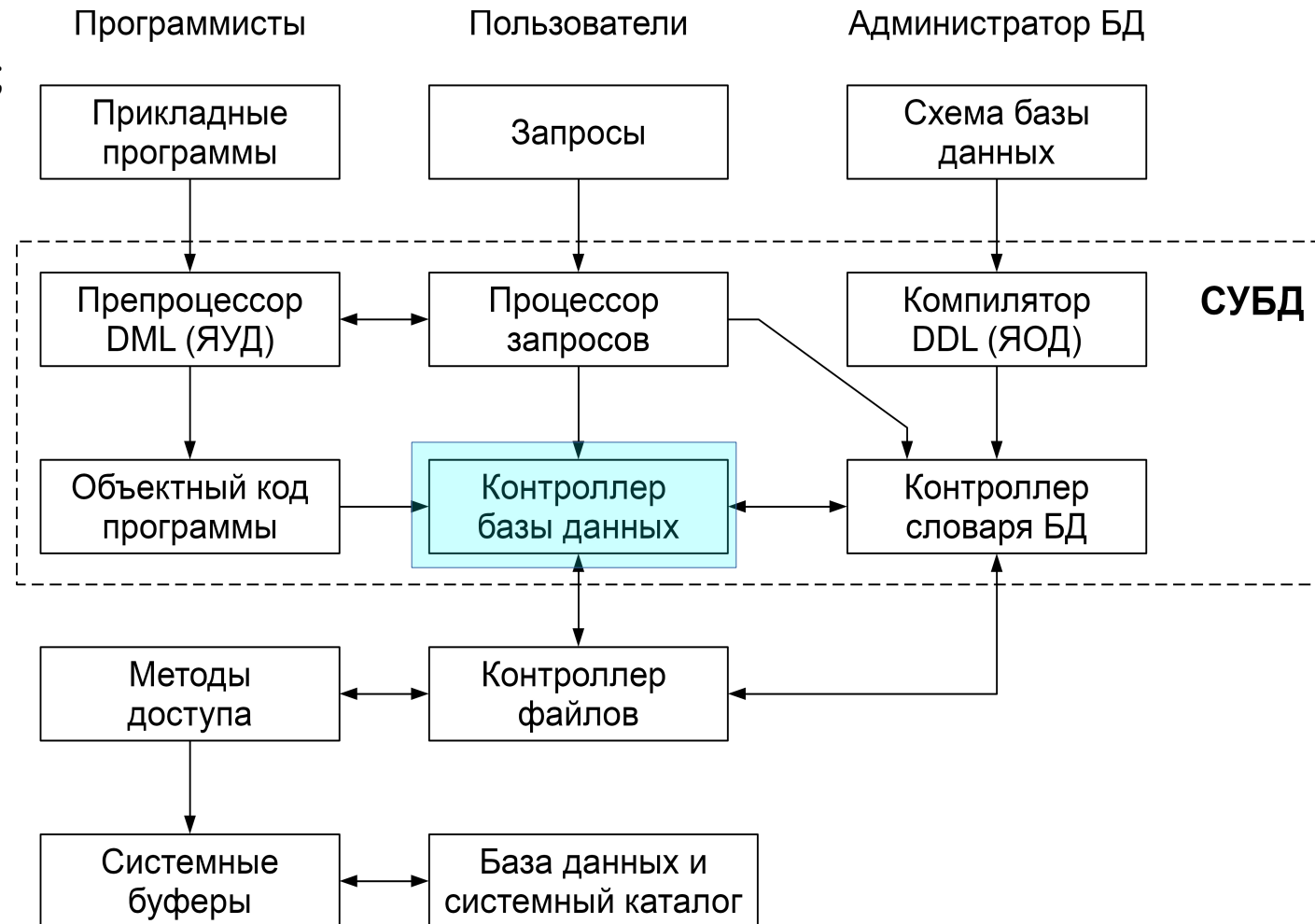
Процессор запросов – основной компонент СУБД – преобразует запросы в последовательность низкоуровневых инструкций для контроллера базы данных;



Контроллер базы данных – принимает запросы, проверяет внешние и концептуальные схемы для определения тех концептуальных записей, которые необходимы для выполнения запроса.

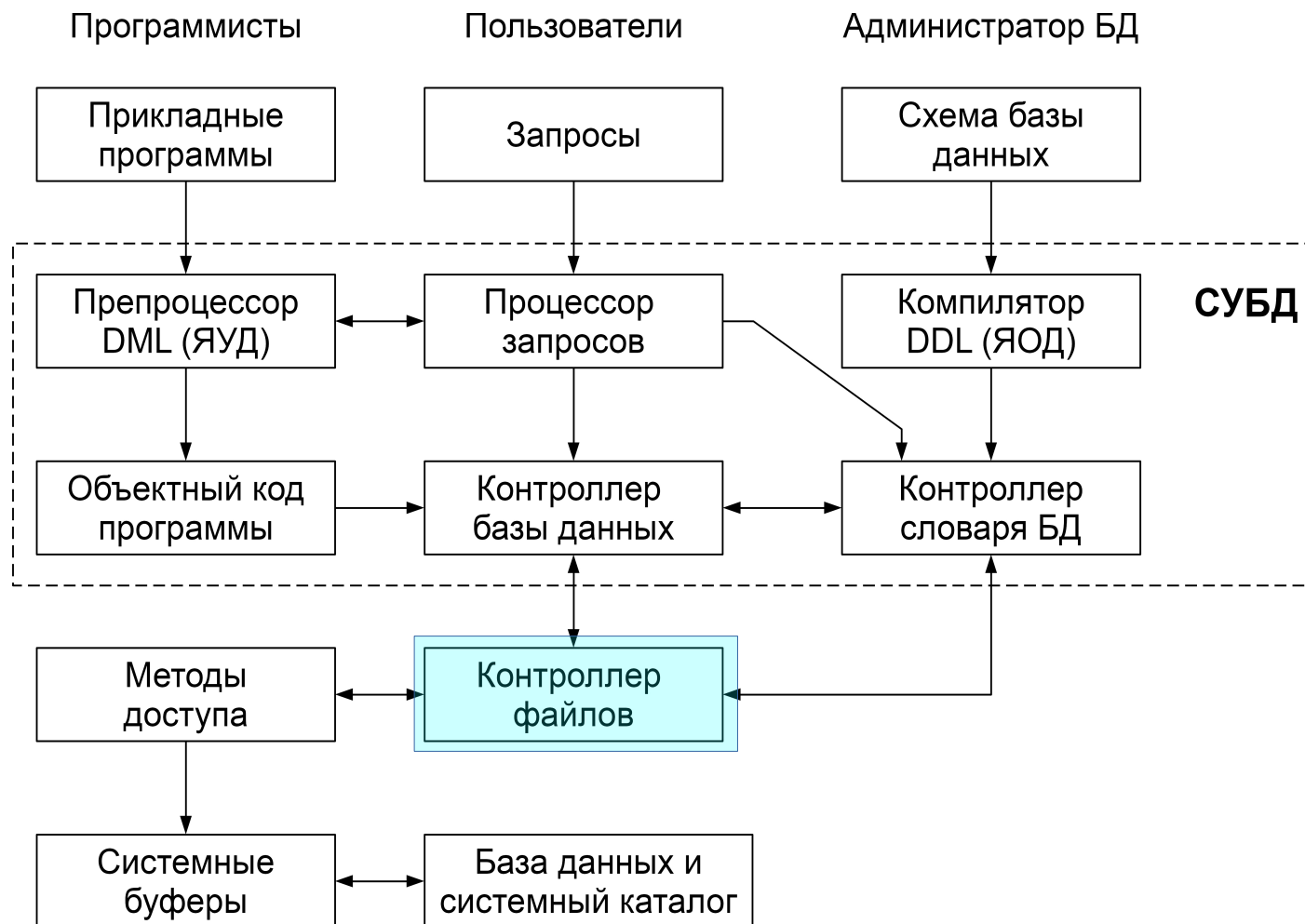
В запрос входят обычно следующие основные компоненты:

контроль прав доступа;
процессор команд;
средства контроля целостности;
оптимизатор запросов;
контроллер транзакций;
планировщик;
контроллер восстановления;
контроллер буферов (память-диск);



Контроллер файлов – манипулирует файлами БД.

Создает и поддерживает список структур и индексов, определенных во внутренней схеме, но не управляет вводом-выводом данных непосредственно, а лишь передает запросы соответствующим методам доступа, которые обмениваются данными между системными буферами и диском;



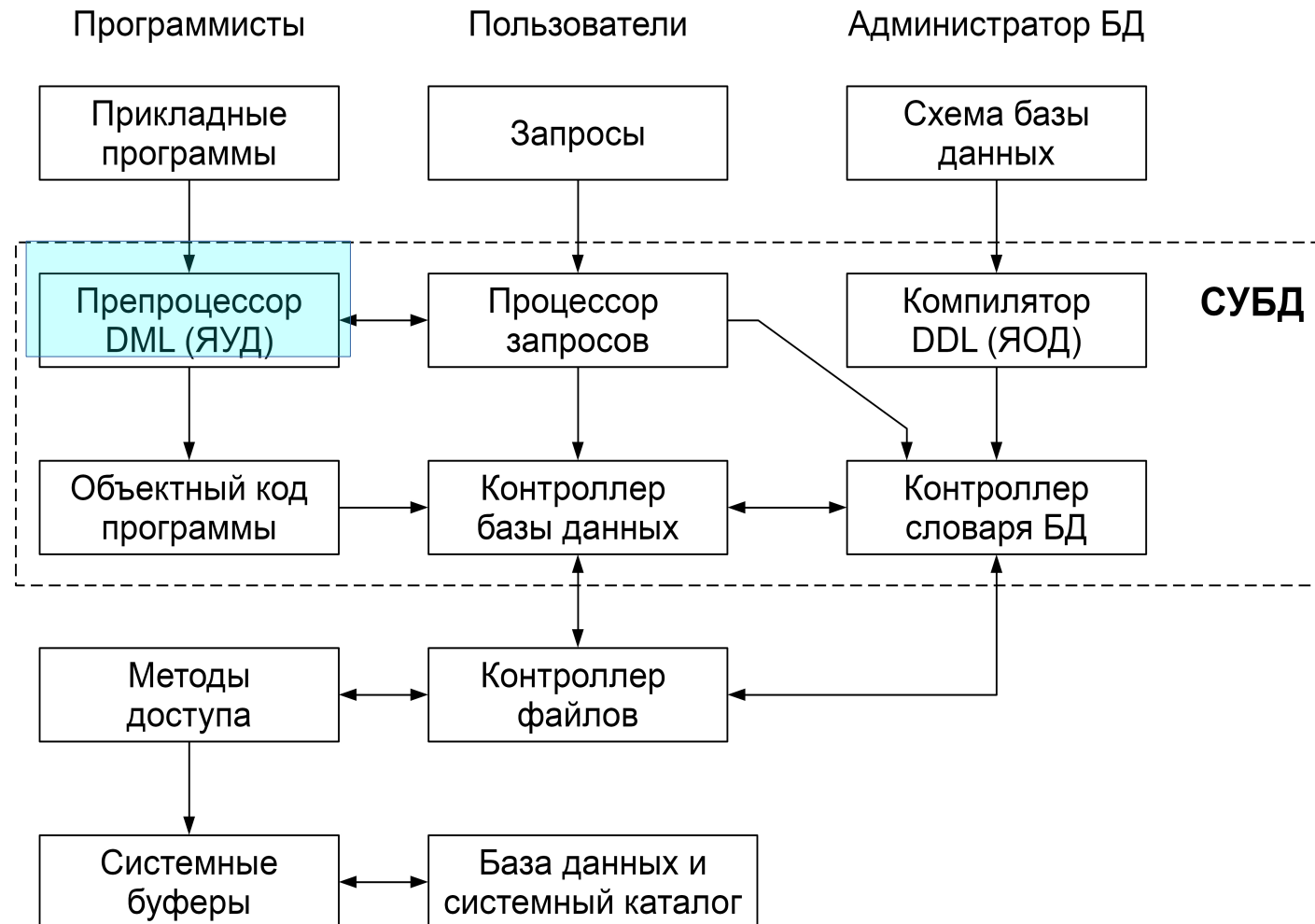
Преппроцессор языка DML – взаимодействует с процессором запросов и преобразует внедренные в прикладные программы DML-операторы в вызовы стандартных функций базового языка.

DML (Data Manipulation Language) – язык, содержащий набор операторов для поддержки основных операций манипулирования содержащимися в базе данными:

- вставка;
- модификация;
- извлечение данных
- удаление данных.

DML может быть двух типов:
процедурный – указывает, как можно получить результат оператора DML. Обычно оперирует данными по одной записи;

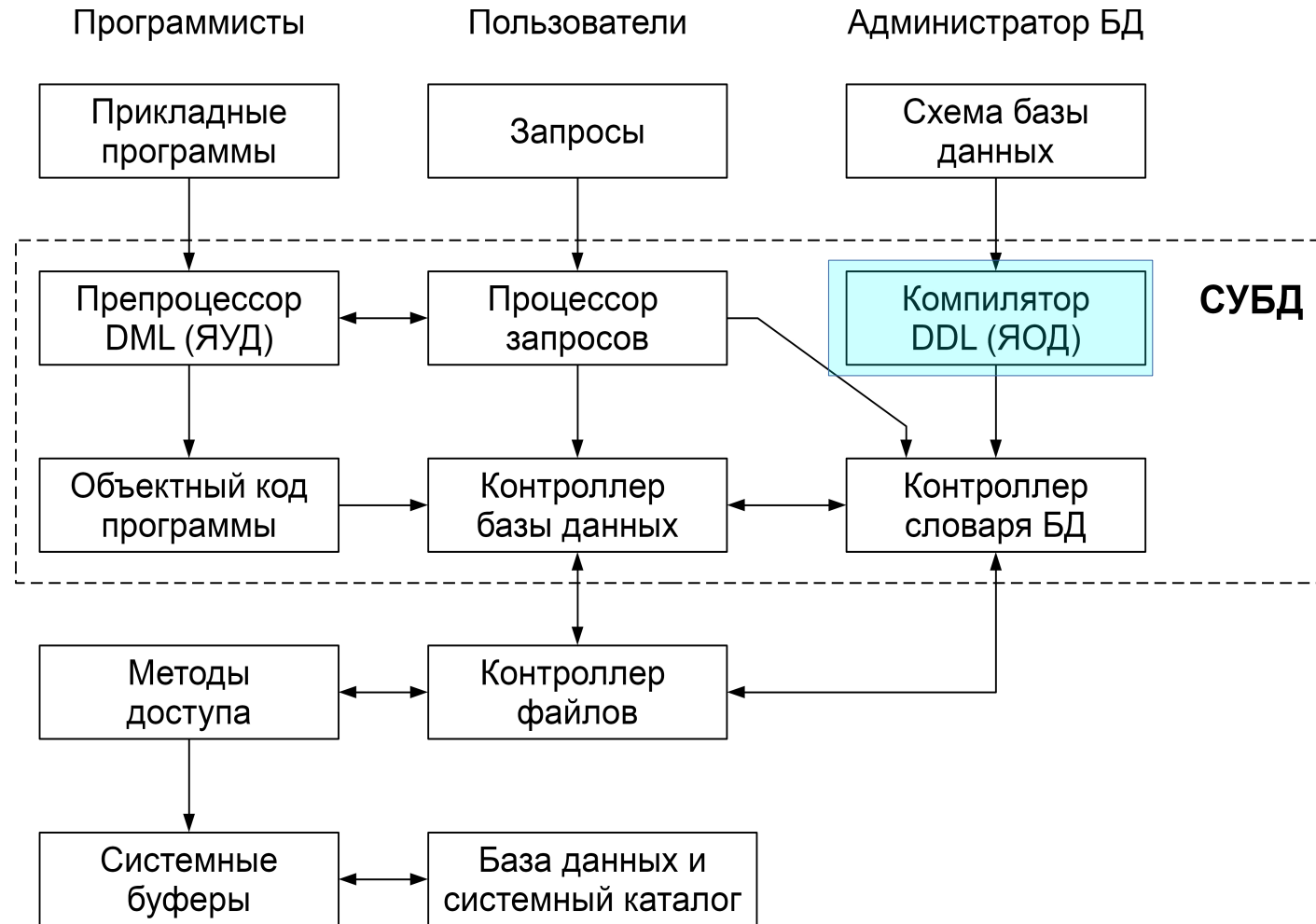
непроцедурный – описывает, какой результат будет получен. Обычно оперирует наборами записей (SQL).



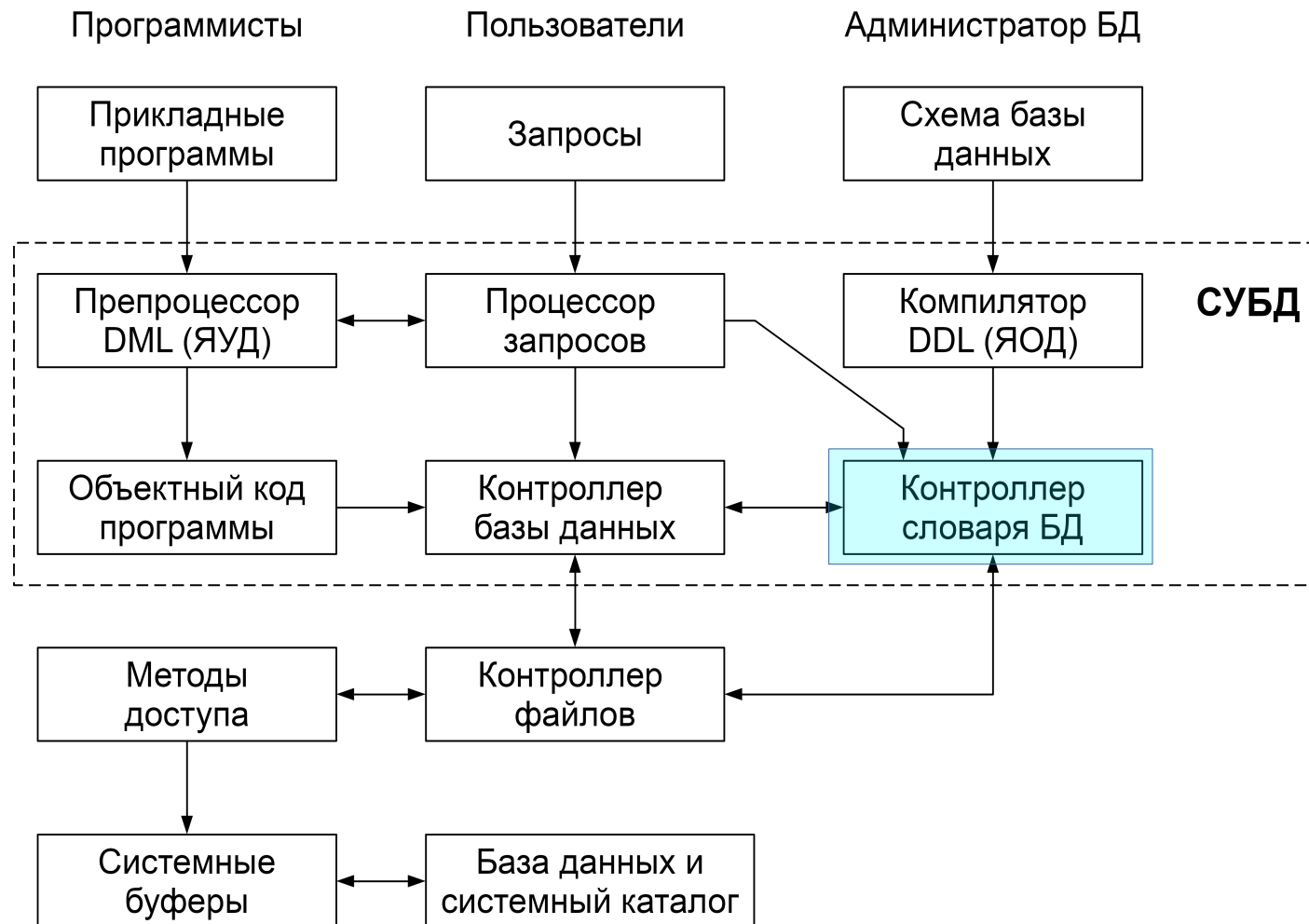
Компилятор языка DDL – преобразует DDL-команды в набор таблиц, содержащих метаданные, которые сохраняются в системном каталоге и частично в заголовках файлов с данными.

DDL (Data Definition Language) - описательный язык, который позволяет описывать сущности (объекты), необходимые для работы некоторого приложения, а также связи, имеющиеся между различными сущностями.

Теоретически, в архитектуре ANSI-SPARC, можно выделить языки DDL для каждой схемы, но на практике есть один, позволяющий задавать спецификации как минимум для внешней и концептуальной схем.



Контроллер словаря – управляет доступом к системному каталогу и обеспечивает работу с ним. Почти все компоненты СУБД имеют доступ к системному каталогу.



Администратор базы данных

Администратор данных — человек, отвечающий за стратегию и политику принятия решений, связанных с данными предприятия.

Администратор данных решает, **какая** именно информация должна храниться в базе данных, т.е. указывает те типы сущностей, в которых заинтересовано предприятие, а также определяет, какую информацию об этих сущностях необходимо вводить в базу данных.

Этот процесс обычно называют логическим (или концептуальным) проектированием базы данных.

Администратор базы данных (АБД) — это человек, обеспечивающий необходимую техническую поддержку для реализации принятых решений.

АБД отвечает за общее управление системой на техническом уровне.

Функции АБД включают:

- определение концептуальной схемы;
- определение внутренней схемы.

Определение концептуальной схемы

После того как содержимое базы данных на абстрактном уровне будет определено администратором данных, АБД создает соответствующую концептуальную схему, используя концептуальный язык определения данных.

Объектная (откомпилированная) форма этой схемы будет использоваться в СУБД для получения ответов на запросы, связанные с доступом к данным.

Исходная (не откомпилированная) форма будет играть роль справочного документа для пользователей системы.

Иногда концептуальную схему создает АД, а логическим проектированием занимается АБД.

Определение внутренней схемы

Администратор базы данных должен также решить, **как** данные будут представлены в хранимой базе данных. Этот процесс обычно называют физическим проектированием базы данных.

После завершения физического проектирования АБД создает соответствующие структуры хранения, используя внутренний язык определения данных, т.е. описывает *внутреннюю схему*.

Также он определяет соответствующее отображение между внутренней и концептуальной схемами.

Как и концептуальная схема, внутренняя схема и соответствующее отображение будут существовать в исходной и объектной формах.

Для определения всех трех схем используются соответствующие языки определения данных. Часто языки определения данных на концептуальном и/или внутреннем уровне могут включать в себя средства определения отображений, тем не менее, две функции – создание схемы и определение отображений – должны быть четко разделены.

Итак, проектирование осуществляется в определенной последовательности – вначале необходимо установить, какие данные требуются, а затем решить, как следует представить их в памяти.

Физическое проектирование выполняется после логического.

Взаимодействие с пользователями

В обязанности АБД входит также взаимодействие с пользователями для предоставления им необходимых данных и подготовки требуемых внешних схем или оказания помощи пользователям в их подготовке с применением прикладного внешнего языка определения данных.

Также функции АБД включают:

- консультации по разработке приложений;
- обеспечение требуемого технического обучения;
- предоставление помощи в выявлении и устранении возникающих проблем;
- прочие виды профессионального обслуживания.

Определение требований защиты и обеспечения целостности данных

Требования защиты и поддержки целостности данных рассматриваются как часть определения концептуальной схемы.

Концептуальный язык определения данных обычно включает в себя средства описания этих требований.

Определение процедур резервного копирования и восстановления

Предприятие, хранящее свои данные в СУБД полностью зависимо от бесперебойного функционирования этой СУБД.

В случае повреждения какой-либо части базы данных вследствие ошибки человека, отказа оборудования или сбоя программ операционной системы очень необходимо иметь возможность восстановить утраченные данные с минимальной задержкой и с наименьшим воздействием на остальную часть системы.

В идеале, данные, которые не были повреждены, совсем не должны затрагиваться в процессе восстановления.

АБД должен разработать и реализовать, во-первых, подходящую *схему восстановления*, включающую периодическую выгрузку базы данных на устройство резервного копирования (получение дампа), а во-вторых, процедуры загрузки базы данных из последнего созданного дампа в случае необходимости.

Требование *быстрого восстановления* поврежденных данных является одной из тех причин, по которым желательно организовать хранение данных не в каком-либо одном месте, а распределить их по нескольким отдельным базам данных.

Каждая из таких баз данных будет представлять собой оптимальный объект выгрузки или перезагрузки.

Существуют системы, хранящие десятки триллионов байт данных и имеет место тенденция к их росту. Такие системы очень больших баз данных (Very Large DataBase – VLDB) требуют тщательного и продуманного администрирования, особенно если необходимо обеспечить для пользователей постоянный доступ к базе данных.

Управление производительностью и реагирование на изменяющиеся требования

АБД отвечает за такую организацию системы, при которой можно поддерживать производительность, оптимальную для всего предприятия в целом, а также за корректировку работы системы (настройку) в соответствии с изменяющимися требованиями.

Например, может возникнуть необходимость в периодической реорганизации хранимой базы данных для обеспечения того, чтобы производительность системы всегда поддерживалась на приемлемом уровне.

Изменения на уровне физического хранения данных (внутреннем уровне) должны сопровождаться соответствующими изменениями в определении его отображения на концептуальный уровень, чтобы по возможности сохранять концептуальную схему неизменной.

Архитектура многопользовательских СУБД

Различают следующие архитектуры (по мере развития):

Телеобработка

В данном случае используется один компьютер, соединенный с терминалами пользователей.

Терминал – оконечное устройство, предназначенное только для ввода и отображения информации.

Пользователь через терминал обращается к пользовательскому приложению, а то в свою очередь – к СУБД, затем результат идет в обратном порядке к пользователю. Основной пользовательский интерфейс на мейнфреймах.

Файловый сервер

Обработка данных распределена в сети. Это обычно локальная вычислительная сеть (ЛВС).

Пользовательские приложения и сама СУБД размещены и функционируют на рабочих станциях и обращаются к файловому серверу только при необходимости доступа к данным, которые располагаются в файлах.

Недостатки:

- большой объем сетевого трафика;
- необходимость в наличии СУБД на каждой рабочей станции;
- управление параллельностью, восстановлением и целостностью данных усложняется из-за доступа к данным от нескольких экземпляров СУБД.

Технология «клиент/сервер»

Единая двухуровневая система, состоящая из:

- Процесс-клиент (толстый или интеллектуальный клиент). Выполняет запросы на доступ к некоторым ресурсам. Внешний компонент, или внешний интерфейс.
- Процесс-сервер, предоставляющий некоторые ресурсы. Внутренний компонент, или машина базы данных.

Процессы совсем необязательно должны быть размещены на одном компьютере.

Обычно сервер располагается на одном узле ЛВС, а клиенты – на других узлах.

Сервер— это сама СУБД.

Он поддерживает все основные функции СУБД:

- определение данных;
- манипулирование данными;
- защиту данных
- поддержание целостности данных и т.д.

В частности, он предоставляет полную поддержку внешнего, концептуального и внутреннего уровней. Поэтому сервер в этом контексте — это просто другое название для СУБД.

Сервер принимает и обрабатывает запросы к базе данных, включая проверку полномочий клиента, обеспечение требований целостности, поддержку системного каталога, поддержку параллельного доступа, выполнение запроса и обновление данных, а затем передает полученные результаты обработки обратно клиенту.

Клиенты — это различные приложения, которые выполняются с помощью СУБД.

Таковыми являются как приложения, написанные пользователями, так и встроенные приложения, предоставляемые поставщиками СУБД или некоторыми сторонними поставщиками программного обеспечения.

В самом сервере разница между встроенными приложениями и приложениями, написанными пользователем, не обнаруживается, поскольку все эти приложения используют один и тот же интерфейс сервера, а именно **интерфейс внешнего уровня**.

Некоторые специальные "служебные" приложения могут оказаться исключениями. Они иногда работают непосредственно на внутреннем уровне системы. Подобные утилиты правильнее относить к встроенным компонентам СУБД, а не к приложениям в обычном смысле.

Клиент управляет пользовательским интерфейсом и логикой приложения. Обычно это рабочая станция с пользовательским приложением, которое:

- принимает от пользователя запрос;
- проверяет синтаксис;
- генерирует запрос к базе данных;
- передает сообщение серверу;
- ожидает поступление ответа;
- форматирует полученные данные для визуализации.

Приложения

- приложения, написанные пользователями;
- приложения, предоставляемые поставщиками.

Приложения, написанные пользователями

В основном, это обычные прикладные программы, чаще всего написанные либо на языке программирования общего назначения, таком как C++ или Java (COBOL), либо на одном из специализированных языков четвертого поколения:

- общего использования (Cliper, Clarion, dBase, 4GL Cobol/PLI generator, ...);
- запросов к базам данных (SQL, Informix-4GL, ...);
- генераторы отчетов (Oracle Reports, ..);
- языки манипулирования данными (Clarion, IDL, PL/SQL, R, ...)\$
- ...

Приложения, предоставляемые поставщиками

Их часто называют *инструментальными средствами*.

В целом, назначение таких средств — упрощать процесс создания и выполнения других приложений, т.е. приложений, которые разрабатываются специально для решения некоторой конкретной задачи.

Часто эти приложения могут выглядеть вовсе не так, как приложения в общепринятом смысле.

Назначение инструментальных средств также состоит в предоставлении пользователям, особенно конечным, возможности создавать приложения без написания традиционных программ.

Например, одно из предоставляемых поставщиком СУБД инструментальных средств может быть генератором отчетов, с помощью которого конечный пользователь сможет получить отформатированный отчет, выполнив обычный запрос к системе.

Каждый такой запрос является по существу небольшим специальным приложением, написанным на языке очень высокого уровня (с конкретным назначением), а именно — на языке формирования отчетов.

Отдельно поставляемые инструментальные средства делятся на несколько самостоятельных классов:

- процессоры языков запросов;
- генераторы отчетов;
- графические бизнес-подсистемы;
- электронные таблицы;
- процессоры команд на естественных языках;
- статистические пакеты (R);
- средства управления копированием или средства извлечения данных;
- генераторы приложений, включая процессоры языков четвертого поколения;
- другие средства разработки приложений, включая CASE-инструменты (CASE, или Computer-Aided Software Engineering – автоматизированное проектирование и создание программ);
- инструментальные средства интеллектуального анализа данных и визуализации.

Преимущества технологии «клиент/сервер»

- более широкий доступ к существующим базам данных;
- повышение производительности, по сравнению с телеобработкой и/или ФС;
- снижение стоимости аппаратного обеспечения (серверная машина – более мощная, чем клиентские);
- снижение сетевого трафика (по сети идут только необходимые данные);
- повышается уровень непротиворечивости данных (проверка целостности данных выполняется централизованно на сервере, а не у клиентов).

Расширение двухуровневой технологии «клиент/сервер» трехуровневая структура.

- тонкий (неинтеллектуальный) клиент, управляющий только пользовательским интерфейсом;
- средний уровень (клиент), управляющий логикой пользовательского приложения;
- сервер базы данных.

Низкоуровневые операции логического уровня в таблицах

Обычно они реализованы, как операторы 4GL языков в СУБД на основе файловой системы.
Мнемоника обобщенная.

```
TABLE LIST (ID, NAME, DATE, ...)
```

```
TABLE ITEM (ID, LIST_ID, PART_NO, NAME, RPOVIDER, QUANT, ...)
```

```
PRIMKEY LIST:BY_ID (ID)                # LIST:ID
```

```
PRIMKEY ITEM:BY_ID (ID)                # IITM:ID
```

```
INDEX LIST:BY_NAME                      (NAME)
```

```
INDEX LIST:BY_DATE                      (DATE)
```

```
INDEX ITEM:BY_NAME                      (NAME)
```

```
INDEX ITEM:BY_NAME_INLIST (LIST_ID, NAME)
```

```
INDEX ITEM:BY_PNO_INLIST (LIST_ID, PART_NO) # кандидат в PrimKey (?поставщик?)
```

Простейшие сериальные операции

SET(TABLE) – установка указателя в «сыром» порядке записей

NEXT(TABLE) – просмотр таблицы в направлении с начала к концу

PREV(TABLE) – просмотр таблицы в направлении с конца к началу

INSERT(RECORD, TABLE) – вставка в таблицу без обновления индексов (APPEND)

Прямой доступ по первичному ключу

GET(RECORD, TABLE, KEY) – получить строку (запись) из таблицы по значению перв. ключа

PUT(RECORD, TABLE, KEY) – обновить строку (запись) в таблице по значению перв. ключа

Добавление/удаление

ADD(RECORD, TABLE) – добавить строку (запись) в таблицу и обновить все индексы

DEL(RECORD, TABLE) – удалить строку (запись) из таблицы и обновить все индексы

Прямой доступ по ключу

GET(RECORD, TABLE, INDEX, KEY_VALUE) – получить строку по значению ключа

PUT(RECORD, TABLE, INDEX, KEY_VALUE) – обновить строку по значению ключа

Сериальные операции в порядке индексов

SET(TABLE, KEY, KEY_VALUE) – установить указатель доступа в порядке индекса

NEXT(TABLE, KEY) – просмотр таблицы в направлении с начала к концу

PREV(TABLE, KEY) – просмотр таблицы в направлении с конца к началу

Базы данных

Лекция 04 – Реляционная модель данных

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2022.09.22

Оглавление

Дореляционные модели данных.....	3
Иерархическая модель данных.....	3
Сетевая модель данных.....	5
Реляционная модель данных.....	7
Табличное представление.....	8
Ключи.....	12
Древовидные структуры.....	13
Реляционные базы данных.....	14
Смысл отношений.....	19
Структурные свойства.....	22
Оптимизация.....	29
Каталог базы данных.....	30

Дореляционные модели данных

Иерархическая модель данных

Иерархическая модель данных возникла из практики работы с данными и создавалась по сути дела программистами. Стандартов на иерархические модели не существует.

Наиболее распространенной СУБД реализующей иерархическую модель являлась СУБД IMS.

Ключевым фактором при создании IMS была необходимость управления большим количеством деталей, связанных друг с другом иерархическим образом (детали → узлы → модули и т.п.).

Иерархическая модель – модель данных на основе записей, в которой все отношения между объектами структурированы в виде деревьев:

- 1) все записи хранятся в общей древовидной структуре, имеющей одну корневую запись;
- 2) каждый узел дерева – запись, описывающая экземпляр объекта;
- 3) соединения между записями в дереве определяют связи между экземплярами объектов.

В иерархической модели связи называются отношениями «предок-потомок», а записи – сегментами.

Потомок может быть связан только с одним предком и для реализации связи с разными предками требуется построение разных деревьев.

Отношение «предок-потомок» реализуют связи «один-ко-многим»;

Иерархическая БД – совокупность деревьев.

Все отношения, которые могут быть представлены в иерархической модели данных – бинарные.

Бинарное отношение — отношение между двумя множествами A и B , то есть всякое подмножество декартова произведения этих множеств: $R \subseteq A \times B$. Бинарное отношение на множестве A – любое подмножество $R \subseteq A \times A$ (равенство, неравенство, эквивалентность, отношение порядка, ...).

Алгоритм перевода ER-диаграммы в иерархическую модель

Алгоритм перевода ER-диаграммы в иерархическую модель данных представлен следующими шагами:

1) Для каждого объекта ER-диаграммы создается тип сегмента в иерархической модели.

Название типа сегмента соответствует названию объекта.

Атрибуты объекта задают поля записи сегмента (название, тип данных).

2) В диаграммах деревьев отношения **один-ко-многим** задаются с помощью отношений «предок-потомок», причем объект со стороны много в связи становится потомком, а объект со стороны один – предком.

3) Для реализации отношений **многие-ко-многим** создается два разных отношения «предок-потомок», в этих отношениях один и тот же объект выступает в качестве предка (первое отношение «предок-потомок») и в качестве потомка (второе отношение «предок-потомок»).

В одном из таких отношений «предок-потомок» реальные данные для экономии места заменяют на ссылки (указатели) на данные из второго отношения.

4) Если отношение мощности многие-ко-многим на ER-диаграмме обладает атрибутом, то в диаграммах деревьев создается сегмент пересечения, который располагается между предком и потомком и содержит значение атрибута.

Основное достоинство иерархической модели данных — представление данных в виде иерархических связей и автоматическое ограничения целостности (не существует потомков без родителей).

К **основным недостаткам** иерархической модели данных относятся:

- нет разделения логической и физической модели данных;
- непредвиденные запросы требуют реорганизации БД;
- для не иерархических связей описание сильно усложняется и требует много памяти;
- сложность составления программ обработки данных;
- нет стандарта на построение подобных систем.

Сетевая модель данных

Сетевая модель данных – модель данных на основе записей, в которой данные представлены сетевыми структурами типа направленного графа.

Разница между *иерархической моделью данных* и *сетевой* состоит в том, что в иерархических структурах запись-потомок должна иметь в точности одного предка, а в сетевой структуре данных у потомка может иметься любое число предков.

Узлы в такой модели представляют объекты и называются – *наборами типов записей данных*.

Ребра графа представляют отношения между объектами и называются *наборами типов связей*.

Наборы выражают отношения «один-ко-многим» или «один-к-одному» между двумя типами записей, при этом выделяют тип записи *владельца* – со стороны «один» связи, и тип записи *члена* набора – со стороны «многие» связи.

Сетевая БД состоит из *наборов отношений*.

Все отношения, которые могут быть представлены в сетевой модели данных – бинарные.

Алгоритм перевода ER-диаграммы в сетевую модель данных:

1) Для каждого объекта ER-диаграммы создается тип записей в сетевой модели.

Название типа записей соответствует названию объекта.

Атрибуты объекта задают поля типа записи (название, тип данных).

2) Для отношения «**один-ко-многим**» тип записи со стороны «один» отношения становится владельцем набора, а тип записи со стороны «многие» отношения – членом набора.

Если мощность отношения «**один-к-одному**», то типы записей в наборе выбираются произвольно.

3) Для каждого n-арного отношения, где $n > 2$, создается связывающая запись, которая становится типом записи члена для n наборов.

Владельцами наборов назначаются типы записей со стороны «один», полученных в результате преобразований отношений «один-ко-многим».

4) Для реализации отношений **«многие-ко-многим»** создается связывающая запись, которая становится типом записи члена для двух наборов, владельцами которых являются типы записей, соответствующие объектам связи.

5) Если отношение мощности «многие-ко-многим» на ER-диаграмме обладает атрибутом, то этот атрибут добавляется в связывающую запись.

Основные достоинства сетевой модели данных:

- высокая производительность в статичных системах с простой структурой;
- наличие ограничений целостности в отношении «зависших» потомков;
- **наличие стандарта (DBTG).**

К **основным недостаткам** сетевой модели данных относятся:

- структура БД определяется запросами пользователей — смена запросов ведет к реорганизации всей БД;
- производительность непредвиденных приложений низкая — нет возможности расширения;
- определение схемы влияет на физический уровень БД (формат записей);
- сложность составления программ обработки данных;
- сложно установить ограничения на образование связей.

Операции

- вставка, чтение, обновление и удаление записей;
- вставка, модификация (переключение) и удаление связей между владельцем и членом.

Реляционная модель данных

Замечание о терминологии

SQL и реляционная модель – вовсе не одно и то же.

Дейт уточняет:

«Если знания о реляционной модели проистекают исключительно из знания SQL, то эти знания могут оказаться неверными. Отсюда, в частности, следует, что кое-что из уже известного следует забыть.»

В рамках реляционной теории употребляются формальные термины — «отношение», «кортеж» и «атрибут».

В SQL эти термины не используются, там применяются более «дружественные» слова — таблицы, строка и столбец/колонка.

Истина заключается в том, что отношение – это не таблица, кортеж – не строка, а атрибут – не столбец.

SQL пытался упростить один набор терминов, но он сделал все возможное для усложнения другого. Имеются в виду термины *оператор*, *функция*, *процедура*, *подпрограмма* и *метод* — все они обозначают по существу одно и то же (быть может, с незначительными вариациями).

Реляционная модель для всего вышеперечисленного использует термин ***оператор***.

Представление данных в виде деревьев, сетевых и списковых структур в общем случае препятствует многим изменениям данных, необходимым при росте базы.

Рост базы может привести к нарушению логического представления данных и, как следствие, к изменениям в прикладных программах.

Избежать растущей сложности древовидных и сетевых структур можно с помощью метода, называемого *нормализацией*. Этот метод был разработан и активно применялся Коддом.

Нормализация относится к представлению данных пользователем или к логическому описанию данных; она не связана с физическим представлением данных.

Табличное представление

Один из самых естественных способов представления данных для обычного пользователя — это двумерная таблица.

Имя атрибута:	Номер-служашего	Имя-служашего	Пол	Звание	Дата-рождения	Отдел	Код-профессии	Должность	Зарплата
Форма представления:	N5	AV	B1	N2	N6	N3	N2	AV	N4
Значение атрибута:	53730	ДЕСНС БИЛЛ У	1	03	100335	044	73	БУХГАЛТЕР	2000
	28719	БЛЭНАГАН ДЖОЙ Е	1	05	101019	172	43	МОНТАЖНИК	1800
Запись, сегмент, кортеж:→	53550	ЛОРАНС МАРИГОЛД	0	07	090938	044	02	КОНТОРСКИЙ СЛУЖ.	1100
	79632	РОКФЕЛЛЕР ФРЕД	1	11	011132	090	11	КОНСУЛЬТАНТ	5000
	15971	РОШЛИ ЭД С	1	13	021242	172	43	МОНТАЖНИК	1700
	51883	СМИТ ТОМ П У	1	03	091130	044	73	БУХГАЛТЕР	2000
	36453	РАЛНЕР УИЛЬЯМ К	1	03	110941	044	02	КОНТОРСКИЙ СЛУЖ.	1200
	41618	ХОРСРЕДИШ ФРЕДА	0	07	071235	172	07	ИНЖЕНЕР	2500
	61903	ХОЛЛ АЛЬБЕРТ	1	11	011030	172	21	АРХИТЕКТОР	3700
	72921	ФАЙР КАРОЛИН	0	03	020442	090	93	ПРОГРАММИСТ	2100

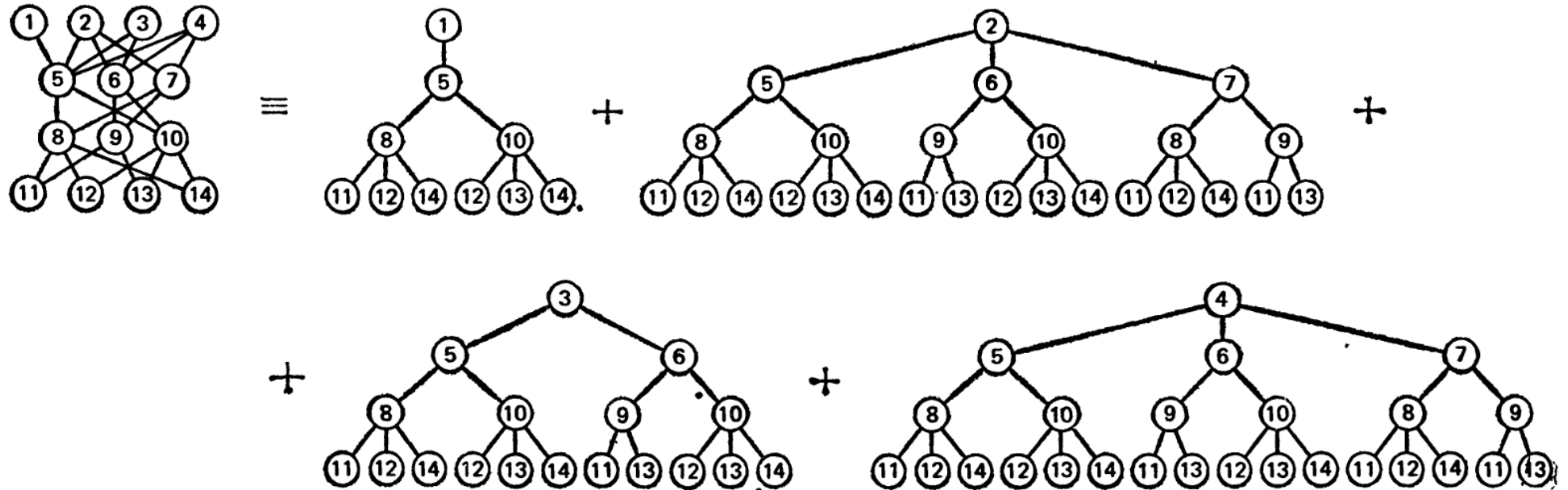
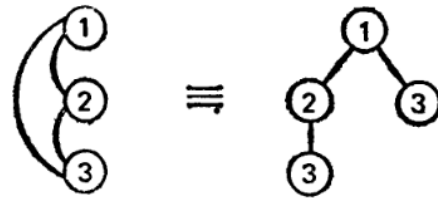
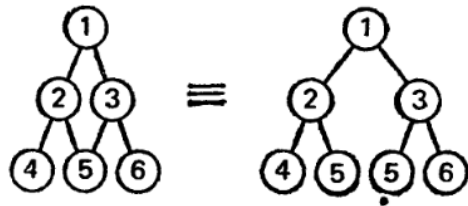
Идентификатор объекта

Набор величин одного элемента данных (домен)

Некоторые атрибуты сами представляют собой идентификаторы объекта в другом файле

Она привычна для пользователя, понятна и обозрима, ее структуру легко запомнить.

Любая сетевая структура может быть с некоторой избыточностью разложена в совокупность древо-видных структур.



Любое представление данных может быть сведено с некоторой избыточностью к двумерным плоским файлам.

Связи между данными могут быть представлены в форме двумерных таблиц.

Этот процесс, выполняемый шаг за шагом для каждой связи между данными в базе, называется *нормализацией*.

Таблицы могут быть построены таким образом, что не будет утеряна информация о связях между элементами данных. Рассматриваемые таблицы — это прямоугольные массивы, которые можно описать математически.

Таблица обладает следующими свойствами

- 1) Каждый элемент таблицы представляет собой один элемент данных; повторяющиеся группы отсутствуют.
- 2) Все столбцы в таблице однородные; это означает, что элементы столбца имеют одинаковую природу.
- 3) Столбцам однозначно присвоены имена.
- 4) В таблице нет двух одинаковых строк.
- 5) В операциях с такой таблицей ее строки и столбцы могут просматриваться в любом порядке и в любой последовательности безотносительно к их информационному содержанию и смыслу.

Выделение повторяющейся группы в отдельную таблицу путем расщепления

Заказ-на-закупку

No заказа	No поставщика	Дата заказа	Дата поставки	ИТОГО		
				No изделия	Цена	Кол-во

Заказ-на-закупку

No заказа	No поставщика	Дата заказа	Дата поставки	ИТОГО

Ключ

Партия-товара

No заказа	No изделия	Цена	Кол-во

Ключ

Ключи

Каждый кортеж должен иметь ключ — уникальный идентификатор.

Иногда кортеж может идентифицироваться значением одного атрибута. Например, атрибут НОМЕР-ЗАКАЗА однозначно определяет кортеж файла ЗАКАЗ-НА-ЗАКУПКУ.

Но бывает и так, что для идентификации кортежа необходимы значения нескольких атрибутов.

Для определения кортежа файла ПАРТИЯ-ТОВАРА недостаточно одного атрибута, и ключ должен состоять из двух атрибутов:

НОМЕР-ЗАКАЗА + НОМЕР-ИЗДЕЛИЯ.

Ключ должен обладать двумя свойствами:

1) Однозначная идентификация кортежа — кортеж должен однозначно определяться значением ключа.

2) Отсутствие избыточности — никакой атрибут нельзя удалить из ключа, не нарушая при этом свойства однозначной идентификации.

Для кортежей одного отношения может существовать несколько наборов атрибутов, удовлетворяющих двум приведенным свойствам.

Такие наборы называются *возможными ключами*.

Тот ключ, который фактически будет использоваться для идентификации записей, называется **первичным ключом**.

Правила выбора атрибутов первичного ключа

Для первичного ключа атрибуты следует выбирать так, чтобы для них был заранее известен диапазон значений, а их количество было бы как можно меньше.

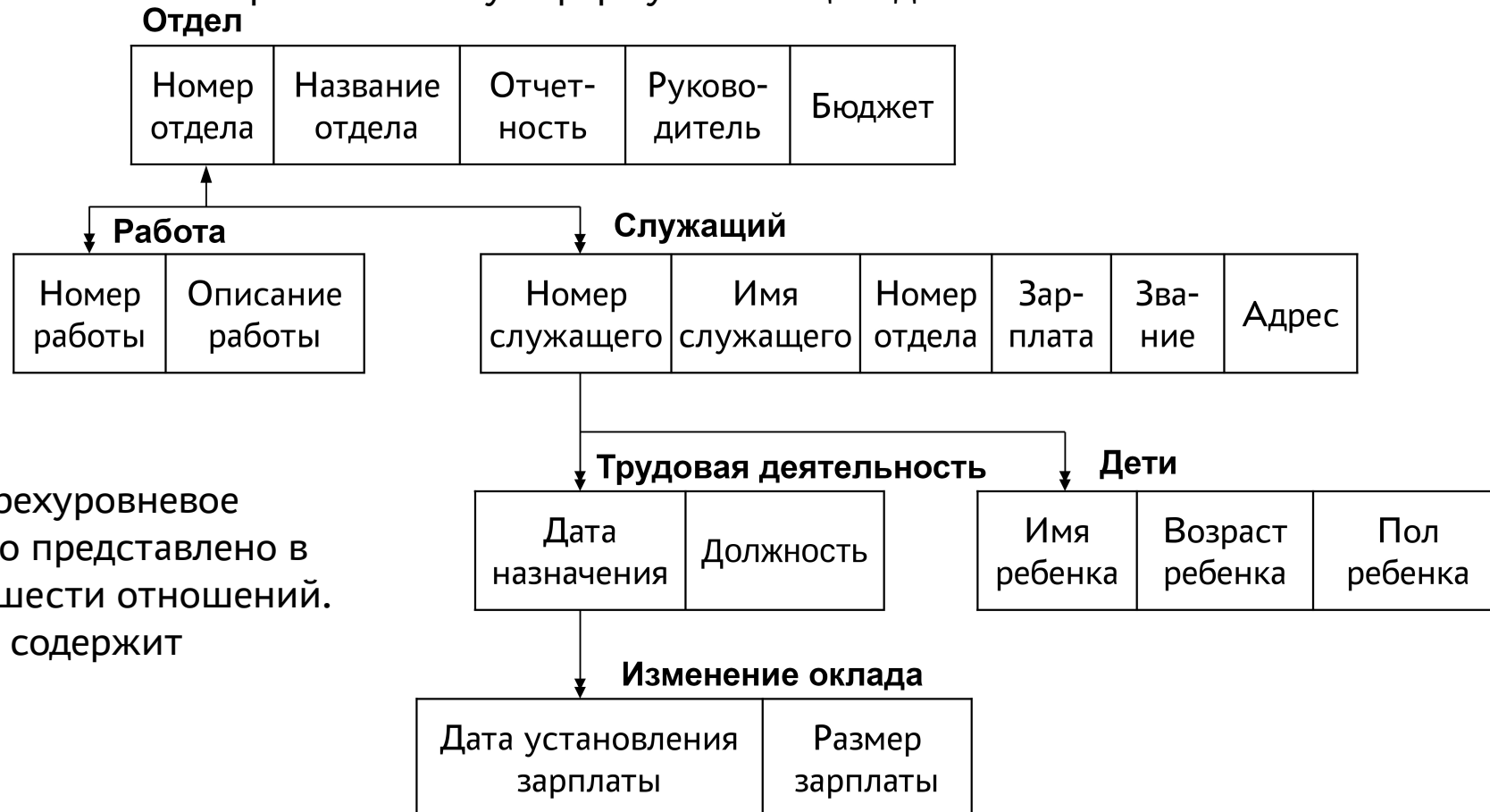
Обозначения

Заказ-на-закупку(Номер-заказа, Номер-поставщика, Дата-заказа, Дата-поставки, Итого)

Партия-товара(Номер-заказа, Номер-изделия, Цена, Количество)

Древовидные структуры

Преобразование в нормализованную форму с помощью добавления ключевых элементов



Четырехуровневое
дерево представлено в
виде шести отношений.
Ключ содержит

Отдел(Номер-отдела, Название-отдела, Отчетность, Руководитель, Бюджет)

Работа(Номер-отдела, Номер-работы, Описание-работы)

Служащий(Номер-служащего, Имя-служащего, Номер-отдела, Зарплата, Звание, Адрес)

Изменение-оклада(Номер-служащего, Дата-установления-зарплаты, Размер-зарплаты)

Дети(Номер-служащего, Имя-ребенка, Возраст-ребенка, Пол-ребенка)

Трудовая-деятельность(Номер-служащего, Дата-назначения, Должность)

Реляционные базы данных

Сторонники нормализации ввели собственную терминологию и называют простые вещи специальными словами.

Таблица такого вида, как представлено выше, называется *отношением*.

База данных, построенная с помощью отношений, называется *реляционной базой данных*.

Таким образом, реляционная база данных строится из «плоских» (двумерных) наборов элементов данных.

Отношение, или *таблица*, — это набор кортежей.

Кортежи являются n -мерными, т. е. если таблица имеет n столбцов, отношение называется *отношением степени n* .

Отношение степени 2 называется бинарным, отношение степени 3 — тернарным, а отношение степени n — n -арным.

Набор значений элементов данных одного типа, т. е. один столбец таблицы, называется *доменом*.

Столбец с номером k называется k -м доменом отношения.

В математике R определяется как отношение, заданное на n множествах $S_1, S_2, \dots, S_n$ (не обязательно различных), если оно представляет собой набор кортежей, таких, что первый элемент каждого кортежа есть элемент из S_1 , второй — элемент из S_2 и т. д.

Для описания таких отношений и операций над ними существуют точные математические обозначения, основанные на алгебре отношений или исчислении отношений (реляционная алгебра).

Реляционная алгебра — замкнутая система операций над отношениями в реляционной модели данных.

Операции реляционной алгебры также называют реляционными операциями.

Кортеж — определение

Если дана коллекция типов $T_i (i = 1, 2, \dots, n)$, которые не обязательно все должны быть разными, то значением кортежа (или кратко кортежем), определенным с помощью этих типов (назовем его t), является множество упорядоченных троек в форме $\langle A_i, T_i, v_i \rangle$, где A_i — имя атрибута, T_i — имя типа и v_i — значение типа T_i .

Кроме того, кортеж t должен соответствовать приведенным ниже требованиям:

- Значение n является **степенью**, или **арностью** кортежа t .
- Упорядоченная тройка $\langle A_i, T_i, v_i \rangle$ является компонентом (частью) t .
- Упорядоченная пара $\langle A_i, T_i \rangle$ представляет собой **атрибут** t и однозначно определяется именем атрибута A_i (имена атрибутов A_i и A_j совпадают, только если $i = j$).
- Значение v_i — это **значение атрибута**, соответствующее атрибуту A_i кортежа t .
- Тип T_i — это соответствующий **тип атрибута**.
- Полное множество атрибутов составляет **заголовок кортежа** t .
- Тип кортежа t определен заголовком t , а сам заголовок и этот тип кортежа имеют такие же атрибуты (и поэтому такие же имена и типы атрибутов) и такую же степень, как и кортеж t .

Точное определение имени типа кортежа:

TUPLE {A1 T1, A2 T2, ..., An Tn}

Пример кортежа:

MAJOR_PNO : PARTNO	MINOR_PNO : PARTNO	QTY : QUANTITY
P2	P4	7

Имена атрибутов: MAJOR_PNO, MINOR_PNO, QTY

Имена типов: PARTNO, QUANTITY

Значения: PARTNO{'P2'}, PARTNO{'P4'}, QUANTITY{7}

TUPLE {MAJOR_PNO PARTNO, MINOR_PNO PARTNO, QTY QUANTITY}

В неформальном изложении принято исключать имена типов из заголовка кортежа и показывать только имена атрибутов. Поэтому показанный выше кортеж может быть неформально представлен:

MAJOR_PNO	MINOR_PNO	QTY
P2	P4	7

Термин «кортеж» приблизительно соответствует понятию строки, так же, как термин «отношение» приблизительно соответствует понятию таблицы.

В реляционной модели не используется термин «поле», вместо него используется термин «атрибут», который в рассматриваемом контексте примерно соответствует понятию столбца таблицы.

Пусть есть БД отделов и сотрудников:

DEPT (Отделы)

DEPT#	DNAME	BUDGET
D1	Marketing	10M
D2	Development	12M
D3	Research	5M

EMP (Служащие)

EMP#	ENAME	DEPT#	SALARY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
E3	Finci	D2	30K
E4	Saito	D2	35K

Δ

EMP.DEPT# ссылается на DEPT. DEPT#

Таблицы **DEPT** и **EMP** в базе данных фактически являются *переменными отношениями*.

Соответственно, как любые переменные, имеют значения. Их значения — *значения отношения*, которые они принимают в разное время.

Предположим, например, что таблица **EMP** в данный момент имеет значение (значение отношения), которое показано выше, и допустим, что мы удалили строку о сотруднике с фамилией Saito.

DELETE EMP WHERE EMP# = EMP# ('E4') ;

Результат выполнения этой операции:

EMP (Служащие)

EMP#	ENAME	DEPT#	SALARY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
E3	Finci	D2	30K

Концептуально это действие можно описать таким образом — **старое значение отношения EMP было заменено в целом совершенно другим, новым значением отношения.**

Старое значение с четырьмя строками и новое значение с тремя очень похожи, но в действительности они являются разными.

Можно считать, что данная операция удаления строки — это просто альтернативный, упрощенный способ записи для определенной операции реляционного присваивания, которая могла бы выглядеть примерно следующим образом¹.

```
EMP := EMP WHERE NOT (EMP# = EMP#('E4'));
```

Сначала вычисляется выражение, расположенное справа от знака присваивания, а затем его значение присваивается переменной, которая записана перед знаком присваивания.

В целом задача заключается в том, чтобы заменить «старое» значение отношения EMP «новым».

Таким же образом операции INSERT и UPDATE также являются просто сокращенной формой записи соответствующих реляционных операций присваивания.

В реляционной теории (РТ) следует четко различать *переменные отношения* и сами *отношения*.

Для переменной отношения (*relation variable*) иногда используется термин *relvar*².

Термин «*переменная отношения*» (*relvar*) не является общепринятым и в документации на БД практически не встречается, хотя различать сами отношения, т.е. значения отношений, и переменные отношения как таковые, с точки зрения РТ очень важно. Например, ограничения целостности и операции обновления, применяются к переменным отношения, а не к отношениям.

1) Оператор DELETE и равносильный ему оператор присваивания записаны на языке Tutorial D

2) Различие между значениями отношения и переменными отношения фактически представляют собой особый случай различия между значениями и переменными в целом.

Смысл отношений

Столбцы в отношениях связаны с типами данных. Реляционная модель допускает неограниченный набор типов [данных]. Это означает, что пользователи могут как применять определяемые системой или встроенные типы, так и определять собственные. Например, определять типы можно так («...» здесь заменяет сами определения, которые на сей момент не важны).

```
TYPE EMP_NO ... ;  
TYPE NAME ... ;  
TYPE DPT_NO ... ;  
TYPE MONEY ... ;
```

Тип **EMP_NO**, например, можно рассматривать, как множество всевозможных табельных номеров, тип **NAME** — как множество всевозможных имен и т.д.

Каждое отношение (каждое значение отношения) состоит из двух частей:

- 1) набор пар (<имя_столбца> : <имя_типа>) — заголовок;
- 2) набор строк, согласованных с этим заголовком — тело.

Пример — часть отношения EMP, дополненного обозначениями типов столбцов.

EMP (Служащие)

EMP# : EMP_NO	ENAME : NAME	DEPT# : DEPT_NO	SALARY : MONEY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
E3	Finci	D2	30K
E4	Saito	D2	35K

Хотя на практике компоненты заголовка <имя_типа> обычно опускают, необходимо учитывать, что концептуально они всегда присутствуют.

Отношения можно представлять также следующим образом:

1) В определенном отношении R его заголовок представляет собой некоторый **предикат** (функция, возвращающая значения истинности и принимающая ряд формальных параметров).

2) Каждая строка в теле отношения R представляет собой определенное **истинное высказывание**, полученное из предиката путем подстановки определенных значений фактических параметров соответствующего типа вместо формальных параметров этого предиката (путем конкретизации).

EMP (Служащие)

EMP# : EMP_NO	ENAME : NAME	DEPT# : DEPT_NO	SALARY : MONEY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
...	...	...	...

Для отношения EMP предикат будет следующим:

«Служащий с табельным номером **EMP#** и фамилией **ENAME** работает в отделе с номером **DEPT#** и получает зарплату **SALARY**».

Здесь формальные параметры – EMP#, ENAME, DEPT# И SALARY. Они соответствуют четырем столбцам переменной отношения **EMP**.

Соответствующие истинные утверждения:

«Служащий с табельным номером E1 и фамилией Lopez работает в отделе с номером D1 и получает зарплату 40 тыс. долл. в год.»

«Служащий с номером E2 и фамилией Cheng работает в отделе с номером D1 и получает зарплату 42 тыс. долл. в год.»

Иными словами, **типы** — это объекты (множества объектов), которые могут стать предметом обсуждения;

отношения — это факты (множества фактов), касающиеся объектов, которые могут стать предметом обсуждения.

Типы связаны с отношениями точно так же, как существительные связаны с предложениями на естественном языке.

В примере, приведенном выше, предметом обсуждения являются:

- табельные номера служащих EMP_NO;
- имена NAME;
- номера отделов DEPT_NO;
- значения денежных сумм MONEY.

Об обсуждаемом же предмете можно привести истинное высказывание следующего вида:

«Служащий с определенным табельным номером имеет определенное имя, работает в определенном отделе и получает определенную зарплату».

Структурные свойства

В оригинальной модели три основных компонента:

- структура;
- целостность;
- средства манипулирования.

Структура

Главным структурным свойством является само отношение.

Отношения принято изображать на бумаге в виде таблиц.

DEPT

DNO	DNAME	BUDGET
D1	Marketing	10M
D2	Development	12M
D3	Research	5M

Δ

EMP

ENO	ENAME	DNO	SALARY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
E3	Finci	D2	30K
E4	Saito	D3	35K

DEPT.DNO ссылается на EMP.DNO

Отношения определены над *типами* (типы называют также *доменами*).

Тип представляет собой *концептуальное множество значений*, которые могут принимать фактические (конкретные) значения атрибутов фактических (конкретных) отношений.

В простой БД об отделах и служащих можно выделить тип **DNO** («номера отделов»), представляющий все допустимые номера отделов, и тогда атрибут **DNO** в отношении **DEPT** и атрибут **DNO** в отношении **EMP** будут принимать значения из соответствующего ему концептуального множества.

Атрибуты не обязательно называть так же, как соответствующий тип.

Структурный аспект

Данные в базе воспринимаются пользователем, как таблицы и никак иначе.

Аспект целостности

Эти таблицы отвечают определенным условиям целостности.

Аспект обработки

В распоряжении пользователя имеются операторы манипулирования таблицами, например, предназначенные для поиска данных, которые генерируют новые таблицы на основании уже имеющихся и среди которых есть, по крайней мере, операторы:

- сокращения (restrict);
- проекции (project);
- объединения (join).

Операция сокращения извлекает указанные строки из таблицы. В названии этой операции подразумевается, что *кардинальность* ее результата меньше или равна кардинальности исходной таблицы.

Операцию сокращения иногда называют *выборкой* (select). Однако, соответствующий оператор соответствует оператору SELECT языка SQL не полностью.

Операция проекции предназначена для извлечения определенных столбцов из таблицы.

Операция соединения предназначена для получения комбинации двух таблиц на основе общих значений в общих столбцах.

Операция соединения

DEPT

DEPT#	DNAME	BUDGET
D1	Marketing	10M
D2	Development	12M
D3	Research	5M

Δ

EMP

EMP#	ENAME	DEPT#	SALARY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
E3	Finci	D2	30K
E4	Saito	D2	35K

DEPT.DNO ссылается на EMP.DNO

В таблицах DEPT и EMP есть общий столбец DEPT#, а следовательно, для этих таблиц можно выполнить операцию соединения на основе общих значений в этом столбце. При выполнении данной операции строка таблицы DEPT соединяется со строкой таблицы EMP и образуется более длинная строка, но подобное происходит тогда и только тогда, когда эти две строки имеют одинаковое значение поля DEPT#.

Например, можно соединить в результирующую строку следующие строки таблиц DEPT и EMP (названия столбцов приведены для наглядности). Это возможно, так как в общем столбце рассматриваемых строк имеется одно и то же значение D1.

DEPT#	DNAME	BUDGET	EMP#	ENAME	SALARY
D1	Marketing	10M	E1	Lopez	40K
D1	Marketing	10M	E2	Cheng	42K
D2	Development	12M	E3	Finci	30K
D2	Development	12M	E4	Saito	35K

Общий результат состоит из множества всех таких соединенных строк.

Столбец DEPT# в каждой результирующей строке встречается один раз, а не два.

Следует также отметить, что в поле DEPT# таблицы EMP отсутствует значение D3, т.е. нет служащих, работающих в отделе D3, поэтому в данном поле нет и результирующих строк со значением D3, хотя строка со значением D3 в таблице DEPT присутствует.

Сокращение (извлекает указанные строки из таблицы)

DEPTs where BUDGET > 8M

DEPT#	DNAME	BUDGET
D1	Marketing	10M
D2	Development	12M

Проекция (извлекает определенные столбцы из таблицы)

DEPTs over DEPT#, BUDGET

DEPT#	BUDGET
D1	10M
D2	12M
D3	5M

Важная особенность — результат выполнения каждой из трех представленных операций — это еще одна таблица, т.е. операторы сокращения, проекции и соединения порождают таблицы из таблиц.

Это — пример проявления реляционного свойства замкнутости. Оно имеет очень большое значение, главным образом потому, что результатом выполнения операции является объект того же рода, что и объект, над которым производилась операция, а именно — таблица.

Это значит, что

К результатам операций можно снова применить какую-либо операцию.

Например, можно сформировать проекцию соединения, соединение двух сокращений, сокращение проекции и т.д.

То есть, можно использовать вложенные выражения, т.е. выражения, в которых операнды представлены выражениями, а не простыми именами таблиц.

Промежуточные результаты

Тот факт, что результатом выполнения каждой операции является таблица, не подразумевает, что система обязательно должна полностью материализовать результат каждой отдельной операции в виде реальной таблицы — промежуточный результат, который создается операцией соединения, возможно, никогда и не будет существовать в виде полной материализованной таблицы, как таковой.

На практике, как правило, разработчики каждой системы настойчиво стремятся избежать полной материализации промежуточных результатов в целях повышения производительности.

Если промежуточные результаты материализуются полностью, то стратегия вычисления выражения в целом называется **материализованными вычислениями** (materialized evaluation).

Если промежуточные результаты передаются последующей операции по частям, то этот подход называется **конвейерными вычислениями** (pipelined evaluation).

Важно — операции применяются сразу ко всему множеству строк, а не к отдельной строке за один раз, т.е. операндами и результатами являются не отдельные строки, а целые таблицы, которые содержат множество строк.

Особенности

Определение реляционной системы требует, чтобы база данных только воспринималась пользователем как набор таблиц.

Таблицы в реляционной системе являются **логическими**, а не **физическими** структурами.

На самом деле, на физическом уровне система может использовать любую из существующих структур памяти (последовательный файл, индексирование, хэширование, цепочку указателей, сжатие и т.п.), лишь бы существовала возможность отображать эти структуры в виде таблиц на логическом уровне.

Таблицы представляют собой **абстракцию** способа физического хранения данных, в которой все нюансы реализации на уровне физической памяти скрыты от пользователя, в частности это:

- 1) размещение хранимых записей;
- 2) упорядочение хранимых записей;
- 3) кодировка хранимых данных;
- 4) префиксы хранимых записей;
- 5) хранимые структуры доступа, такие как индексы и т.д.

Термин «логическая структура» в терминологии ANSI/SPARC (три уровня) относится как к концептуальному, так и ко внешнему уровням:

- концептуальный, и внешний уровни в реляционной системе являются одинаково реляционными;
- внутренний или физический уровень реляционными не является.

Реляционная теория не может определить внутренний уровень, она определяет лишь то, как база данных представлена пользователю. Единственное требование состоит в следующем — какая бы физическая структура не была выбрана, она должна полностью реализовывать существующую логическую структуру.

У реляционных баз данных есть одно замечательное свойство, так называемый информационный принцип — все информационное наполнение базы данных представлено одним и только одним способом, а именно — **явным заданием значений, помещенных в позиции столбцов в строках таблицы**. Этот метод представления — единственно возможный для реляционных баз данных (на логическом уровне). В частности, нет никаких указателей, связывающих одну таблицу с другой.

Аспект целостности

На практике для БД отделов и сотрудников потребовалось бы определить несколько ограничений поддержки целостности базы. Например:

- зарплата служащих не должна выходить за пределы от 25 до 95 тыс. долл. в год;
- бюджет отдела должен находиться в пределах от 1 до 15 млн.\$
- и т.д.

Некоторые из таких правил имеют настолько важное практическое значение, что получили специальные названия.

1) Каждая строка в таблице DEPT должна включать уникальное значение столбца DEPT#; аналогично, каждая строка в таблице EMP должна включать уникальное значение столбца EMP#.

Говорят, что DEPT# и EMP# в таблице EMP являются **первичными ключами для своих таблиц**.

2) Каждое значение столбца DEPT# в таблице EMP **должно** быть представлено и в виде значения столбца DEPT# в таблице DEPT в соответствии с тем фактом, что каждый служащий должен быть приписан к существующему отделу. Говорят, что столбец DEPT# в таблице EMP является **внешним ключом**, ссылающимся на первичный ключ таблицы DEPT .

Оптимизация

Все реляционные операции, такие как сокращение, проекция и соединение, выполняются на уровне множеств. Поэтому реляционные языки часто называют *непроцедурными*, а *декларативными*, так как пользователь указывает, **что** делать, а не **как** делать.

Фактически пользователь сообщает лишь, что ему нужно, без указания процедуры получения результата.

Процесс навигации (перемещения) по хранимой базе данных в целях удовлетворения запроса пользователя выполняется системой автоматически, а не пользователем вручную, поэтому реляционные системы иногда называют *системами автоматической навигации*.

В *нереляционных* системах за навигацию по базе данных в основном несет ответственность сам пользователь.

Следует отметить, что *непроцедурный* — это хотя и общепринятый, но не совсем точный термин, потому что *процедурный* и *непроцедурный* — понятия относительные.

Обычно можно с уверенностью определить лишь то, является ли язык А более (или менее) процедурным, чем язык Б.

Поэтому точнее будет сказать, что реляционные языки, такие как SQL, характеризуются более высоким уровнем абстракции, чем типичные языки программирования, подобные C++ или COBOL.

В принципе, именно более высокий уровень абстракции способствует повышению продуктивности труда программистов, свойственному для реляционных систем.

Ответственность за организацию выполнения автоматической навигации возложена на очень важный компонент СУБД — *оптимизатор*. Его работа заключается в том, чтобы выбрать самый эффективный способ выполнения для каждого запроса пользователя.

Каталог базы данных

Каждая СУБД должна поддерживать функции каталога, или словаря.

Каталог обычно размещается там, где хранятся различные схемы (внешние, концептуальные, внутренние) и все, что относится к отображениям («внешний-концептуальный», «концептуальный-внутренний», «внешний-внешний»).

В каталоге содержится подробная информация, касающаяся различных объектов, имеющих значение для самой системы (описательная информация или метаданные):

- переменные отношения;
- индексы;
- ограничения для поддержки целостности;
- ограничения защиты;
- и пр.

Эта информация необходима для обеспечения правильной работы системы.

Например, оптимизатор использует информацию каталога об индексах и других физических структурах хранения данных, а также прочую информацию, необходимую ему для принятия решения о том, как выполнить тот или иной запрос пользователя.

Подсистема защиты использует информацию каталога о пользователях и установленных ограничениях защиты, чтобы разрешить или запретить выполнение поступившего запроса.

Свойством реляционных систем является то, что их каталог также состоит из переменных отношений, таких же как и пользовательские.

Чтобы отличать их от пользовательских их называют системными переменными отношения.

В результате пользователь может обращаться к каталогу так же, как к своим данным.

Например, в каталоге любой системы SQL обычно содержатся системные переменные отношения **TABLES** и **COLUMNS**, назначение которых — описание известных системе таблиц, т.е. переменных отношений, и столбцов этих таблиц.

Каталог базы данных отделов и служащих может быть схематически представлен в **TABLES** и **COLUMNS** в следующем виде:

TABLE

TAB_NAME	COLS	ROWS	
DEPT	3	3	
EMP	4	4	
TABLE	3	123	

COLUMNS

TAB_NAME	COL_NAME	COL_TYPE	
DEPT	DEPT#	DPT_NO	
DEPT	DNAME	NAME	
DEPT	BUDGET	MONEY	
EMP	EMP#	EMP_NO	
EMP	ENAME	NAME	
EMP	DEPT#	DPT_NO	
EMP	SALARY	MONEY	
TABLE	TAB_NAME	TAB_NAME_T	
TABLE	COLS	COLS_T	
TABLE	ROWS	ROWS_T	

Каталог также должен описывать сам себя, т.е. включать записи, описывающие переменные отношения самого каталога.

Какие столбцы содержит переменная отношения **DEPT**³:

(COLUMNS WHERE TABNAME = 'DEPT'){COLNAME}

В каких переменных отношения есть столбец **EMP#**.

(COLUMNS WHERE COLNAME = 'EMP#'){TABNAME}

3) предполагается, что по каким-то причинам пользователь не имеет этой информации

Базы данных

Лекция 05 – Основы SQL

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2022.09.29

Оглавление

Язык SQL.....	3
Синтаксис.....	5
Идентификаторы и ключевые слова.....	6
Выделенные идентификаторы или идентификаторы в кавычках.....	7
Константы.....	8
Числовые константы.....	11
Операторы.....	13
Специальные знаки.....	15
Комментарии.....	16
Приоритеты операторов.....	17

Язык SQL

В языке SQL вместо терминов «отношение» и «переменная отношения» используются термины «таблица», а вместо терминов «кортеж» и «атрибут» — «строка» и «столбец».

Эти термины используются в стандарте языка SQL и в поддерживающих его продуктах.

SQL — язык очень большого объема. Его спецификация¹ содержит несколько тысяч страниц.

SQL не является в полной мере реляционным языком.

В языке SQL имеются операции как определения данных, так и манипулирования ими.

SQL (Structured Query Language — «язык структурированных запросов») — **декларативный язык программирования**, применяемый для создания, модификации и управления данными в реляционной базе данных под управлением СУБД (DBMS).

Предназначен для описания, изменения и извлечения данных, хранимых в реляционных базах данных. «Чистый» SQL без современных расширений считается языком программирования не полным по Тьюрингу, но в стандарте языка есть спецификация SQL/PSM, которая предусматривает возможность его процедурных расширений.

Изначально SQL был основным способом работы пользователя с базой данных и позволял выполнять следующий набор операций:

- создание в БД новой таблицы;
- добавление в таблицу новых записей;
- изменение записей;
- удаление записей;
- выборка записей из одной или нескольких таблиц в соответствии с заданным условием;
- изменение структур таблиц.

1) International Organization for Standardization (ISO): Information Technology — Database Languages— SQL, Document ISO/IEC 9075:2023

Современный SQL существенно усложнился — появились возможности описания и управления такими хранимыми объектами, как индексы, представления, триггеры и хранимые процедуры.

В результате стал приобретать черты, свойственные языкам программирования.

В данный момент SQL представляет собой совокупность операторов, инструкций и вычисляемых функций. Операторы SQL делятся на:

операторы определения данных (Data Definition Language, DDL):

CREATE создаёт объект базы данных (саму базу, таблицу, представление, пользователя и т.п.);

ALTER изменяет объект;

DROP удаляет объект.

операторы манипуляции данными (Data Manipulation Language, DML):

SELECT выбирает данные, удовлетворяющие заданным условиям;

INSERT добавляет новые данные;

UPDATE изменяет существующие данные;

DELETE удаляет данные.

операторы определения доступа к данным (Data Control Language, DCL):

GRANT предоставляет пользователю (группе) разрешения на определённые операции с объектом;

REVOKE отзывает ранее выданные разрешения;

DENY задаёт запрет, имеющий приоритет над разрешением.

операторы управления транзакциями (Transaction Control Language, TCL):

COMMIT регистрирует транзакцию;

ROLLBACK откатывает все изменения, сделанные в контексте текущей транзакции;

SAVEPOINT делит транзакцию на более мелкие участки.

Синтаксис

SQL-программа состоит из последовательности команд.

Команда — последовательность компонентов, оканчивающуюся точкой с запятой '; '.

Конец входного потока также считается концом команды.

Какие именно компоненты допустимы для конкретной команды, зависит от её синтаксиса.

Компонентом команды может быть:

- ключевое слово;
- идентификатор;
- идентификатор в кавычках;
- строка (или константа);
- специальный символ.

Компоненты обычно разделяются пробельными символами (пробел, табуляция, перевод строки), но это не требуется, если нет неоднозначности, например, когда спецсимвол оказывается рядом с компонентом другого типа.

Пример

```
SELECT * FROM MY_TABLE;  
UPDATE MY_TABLE SET A = 5;  
INSERT INTO MY_TABLE VALUES (3, 'hi there');
```

SQL-программы могут содержать **комментарии** – они не являются компонентами команд и пропускаются при интерпретации.

Идентификаторы – идентифицируют имена таблиц, столбцов или других объектов БД, в зависимости от того, где они используются. Иногда их называют «именами».

Ключевые слова и идентификаторы имеют одинаковую лексическую структуру, то есть, не зная языка, нельзя определить, является ли некоторый компонент ключевым словом или идентификатором. Ключевых слов около тысячи.

Идентификаторы и ключевые слова

Идентификаторы и ключевые слова SQL должны начинаться с буквы (**a-z**). Допускаются также не латинские буквы и буквы с диакритическими знаками или подчёркивания (**_**). Последующими символами в идентификаторе или ключевом слове могут быть буквы, цифры (**0-9**), знаки доллара (**\$**) или подчёркивания.

Стандарт SQL не включает использование знака доллара в идентификаторах, поэтому их использование влияет на переносимость приложений.

В стандарте SQL гарантированно не будет ключевых слов с цифрами и начинающихся или заканчивающихся подчёркиванием.

Система выделяет для идентификатора не более **NAMEDATALEN-1** байт, а более длинные имена усекаются.

По умолчанию **NAMEDATALEN** равно 64, так что максимальная длина идентификатора равна 63 байта.

Если этого недостаточно, этот предел можно увеличить, изменив константу **NAMEDATALEN** в файле **src/include/pg_config_manual.h**.

**Ключевые слова и идентификаторы без кавычек
воспринимаются системой без учёта регистра.**

```
UPDATE MY_TABLE SET A = 5;  
update my_table set a = 5;
```

Обычно используется неформальное соглашение записывать ключевые слова заглавными буквами, а имена строчными:

```
UPDATE my_table SET a = 5;
```

Выделенные идентификаторы или идентификаторы в кавычках

Обычный набор символов, заключенный в двойные кавычки (").

Такие идентификаторы всегда будут считаться идентификаторами, но не ключевыми словами.

Таким образом **"update"** можно использовать для обозначения столбца или таблицы, в то время как **upadte** без кавычек будет воспринят как ключевое слово.

```
UPDATE "my_table" SET "a" = 5;
```

Идентификаторы в кавычках могут содержать любые символы, за исключением символа '\0'.

Чтобы включить в такой идентификатор кавычки, их нужно продублировать.

Кавычки позволяют создавать таблицы и столбцы с именами, которые иначе были бы невозможны, например, с пробелами или амперсандами. Ограничение длины при этом сохраняется.

Идентификатор, заключённый в кавычки, становится зависимым от регистра.

Идентификаторы без кавычек в Postgres всегда переводятся в нижний регистр, что несовместимо со стандартом SQL, где говорится о том, что имена должны приводиться к верхнему регистру.

То есть, согласно стандарту **foo** должно быть эквивалентно **"F00"**, но не **"foo"**. Поэтому при создании переносимых приложений рекомендуется либо всегда заключать определённое имя в кавычки, либо не заключать никогда.

Вариант идентификаторов в кавычках позволяет использовать символы Unicode по их кодам.

Такой идентификатор начинается с **U&**, а затем сразу без пробелов идёт двойная кавычка, например **U&"foo"**.

В кавычках можно записывать символы Unicode двумя способами — обратная косая черта, а за ней код символа из четырёх шестнадцатеричных цифр, либо обратная косая черта, знак плюс, а затем код из шести шестнадцатеричных цифр. Например, идентификатор "data" можно записать так:

```
U&"d\0061t\+000061"
```


Константы

В PostgreSQL есть три типа констант подразумеваемых типов — строки, битовые строки и числа. Константы можно также записывать, указывая типы явно.

Строковые константы

Строковая константа в SQL — это обычная последовательность символов, заключённая в апострофы ('), например:

```
'Это строка'
```

Чтобы включить апостроф в строку, нужно использовать два апострофа рядом:

```
'Жанна д' 'Арк'
```

Две строковые константы, разделённые пробельными символами и *минимум* одним переводом строки, объединяются в одну и обрабатываются, как если бы строка была записана в одной константе:

```
SELECT 'foo'  
'bar';
```

эквивалентно:

```
SELECT 'foobar';
```

но будет синтаксической ошибкой:

```
SELECT 'foo'      'bar';
```

Строковые константы со спецпоследовательностями в стиле C

Postgres принимает «спецпоследовательности», что является расширением стандарта SQL.

Строка со спецпоследовательностями начинается с буквы **E** (заглавной или строчной), стоящей непосредственно перед апострофом:

```
E 'foo'
```

Внутри таких строк символ `'\'` начинает C-подобные спецпоследовательности:

Спецпоследовательность	Интерпретация
<code>\b</code>	символ «забой»
<code>\f</code>	подача формы
<code>\n</code>	новая строка
<code>\r</code>	возврат каретки
<code>\t</code>	табуляция
<code>\o, \oo, \ooo (o = 0–7)</code>	восьмеричное значение байта
<code>\xh, \xhh (h = 0–9, A–F)</code>	шестнадцатеричное значение байта
<code>\uxxxx, \Uxxxxxxxx (x = 0–9, A–F)</code>	16- или 32-битный шестнадцатеричный код символа Unicode

Любой другой символ, идущий после обратной косой черты, воспринимается буквально.

Таким образом, чтобы включить в строку обратную косую черту, нужно написать две косых черты (`\\`). Так же можно включить в строку апостроф, написав `\'`, в дополнение к `'`.

Строковые константы, заключённые в доллары

Стандартный синтаксис для строковых констант может плохо читаться, когда строка содержит много апострофов или обратных косых черт, поскольку каждый такой символ приходится дублировать.

Чтобы и в таких случаях запросы оставались читаемыми, Postgres предлагает способ записи строковых констант — «заключение строк в доллары».

Строковая константа, заключённая в доллары, начинается со знака доллара '\$', необязательного *тега* из нескольких символов и ещё одного знака доллара, затем содержит обычную последовательность символов, составляющую строку, и оканчивается знаком доллара, тем же тегом и замыкающим знаком доллара.

Например, строку «Жанна д'Арк» можно записать в долларах двумя способами:

```
$$Жанна д'Арк$$  
$SomeTag$Жанна д'Арк$SomeTag$
```

Битовые строковые константы

Битовые строковые константы похожи на обычные с дополнительной буквой **B** (заглавной или строчной), добавленной непосредственно перед открывающим апострофом без пробелов:

```
B'1001'.
```

В битовых строковых константах допускаются лишь символы 0 и 1.

Битовые константы могут быть записаны в шестнадцатеричном виде, с начальной буквой **X** (заглавной или строчной):

```
X'1FF'
```

Числовые константы

<цифры>

<цифры>.[<цифры>][e[+-]<цифры>]

[<цифры>].<цифры>[e[+-]<цифры>]

<цифры>e[+-]<цифры>

где <цифры> — это одна или несколько десятичных цифр (0..9).

До или после десятичной точки (при её наличии) должна быть минимум одна цифра.

Как минимум одна цифра должна следовать за обозначением экспоненты (e), если она присутствует. В числовой константе не может быть пробелов или других символов. Любой знак минус или плюс в начале строки не считается частью числа — это оператор, применённый к константе.

Примеры

42

3.5

4.

.001

5e2

1.925e-3

Числовая константа, не содержащая точки и экспоненты, изначально рассматривается как константа типа **integer**, если её значение уместается в 32-битный тип **integer**, затем как константа типа **bigint**, если её значение уместается в 64-битный **bigint**, в противном случае она принимает тип **numeric**.

Константы, содержащие десятичные точки и/или экспоненты, всегда считаются константами типа **numeric**.

Константы других типов

Константу обычного типа можно ввести одним из следующих способов:

```
type 'string'           -- REAL '123.45'  
'string'::type         -- '123.45'::REAL  
CAST ( 'string' AS type ) -- CAST ( '123.45' AS REAL )
```

Явное приведение типа можно опустить, если нужный тип константы определяется однозначно (например, когда она присваивается непосредственно столбцу таблицы), так как в этом случае приведение происходит автоматически.

Операторы

Имя оператора представляет собой последовательность не более чем **NAMEDATALEN-1** (по умолчанию 63) символов из следующего списка:

`+ - * / < > = ~ ! @ # % ^ & | ` ?`

Однако для имён операторов есть ещё несколько ограничений:

- Сочетания символов `--` и `/*` не могут присутствовать в имени оператора (начало комментария).
- Многосимвольное имя оператора не может заканчиваться знаком `+` или `-`, если только оно не содержит также один из этих символов:

`~ ! @ # % ^ & | ` ?`

@- — допустимое имя оператора

*- — недопустимое имя оператора.

Благодаря этому ограничению, Postgres может разбирать корректные SQL-запросы без пробелов между компонентами.

```
CREATE OPERATOR имя (  
    {FUNCTION|PROCEDURE} = имя_функции  
    [, LEFTARG = тип_слева ] [, RIGHTARG = тип_справа ]  
    [, COMMUTATOR = коммут_оператор ] [, NEGATOR = обратный_оператор ]  
    [, RESTRICT = процедура_ограничения ] [, JOIN = процедура_соединения ]  
    [, HASHES ] [, MERGES ]  
)
```

Пользовательские операторы

Любой оператор представляет собой просто «синтаксический сахар» для вызова нижележащей функции, которая выполняет реальную работу.

Поэтому прежде чем можно создать оператор, необходимо создать нижележащую функцию.

Однако оператор не только синтаксический сахар — он несёт и дополнительную информацию, помогающую планировщику запросов оптимизировать запросы с этим оператором.

Postgres поддерживает префиксные и инфиксные операторы.

$A + B$ — инфиксная запись (требуется скобка для смены приоритетов) $// A + B * C \rightarrow A + (B * C)$

$+ A B$ — префиксная запись $+ A * B C$

$A B +$ — постфиксная запись $B C * A +$ или $A B C * +$ (стековые компьютеры)

Операторы могут быть перегружены — то есть одно имя оператора могут иметь различные операторы с разным количеством и типами операндов.

Когда выполняется запрос, система определяет, какой оператор вызвать, по количеству и типам предоставленных операндов.

Специальные знаки

Некоторые не алфавитно-цифровые символы имеют специальное значение, но при этом не являются операторами.

- Знак доллара '\$', предваряющий число, используется для представления позиционного параметра в теле определения функции или подготовленного оператора. В других контекстах знак доллара может быть частью идентификатора или строковой константы, заключённой в доллары.

- Круглые скобки '(')' имеют обычное значение и применяются для группировки выражений и повышения приоритета операций. В некоторых случаях скобки — это необходимая часть синтаксиса определённых SQL-команд.

- Квадратные скобки '[']' применяются для выделения элементов массива.

- Запятые ',' используются в некоторых синтаксических конструкциях для разделения элементов списка.

- Точка с запятой ';' завершает команду SQL. Она не может находиться нигде внутри команды, за исключением строковых констант или идентификаторов в кавычках.

- Двоеточие ':' применяется для выборки «срезов» массивов (slice). В некоторых диалектах SQL, например, в Embedded SQL, двоеточие может быть префиксом в имени переменной.

- Звёздочка '*' используется в некоторых контекстах как обозначение всех полей строки или составного значения. Она также имеет специальное значение, когда используется как аргумент некоторых агрегатных функций, а именно функций, которым не нужны явные параметры.

- Точка '.' используется в числовых константах, а также для отделения имён схемы, таблицы и столбца.

Комментарии

Комментарий — это последовательность символов, которая начинается с двух минусов и продолжается до конца строки:

```
-- Это стандартный комментарий SQL
```

Кроме этого, можно записывать блочные комментарии в стиле C:

```
/*  
 * многострочный комментарий  
 * с вложенностью: /* вложенный блок комментария */  
*/
```

Здесь комментарий начинается с `/*` и продолжается до соответствующего вхождения `*/`.

Блочные комментарии можно вкладывать друг в друга — это разрешено по стандарту SQL и позволяет комментировать большие блоки кода, которые при этом уже могут содержать блоки комментариев.

Комментарий удаляется из входного потока в начале синтаксического анализа и фактически заменяется пробелом.

Приоритеты операторов

Большинство операторов имеют одинаковый приоритет и вычисляются слева направо.

Приоритет и очерёдность операторов жёстко фиксированы в синтаксическом анализаторе.

Если необходимо, чтобы выражение с несколькими операторами разбиралось не в том порядке, который диктуют приоритеты, следует использовать скобки.

Оператор/элемент	Очерёдность	Описание
.	слева-направо	разделитель имён таблицы и столбца
::	слева-направо	приведение типов в стиле PostgreSQL
[]	слева-направо	выбор элемента массива
+ -	справа-налево	унарный плюс, унарный минус
^	слева-направо	возведение в степень
* / %	слева-направо	умножение, деление, остаток от деления
+ -	слева-направо	сложение, вычитание
(любой другой оператор)	слева-направо	все другие встроенные и пользовательские операторы
BETWEEN IN LIKE ILIKE SIMILAR		проверка диапазона, проверка членства, сравнение строк
< > = <= >= <>		операторы сравнения
IS ISNULL NOTNULL		IS TRUE, IS FALSE, IS NULL, IS DISTINCT FROM и т. д.
NOT	справа-налево	логическое отрицание
AND	слева-направо	логическая конъюнкция
OR	слева-направо	логическая дизъюнкция

Правила приоритета операторов также применяются к операторам, определённым пользователем с теми же именами, что и вышеперечисленные встроенные операторы.

Базы данных

Лекция 05 – Реляционная модель данных

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2022

2022.10.08

Оглавление

Реляционная модель данных.....	3
Отношения и переменные отношения.....	3
Смысл отношений.....	8
Оптимизация.....	11
Каталог базы данных.....	14
Низкоуровневые операции логического уровня в таблицах.....	16

Реляционная модель данных

Отношения и переменные отношения

Реляционная база данных — это база данных, в которой данные представлены в виде таблиц.

Вопрос: почему такая БД называется именно реляционной, а не, например, табличной?

Ответ: термин «relation» (отношение) — это формальное название *определенного* вида таблиц.

relation — связь, отношение, соотношение, зависимость, ... (перевод с англ. в п.ч.в.)

Например, можно сказать, что база данных отделов и служащих, представленная ниже, содержит два отношения.

DEPT (Отделы)

DEPT#	DNAME	BUDGET
D1	Marketing	10M
D2	Development	12M
D3	Research	5M

Δ

EMP (Служащие)

EMP#	ENAME	DEPT#	SALARY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
E3	Finci	D2	30K
E4	Saito	D2	35K

EMP.DEPT# ссылается на DEPT.DEPT#

Теория реляционной модели была первоначально сформулирована Коддом (IBM) и при этом умышленно применялись лишь определенные, тщательно подобранные термины, не допускающие многозначности. Причина — многие распространенные в те времена термины были очень нечеткими. Им не хватало точности, необходимой для той формальной теории, которую предложил Кодд.

Например, термин «*запись*». В разное время и в различных контекстах он мог означать экземпляр записи или тип записи, логическую запись или физическую запись, хранимую запись или виртуальную запись, ...

В настоящее время в неформальном контексте термины «отношение» и «таблица» принято считать синонимами. На практике в подобном контексте термин таблица используется гораздо чаще, чем термин отношение.

Дейт в своей книге « Введение в системы баз данных» объясняет, почему термин «отношение» поставлен на первое место.

- реляционные системы основаны на реляционной модели. Реляционная модель — это абстрактная теория данных, основанная на таких областях математики, как теория множеств и логика предикатов (исчисление высказываний). Поэтому в формальной реляционной модели термин «запись» не используется вообще — вместо него применяется термин «кортеж» (tuple), точное определение которого дал сам Кодд, когда ввел его впервые.

Кортеж.

Если дана коллекция типов $T_i (i = 1, 2, \dots, n)$, которые не обязательно все должны быть разными, то значением кортежа (или кратко кортежем), определенным с помощью этих типов (назовем его t), является множество упорядоченных троек в форме $\langle A_i, T_i, v_i \rangle$, где A_i — имя атрибута, T_i — имя типа и v_i — значение типа T_i .

Кроме того, кортеж t должен соответствовать приведенным ниже требованиям:

- Значение n является **степенью**, или **арностью** кортежа t .
- Упорядоченная тройка $\langle A_i, T_i, v_i \rangle$ является компонентом (частью) t .
- Упорядоченная пара $\langle A_i, T_i \rangle$ представляет собой **атрибут** t и однозначно определяется именем атрибута A_i (имена атрибутов A_i и A_j совпадают, только если $i = j$).
- Значение v_i — это **значение атрибута**, соответствующее атрибуту A_i кортежа t .
- Тип T_i — это соответствующий **тип атрибута**.
- Полное множество атрибутов составляет **заголовок кортежа** t .
- Тип кортежа t определен заголовком t , а сам заголовок и этот тип кортежа имеют такие же атрибуты (и поэтому такие же имена и типы атрибутов) и такую же степень, как и кортеж t .

Точное определение имени типа кортежа:

TUPLE {A1 T1, A2 T2, ..., An Tn}

Пример кортежа:

MAJOR_PNO : PARTNO	MINOR_PNO : PARTNO	QTY : QUANTITY
P2	P4	7

Имена атрибутов: MAJOR_PNO, MINOR_PNO, QTY

Имена типов: PARTNO, QUANTITY

Значения: PARTNO{'P2'}, PARTNO{'P4'}, QUANTITY{7}

TUPLE {MAJOR_PNO PARTNO, MINOR_PNO PARTNO, QTY QUANTITY}

В неформальном изложении принято исключать имена типов из заголовка кортежа и показывать только имена атрибутов. Поэтому показанный выше кортеж может быть неформально представлен:

MAJOR_PNO	MINOR_PNO	QTY
P2	P4	7

Термин «кортеж» приблизительно соответствует понятию строки, так же, как термин «отношение» приблизительно соответствует понятию таблицы.

В реляционной модели не используется термин «поле», вместо него используется термин «атрибут», который в рассматриваемом контексте примерно соответствует понятию столбца таблицы.

Снова обратимся к БД отделов и сотрудников:

DEPT (Отделы)

DEPT#	DNAME	BUDGET
D1	Marketing	10M
D2	Development	12M
D3	Research	5M

EMP (Служащие)

EMP#	ENAME	DEPT#	SALARY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
E3	Finci	D2	30K
E4	Saito	D2	35K

Δ

EMP.DEPT# ссылается на DEPT. DEPT#

Таблицы **DEPT** и **EMP** в базе данных фактически являются *переменными отношениями*.

Соответственно, как любые переменные, имеют значения. Их значения — *значения отношения*, которые они принимают в разное время.

Предположим, например, что таблица **EMP** в данный момент имеет значение (значение отношения), которое показано выше, и допустим, что мы удалили строку о сотруднике с фамилией Saito.

DELETE EMP WHERE EMP# = EMP# ('E4') ;

Результат выполнения этой операции:

EMP (Служащие)

EMP#	ENAME	DEPT#	SALARY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
E3	Finci	D2	30K

Концептуально это действие можно описать таким образом — **старое значение отношения EMP было заменено в целом совершенно другим, новым значением отношения.**

Старое значение с четырьмя строками и новое значение с тремя очень похожи, но в действительности они являются разными.

Можно считать, что данная операция удаления строки — это просто альтернативный, упрощенный способ записи для определенной операции реляционного присваивания, которая могла бы выглядеть примерно следующим образом¹.

```
EMP := EMP WHERE NOT (EMP# = EMP#( 'E4' ));
```

Сначала вычисляется выражение, расположенное справа от знака присваивания, а затем его значение присваивается переменной, которая записана перед знаком присваивания.

В целом задача заключается в том, чтобы заменить «старое» значение отношения EMP «новым».

Таким же образом операции INSERT и UPDATE также являются просто сокращенной формой записи соответствующих реляционных операций присваивания.

В реляционной теории (РТ) следует четко различать *переменные отношения* и сами *отношения*.

Для переменной отношения (*relation variable*) иногда используется термин *relvar*².

Термин «*переменная отношения*» (*relvar*) не является общепринятым и в документации на БД практически не встречается, хотя различать сами отношения, т.е. значения отношений, и переменные отношения как таковые, с точки зрения РТ очень важно. Например, ограничения целостности и операции обновления, применяются к переменным отношения, а не к отношениям.

1) Оператор DELETE и равносильный ему оператор присваивания записаны на языке Tutorial D

2) Различие между значениями отношения и переменными отношения фактически представляют собой особый случай различия между значениями и переменными в целом.

Смысл отношений

Столбцы в отношениях связаны с типами данных. Реляционная модель допускает неограниченный набор типов [данных]. Это означает, что пользователи могут как применять определяемые системой или встроенные типы, так и определять собственные. Например, определять типы можно так («...» здесь заменяет сами определения, которые на сей момент не важны).

```
TYPE EMP_NO ... ;  
TYPE NAME ... ;  
TYPE DPT_NO ... ;  
TYPE MONEY ... ;
```

Тип **EMP_NO**, например, можно рассматривать, как множество всевозможных табельных номеров, тип **NAME** — как множество всевозможных имен и т.д.

Каждое отношение (каждое значение отношения) состоит из двух частей:

- 1) набор пар (<имя_столбца> : <имя_типа>) — заголовок;
- 2) набор строк, согласованных с этим заголовком — тело.

Пример — часть отношения EMP, дополненного обозначениями типов столбцов.

EMP (Служащие)

EMP# : EMP_NO	ENAME : NAME	DEPT# : DEPT_NO	SALARY : MONEY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
E3	Finci	D2	30K
E4	Saito	D2	35K

Хотя на практике компоненты заголовка <имя_типа> обычно опускают, необходимо учитывать, что концептуально они всегда присутствуют.

Отношения можно представлять также следующим образом:

- 1) В определенном отношении R его заголовок представляет собой некоторый **предикат** (функция, возвращающая значения истинности и принимающая ряд формальных параметров).
- 2) Каждая строка в теле отношения R представляет собой определенное **истинное высказывание**, полученное из предиката путем подстановки определенных значений фактических параметров соответствующего типа вместо формальных параметров этого предиката (путем конкретизации).

EMP (Служащие)

EMP# : EMP_NO	ENAME : NAME	DEPT# : DEPT_NO	SALARY : MONEY
E1	Lopez	D1	40K
E2	Cheng	D1	42K
...	...	...	...

Для отношения EMP предикат будет следующим:

«Служащий с табельным номером **EMP#** и фамилией **ENAME** работает в отделе с номером **DEPT#** и получает зарплату **SALARY**».

Здесь формальные параметры – EMP#, ENAME, DEPT# И SALARY. Они соответствуют четырем столбцам переменной отношения **EMP**.

Соответствующие истинные утверждения:

«Служащий с табельным номером E1 и фамилией Lopez работает в отделе с номером D1 и получает зарплату 40 тыс. долл. в год.»

«Служащий с номером E2 и фамилией Cheng работает в отделе с номером D1 и получает зарплату 42 тыс. долл. в год.»

Иными словами, **типы** — это объекты (множества объектов), которые могут стать предметом обсуждения;

отношения — это факты (множества фактов), касающиеся объектов, которые могут стать предметом обсуждения.

Типы связаны с отношениями точно так же, как существительные связаны с предложениями на естественном языке.

В примере, приведенном выше, предметом обсуждения являются:

- табельные номера служащих EMP_NO;
- имена NAME;
- номера отделов DEPT_NO;
- значения денежных сумм MONEY.

Об обсуждаемом же предмете можно привести истинное высказывание следующего вида:

«Служащий с определенным табельным номером имеет определенное имя, работает в определенном отделе и получает определенную зарплату».

Оптимизация

Все реляционные операции, такие как сокращение, проекция и соединение, выполняются на уровне множеств. Поэтому реляционные языки часто называют **непроцедурными**, а **декларативными**, так как пользователь указывает, **что** делать, а не **как** делать.

Фактически пользователь сообщает лишь, что ему нужно, без указания процедуры получения результата.

Процесс навигации (перемещения) по хранимой базе данных в целях удовлетворения запроса пользователя выполняется системой автоматически, а не пользователем вручную, поэтому реляционные системы иногда называют **системами автоматической навигации**.

В **нереляционных** системах за навигацию по базе данных в основном несет ответственность сам пользователь.

Следует отметить, что **непроцедурный** — это хотя и общепринятый, но не совсем точный термин, потому что **процедурный** и **непроцедурный** — понятия относительные.

Обычно можно с уверенностью определить лишь то, является ли язык А более (или менее) процедурным, чем язык Б.

Поэтому точнее будет сказать, что реляционные языки, такие как SQL, характеризуются более высоким уровнем абстракции, чем типичные языки программирования, подобные C++ или COBOL.

В принципе, именно более высокий уровень абстракции способствует повышению продуктивности труда программистов, свойственному для реляционных систем.

Ответственность за организацию выполнения автоматической навигации возложена на очень важный компонент СУБД — **оптимизатор**. Его работа заключается в том, чтобы выбрать самый эффективный способ выполнения для каждого запроса пользователя.

Например, пользователь сделал следующий запрос (язык Tutorial D).

(EMP WHERE EMP# = EMP#('E4')){SALARY} (1)

Выражение **(EMP WHERE ...)** означает операцию сокращения текущего значения переменной отношения **EMP**, касающаяся той строки, в которой значение столбца **EMP#** равно **E4**.

R{SALARY} означает проекцию отношения **R** по столбцу **SALARY**.

В данном случае используется реляционное свойство замкнутости — выражение (1) является вложенным, в котором результат **R** операции сокращения **EMP** используется в качестве входных данных для операции проекции. Результатом (1) становится отношение, состоящее из одного столбца и одной строки, которое содержит данные о зарплате служащего **E4**.

В этом примере могут применяться, как минимум, два способа доступа к необходимым данным:

Наивный — последовательный физический просмотр (хранимой версии) переменной отношения **EMP**, пока требуемая запись не будет найдена.

Индексный — если есть индекс для столбца **EMP#** переменной отношения **EMP**³, то переход с помощью этого индекса к данным служащего с номером **E4** будет прямым.

Оптимизатор определит, какую из двух возможных стратегий следует применить. В общем случае для реализации любого конкретного реляционного запроса оптимизатор будет осуществлять выбор стратегии, исходя из соображений, подобных следующим:

- на какие переменные отношения есть ссылки в запросе;
- насколько велики эти переменные отношения в настоящее время;
- какие существуют индексы;
- насколько избирательны эти индексы;
- как физически группируются данные на диске;
- какие реляционные операции используются; и т.д.

3) этот индекс скорее всего существует, поскольку столбец является уникальным, а большинство систем поддерживает индекс для обеспечения уникальности.

Резюме

Пользователь указывает в запросе, какие данные ему требуются, а не как их получить, а стратегия доступа к данным выбирается оптимизатором (автоматическая навигация).

В результате пользователи и пользовательские программы становятся независимыми от применяемых стратегий доступа и от физического представления данных.

Каталог базы данных

Каждая СУБД должна поддерживать функции каталога, или словаря.

Каталог обычно размещается там, где хранятся различные схемы (внешние, концептуальные, внутренние) и все, что относится к отображениям («внешний-концептуальный», «концептуальный-внутренний», «внешний-внешний»).

В каталоге содержится подробная информация, касающаяся различных объектов, имеющих значение для самой системы (описательная информация или метаданные):

- переменные отношения;
- индексы;
- ограничения для поддержки целостности;
- ограничения защиты;
- и пр.

Эта информация необходима для обеспечения правильной работы системы.

Например, оптимизатор использует информацию каталога об индексах и других физических структурах хранения данных, а также прочую информацию, необходимую ему для принятия решения о том, как выполнить тот или иной запрос пользователя.

Подсистема защиты использует информацию каталога о пользователях и установленных ограничениях защиты, чтобы разрешить или запретить выполнение поступившего запроса.

Свойством реляционных систем является то, что их каталог также состоит из переменных отношений, таких же как и пользовательские.

Чтобы отличать их от пользовательских их называют системными переменными отношения.

В результате пользователь может обращаться к каталогу так же, как к своим данным.

Например, в каталоге любой системы SQL обычно содержатся системные переменные отношения **TABLES** и **COLUMNS**, назначение которых — описание известных системе таблиц, т.е. переменных отношений, и столбцов этих таблиц.

Каталог базы данных отделов и служащих может быть схематически представлен в **TABLES** и **COLUMNS** в следующем виде:

TABLE

TAB_NAME	COLS	ROWS	
DEPT	3	3	
EMP	4	4	
TABLE	3	123	

COLUMNS

TAB_NAME	COL_NAME	COL_TYPE	
DEPT	DEPT#	DPT_NO	
DEPT	DNAME	NAME	
DEPT	BUDGET	MONEY	
EMP	EMP#	EMP_NO	
EMP	ENAME	NAME	
EMP	DEPT#	DPT_NO	
EMP	SALARY	MONEY	
TABLE	TAB_NAME	TAB_NAME_T	
TABLE	COLS	COLS_T	
TABLE	ROWS	ROWS_T	

Каталог также должен описывать сам себя, т.е. включать записи, описывающие переменные отношения самого каталога.

Какие столбцы содержит переменная отношения **DEPT**⁴:

(COLUMNS WHERE TABNAME = 'DEPT'){COLNAME}

В каких переменных отношения есть столбец **EMP#**.

(COLUMNS WHERE COLNAME = 'EMP#'){TABNAME}

4) предполагается, что по каким-то причинам пользователь не имеет этой информации

Низкоуровневые операции логического уровня в таблицах

TABLE LIST:LST (ID, NAME, DATE, ...)

TABLE ITEM:ITM (ID, LST_ID, PART_NO, NAME, RPROVIDER, QUANT, ...)

PRIMKEY LST:BY_ID (ID) # LST:ID

PRIMKEY ITM:BY_ID (ID) # ITM:ID

INDEX LST:BY_NAME (NAME)

INDEX LST:BY_DATE (DATE)

INDEX ITM:BY_NAME (NAME)

INDEX ITM:BY_NAME_INLIST (LST_ID, NAME)

INDEX ITM:BY_PNO_INLIST (LST_ID, PART_NO) # кандидат в PrimKey (?поставщик?)

Простейшие сериальные операции

SET(TABLE) – установка курсора в «сыром» порядке записей

NEXT(TABLE) – просмотр таблицы в направлении с начала к концу

PREV(TABLE) – просмотр таблицы в направлении с конца к началу

INSERT(RECORD, TABLE) – вставка в таблицу без обновления индексов

Прямой доступ по первичному ключу

GET(RECORD, TABLE, KEY) – получить строку (запись) из таблицы по значению перв. ключа

PUT(RECORD, TABLE, KEY) – обновить строку (запись) в таблице по значению перв. ключа

Добавление/удаление

ADD(RECORD, TABLE) – добавить строку (запись) в таблицу и обновить все индексы

DEL(RECORD, TABLE, KEY) – удалить строку (запись) из таблицы и обновить все индексы

Прямой доступ по ключу

GET(RECORD, TABLE, KEY, KEY_VALUE) — получить строку по значению ключа

PUT(RECORD, TABLE, KEY, KEY_VALUE) — обновить строку по значению ключа

ADD(RECORD, TABLE) — добавить строку (запись) в таблицу и обновить все индексы

DEL(RECORD, TABLE, KEY) — удалить строку (запись) из таблицы и обновить все индексы

Сериальные операции в порядке индексов

SET(TABLE, KEY, KEY_VALUE) — установить курсор доступа в порядке индекса

NEXT(TABLE, KEY) — просмотр таблицы в направлении с начала к концу

PREV(TABLE, KEY) — просмотр таблицы в направлении с конца к началу

Базы данных

Лекция 06 – Основы SQL

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

Оглавление

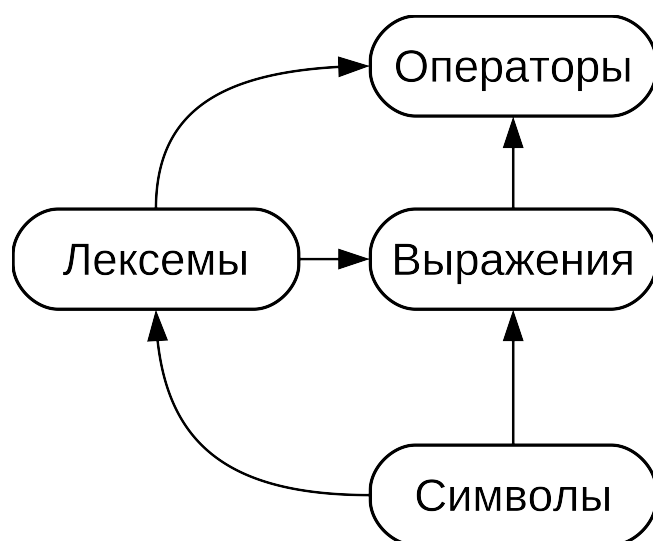
Язык SQL.....	3
Выражения.....	3
Константа или непосредственное значение.....	5
Ссылка на столбец.....	5
Позиционные параметры.....	5
Выражение с индексом.....	6
Выражение выбора поля.....	7
Применение оператора.....	8
Вызов функции.....	9
Агрегатное выражение.....	13
Приведение типов.....	14
Применение правил сортировки.....	16
Скалярный подзапрос.....	20
Конструктор массива.....	21
Конструктор табличной строки.....	24
Правила вычисления выражений.....	25
Типы.....	27
Числовые типы.....	28
Денежные типы.....	34
LC_NUMERIC & LC_MONETARY.....	36
Символьные типы.....	38
Двоичные типы данных.....	42

Язык SQL

Выражения

Состав языка программирования

Естественный язык	Алгоритмический язык
Предложения Словосочетания Слова Символы	Операторы Выражения Лексемы Символы



Алфавит языка – набор неделимых знаков (символов), с помощью которых пишутся все тексты на языке.

Лексема – элементарная конструкция, минимальная единица языка, имеющая самостоятельный смысл.

Выражение задает правила **вычисления некоторого значения**.

Оператор задает законченное описание **некоторого действия**.

В SQL используется два вида выражений:

- выражения величины (value expressions);
- табличные выражения.

Результатом табличного выражения является таблица, а результатом выражения величины (value) является некоторое значение, которое иногда называют скаляром. Выражения величины часто называют скалярными или просто выражениями. Синтаксис таких выражений позволяет вычислять значения из примитивных частей, используя арифметические, логические и другие операции.

Выражения величины применяются в SQL самых разных контекстах, например в списке результатов команды **SELECT**, в значениях столбцов в **INSERT** или **UPDATE**, в условиях поиска во многих командах.

Выражениями величины являются:

- константа или непосредственное значение;
- ссылка на столбец;
- ссылка на позиционный параметр;
- выражение с индексом;
- выражение выбора поля;
- применение оператора;
- вызов функции;
- агрегатное выражение;
- вызов оконной функции;
- приведение типов;
- применение правил сортировки;
- скалярный подзапрос;
- конструктор массива;
- конструктор табличной строки.

Кроме того, выражением значения являются скобки, которые используются для группировки подвыражений и переопределения приоритета операторов.

Также существует ещё несколько конструкций, которые можно классифицировать как выражения, хотя они не соответствуют общим синтаксическим правилам. Они обычно имеют вид функции или оператора. Пример такой конструкции — предложение **IS NULL**.

Строки, битовые строки и числа (см. LK05)

Ссылку на столбец можно записать в форме:

tablename — имя таблицы или её псевдоним (определённый в предложении **FROM**).

Позиционные параметры

Ссылка на позиционный параметр может появляться в теле определения функции или оператора.

Ссылка на позиционный параметр применяется для обращения к значению, которое передается в SQL-оператор извне.

Параметры используются в определениях SQL-функций и операторов, составляемых пользователем. Некоторые клиентские библиотеки также поддерживают передачу значений данных отдельно от самой SQL-команды, и в этом случае параметры позволяют ссылаться на такие значения. Ссылка на параметр записывается в следующей форме:

\$number

Например, определение некоторой функции **dept**:

```
CREATE FUNCTION dept(text) RETURNS dept
AS $$ SELECT * FROM dept WHERE name = $1 $$ -- $1 ссылается на значение
LANGUAGE SQL; -- первого аргумента функции
```


Выражение с индексом

```
CREATE TABLE sal_emp (  
    name          text,  
    pay_by_quarter integer[], -- квартальная зарплата  
    schedule      text[][]   -- недельный график  
);
```

Если результат выражения является массивом, то можно извлечь определённый его элемент:

expression [subscript]

или несколько соседних элементов («срез массива»):

expression [lower_subscript : upper_subscript]

Каждый индекс сам по себе является выражением, результат которого округляется к ближайшему целому.

В общем случае выражение, результатом которого является массив, должно заключаться в круглые скобки, но их можно опустить, если выражение массива используется с индексом — это просто ссылка на столбец или позиционный параметр.

Если исходный массив многомерный, можно соединить несколько индексов.

```
mytable.arraycolumn[4]  
mytable.two_d_column[17][34]  
$1[10:42]                -- позиционный параметр  
(arrayfunction(a,b))[42] -- функция, возвращающая массив
```

В последней строке круглые скобки необходимы.

Выражение выбора поля

Если результат выражения — значение составного типа, например, строка таблицы, то определённое поле этой строки можно извлечь следующим образом:

expression . fieldname

В общем случае выражение такого типа должно заключаться в круглые скобки, но их можно опустить, если выражение — это ссылка на таблицу или позиционный параметр:

```
anytable.anycolumn      -- ссылка на столбец в таблицу
$1.somecolumn           -- позиционный параметр
(rowfunction(a,b)).col3  -- функция, возвращающая строку
```

Полная ссылка на столбец — это частный случай выбора поля.

Важный особый случай — извлечение поля из столбца составного типа:

```
(compositecol).somefield
(mytable.compositecol).somefield
```

Скобки требуются, чтобы показать, что **compositecol** — это имя столбца, а не таблицы, и что **mytable** — имя таблицы, а не схемы.

Все поля составного столбца запрашиваются с помощью *:

(compositecol) . *

Применение оператора

Существуют два возможных синтаксиса применения операторов:

expression operator expression -- бинарный инфиксный оператор
operator expression -- унарный префиксный оператор

где **оператор** соответствует синтаксическим правилам, описанным в LK05, либо это одно из ключевых слов **AND**, **OR** и **NOT**, либо полное имя оператора в форме:

OPERATOR (*schema . operatorname*)

Существуют встроенные операторы и операторы, определенные системой и пользователем. Унарный или бинарный некоторый конкретный оператор, зависит от типа оператора.

Вызов функции

Вызов функции записывается просто как имя функции (возможно, дополненное именем схемы) и список аргументов в скобках:

function_name ([*expression* [, *expression* ...]])

Например, вычисление квадратного корня из двух:

sqrt(2)

Помимо встроенных функций пользователь может определить и другие функции. Аргументам при этом могут быть присвоены необязательные имена.

```
CREATE FUNCTION concat_lower_or_upper(a text,  
                                     b text,  
                                     uppercase boolean DEFAULT false)  
RETURNS text AS $$  
    SELECT CASE  
        WHEN $3 THEN UPPER($1 || ' ' || $2)  
        ELSE LOWER($1 || ' ' || $2)  
    END;  
$$  
LANGUAGE SQL IMMUTABLE STRICT;
```

Позиционная передача параметров – аргументы указываются в заданном порядке

```
CREATE FUNCTION concat_lower_or_upper(a text,  
                                     b text,  
                                     uppercase boolean DEFAULT false)
```

Вызовы

```
SELECT concat_lower_or_upper('Hello', 'World', true);  
concat_lower_or_upper  
-----  
HELLO WORLD
```

```
SELECT concat_lower_or_upper('Hello', 'World'); -- $3 опущен и исп. знач. по-умолчан.  
concat_lower_or_upper  
-----  
HELLO WORLD
```

В позиционной записи любые аргументы с определённым значением по умолчанию можно опускать справа налево.

Именная передача — для указания аргумента используется имя, которое отделяется от выражения значения символами `=>`:

```
SELECT concat_lower_or_upper(a => 'Hello', b => 'World'); -- uppercase опущен
concat_lower_or_upper
-----
hello world
```

Преимуществом такой записи является возможность записывать аргументы в любом порядке:

```
SELECT concat_lower_or_upper(a => 'Hello', uppercase => true, b => 'World');
concat_lower_or_upper
-----
HELLO WORLD
```

Для обратной совместимости поддерживается и старый синтаксис с «:=»:

```
SELECT concat_lower_or_upper(a := 'Hello', uppercase := true, b := 'World');
concat_lower_or_upper
-----
HELLO WORLD
```

Смешанная передача — параметры передаются и по именам, и позиционно. При этом именованные аргументы не могут стоять перед позиционными:

```
SELECT concat_lower_or_upper('Hello', 'World', uppercase => true);
concat_lower_or_upper
-----
HELLO WORLD
```

Аргументы **a** и **b** передаются позиционно, а **uppercase** — по имени.

Q: Зачем?

A1: Вызов стал чуть более читаемым.

A2: Для сложных функций с множеством аргументов, часть из которых имеют значения по умолчанию, именная или смешанная передача позволяет записать вызов эффективнее и уменьшить вероятность ошибок.

Агрегатное выражение

Агрегатное выражение представляет собой применение агрегатной функции к строкам, которые выбраны запросом.

Агрегатная функция сводит множество входных значений к одному выходному, например, к сумме или среднему.

Обычно строки данных передаются агрегатной функции в неопределённом порядке и во многих случаях это не имеет значения, например функция **min** выдаёт один и тот же результат независимо от порядка поступающих данных.

Есть некоторые агрегатные функции (**array_agg**, **string_agg**), которые выдают результаты, зависящие от порядка данных. Для таких агрегатных функций в список аргументов можно добавить предложение **ORDER BY**, чтобы задать нужный порядок.

Приведение типов

Приведение типа определяет преобразование данных из одного типа в другой. Postgres воспринимает две эквивалентные формы приведения типов:

CAST (*expression* AS *type*)
expression* :: *type

Запись с **CAST** соответствует стандарту SQL.
Вариант с **::** — историческое наследие PostgreSQL.

Когда приведению подвергается значение выражения известного типа, происходит преобразование типа во время выполнения. Это приведение будет успешным, только если определён подходящий оператор преобразования типов.

Явное приведение типа можно опустить, если возможно однозначно определить, какой тип должно иметь выражение, например, когда оно присваивается столбцу таблицы. В таких случаях система преобразует тип автоматически.

Однако автоматическое преобразование выполняется только для приведений с пометкой «OK to apply implicitly» в системных каталогах. Все остальные приведения должны записываться явно. Это ограничение позволяет избежать сюрпризов с неявным преобразованием.

Приведение типа можно записать как вызов функции:

***typename* (*expression*)**

Однако это будет работать только для типов, имена которых являются также допустимыми именами функций. Например, **double precision** нельзя использовать таким образом, а **float8** (альтернативное название того же типа) — можно.

При использовании любой из форм записи внутренне происходит вызов зарегистрированной функции (**CREATE CAST**), выполняющей реальное преобразование. Именем такой функции преобразования является имя выходного типа, и таким образом функциональная форма есть не что иное, как прямой вызов нижележащей функции преобразования.

При создании переносимого приложения на это поведение, конечно, не следует рассчитывать.

Запись приведения типа в функциональной форме лучше не применять, поскольку это может вызвать разного рода несоответствия и, как следствие, трудно обнаруживаемые проблемы.

Применение правил сортировки

Предложение **COLLATE** переопределяет правило сортировки выражения. Оно добавляется после выражения:

expr COLLATE collation

где **collation** — идентификатор правила, возможно дополненный именем схемы.

Предложение **COLLATE** «связывает» выражение сильнее, чем операторы, поэтому при необходимости следует использовать скобки.

Если правило сортировки явно не определено, система либо выбирает в зависимости от столбцов, которые используются в выражении, либо, если в выражении столбцов нет, переключается на установленное для базы данных правило сортировки по умолчанию.

Предложение **COLLATE** имеет два распространённых применения:

1) переопределение порядка сортировки в предложении **ORDER BY**, например:

```
SELECT a, b, c FROM tbl WHERE ... ORDER BY a COLLATE "C";
```

2) переопределение правил сортировки при вызове функций или операторов, возвращающих зависимости от локали результаты, например:

```
SELECT * FROM tbl WHERE a > 'foo' COLLATE "C";
```

В последнем случае предложение **COLLATE** добавлено к аргументу оператора, на действие которого нужно повлиять. Не имеет значения, к какому именно аргументу оператора или функции добавляется **COLLATE**, поскольку правило сортировки, применяемое к оператору или функции, выбирается при анализе всех аргументов, а явное предложение **COLLATE** переопределяет правила сортировки.

```
SELECT * FROM tbl WHERE a COLLATE "C" > 'foo';
```

Но будет ошибкой:

```
SELECT * FROM tbl WHERE (a > 'foo') COLLATE "C";
```

поскольку правило сортировки нельзя применить к результату оператора `>`, который имеет несравнимый тип данных **boolean**.

Локаль

Локаль — это сочетание языковых и культурных аспектов. Они включают в себя:

- язык сообщений;
- различные наборы символов;
- лексикографические соглашения;
- форматирование чисел;
- и прочее.

В локали существуют различные категории информации, которые программа может использовать — они описаны как макросы.

LC_ALL	— локаль целиком
LC_COLLATE	— сортировку строк
LC_CTYPE	— классы символов
LC_MESSAGES	— локализованные сообщения на родном языке
LC_MONETARY	— форматирование значений денежных единиц
LC_NUMERIC	— форматирование не денежных числовых значений
LC_TIME	— форматирование значений дат и времени

LC_COLLATE — эта категория определяет правила сравнения, используемые при сортировке и регулярных выражениях, включая равенство классов символов и сравнение многосимвольных элементов.

Эта категория локали изменяет поведение функций, которые используются для сравнения строк с учётом местного алфавита. Например, немецкая **ß** эсцет (sharp s) рассматривается как «ss».

```
strcoll("Das Große Eszett", "Das Grosse Eszett"); //
```

LC_CTYPE — эта категория определяет интерпретацию последовательности байт в символы (например, одиночный или многобайтовый символ), классификацию символов (например, буквенный или цифровой) и поведение классов символов.

LC_MONETARY — эта категория определяет форматирования, используемое для денежных значений. Она изменяет информацию, возвращаемую функцией, которая описывает способ отображения числа, например, необходимо ли использовать в качестве десятичного разделителя точку или запятую.

LC_MESSAGES — эта категория изменяет язык отображаемых сообщений и указывает, как должны выглядеть положительный и отрицательный ответы.

LC_NUMERIC — эта категория определяет правила форматирования, используемые для не денежных значений, например, разделительный символ тысяч и дробной части (точка в англоязычных странах, и запятая во многих других).

LC_TIME — эта категория управляет форматированием значений даты и времени. Например, большая часть Европы использует 24-часовой формат, тогда как в США используют 12-часовой.

LC_ALL — всё вышеперечисленное.

Список поддерживаемых категорий можно получить с помощью утилиты оболочки **locale**

```
$ locale
LANG=ru_RU.utf8
LC_CTYPE="ru_RU.utf8"
LC_NUMERIC="ru_RU.utf8"
LC_TIME="ru_RU.utf8"
LC_COLLATE="ru_RU.utf8"
LC_MONETARY="ru_RU.utf8"
LC_MESSAGES="ru_RU.utf8"
LC_PAPER="ru_RU.utf8"
LC_NAME="ru_RU.utf8"
LC_ADDRESS="ru_RU.utf8"
LC_TELEPHONE="ru_RU.utf8"
LC_MEASUREMENT="ru_RU.utf8"
LC_IDENTIFICATION="ru_RU.utf8"
LC_ALL=
```

Скалярный подзапрос

Скалярный подзапрос — это обычный запрос **SELECT** в скобках, который возвращает ровно одну строку и один столбец. Результат такого запроса **SELECT** используется в окружающем его выражении.

В качестве скалярного подзапроса нельзя использовать запросы, возвращающие более одной строки или столбца.

Если в результате выполнения подзапрос не вернёт никаких строк, скалярный результат считается равным **NULL**.

В подзапросе можно ссылаться на переменные из окружающего запроса — в процессе одного вычисления подзапроса они будут считаться константами.

Например, следующий запрос находит самый населённый город в каждом штате:

```
SELECT name, (SELECT max(pop) FROM cities WHERE cities.state = states.name) FROM states;
```

Конструктор массива

Конструктор массива — это выражение, которое создаёт массив, определяя значения его элементов. Конструктор простого массива состоит из ключевого слова **ARRAY**, открывающей квадратной скобки **[**, списка выражений (разделённых запятыми), задающих значения элементов массива, и закрывающей квадратной скобки **]**. Например:

```
SELECT ARRAY[ 1, 2, 3 + 4 ];  
      array  
-----  
    {1,2,7}
```

По умолчанию типом элементов массива считается общий тип для всех выражений, определённый по правилам, действующим и для конструкций **UNION** и **CASE**.

Можно переопределить тип по умолчанию, указав явно в конструкторе требуемый тип:

```
SELECT ARRAY[1,2,22.7]::integer[];  
      array  
-----  
    {1,2,23}
```

Это эквивалентно тому, как привести к нужному типу каждое выражение по отдельности.

Многомерные массивы – их можно создавать, вкладывая конструкторы друг в друга. При этом во внутренних конструкторах слово **ARRAY** можно опускать:

```
SELECT ARRAY[ARRAY[1,2], ARRAY[3,4]];
      array
-----
{{1,2},{3,4}}
```

эквивалентно:

```
SELECT ARRAY[[1,2],[3,4]];
      array
-----
{{1,2},{3,4}}
```

Многомерные массивы должны быть прямоугольными, и поэтому внутренние конструкторы одного уровня должны создавать вложенные массивы одинаковой размерности.

Любое приведение типа, применённое к внешнему конструктору **ARRAY**, автоматически распространяется на все внутренние.

Элементы многомерного массива можно создавать не только вложенными конструкторами **ARRAY**, но и другими способами, позволяющими получить массивы нужного типа:

```
CREATE TABLE arr(f1 int[], f2 int[]);
INSERT INTO arr VALUES (ARRAY[[1,2],[3,4]], ARRAY[[5,6],[7,8]]);
SELECT ARRAY[f1, f2, '{{9,10},{11,12}}'::int[]] FROM arr;
      array
-----
{{{1,2},{3,4}},{5,6},{7,8}},{9,10},{11,12}}
```

Можно создать и пустой массив, но так как массив не может быть не типизированным, необходимо явно привести пустой массив к нужному типу:

```
SELECT ARRAY[]::integer[];  
array  
-----  
{}
```

Также возможно создать массив из результатов подзапроса. В этом случае конструктор массива записывается так же с ключевым словом **ARRAY**, за которым в круглых скобках следует подзапрос:

```
SELECT ARRAY(SELECT oid FROM pg_proc WHERE proname LIKE 'bytea%');  
array  
-----  
{2011,1954,1948,1952,1951,1244,1950,2005,1949,1953,2006,31,2412}
```

```
SELECT ARRAY(SELECT ARRAY[i, i*2] FROM generate_series(1,5) AS a(i));  
array  
-----  
{{1,2},{2,4},{3,6},{4,8},{5,10}}
```

Индексы массива, созданного конструктором **ARRAY**, всегда начинаются с 1.

Конструктор табличной строки

Конструктор строки таблицы — это выражение, создающее строку (кортеж), также называемую составным значением (composite value) из значений ее полей.

Конструктор строки состоит из ключевого слова **ROW**, открывающей круглой скобки, нуля или нескольких выражений (разделённых запятыми), определяющих значения полей, и закрывающей скобки:

```
SELECT ROW ( 1, 2.5, 'this is a test' ) ;
```

Если в списке более одного выражения, ключевое слово **ROW** можно опустить.

Конструктор строки поддерживает синтаксис **rowvalue.***, при этом данное выражение будет возвращено в список элементов.

Например, если таблица **t** содержит столбцы **f1** и **f2**, следующие операторы эквивалентны:

```
SELECT ROW(t.*, 42) FROM t;  
SELECT ROW(t.f1, t.f2, 42) FROM t;
```

Используя конструктор строк (кортежей), можно создавать составное значение для сохранения в столбце составного типа или для передачи аргумента в функцию, принимающую составной параметр.

Можно сравнивать два составных значения или проверить их с помощью **IS NULL** или **IS NOT NULL**, например:

```
SELECT ROW(1,2.5,'this is a test') = ROW(1, 3, 'not the same');  
  
-- выбрать все строки, содержащие только NULL  
SELECT ROW(table.*) IS NULL FROM table;
```

Правила вычисления выражений

Порядок вычисления подвыражений не определён. В частности, аргументы оператора или функции не обязательно вычисляются слева направо или в любом другом фиксированном порядке.

Более того, если результат выражения можно получить, вычисляя только некоторые его части, тогда другие подвыражения не будут вычисляться вовсе. Например, если написать:

```
SELECT true OR somefunc( );
```

функция **somefunc()** скорее всего не будет вызываться. То же самое справедливо для записи:

```
SELECT somefunc( ) OR true;
```

Поэтому в сложных выражениях не стоит использовать функции с побочными эффектами.

Особенно опасно рассчитывать на порядок вычисления или побочные эффекты в предложениях **WHERE** и **HAVING**, так как эти предложения тщательно оптимизируются при построении плана выполнения.

Логические выражения (сочетания **AND/OR/NOT**) в этих предложениях могут быть видоизменены любым способом, допустимым законами Булевой алгебры.

Когда порядок вычисления важен, его можно зафиксировать с помощью конструкции **CASE**. Например, можно запросто получить деление на ноль в предложении **WHERE**:

```
SELECT ... WHERE x > 0 AND y/x > 1.5;
```

Безопасный вариант:

```
SELECT ... WHERE CASE WHEN x > 0 THEN y/x > 1.5 ELSE false END;
```

Применяемая так конструкция **CASE** защищает выражение от оптимизации, поэтому использовать её нужно только при необходимости.

В данном случае было бы лучше решить проблему, переписав условие как $y > 1.5 \cdot x$.

CASE не предотвращает раннее вычисление константных подвыражений — функции и операторы, помеченные как **IMMUTABLE**¹, могут вычисляться при планировании, а не при выполнении запроса. Поэтому в примере

```
SELECT CASE WHEN x > 0 THEN x ELSE 1/0 END FROM tab;
```

скорее всего, произойдёт деление на ноль в процессе упрощения планировщиком константного подвыражения, несмотря на то, что во всех строках в таблице $x > 0$, и во время выполнения ветвь **ELSE** никогда бы не выполнилась.

Похожие ситуации, в которых неявно появляются константы, могут возникать и в запросах внутри функций, так как значения аргументов функции и локальных переменных при планировании могут быть заменены константами. Поэтому, для защиты от рискованных вычислений вместо выражения **CASE** безопаснее использовать конструкцию **IF-THEN-ELSE**.

1) неизменяемый, фиксированный

Типы

Типы бывают встроенные и определяемые пользователем командой **CREATE TYPE**. У типа есть имя. В Postgres некоторые типы имеют альтернативные имена — псевдонимы. Типы делятся на:

- числовые типы;
- денежные типы;
- символьные типы;
- типы даты/времени;
- логический тип;
- типы перечислений;
- типы, предназначенные для текстового поиска;
- массивы;
- составные типы;
- диапазонные типы;
- типы доменов;
- идентификаторы объектов;
- тип pg_lsn;- двоичные типы данных;
- геометрические типы;
- типы, описывающие сетевые адреса;
- битовые строки;
- тип UUID;
- тип XML;
- типы JSON;
- псевдотипы.

Каждый тип данных имеет внутреннее представление, скрытое функциями ввода и вывода.

Многие встроенные типы стандартны и имеют очевидные внешние форматы.

Есть типы, уникальные для Postgres (геометрические пути).

Числовые типы

- целочисленные типы
- числа с произвольной (задаваемой) точностью
- типы с плавающей точкой
- последовательные типы

Имя	Размер	Описание	Диапазон
smallint	2 байта	целое в небольшом диапазоне	-32768 .. +32767
integer	4 байта	типичный выбор для целых чисел	-2147483648 .. +2147483647
bigint	8 байт	целое в большом диапазоне	-9223372036854775808 .. 9223372036854775807
decimal	переменный	вещественное число с указанной точностью	до 131072 цифр до десятичной точки и до 16383 — после
numeric	переменный	вещественное число с указанной точностью	до 131072 цифр до десятичной точки и до 16383 — после
real	4 байта	вещественное число с переменной точностью	точность в пределах 6 десятичных цифр
double precision	8 байт	вещественное число с переменной точностью	точность в пределах 15 десятичных цифр
smallserial	2 байта	небольшое целое с автоувеличением	1 .. 32767
serial	4 байта	целое с автоувеличением	1 .. 2147483647
bigserial	8 байт	большое целое с автоувеличением	1 .. 9223372036854775807

Тип **numeric** позволяет хранить числа с очень большим количеством цифр.

Рекомендуется для хранения денежных сумм и других величин, где важна точность.

Вычисления с типом **numeric** дают точные результаты, где это возможно, например, при сложении, вычитании и умножении. Однако операции со значениями **numeric** выполняются гораздо медленнее, чем с целыми числами или с типами с плавающей точкой.

Столбец типа **numeric** объявляется следующим образом:

NUMERIC ([*precision* [, *scale*]])

precision — (точность) общее количество значимых цифр в числе > 0 .

scale — (масштаб) определяет количество десятичных цифр в дробной части, справа от десятичной точки ≥ 0 .

Например, число 123.45678 имеет точность 8 и масштаб 5.

Целочисленные значения можно считать числами с масштабом 0.

NUMERIC(*precision*)

Форма без указания точности:

NUMERIC

создаёт столбец типа «неограниченное число», в котором можно сохранять числовые значения любой длины до предела, обусловленного реализацией. В столбце этого типа входные значения не будут приводиться к какому-либо масштабу, тогда как в столбцах **numeric** с явно заданным масштабом все значения подгоняются под этот масштаб.

Стандарт SQL утверждает, что по умолчанию должен устанавливаться масштаб 0, т. е. значения должны приводиться к целым числам.

Числовые значения физически хранятся без каких-либо дополняющих нулей слева или справа. Таким образом, объявленные точность и масштаб столбца определяют максимальный, а не фиксированный размер хранения.

В дополнение к обычным числовым значениям тип **numeric** принимает следующие специальные значения IEEE 754 — **Infinity**, **-Infinity**, **NaN**. В команде SQL, их нужно заключать в апострофы:

```
UPDATE table SET x = '-Infinity'
```

Регистр символов в этих строках не важен.

В качестве альтернативы значения бесконечности могут быть записаны как **inf** и **-inf**.

В IEEE 754 и большинстве реализаций **NaN** считается не равным любому другому значению, в том числе и самому **NaN**.

Однако, чтобы значения **numeric** можно было сортировать и использовать в древовидных индексах, Postgres считает, что значения **NaN** равны друг другу и при этом больше любых числовых значений, которые не **NaN**.

При округлении значений тип **numeric** выдаёт число, большее по модулю, тогда как (на большинстве платформ по умолчанию) типы **real** и **double precision** выдают ближайшее «чётное» число:

```
SELECT x, round(x::numeric) AS num_round, round(x::double precision) AS dbl_round
FROM generate_series(-3.5, 3.5, 1) as x;
```

x	num_round	dbl_round
-3.5	-4	-4
-2.5	-3	-2
-1.5	-2	-2
-0.5	-1	-0
0.5	1	0
1.5	2	2
2.5	3	2
3.5	4	4

Типы с плавающей точкой

Типы данных **real** и **double precision** хранят приближённые числовые значения с переменной точностью. На всех поддерживаемых в настоящее время платформах эти типы реализуют стандарт IEEE 754 для двоичной арифметики с плавающей точкой с одинарной и двойной точностью, соответственно, в той мере, в какой его поддерживают процессор, операционная система и компилятор.

- Если нужна точность при хранении и вычислениях (например, для денежных сумм), необходимо использовать тип **numeric**.
- Если необходимо выполнять с этими типами сложные вычисления, имеющие большую важность, следует тщательно изучить реализацию операций в целевой среде и поведение в крайних случаях (бесконечность, антипереполнение).
- Проверка равенства двух чисел с плавающей точкой может не всегда давать ожидаемый результат.

В дополнение к обычным числовым значениям типы с плавающей точкой принимают следующие специальные значения:

Infinity
-Infinity
NaN

Они представляют особые значения, описанные в IEEE 754 (см. **numeric**).

Последовательные (серийные) типы

Часто в Postgres используются для создания столбца с автоувеличением.

Способ, соответствующий стандарту SQL, заключается в использовании столбцов идентификации в **CREATE TABLE**.

Типы данных **smallserial**, **serial** и **bigserial** не являются настоящими типами, а представляют собой просто удобное средство для создания столбцов с уникальными идентификаторами (подобное свойству **AUTO_INCREMENT** в некоторых СУБД). В реализации 14 запись:

```
CREATE TABLE tablename (  
    colname SERIAL  
);
```

эквивалентна следующим командам:

```
CREATE SEQUENCE tablename_colname_seq AS integer;  
CREATE TABLE tablename (  
    colname integer NOT NULL DEFAULT nextval('tablename_colname_seq')  
);  
ALTER SEQUENCE tablename_colname_seq OWNED BY tablename.colname;
```

Т.е. создаётся целочисленный столбец со значением по умолчанию, извлекаемым из генератора последовательности.

Чтобы в столбец нельзя было вставить **NULL**, в его определение добавляется ограничение **NOT NULL**. Во многих случаях также имеет смысл добавить для этого столбца ограничения **UNIQUE** или **PRIMARY KEY** для защиты от ошибочного добавления дублирующихся значений, поскольку автоматически это не происходит.

Последняя команда определяет, что последовательность «принадлежит» столбцу, так что она будет удалена при удалении столбца или таблицы.

Поскольку типы **smallserial**, **serial** и **bigserial** реализованы через последовательности, в чи-
словом ряду значений столбца могут образовываться пропуски, даже если никакие строки не удаля-
лись — значение, выделенное из последовательности, считается «использованным», даже если строку
с этим значением не удалось вставить в таблицу. Это может произойти, например, при откате транзак-
ции, добавляющей данные.

Чтобы вставить в столбец **serial** следующее значение последовательности, ему нужно присвоить
значение по умолчанию. Это можно сделать, либо исключив его из списка столбцов в операторе
INSERT, либо с помощью ключевого слова **DEFAULT**.

Имена типов **serial** и **serial4** эквивалентны — они создают столбцы типа **integer**.
bigserial и **serial8** создают столбцы **bigint**.

Тип **bigserial** следует использовать, если за всё время жизни таблицы планируется использовать
больше чем 2^{31} значений.

smallserial и **serial2** создают столбец **smallint**.

Последовательность, созданная для столбца **serial**, автоматически удаляется при удалении свя-
занного с ней столбца.

Последовательность можно удалить и отдельно от столбца, но при этом также будет удалено опре-
деление значения по умолчанию.

Денежные типы

Тип **money** хранит денежную сумму с фиксированной дробной частью.

Имя	Размер	Описание	Диапазон
money	8 байт	денежная сумма	-92 233 720 368 547 758.08 .. +92 233 720 368 547 758.07

Точность дробной части определяется на уровне базы данных параметром **LC_MONETARY**.

Для диапазона, показанного в таблице, предполагается, что число содержит два знака после запятой.

Входные данные могут быть записаны по-разному, в том числе в виде целых и дробных чисел, а также в виде строки в денежном формате, например **'\$1,000.00'**.

Выводятся эти значения обычно в денежном формате, зависящем от региональных стандартов.

Выводимые значения этого типа зависят от региональных стандартов, поэтому попытка загрузить данные типа **money** в базу данных (например, при восстановлении из бэкапа) с другим параметром **LC_MONETARY** может быть неудачной.

Значения типов **numeric**, **int** и **bigint** можно привести к типу **money**.

Преобразования типов **real** и **double precision** возможны через тип **numeric**:

```
SELECT '12.34'::float8::numeric::money;
```

Использовать числа с плавающей точкой для денежных сумм не рекомендуется из-за возможных ошибок округления.

Значение **money** можно привести к типу **numeric** без потери точности. Преобразование в другие типы может быть неточным и также должно выполняться в два этапа:

```
SELECT '52093.89'::money::numeric::float8;
```

При делении значения типа **money** на целое число выполняется отбрасывание дробной части и получается целое, ближайшее к нулю.

Чтобы получить результат с округлением, необходимо либо:

- выполнять деление значения с плавающей точкой;
- до деления привести значение типа **money** к **numeric**, после чего привести результат к типу **money**.

Последний вариант предпочтительнее — он исключает риск потери точности.

Когда значение **money** делится на другое значение **money**, результатом будет значение типа **double precision**, то есть просто число, а не денежная величина.

Денежные единицы измерения при делении сокращаются.

LC_NUMERIC & LC_MONETARY

```
$ cat /usr/include/locale.h
#ifndef _LOCALE_H
#define _LOCALE_H    1

#include <features.h>

#define __need_NULL
#include <stddef.h>
#include <bits/locale.h>

__BEGIN_DECLS

/* These are the possibilities for the first argument to setlocale.
   The code assumes that the lowest LC_* symbol has the value zero.  */
#define LC_CTYPE        __LC_CTYPE
#define LC_NUMERIC      __LC_NUMERIC
#define LC_TIME         __LC_TIME
#define LC_COLLATE      __LC_COLLATE
#define LC_MONETARY     __LC_MONETARY
#define LC_MESSAGES     __LC_MESSAGES
#define LC_ALL          __LC_ALL
#define LC_PAPER        __LC_PAPER
#define LC_NAME         __LC_NAME
#define LC_ADDRESS      __LC_ADDRESS
#define LC_TELEPHONE    __LC_TELEPHONE
#define LC_MEASUREMENT  __LC_MEASUREMENT
#define LC_IDENTIFICATION __LC_IDENTIFICATION
```

```

/* Structure giving information about numeric and monetary notation. */
struct lconv { /* Numeric (non-monetary) information. */
    char *decimal_point; /* Decimal point character. */
    char *thousands_sep; /* Thousands separator. */
    ...

    /* Monetary information. */
    /* First three chars are a currency symbol from ISO 4217. */
    /* Fourth char is the separator. Fifth char is '\0'. */
    char *int_curr_symbol;
    char *currency_symbol; /* Local currency symbol. */
    char *mon_decimal_point; /* Decimal point character. */
    char *mon_thousands_sep; /* Thousands separator. */
    char *mon_grouping; /* Like `grouping' element (above). */
    char *positive_sign; /* Sign for positive values. */
    char *negative_sign; /* Sign for negative values. */
    ...
};

/* Set and/or return the current locale. */
extern char *setlocale (int __category, const char *__locale) __THROW;
/* Return the numeric/monetary information for the current locale. */
extern struct lconv *localeconv (void) __THROW;
...
__END_DECLS
#endif /* locale.h */

```


Символьные типы

Имя	Описание
<code>character varying(<i>n</i>)</code> , <code>varchar(<i>n</i>)</code>	строка ограниченной переменной длины
<code>character(<i>n</i>)</code> , <code>char(<i>n</i>)</code>	строка фиксированной длины, дополненная пробелами
<code>text</code>	строка неограниченной переменной длины

SQL определяет два основных символьных типа:

`character varying(n)`
`character(n)`

где *n* — положительное число.

Оба эти типа могут хранить текстовые строки длиной до *n* символов (**не байт !!!**).

Попытка сохранить в столбце такого типа более длинную строку приведёт к ошибке, но только если все лишние символы не являются пробелами — пробелы будут усечены до максимально допустимой длины (SQL).

Если длина сохраняемой строки оказывается меньше объявленной, значения типа **`character`** будут дополняться пробелами, а тип **`character varying`** просто сохранит короткую строку.

При приведении значения к типу **`character varying(n)`** или **`character(n)`**, часть строки, выходящая за границу в *n* символов, удаляется, не вызывая ошибки (SQL).

Postgres

Записи **varchar(n)** и **char(n)** являются синонимами **character varying(n)** и **character(n)**, соответственно.

Записи **character** без указания длины соответствует **character(1)**.

Если длина не указана для **character varying**, этот тип будет принимать строки любого размера.

Тип **text** позволяет хранить строки произвольной длины.

Тип **text** не в стандарте SQL, но его поддерживают многие СУБД SQL.

Значения типа **character** физически дополняются пробелами до **n** символов, хранятся и отображаются в таком виде.

При сравнении двух значений типа character дополняющие пробелы считаются незначащими и игнорируются.

С правилами сортировки, где пробельные символы являются значащими, это поведение может приводить к неожиданным результатам:

```
SELECT 'a '::CHAR(2) collate "C" < E'a\n'::CHAR(2)
```

вернёт **true**, хотя в локали «C» символ пробела считается больше символа новой строки.

При приведении значения character к другому символьному типу дополняющие пробелы отбрасываются.

Дополняющие пробелы несут смысловую нагрузку в типах **character varying** и **text**, в проверках по шаблонам, и в регулярных выражениях.

Какие именно символы можно сохранить в этих типах данных, зависит от того, какой набор символов был выбран при создании базы данных.

В любом случае символ с кодом 0 (NUL) сохранить нельзя, вне зависимости от выбранного набора символов.

Для хранения короткой строки (до 126 байт) требуется дополнительный 1 байт плюс размер самой строки, включая дополняющие пробелы для типа **character**. Для строк длиннее требуется не 1, а 4 дополнительных байта. Система может автоматически сжимать длинные строки, так что физический размер на диске может быть меньше. Очень длинные текстовые строки переносятся в отдельные таблицы, чтобы они не замедляли работу с другими столбцами.

Примеры

```
CREATE TABLE test1 (a character(4));
INSERT INTO test1 VALUES ('ok');
SELECT a, char_length(a) FROM test1; -- (1)
```

a	char_length
ok	2

```
CREATE TABLE test2 (b varchar(5));
INSERT INTO test2 VALUES ('ok');
INSERT INTO test2 VALUES ('good');
INSERT INTO test2 VALUES ('too long');
ОШИБКА: значение не укладывается в тип character varying(5)
INSERT INTO test2 VALUES ('too long'::varchar(5)); -- явное усечение
SELECT b, char_length(b) FROM test2;
```

b	char_length
ok	2
good	5
too l	5

Максимально возможный размер строки составляет около 1 ГБ.

Допустимое значение n в объявлении типа данных будет меньше этого числа — в зависимости от кодировки каждый символ может занимать несколько байт.

Если необходимо хранить строки без определённого предела длины, следует использовать типы **text** или **character varying** без указания длины.

В Postgres есть ещё два символьных типа фиксированной длины.

Имя	Размер	Описание
"char"	1 байт	внутренний однобайтный тип
name	64 байта	внутренний тип для имён объектов

Тип **name** создан только для хранения идентификаторов во внутренних системных таблицах и не предназначен для обычного применения пользователями.

В настоящее время его длина составляет 64 байта (63 ASCII-символа плюс терминатор).

В исходном коде на C длина задаётся константой **NAMEDATALEN**. Эта константа определяется во время компиляции и её можно менять в некоторых случаях.

Максимальная длина по умолчанию в следующих версиях может быть увеличена.

Тип **"char"** (двойные кавычки) отличается от **char(1)** тем, что он фактически хранится в одном байте. Используется во внутренних системных таблицах для простых перечислений.

Двоичные типы данных

- шестнадцатеричный формат **bytea**
- формат спецпоследовательностей **bytea**

Для хранения двоичных данных предназначен тип **bytea**

Имя	Размер	Описание
bytea	1 или 4 байта плюс сама двоичная строка	двоичная строка переменной длины

Двоичные строки представляют собой последовательность октетов (ASN.1) и имеют два отличия от текстовых строк.

1) в двоичных строках можно хранить байты с кодом 0 и другими «непечатаемыми» значениями (обычно это значения вне десятичного диапазона 32..126).

В текстовых строках нельзя сохранять нулевые байты, а также значения и последовательности значений, не соответствующие выбранной кодировке базы данных.

2) в операциях с двоичными строками обрабатываются байты в чистом виде, тогда как текстовые строки обрабатываются в зависимости от языковых стандартов. То есть, двоичные строки больше подходят для данных, которые программист видит как «просто байты», а символьные строки — для хранения текста.

Тип **bytea** поддерживает два формата ввода и вывода — шестнадцатеричный и традиционный для PostgreSQL формат спецпоследовательностей.

Строка со спецпоследовательностями начинается с буквы E (заглавной или строчной), стоящей непосредственно перед апострофом:

```
E'foo'
```

Внутри таких строк символ '\' начинает C-подобные спецпоследовательности:

Входные данные принимаются в обоих форматах, а формат выходных данных зависит от параметра конфигурации **bytea_output**; по умолчанию шестнадцатеричный.

Стандарт SQL определяет другой тип двоичных данных — **BLOB (BINARY LARGE OBJECT)** — большой двоичный объект.

Его входной формат отличается от форматов **bytea**, но функции и операторы в основном те же.

Базы данных

Лекция 07 – Основы SQL

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2022.10.13

Оглавление

Язык SQL.....	3
Типы.....	3
Числовые типы.....	4
Представление данных в x86/x86_64.....	5
Типы даты/времени.....	6
Логический тип.....	11
Геометрические типы.....	12
Типы, описывающие сетевые адреса.....	13
Двоичные типы данных.....	16
Битовые строки.....	17
Тип UUID.....	18
Массивы.....	19
Составные типы.....	27
Идентификаторы объектов.....	31
Псевдотипы.....	32

Язык SQL

Типы

Типы бывают встроенные и определяемые пользователем командой **CREATE TYPE**. У типа есть имя. В Postgres некоторые типы имеют альтернативные имена — псевдонимы. Типы делятся на:

- числовые типы;
- денежные типы;
- символьные типы;
- типы даты/времени;
- логический тип;
- типы перечислений;
- типы, предназначенные для текстового поиска;
- массивы;
- составные типы;
- диапазонные типы;
- типы доменов;
- идентификаторы объектов;
- тип pg_lsn;
- двоичные типы данных;
- геометрические типы;
- типы, описывающие сетевые адреса;
- битовые строки;
- тип UUID;
- тип XML;
- типы JSON;
- псевдотипы.

Числовые типы

Имя	Размер	Описание	Диапазон
smallint	2 байта	целое в небольшом диапазоне	-32768 .. +32767
integer	4 байта	типичный выбор для целых чисел	-2147483648 .. +2147483647
bigint	8 байт	целое в большом диапазоне	-9223372036854775808 .. 9223372036854775807
decimal	переменный	вещественное число с указанной точностью	до 131072 цифр до десятичной точки и до 16383 — после
numeric	переменный	вещественное число с указанной точностью	до 131072 цифр до десятичной точки и до 16383 цифр после десятичной точки
real	4 байта	вещественное число с переменной точностью	точность в пределах 6 десятичных цифр
double precision	8 байт	вещественное число с переменной точностью	точность в пределах 15 десятичных цифр
smallserial	2 байта	небольшое целое с автоувеличением	1 .. 32767
serial	4 байта	целое с автоувеличением	1 .. 2147483647
bigserial	8 байт	большое целое с автоувеличением	1 .. 9223372036854775807

Представление данных в x86/x86_64

Segment word size	32 bit						64 bit			
compiler	MS	Intel Win	Borla	Watc	GCC,Clang	Int Linux	MS	Intel Win	GCC,Clang	Int Linux
bool	1	1	1	1	1	1	1	1	1	1
char	1	1	1	1	1	1	1	1	1	1
wchar_t	2	2	2	2	2	2	2	2	4	4
short int	2	2	2	2	2	2	2	2	2	2
int	4	4	4	4	4	4	4	4	4	4
long int	4	4	4	4	4	4	4	4	8	8
int64_t	8	8			8	8	8	8	8	8
enum (typical)	4	4	4	4	4	4	4	4	4	4
float	4	4	4	4	4	4	4	4	4	4
double	8	8	8	8	8	8	8	8	8	8
long double	8	16	10	8	12	12	8	16	16	16
__m64	8	8			8	8		8	8	8
__m128	16	16			16	16	16	16	16	16
__m256	32	32			32	32	32	32	32	32
__m512	64	64			64	64	64	64	64	64
pointer	4	4	4	4	4	4	8	8	8	8
function pointer	4	4	4	4	4	4	8	8	8	8
data member ptr (min)	4	4	8	4	4	4	4	4	8	8
data member ptr (max)	12	12	8	12	4	4	12	12	8	8
member func ptr (min)	4	4	12	4	8	8	8	8	16	16
member func ptr (max)	16	16	12	16	8	8	24	24	16	16

Типы даты/времени

- ввод даты/времени
- вывод даты/времени
- часовые пояса
- ввод интервалов
- вывод интервалов

Юлианская дата — число суток, прошедших начиная с полудня понедельника, 1 января 4713 года до н. э. *пролептического* юлианского календаря или, что то же самое, 24 ноября 4714 года до н. э. *пролептического* григорианского календаря (соответственно, −4712 год и −4713 год по астрономическому счёту лет).

Даты сменяются в полдень.

В гражданском счёте первому году н. э. предшествует первый год до н. э.

В астрономическом счёте первому году н. э. предшествует нулевой год.

Юлианский календарь введен 1 января 45 года до н. э.

Григорианский календарь уточняет юлианский. Введен 4 октября 1582 года взамен юлианского, при этом следующим днём после четверга 4 октября стала пятница 15 октября (+11).

```
typedef struct {
    int    year; //
    int    ymon; // месяц года григорианский
    int    mday; // день месяца 1..31
    int    hour; // 0..23
    int    min;   // 0..59
    double sec;  // 0..60 -- (високосная секунда)
    double yday; // день года 1..365
} UTC;
```

PostgreSQL поддерживает полный набор типов даты и времени SQL.

Все даты считаются по Григорианскому календарю, даже для времени до его введения.

Имя	Размер	Описание	Наименьшее значение	Наибольшее значение	Точность
timestamp [(p)] [without time zone]	8 байт	дата и время (без часового пояса)	4713 до н. э.	294276 н. э.	1 мкс
timestamp [(p)] with time zone	8 байт	дата и время (с часовым поясом)	4713 до н. э.	294276 н. э.	1 мкс
date	4 байта	дата (без времени суток)	4713 до н. э.	5874897 н. э.	1 день
time [(p)] [without time zone]	8 байт	время суток (без даты)	00:00:00	24:00:00	1 мкс
time [(p)] with time zone	12 байт	время дня (без даты), с часовым поясом	00:00:00+1559	24:00:00-1559	1 мкс
interval [поля] [(p)]	16 байт	временной интервал	-178000000 лет	178000000 лет	1 мкс

Типы **time**, **timestamp** и **interval** принимают необязательное значение точности **p**, определяющее, сколько знаков после запятой должно сохраняться в секундах.

По умолчанию точность не ограничивается. Допустимые значения **p** лежат в интервале от 0 до 6.

Тип **time with time zone** определён стандартом SQL, но ценность его сомнительна.

В большинстве случаев сочетание типов **date**, **time**, **timestamp without time zone** и **timestamp with time zone** удовлетворяет все потребности в функционале дат/времени, возникающие в приложениях.

Значения даты и времени принимаются во многих форматах, включая ISO 8601.

```
type [ (p) ] 'value'
```

Пример

Стандарт SQL различает константы типов **timestamp without time zone** и **timestamp with time zone** по знаку «+» или «-» и смещению часового пояса, добавленному после времени.

```

TIMESTAMP '2004-10-19 10:23:54'      — тип timestamp without time zone
TIMESTAMP '2004-10-19 10:23:54+02'   — тип timestamp with time zone.

```

TIMESTAMP WITH TIME ZONE '2004-10-19 10:23:54+02'

Значения **timestamp with time zone** внутри всегда хранятся в UTC (Universal Coordinated Time – Всемирное скоординированное время). Вводимое значение, в котором часовой пояс указан явно, переводится в UTC с учётом смещения часового пояса. Если часовой пояс не указан, подразумевается заданный системным параметром **timezone** и время пересчитывается в UTC со смещением **timezone**.

```
$ timedatectl show
Timezone=Europe/Minsk
LocalRTC=no
CanNTP=yes
NTP=yes
NTPSynchronized=no
TimeUSec=Sat 2022-10-29 00:22:42 +03
RTCTimeUSec=Sat 2022-10-29 00:22:43 +03
```

Специальные значения

Postgres поддерживает несколько специальных значений даты/времени. Значения **infinity** и **-infinity** имеют особое представление в системе и они отображаются в том же виде. Другие преобразуются в значения даты/времени. Для использования специальных значений в качестве констант в командах SQL, их нужно заключать в апострофы.

Вводимая строка	Допустимые типы	Описание
epoch	date, timestamp	1970-01-01 00:00:00+00 (точка отсчёта времени в Unix)
infinity	date, timestamp	время после максимальной допустимой даты
-infinity	date, timestamp	время до минимальной допустимой даты
now	date, time, timestamp	время начала текущей транзакции
today	date, timestamp	время начала текущих суток (00:00)
tomorrow	date, timestamp	время начала следующих суток (00:00)
yesterday	date, timestamp	время начала предыдущих суток (00:00)
allballs	time	00:00:00.00 UTC

Ввод интервалов

Значения типа **interval** могут быть записаны в следующей расширенной форме:

[@] quantity unit [quantity unit...] [direction]

quantity — это число (возможно, со знаком);

unit — одно из значений: **microsecond, millisecond, second, minute, hour, day, week, month, year, decade, century, millennium**, которые обозначают соответственно микросекунды, миллисекунды, секунды, минуты, часы, дни, недели, месяцы, годы, десятилетия, века и тысячелетия.

Либо эти же слова во множественном числе, либо их сокращения;

direction может принимать значение **ago** (назад) или быть пустым.

Знак @ является необязательным.

Все заданные величины различных единиц суммируются вместе с учётом знака чисел.

Указание **ago** меняет знак всех полей на противоположный.

Количества дней, часов, минут и секунд можно определить, не указывая явно соответствующие единицы, например **'1 12:59:10'** эквивалентно **'1 day 12 hours 59 min 10 sec'**.

Сочетание года и месяца также можно записать через минус, например **'200-10'** означает то, же что и **'200 years 10 months'**.

Эти краткие формы только и разрешены стандартом SQL и они используются postgres при выводе, когда **IntervalStyle** имеет значение **sql_standard**.

Логический тип

Тип **boolean** — стандартный SQL-тип.

Тип **boolean** может иметь следующие состояния — «**true**», «**false**» и третье состояние «**unknown**», которое представляется SQL-значением **NULL**. Занимает 1 байт.

Логические константы могут представляться в SQL-запросах следующими ключевыми словами SQL — **TRUE**, **FALSE** и **NULL**.

Функция ввода данных типа **boolean** воспринимает следующие строковые представления состояния «**true**» — **true, yes, on, 1** и следующие представления состояния «**false**» — **false, no, off, 0**.

Принимаются и уникальные префиксы этих строк, например **t** или **n**.

Регистр символов не имеет значения. Пробельные символы в начале и в конце строки игнорируются.

Функция вывода данных типа **boolean** всегда выдает **t** или **f**:

```
CREATE TABLE test1 (  
    a boolean,  
    b text  
);  
INSERT INTO test1 VALUES (TRUE, 'sic est');  
INSERT INTO test1 VALUES (FALSE, 'non est');  
SELECT * FROM test1;  
a | b  
---+-----  
t | sic est  
f | non est  
  
SELECT * FROM test1 WHERE a;  
a | b  
---+-----  
t | sic est
```

Геометрические типы

- точки;
- прямые;
- отрезки;
- прямоугольники;
- пути;
- многоугольники;
- окружности.

Геометрические типы данных представляют объекты в двумерном пространстве.

Имя	Размер	Описание	Представление
point	16 байт	Точка на плоскости	(x, y)
line	32 байта	Бесконечная прямая	$\{A, B, C\}$
lseg	32 байта	Отрезок	$((x_1, y_1), (x_2, y_2))$
box	32 байта	Прямоугольник	$((x_1, y_1), (x_2, y_2))$
path	16+16n байт	Закрытый путь (подобен многоугольнику)	$((x_1, y_1), \dots)$
path	16+16n байт	Открытый путь	$[(x_1, y_1), \dots]$
polygon	40+16n байт	Многоугольник (подобен закрытому пути)	$((x_1, y_1), \dots)$
circle	24 байта	Окружность	$\langle (x, y), r \rangle$ (центр окружности и радиус)

Для выполнения различных геометрических операций, в частности масштабирования, вращения и определения пересечений, Postgres имеет богатый набор функций и операторов.

Типы, описывающие сетевые адреса

Для хранения сетевых адресов лучше использовать эти специальные типы, а не простые текстовые строки, поскольку Postgres проверяет вводимые значения данных типов и предоставляет специализированные операторы и функции для работы с ними.

Имя	Размер	Описание
<code>cidr</code>	7 или 19 байт	Сети IPv4 и IPv6
<code>inet</code>	7 или 19 байт	Узлы и сети IPv4 и IPv6
<code>macaddr</code>	6 байт	MAC-адреса
<code>macaddr8</code>	8 байт	MAC-адреса (в формате EUI-64)

```
$ ifconfig
wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.100.21 netmask 255.255.255.0 broadcast 192.168.100.255
    inet6 fe80::8bde:9f77:3315:fc03 prefixlen 64 scopeid 0x20<link>
    ether f4:4e:e3:91:75:35 txqueuelen 1000 (Ethernet)
```

`inet`

Тип **`inet`** содержит IPv4- или IPv6-адрес узла и может в этом же поле содержать также и его подсеть. Подсеть представляется числом бит, определяющих адрес сети в адресе узла (маска сети).

Если маска для сетевого адреса IPv4 равна 32, такое значение представляет не подсеть, а определённый узел.

Адреса IPv6 имеют длину 128 бит, поэтому уникальный адрес узла задаётся с маской 128 бит.

Если требуются только адреса сетей, следует использовать тип **`cidr`**, а не **`inet`**.

Формат:

`address/y`

`y` — число бит в маске сети.

cidr

Тип **cidr** содержит определение сети IPv4 или IPv6.

Входные и выходные форматы соответствуют соглашениям CIDR (Classless Internet Domain Routing).

Определение сети записывается в формате

address/y

где **address** — минимальный адрес в сети, представленный в виде адреса IPv4 или IPv6, а **y** — число бит в маске сети.

Если **y** не указывается, это значение вычисляется по старой классовой схеме нумерации сетей, но при этом оно может быть увеличено, чтобы в него вошли все байты введённого адреса.

Если в сетевом адресе справа от маски сети окажутся биты со значением 1, он будет считаться ошибочным.

Различия inet и cidr

inet принимает значения с ненулевыми битами справа от маски сети, а **cidr** — нет.

Например, значение 192.168.0.1/24 является допустимым для типа **inet**, но не для **cidr**.

macaddr

Тип **macaddr** предназначен для хранения MAC-адреса, примером которого является адрес сетевого интерфейса.

Вводимые значения могут задаваться в следующих форматах:

'08:00:2b:01:02:03' (IEEE 802-2001 MSB)

'08-00-2b-01-02-03' (IEEE 802-2001 LSB)

'08002b:010203'

'08002b-010203'

'0800.2b01.0203'

'0800-2b01-0203'

'08002b010203'

macaddr8

Тип **macaddr8** хранит MAC-адреса в формате EUI-64, применяющиеся, например, для аппаратных адресов сетевых и прочих коммуникационных интерфейсов.

Этот тип может принять и 6-байтовые, и 8-байтовые MAC-адреса и сохраняет их в 8 байтах.

MAC-адреса, заданные в 6-байтовом формате, хранятся в формате 8 байт, а 4-ый и 5-ый байт содержат FF и FE, соответственно. Ниже показаны примеры допустимых входных строк в стандарте:

'08:00:2b:01:02:03:04:05'

'08-00-2b-01-02-03-04-05'

Двоичные типы данных

- шестнадцатеричный формат **bytea**
- формат спецпоследовательностей **bytea**

Для хранения двоичных данных предназначен тип **bytea**

Имя	Размер	Описание
bytea	1 или 4 байта плюс сама двоичная строка	двоичная строка переменной длины

Двоичные строки представляют собой последовательность октетов (ASN.1) и имеют два отличия от текстовых строк.

1) в двоичных строках можно хранить байты с кодом 0 и другими «непечатаемыми» значениями (обычно это значения вне десятичного диапазона 32..126).

В текстовых строках нельзя сохранять нулевые байты, а также значения и последовательности значений, не соответствующие выбранной кодировке базы данных.

2) в операциях с двоичными строками обрабатываются байты в чистом виде, тогда как текстовые строки обрабатываются в зависимости от языковых стандартов. То есть, двоичные строки больше подходят для данных, которые программист видит как «просто байты», а символьные строки — для хранения текста.

Битовые строки

Битовые строки представляют собой последовательности из 1 и 0. Их можно использовать для хранения или отображения битовых масок. В SQL есть два битовых типа:

- **bit(n)**

- **bit varying(n)**

где **n** — положительное целое число.

Длина значения типа **bit** должна в точности равняться **n** — при попытке сохранить более длинные или более короткие данные произойдёт ошибка.

Данные типа **bit varying** могут иметь переменную длину, но не превышающую **n** — строки большей длины не будут приняты.

Запись **bit** без указания длины равнозначна записи **bit(1)**.

Запись **bit varying** без указания длины подразумевает строку неограниченной длины.

```
CREATE TABLE test (a BIT(3), b BIT VARYING(5));
INSERT INTO test VALUES (B'101', B'00');
INSERT INTO test VALUES (B'10', B'101');
```

ОШИБКА: длина битовой строки (2) не соответствует типу bit(3)

```
INSERT INTO test VALUES (B'10'::bit(3), B'101');
SELECT * FROM test;
```

a		b
101		00
100		101

Тип UUID

Тип данных **uuid** сохраняет универсальные уникальные идентификаторы (Universally Unique Identifiers, UUID), определённые в RFC 4122, ISO/IEC 9834-8:2005 и связанных стандартах.

В некоторых системах это называется GUID, глобальным уникальным идентификатором.

Этот идентификатор представляет собой 128-битное значение, генерируемое специальным алгоритмом, практически гарантирующим, что этим же алгоритмом оно не будет получено больше нигде в мире.

Таким образом, эти идентификаторы будут уникальными и в распределённых системах, а не только в единственной базе данных, как значения генераторов последовательностей.

```
$ cat /etc/fstab
UUID=77b34e2d-e9f9-42ba-9cab-458fcfca267e /          ext4      defaults          1 1
UUID=66c3886f-c106-4439-a204-f36b01814591 /boot        ext4      defaults          1 2
UUID=400C-7ED1 /boot/efi    vfat      umask=0077,shortname=winnt 0 2
UUID=5bcb7f89-0109-42cd-83ab-478eac528f4f /home        ext4      defaults          1 2
```

Пример UUID в стандартном текстовом виде:

a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11

```
$ uuidgen
df8ae888-6076-4f55-b3fb-677877e5b90d
$ uuidgen -t --sha1 --namespace @dns --name "lsi.bas-net.by"
770990c7-4f1a-5242-8565-d32d339c066d
```

uuid_t

Массивы

Postgres позволяет определять столбцы таблицы как **многомерные массивы переменной длины**. Элементами массивов могут быть любые встроенные или определённые пользователями базовые типы, перечисления, составные типы, типы-диапазоны или домены.

Объявления типов массивов

Для объявления типа массива к названию типа элементов добавляются квадратные скобки `[]`.

Приведенная ниже команда создает таблицу **sal_emp** со столбцами типов **text**, одномерный массив с элементами типа **integer** — квартальная зарплата работников, и двухмерный массив с элементами типа **text** — недельный график работника.

```
CREATE TABLE sal_emp (  
    name          text,          -- ФИО работника  
    pay_by_quarter integer[],     -- квартальная зарплата  
    schedule      text[][]       -- недельный график  
);
```

Также существует и альтернативный синтаксис с ключевым словом **ARRAY** (стандарт SQL). Столбец **pay_by_quarter** можно было бы определить так:

```
pay_by_quarter integer ARRAY[4],
```

Или без указания размера массива:

```
pay_by_quarter integer ARRAY,
```

Ограничений на фактический размер массива .

Ввод значения массива

Чтобы записать значение массива в виде литеральной константы, следует заключить значения элементов в фигурные скобки и разделить их запятыми.

Если элемент содержит запятые или фигурные скобки, его заключают в двойные кавычки.

Общий формат константы массива:

```
'{ val1 delim val2 delim ... }'
```

delim — символ, указанный в качестве разделителя в соответствующей записи в системной таблице **pg_type**. Обычно это запятая (,) или, в случае типа **box**, точка с запятой (;).

Каждое **val** — либо константа типа элемента массива, либо вложенный массив.

```
'{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}'
```

Пример

```
INSERT INTO sal_emp
VALUES ('Virt N.',
       '{10000, 10000, 10000, 10000}',
       '{{"meeting", "lunch"}, {"training", "presentation"}}');
```

```
INSERT INTO sal_emp
VALUES ('Knuth D.',
       '{20000, 25000, 25000, 25000}',
       '{{"breakfast", "consulting"}, {"meeting", "lunch"}}');
```

И результат запроса:

```
SELECT * FROM sal_emp;
name |          pay_by_quarter          |          schedule
-----+-----+-----
Bill | {10000,10000,10000,10000} | {{meeting,lunch},{training,presentation}}
Carol| {20000,25000,25000,25000} | {{breakfast,consulting},{meeting,lunch}}
(2 rows)
```

В многомерных массивов число элементов в каждой размерности должно быть одинаковым. В противном случае возникает ошибка. Например:

```
INSERT INTO sal_emp
VALUES ('Bill',
       '{10000, 10000, 10000, 10000}',
       '{{"meeting", "lunch"}, {"meeting"}}');
ERROR: multidimensional arrays must have array expressions with matching dimensions
```

Обращение к массивам

Указывается индекс элемента в массиве.

Следующий запрос получает имена сотрудников, зарплата которых изменилась во втором квартале:

```
SELECT name FROM sal_emp WHERE pay_by_quarter[1] <> pay_by_quarter[2];
```

```
name
```

```
-----
```

```
Knuth D.
```

Индексы элементов массива записываются в квадратных скобках.

По умолчанию в Postgres действует соглашение о нумерации элементов массива с 1. Т.е. в массиве из **n** элементов первым считается **array[1]**, а последним — **array[n]**.

Следующий запрос выдаёт зарплату всех сотрудников в третьем квартале:

```
SELECT pay_by_quarter[3] FROM sal_emp;
```

```
pay_by_quarter
```

```
-----
```

```
10000
```

```
25000
```

Прямоугольные срезы массива (блоки)

Срез массива (подмассив) для одной или нескольких размерностей обозначается как

lower-bound:upper-bound

Следующий запрос получает первые пункты в графике Билла в первые два дня недели:

```
SELECT schedule[1:2][1:1] FROM sal_emp WHERE name = 'Bill';
```

```
          schedule
```

```
-----  
{{meeting},{training}}}
```

Если одна из размерностей записана в виде среза, то есть содержит двоеточие, тогда срез распространяется на все размерности. Если при этом для размерности указывается только одно число (без двоеточия), в срез войдут элемент от 1 до заданного номера. Например, в следующем примере **[2]** эквивалентно **[1:2]**:

```
SELECT schedule[1:2][2] FROM sal_emp WHERE name = 'Bill';
```

```
          schedule
```

```
-----  
{{meeting,lunch},{training,presentation}}}
```

Поэтому при обращении к одному элементу размерности, срезы нужно записывать явно для всех измерений, например **[1:2][1:1]** вместо **[2][1:1]**.

Значения **lower-bound** и/или **upper-bound** в указании среза можно опустить. В этом случае опущенная граница заменяется нижним или верхним пределом индексов массива. Например:

```
SELECT schedule[:2][2:] FROM sal_emp WHERE name = 'Bill';
```

Текущие размеры значения массива можно получить с помощью функции **array_dims**:

```
SELECT array_dims(schedule) FROM sal_emp WHERE name = 'Carol';
```

```
array_dims
```

```
-----
```

```
[1:2][1:2]
```

Изменение массивов

Значение массива

```
CREATE TABLE sal_emp (  
    name          text,  
    pay_by_quarter integer[],  
    schedule      text[][]  
);
```

можно заменить полностью:

```
UPDATE sal_emp SET pay_by_quarter = '{25000,25000,27000,27000}'  
WHERE name = 'Carol';
```

или используя синтаксис ARRAY:

```
UPDATE sal_emp SET pay_by_quarter = ARRAY[25000,25000,27000,27000]  
WHERE name = 'Carol';
```

Также можно изменить один элемент массива:

```
UPDATE sal_emp SET pay_by_quarter[4] = 15000 WHERE name = 'Bill';
```

или срез:

```
UPDATE sal_emp SET pay_by_quarter[1:2] = '{27000,27000}'  
WHERE name = 'Carol';
```

Поиск значений в массивах

Чтобы найти значение в массиве, необходимо проверить все его элементы. Это можно сделать вручную, если размер массива известен:

```
SELECT * FROM sal_emp WHERE pay_by_quarter[1] = 10000 OR  
                             pay_by_quarter[2] = 10000 OR  
                             pay_by_quarter[3] = 10000 OR  
                             pay_by_quarter[4] = 10000;
```

или

```
SELECT * FROM sal_emp WHERE 10000 = ANY (pay_by_quarter);
```

если размерность велика/неизвестна.

Можно найти строки, в которых массивы содержат только значения, равные 10000:

```
SELECT * FROM sal_emp WHERE 10000 = ALL (pay_by_quarter);
```

Для поиска в массивах postgres предлагает использовать несколько функций.

`array_position(array, val[, idx])` – возвращает позицию первого вхождения `val`
`array_positions(array, val)` – возвращает массив позиций всех вхождений `val`

Составные типы

Составной тип представляется структурой табличной строки или записи.

По сути это просто список имён полей и соответствующих типов данных.

Postgres позволяет использовать составные типы во многом так же, как и простые типы, например, в определении таблицы можно объявить столбец составного типа.

Объявление составных типов

Ниже приведены два простых примера определения составных типов:

```
CREATE TYPE complex AS (  
    r      double precision,  
    i      double precision  
);
```

```
CREATE TYPE inventory_item AS (  
    name          text,      --  
    supplier_id   integer,   -- поставщик  
    price         numeric    --  
);
```

Синтаксис очень похож на **CREATE TABLE**, за исключением того, что он допускает только названия полей и их типы.

Какие-либо ограничения (такие как NOT NULL) в настоящее время не поддерживаются.

Ключевое слово **AS** здесь имеет значение — без него система будет считать, что подразумевается другой тип команды **CREATE TYPE**, и выдаст синтаксическую ошибку.

Определив такие типы, мы можем использовать их в таблицах:

```
CREATE TABLE on_hand (  
    item          inventory_item,  
    count         integer  
);  
  
INSERT INTO on_hand VALUES (ROW('fuzzy dice', 42, 1.99), 1000);
```

или в функциях:

```
CREATE FUNCTION price_extension(inventory_item, integer) RETURNS numeric  
AS 'SELECT $1.price * $2' LANGUAGE SQL;  
  
SELECT price_extension(item, 10) FROM on_hand;
```

Когда создаётся таблица, вместе с ней автоматически создаётся составной тип. Этот тип представляет тип строки таблицы, и его именем становится имя таблицы. Например, при выполнении команды:

```
CREATE TABLE inventory_item (  
    name          text,  
    supplier_id   integer REFERENCES suppliers,  
    price         numeric CHECK (price > 0)  
);
```

в качестве побочного эффекта будет создан составной тип **inventory_item**, в точности соответствующий тому, что был показан выше, и использовать его можно так же.

Конструирование составных значений

Чтобы записать значение составного типа в виде текстовой константы, его поля нужно заключить в круглые скобки и разделить их запятыми. Значение любого поля можно заключить в кавычки, а если оно содержит запятые или скобки, это делать обязательно.

Таким образом, в общем виде константа составного типа записывается так:

```
'( val1 , val2 , ... )'
```

Например, запись:

```
'("fuzzy dice",42,1.99)'
```

будет допустимой для описанного выше типа **inventory_item**.

Чтобы присвоить **NULL** какому-либо из полей, в соответствующем месте в списке нужно оставить пустое место. Например, задаем значение **NULL** для третьего поля:

```
'("fuzzy dice",42,)'
```

Если же требуется вставить пустую строку, нужно записать пару кавычек:

```
'( "",42, )'
```

Здесь в первом поле окажется пустая строка, а в третьем — **NULL**.

Значения составных типов также можно конструировать, используя синтаксис выражения **ROW**.

В большинстве случаев это значительно проще, чем записывать значения в строке, так как при этом не нужно беспокоиться о вложенности кавычек:

```
ROW('fuzzy dice', 42, 1.99)
ROW(' ', 42, NULL)
```

Обращение к составным типам (.)

Чтобы обратиться к полю столбца составного типа, после имени столбца нужно добавить точку и имя поля, подобно тому, как указывается столбец после имени таблицы.

На самом деле, эти обращения неотличимы, так что часто бывает необходимо использовать скобки, чтобы команда была разобрана правильно.

Например, можно попытаться выбрать поле столбца из тестовой таблицы **on_hand** так:

```
SELECT item.name FROM on_hand WHERE item.price > 9.99;
```

Но это не будет работать, так как согласно правилам SQL имя **item** здесь воспринимается как имя таблицы, а не столбца в таблице **on_hand**. Поэтому этот запрос нужно переписать так:

```
SELECT (item).name FROM on_hand WHERE (item).price > 9.99;
```

либо указать и имя таблицы:

```
SELECT (on_hand.item).name FROM on_hand WHERE (on_hand.item).price > 9.99;
```

В этом случае объект в скобках будет правильно интерпретирован как ссылка на столбец **item**, из которого выбирается поле.

При выборке поля из значения составного типа также возможны всякие синтаксические проблемы. Например, чтобы выбрать одно поле из результата функции, возвращающей составное значение, требуется написать что-то подобное:

```
SELECT (my_func(...)).field FROM ...
```

Без дополнительных скобок в этом запросе произойдёт синтаксическая ошибка.

Идентификаторы объектов

Идентификатор объекта (Object Identifier, OID) используется внутри Postgres в качестве первичного ключа различных системных таблиц.

Идентификатор объекта представляется в типе **oid**.

Также существуют различные типы-псевдонимы для **oid**, с именами **reg<something>**.

Имя	Ссылки	Описание	Пример значения
oid	any	числовой идентификатор объекта	564182
regclass	pg_class	имя отношения	pg_type
regcollation	pg_collation	имя правила сортировки	"POSIX"
regconfig	pg_ts_config	конфигурация текстового поиска	english
regdictionary	pg_ts_dict	словарь текстового поиска	simple
regnamespace	pg_namespace	пространство имён	pg_catalog
regoper	pg_operator	имя оператора	+
regoperator	pg_operator	оператор с типами аргументов	*(integer, integer)
regproc	pg_proc	имя функции	sum
regprocedure	pg_proc	функция с типами аргументов	sum(int4)
regrole	pg_authid	имя роли	smith
regtype	pg_type	имя типа данных	integer

Псевдотипы

В систему типов PostgreSQL включены несколько специальных элементов, которые в совокупности называются псевдотипами.

Псевдотип нельзя использовать в качестве типа данных столбца, но можно объявить функцию с аргументом или результатом такого типа.

Каждый из существующих псевдотипов полезен в ситуациях, когда характер функции не позволяет просто получить или вернуть определённый тип данных SQL.

Имя псевдотипа	Описание
any	Указывает, что функция принимает любой вводимый тип данных.
anyelement	Указывает, что функция принимает любой тип данных.
anyarray	Указывает, что функция принимает любой тип массива.
anynonarray	Указывает, что функция принимает любой тип данных, кроме массивов.
anyenum	Указывает, что функция принимает любое перечисление.
anyrange	Указывает, что функция принимает любой диапазонный тип данных.
anymultirange	Указывает, что функция принимает любой мультидиапазонный тип данных.
anycompatible	Указывает, что функция принимает любой тип данных и может автоматически приводить различные аргументы к общему типу данных.
cstring	Указывает, что функция принимает или возвращает строку в стиле C.
internal	Указывает, что функция принимает или возвращает внутренний серверный тип данных.
record	Указывает, что функция принимает или возвращает неопределённый тип строки.
void	Указывает, что функция не возвращает значение.
unknown	Обозначает ещё не распознанный тип, например простую строковую константу.

Функции, написанные на языке C, могут быть объявлены с параметрами или результатами любого из этих псевдотипов. Ответственность за безопасное поведение функции с аргументами таких типов ложится на разработчика функции.

Базы данных

Лекция 08 – Основы SQL

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2022.10.20

Оглавление

Определение данных.....	3
Создание таблицы.....	3
Удаление таблицы.....	5
Значения по умолчанию.....	6
Генерируемые столбцы.....	8
Ограничения.....	10
Ограничения-проверки.....	10
Ограничения NOT NULL.....	15
Ограничения уникальности.....	17
Первичные ключи.....	19
Внешние ключи.....	21
Изменение таблиц (ALTER).....	27
Добавление/удаление столбца.....	28
Удаление столбца.....	28
Добавление/удаление ограничения.....	29
Удаление ограничения.....	29
Изменение значения по умолчанию.....	30
Изменение типа данных столбца.....	31
Переименование столбца.....	32
Переименование таблицы.....	32

Определение данных

Создание таблицы

Для создания таблицы используется команда **CREATE TABLE**.

В этой команде необходимо указать как минимум:

- имя новой таблицы;
- имена и типы данных каждого столбца.

Например:

```
CREATE TABLE dumb_table (  
    first_column    text,  
    second_column   integer  
);
```

Создается таблица **dumb_table** с двумя столбцами.

Первый столбец называется **first_column** и имеет тип данных **text**.

Второй столбец называется **second_column** и имеет тип **integer**.

Имена таблицы и столбцов должны быть в синтаксисе идентификаторов.

Имена типов в большинстве случаев тоже являются идентификаторами (есть исключения).

Список столбцов заключается в скобки, а его элементы разделяются запятыми.

В именах таблиц и столбцов следует отражать, какие данные они будут содержать.

```
CREATE TABLE products (  
    product_no    integer,    -- int32_t  
    name          varchar,    -- как называется  
    price         numeric     -- огромное вещественное число с фиксированной запятой  
);
```

Попытка повторного создания

```
CREATE TABLE dumb_table (  
    first_column    text,  
    second_column   integer  
);
```

ОШИБКА: отношение "dumb_table" уже существует

```
CREATE TABLE IF NOT EXISTS table_name (  
    ...  
);
```

ЗАМЕЧАНИЕ: отношение "table_name" уже существует, пропускается

```
CREATE TABLE
```

Удаление таблицы

```
DROP TABLE dumb_table;  
DROP TABLE products;
```

Попытка удаления несуществующей таблицы считается ошибкой.

Тем не менее в SQL-скриптах часто применяют безусловное удаление таблиц перед созданием, игнорируя все сообщения об ошибках, так что они выполняют свою задачу независимо от того, существовали таблицы или нет.

Можно использовать вариант **DROP TABLE IF EXISTS** (не в стандарте SQL).

Значения по умолчанию

Столбцу можно назначить значение по умолчанию, которое будет присвоено столбцу при добавлении строки в том случае, если значение явно не указано.

Если значение по умолчанию не объявлено явно, им считается значение **NULL**. Это имеет смысл, если полагать, что **NULL** представляет неизвестные данные.

Значения по умолчанию указываются при определении таблицы после типа данных столбца.

```
CREATE TABLE products (  
    product_no      integer,  
    name            varchar,  
    price           numeric      DEFAULT 9.99  
);
```

Значение по умолчанию может быть также выражением, которое вычисляется не когда создаётся таблица, а в момент присваивания значения по умолчанию.

Например, столбцу типа **timestamp** в качестве значения по умолчанию часто присваивается **CURRENT_TIMESTAMP**, чтобы в момент добавления строки в нём оказалось текущее время.

DEFAULT является *ограничением* (CONSTRAINT).

Генерация «последовательных номеров» для всех строк:

```
CREATE SEQUENCE products_product_no_seq AS integer;  
...  
CREATE TABLE products (  
    product_no integer DEFAULT nextval('products_product_no_seq'),  
    ...  
);
```

Функция **nextval()** возвращает следующее значение из генератора последовательности.
Или, что аналогично,

```
CREATE TABLE products (  
    product_no SERIAL,  
    ...  
);
```

Генерируемые столбцы

Генерируемый столбец является столбцом особого рода, который всегда вычисляется из других.

Есть два типа генерируемых столбцов:

- сохранённые;
- виртуальные.

Сохранённый (STORED) генерируемый столбец вычисляется при записи (добавлении или изменении) и занимает место в таблице так же, как и обычный столбец.

Виртуальный генерируемый столбец не занимает места и вычисляется при чтении.

В настоящее время в Postgres реализованы только сохранённые генерируемые столбцы. Для создания генерируемого столбца используется предложение **GENERATED ALWAYS AS**

```
CREATE TABLE people (  
    ...,  
    height_cm numeric, -- рост в сантиметрах  
    height_in numeric GENERATED ALWAYS AS (height_cm / 2.54) STORED  
);
```

Ключевое слово **STORED**, определяющее тип хранения генерируемого столбца, обязательно.

Произвести запись непосредственно в генерируемый столбец нельзя.

Поэтому в командах **INSERT** или **UPDATE** нельзя задать значение для таких столбцов.

Отличия генерируемых столбцов от столбцов со значением по умолчанию

- 1) значение столбца по умолчанию вычисляется один раз, когда в таблицу впервые вставляется строка и никакое другое значение не задано;
- 2) значение генерируемого столбца может меняться при изменении строки и не может быть переопределено;
- 3) выражение значения по умолчанию не может обращаться к другим столбцам таблицы, в то время как генерирующее выражение чаще всего именно это и делают;
- 4) в выражении значения по умолчанию могут вызываться «изменяемые» функции, например, **random()** или функции, зависящие от времени, а для генерируемых столбцов это не допускается.

Ограничения и особенности генерирующих выражений и таблиц, их содержащих:

- 1) в генерирующем выражении могут использоваться только постоянные функции и не могут фигурировать подзапросы или ссылки на какие-либо значения, не относящиеся к данной строке;
- 2) генерирующее выражение не может обращаться к другому генерируемому столбцу;
- 3) генерирующее выражение не может обращаться к системным столбцам, кроме **tableoid**;
- 4) для генерируемого столбца нельзя задать значение по умолчанию;
- 5) генерируемый столбец не может быть частью ключа секционирования.
- 6) есть ограничения по наследованию.
- 7) права доступа к генерируемым столбцам существуют отдельно от прав для столбцов, из которых он вычисляется, поэтому их можно организовать так, чтобы определённый пользователь мог прочитать генерируемый столбец, но не мог читать оригиналы.

Секционирование — таблица разделяется на несколько частей меньшего размера:

- горизонтальное секционирование (части таблицы LOGS содержат строки по месяцам или фикс.)
- вертикальное секционирование (части таблицы NEWS содержат разные ее столбцы).

Ограничения

- ограничение DEFAULT;
- ограничения-проверки (CHECK);
- ограничения NOT NULL;
- ограничения уникальности;
- первичные ключи;
- внешние ключи;
- ограничения-исключения.

Ограничения-проверки

Ограничение-проверка — наиболее общий тип ограничений. В его определении можно указать, что значение данного столбца должно удовлетворять логическому выражению (проверке истинности). Например, цену товара можно ограничить положительными значениями:

```
CREATE TABLE products (  
    product_no integer,  
    name        varchar,  
    price       numeric CHECK (price > 0)  
);
```

Ограничение определяется после объявления типа данных, аналогично тому, как определяется значение по умолчанию.

Значения по умолчанию и ограничения могут указываться в любом порядке.

Ограничение-проверка состоит из ключевого слова **CHECK**, за которым идёт выражение в скобках.

Выражение должно включать столбец, для которого задаётся ограничение, иначе оно не имеет большого смысла.

Ограничению можно присвоить отдельное имя — это улучшит сообщения об ошибках и позволит ссылаться на это ограничение, если придется его изменять:

```
CREATE TABLE products (  
    product_no integer,  
    name        varchar,  
    price        numeric CONSTRAINT positive_price CHECK (price > 0)  
);
```

Синтаксис:

[CONSTRAINT *constraint_name*] *constraint_definition*

Ограничение-проверка может также ссылаться на несколько столбцов. Например, чтобы гарантировать, что цена со скидкой будет всегда меньше обычной:

```
CREATE TABLE products (  
    product_no        integer,  
    name              varchar,  
    price              numeric CHECK (price > 0),           -- ограничение столбца  
    discounted_price   numeric CHECK (discounted_price > 0), -- ограничение столбца  
                                CHECK (price > discounted_price) -- ограничение таблицы  
);
```

Первые два ограничения определяются «постфиксно».

Третье представлено отдельным элементом в списке элементов определения таблицы (огр. табл.).

Определения столбцов и определений ограничений можно переставлять в произвольном порядке.

Первые два ограничения — это ограничения столбцов.

Третье ограничение — ограничение таблицы, поскольку записано отдельно от определений столбцов.

Ограничения столбцов также можно записать в виде ограничений таблицы, тогда как обратное не всегда возможно, так как подразумевается, что ограничение столбца ссылается только на связанный столбец. (Хотя PostgreSQL этого не требует, но для совместимости с другими СУБД лучше следовать этому правилу.) Ранее приведённый пример можно переписать и так:

```
CREATE TABLE products (  
    product_no integer,  
    name          varchar(120),  
    price          numeric,  
    CHECK (price > 0),  
    discounted_price numeric,  
    CHECK (discounted_price > 0),  
    CHECK (price > discounted_price)  
);
```

Или так:

```
CREATE TABLE products (  
    product_no      integer,  
    name            varchar,  
    price           numeric CHECK (price > 0),  
    discounted_price numeric,  
    CHECK (discounted_price > 0 AND price > discounted_price)  
);
```

Ограничениям таблицы можно присваивать имена так же, как и ограничениям столбцов:

```
CREATE TABLE products (  
    product_no      integer,  
    name            varchar(120),  
    price           numeric,  
    CHECK (price > 0),  
    discounted_price numeric,  
    CHECK (discounted_price > 0),  
    CONSTRAINT valid_discount CHECK (price > discounted_price)  
);
```

Ограничение-проверка удовлетворяется, если выражение принимает значение **TRUE** или **NULL**.

Результатом многих выражений с операндами **NULL** будет значение **NULL**, поэтому такие ограничения не будут препятствовать записи **NULL** в связанные столбцы.

Чтобы гарантировать, что столбец не содержит значения **NULL**, следует использовать ограничение **NOT NULL**.

Важное замечание

В Postgres предполагается, что условия ограничений **CHECK** являются постоянными, то есть при одинаковых данных в строке они всегда будут выдавать одинаковый результат. Поэтому postgres поддерживает ограничения **CHECK** только для тех данных, которые относятся к только создаваемой или изменяемой строке.

CHECK, нарушающий это правило, может работать в некоторых простых случаях, однако, в общем случае может случиться, что база данных придёт в состояние, когда условие ограничения окажется ложным. Например, если изменились другие строки, участвующие в его вычислении.

В результате восстановление выгруженных данных может оказаться невозможным.

Во время восстановления возможен сбой, даже если полное состояние базы данных согласуется с условием ограничения, по причине того, что строки загружаются не в том порядке, в котором это условие будет соблюдаться.

Поэтому для определения ограничений, затрагивающих другие строки и другие таблицы, следует использовать ограничения **UNIQUE**, **EXCLUDE** или **FOREIGN KEY**, либо реализовать проверки в триггере. Триггеры из дампа восстанавливаются после восстановления данных, поэтому вышеупомянутого сбоя не будет.

Пользовательские функции в ограничениях

Может случиться так, что в выражении **CHECK** используется пользовательская функция, поведение которой впоследствии будет изменено. Postgres не контролирует этого, и если строки в таблице перестанут удовлетворять ограничению **CHECK**, это останется незамеченным. В итоге при попытке загрузить выгруженные позже данные могут возникнуть проблемы. Поэтому подобные изменения рекомендуется осуществлять следующим образом:

- 1) удалить ограничение (используя **ALTER TABLE**);
- 2) изменить определение функции;
- 3) пересоздать ограничение той же командой, которая при этом перепроверит все строки таблицы.

Ограничения NOT NULL

Ограничение **NOT NULL** просто указывает, что столбцу нельзя присваивать значение NULL:

```
CREATE TABLE products (  
    product_no integer      NOT NULL,  
    name        varchar(120) NOT NULL,  
    price       numeric  
);
```

Ограничение **NOT NULL** всегда записывается как ограничение столбца и функционально эквивалентно ограничению:

CHECK (*column_name* IS NOT NULL)

Однако, если check-ограничению можно назначить имя, то ограничению **NOT NULL** имя назначить нельзя.

Для столбца можно определить больше одного ограничения, для чего их нужно указать одно за другим:

```
CREATE TABLE products (  
    product_no integer      NOT NULL,  
    name        varchar(120) NOT NULL,  
    price       numeric      NOT NULL CHECK (price > 0)  
);
```

Порядок указания ограничений не имеет значения — **порядок проверки ограничений не зависит от порядка из указания.**

Для ограничения **NOT NULL** есть и обратное ограничение — **NULL**.

Суть его состоит в указании, что столбец может иметь значение **NULL** (поведение по умолчанию).

Ограничение **NULL** отсутствует в стандарте **SQL** и было добавлено в **Postgres** только для совместимости с некоторыми другими **СУБД**.

Его можно использовать для быстрого переключения ограничения **NULL/NOT NULL** в скрипте.

```
CREATE TABLE products (  
    product_no integer      NULL,  
    name        varchar(120) NULL,  
    price       numeric      NULL  
);
```

При необходимости в любое из них можно вставить ключевое слово **NOT**, если потребуется.

Тем не менее, при проектировании баз данных чаще всего большинство столбцов должны быть помечены как **NOT NULL**.

Ограничения уникальности

Ограничения уникальности гарантируют, что данные в определённом столбце или группе столбцов уникальны среди всех строк таблицы.

Ограничение в виде ограничения столбца:

```
CREATE TABLE products (  
    product_no integer UNIQUE,  
    name        varchar(120),  
    price       numeric  
);
```

Ограничение в виде ограничения таблицы:

```
CREATE TABLE products (  
    product_no integer,  
    name        varchar(120),  
    price       numeric,  
    UNIQUE (product_no)  
);
```

Чтобы определить ограничение уникальности для группы столбцов, его следует записать в виде ограничения таблицы, при этом имена столбцов перечисляются через запятую:

```
CREATE TABLE example (  
    a integer,  
    b integer,  
    c integer,  
    UNIQUE (a, c)  
);
```

Ограничение уникальности для группы столбцов указывает, что сочетание значений перечисленных столбцов должно быть уникально во всей таблице, тогда как значения каждого столбца по отдельности уникальными не должны быть (и обычно не будут).

Уникальному ограничению можно назначить имя:

```
CREATE TABLE products (  
    product_no integer CONSTRAINT must_be_unique UNIQUE,  
    name        varchar(120),  
    price       numeric  
);
```

Ограничение уникальности нарушается, если в таблице оказывается несколько строк, у которых совпадают значения всех столбцов, включённых в ограничение.

При этом **два значения NULL при сравнении никогда не считаются равными**.

Из этого следует, что даже при наличии ограничения уникальности в таблице можно сохранить строки с дублирующимися значениями, если те содержат **NULL** в одном или нескольких столбцах, участвующих в ограничении.

Это поведение соответствует стандарту SQL, но существуют СУБД, которые ведут себя иначе.

Первичные ключи

Ограничение первичного ключа означает, что образующий его столбец или группа столбцов может быть уникальным идентификатором строк в таблице. Для этого требуется, чтобы значения были одновременно уникальными и отличными от **NULL**. Два следующих определения почти эквивалентны:

```
CREATE TABLE products (  
    product_no integer UNIQUE NOT NULL,  
    name        varchar(120),  
    price       numeric  
);
```

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name        varchar(120),  
    price       numeric  
);
```

Первичные ключи могут включать несколько столбцов.

Синтаксис похож на запись ограничений уникальности:

```
CREATE TABLE example (  
    a integer,  
    b integer,  
    c integer,  
    PRIMARY KEY (a, c)  
);
```

При добавлении первичного ключа автоматически создаётся уникальный индекс — В-дерево для столбца или группы столбцов, перечисленных в первичном ключе, и данные столбцы помечаются как **NOT NULL**.

Таблица может иметь максимум один первичный ключ.

ОШИБКА: таблица "ex2" не может иметь несколько первичных ключей СТРОКА 3: b integer PRIMARY KEY
--

Ограничений уникальности и ограничений **NOT NULL**, которые функционально почти равнозначны первичным ключам, может быть сколько угодно, но назначить ограничением первичного ключа можно только одно.

Теория реляционных баз данных говорит, что первичный ключ должен быть в каждой таблице и ему стоит следовать, даже если это требование не является обязательным в данной реализации.

Внешние ключи

Ограничение внешнего ключа указывает, что значения столбца (или группы столбцов) должны соответствовать значениям в некоторой строке другой таблицы.

Это называется **ссылочной целостностью** двух связанных таблиц.

Например, имеем таблицу:

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name        varchar(120),  
    price       numeric  
);
```

Пусть существует таблица заказов этих продуктов. Нужно, чтобы в таблице заказов содержались только заказы действительно существующих продуктов. Поэтому в ней определяется ограничение внешнего ключа, ссылающееся на таблицу продуктов:

```
CREATE TABLE orders (  
    id            integer PRIMARY KEY,  
    product_no integer REFERENCES products (product_no),  
    quantity     integer  
);
```

С таким ограничением создать заказ со значением **product_no**, отсутствующим в таблице **products** (и не равным **NULL**), будет невозможно.

В такой схеме таблицу **orders** называют *подчинённой* таблицей, а **products** — *главной*.

Соответственно, столбцы называют так же *подчинённым* и *главным* (или *ссылающимся* и *целевым*).

Подчиненную таблицу можно записать короче:

```
CREATE TABLE orders (  
    id            integer PRIMARY KEY,  
    product_no integer REFERENCES products, -- ссылается на products PRIMARY_KEY  
    quantity     integer  
);
```

Если список столбцов опущен, внешний ключ будет связан с первичным ключом главной таблицы *неявно*.

Ограничению внешнего ключа можно назначить имя.

Внешний ключ может ссылаться на группу столбцов. В этом случае его нужно записать в виде обычного ограничения таблицы:

```
CREATE TABLE t1 (  
    a integer PRIMARY KEY,  
    b integer,  
    c integer,  
    FOREIGN KEY (b, c) REFERENCES master_table (c1, c2)  
);
```

Число и типы столбцов в ограничении должны соответствовать числу и типам целевых столбцов.

Дерево

Используя ограничение внешнего ключа, ссылающегося на свою же таблицу, можно организовать иерархическую структуру связей (в направлении к корню):

```
CREATE TABLE tree (  
    node_id    integer PRIMARY KEY,  
    parent_id integer REFERENCES tree,  
    name       varchar(120),  
    ...  
);
```

Для узла верхнего уровня **parent_id** будет равен **NULL**, а записи со значением **parent_id**, отличным от **NULL**, будут ссылаться только на существующие строки таблицы.

Таблица может содержать несколько ограничений внешнего ключа — это используется для связи таблиц в отношении многие-ко-многим. В частности, это можно использовать для организации дерева и даже произвольного графа.

Положим, есть таблицы продуктов и заказов, но нужно, чтобы один заказ мог содержать несколько продуктов (что невозможно в схеме LUT). Это выглядит так:

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name        varchar(120),  
    price       numeric  
);  
  
CREATE TABLE orders (  
    id            integer PRIMARY KEY,  
    shipping_address text,  
    ...  
);  
  
CREATE TABLE order_items (  
    product_no integer REFERENCES products,  
    order_id   integer REFERENCES orders,  
    quantity   integer,  
    PRIMARY KEY (product_no, order_id)  
);
```

В последней таблице первичный ключ покрывает внешние ключи.

Удаление записей, на которые ссылаются вторичные (внешние) ключи

Внешние ключи запрещают создание заказов, не относящихся ни к одному продукту.

Но что делать, если после создания заказов с определённым продуктом мы захотим удалить его?

Варианты поведения — запретить удаление продукта, удалить продукт и связанные с ним заказы, или что-то более умное, например:

- при попытке удаления продукта, на который ссылаются заказы (через таблицу **order_items**), операция запрещается;
- при попытке удалить заказ, он удаляется вместе со всем его содержимым (каскадное удаление).

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name        varchar(120),  
    price       numeric  
);  
  
CREATE TABLE orders (  
    id            integer PRIMARY KEY,  
    shipping_address text,  
    ...  
);  
  
CREATE TABLE order_items (  
    product_no integer REFERENCES products ON DELETE RESTRICT,  
    order_id    integer REFERENCES orders ON DELETE CASCADE,  
    quantity    integer,  
    PRIMARY KEY (product_no, order_id)  
);
```

Ограничивающие и каскадные удаления — два наиболее распространённых варианта.

RESTRICT предотвращает удаление связанной строки.

NO ACTION (поведение по умолчанию) означает, что если зависимые строки продолжают существовать при проверке ограничения, возникает ошибка .

Главным отличием этих двух вариантов друг от друга является то, что **NO ACTION** позволяет отложить проверку в процессе транзакции, а **RESTRICT** — нет.

CASCADE указывает, что при удалении связанных строк зависимые от них будут так же автоматически удалены.

SET NULL и **SET DEFAULT** — при удалении связанных строк они назначают зависимым столбцам в подчинённой таблице значения **NULL** или значения по умолчанию, соответственно.

Следует заметить, что это не будет основанием для нарушения ограничений — если, например, в качестве действия задано **SET DEFAULT**, но значение по умолчанию не удовлетворяет ограничению внешнего ключа, операция закончится ошибкой.

Аналогично указанию **ON DELETE** существует **ON UPDATE**, которое срабатывает при изменении заданного столбца.

При этом возможные действия те же, а **CASCADE** в данном случае означает, что изменённые значения связанных столбцов будут просто скопированы в зависимые строки.

Внешний ключ должен ссылаться на столбцы, образующие первичный ключ или ограничение уникальности. Соответственно, для связанных столбцов всегда будет существовать индекс (определённый соответствующим первичным ключом или ограничением. Это приведет к тому, что проверки соответствия связанной строки будут выполняться эффективно.

Поскольку команды **DELETE** для строк главной таблицы или **UPDATE** для зависимых столбцов требуют просканировать подчинённую таблицу и найти строки, ссылающиеся на старые значения, полезно иметь индекс и для подчинённых столбцов (индекс для первичного ключа строится неявно).

Поскольку это нужно не всегда, и создавать соответствующий индекс можно по-разному, объявление внешнего ключа не создаёт автоматически индекс по связанным столбцам.

Изменение таблиц (ALTER)

Созданную и заполненную данными таблицу можно модифицировать в отношении структуры и/или ограничений. Postgres для этой цели предоставляет набор команд модификации таблиц.

Можно:

- добавлять/удалять столбцы;
- добавлять/удалять ограничения;
- изменять значения по умолчанию;
- изменять типы столбцов;
- переименовывать столбцы;
- переименовывать таблицы.

Добавление/удаление столбца

```
ALTER TABLE table_name ADD COLUMN column_name type [ DEFAULT value ];
```

Новый столбец заполняется заданным для него значением по умолчанию (**DEFAULT**) или значением **NULL**, если ничего не будет указано в качестве **DEFAULT**.

```
ALTER TABLE products ADD COLUMN description text;
```

Можно сразу определить ограничения столбца, используя обычный синтаксис:

```
ALTER TABLE products ADD COLUMN description text CHECK (description <> '');
```

Можно использовать любые конструкции, допустимые в определении столбца в команде CREATE TABLE.

Значение по умолчанию, если есть, должно удовлетворять данным ограничениям. Поэтому сначала можно заполнить столбец и только потом добавить ограничение.

Удаление столбца

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Данные, которые были в удаляемом столбце, исчезают.

Вместе со столбцом удаляются и включающие его ограничения таблицы.

Если на столбец ссылается ограничение внешнего ключа другой таблицы, PostgreSQL не удалит это ограничение, если не предпринять дополнительных мер.

Разрешить удаление всех зависящих от этого столбца объектов можно, добавив указание **CASCADE**:

```
ALTER TABLE products DROP COLUMN description CASCADE;
```

Добавление/удаление ограничения

Для добавления ограничения используется синтаксис ограничения таблицы:

```
ALTER TABLE products ADD CHECK (name <> ' ');  
ALTER TABLE products ADD CONSTRAINT some_name UNIQUE (product_no);  
ALTER TABLE products ADD FOREIGN KEY (product_group_id) REFERENCES product_groups;
```

Ограничение **NOT NULL** нельзя записать в виде ограничения таблицы, поэтому делаем так:

```
ALTER TABLE products ALTER COLUMN product_no SET NOT NULL;
```

Ограничение проходит проверку автоматически и будет добавлено, только если ему удовлетворяют данные таблицы.

Удаление ограничения

Чтобы удалить ограничение, нужно знать его имя.

Если имя не было явно присвоено пользователем, это неявно сделала система. Поэтому нужно сначала выяснить это имя, используя, например, команду `psql`:

```
\d[S+] name -- описание таблицы, представления, последовательности или индекса.
```

Узнав имя, можно удалить ограничение:

```
ALTER TABLE products DROP CONSTRAINT constraint_name;
```

У ограничений **NOT NULL** нет имён, поэтому для его удаления следует делать так:

```
ALTER TABLE products ALTER COLUMN product_no DROP NOT NULL;
```

Изменение значения по умолчанию

Новое значение по умолчанию:

```
ALTER TABLE products ALTER COLUMN price SET DEFAULT 7.77;
```

Это не влияет на существующие строки таблицы, а задает значение по умолчанию для последующих команд **INSERT**.

Удаление значения по умолчанию:

```
ALTER TABLE products ALTER COLUMN price DROP DEFAULT;
```

При этом по сути значению по умолчанию просто присваивается **NULL**. Как следствие, ошибки не будет, если вы попытаетесь удалить значение по умолчанию, не определённое явно, так как неявно оно существует и равно **NULL**.

Изменение типа данных столбца

ALTER TABLE *table_name* ALTER COLUMN *column_name* TYPE *new_type*

Пример:

```
ALTER TABLE products ALTER COLUMN price TYPE numeric(10,2);
```

Операция будет успешна, только если все существующие значения в столбце могут быть неявно приведены к новому типу.

Если требуется более сложное преобразование, необходимо добавить указание **USING**, определяющее, как получить новые значения из старых.

Важно:

Postgres попытается преобразовать к новому типу значение столбца по умолчанию и все связанные с этим столбцом ограничения. Преобразование может оказаться некорректным, и в результате случится нехорошее. Поэтому,

перед тем как менять тип столбца, следует удалить все его ограничения, а потом заново установить ограничения, модифицированные в соответствии с новым типом.

Переименование столбца

ALTER TABLE *table_name* RENAME COLUMN *column_name* TO *new_column_name*;

```
ALTER TABLE products RENAME COLUMN product_no TO product_number;
```

Переименование таблицы

ALTER TABLE *table_name* RENAME TO *new_table_name*;

```
ALTER TABLE products RENAME TO items;
```

Базы данных

Лекция 09 – Основы SQL

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2022.10.27

Оглавление

Модификация данных.....	3
Добавление данных.....	3
Наследование.....	5
Изменение данных. Команда UPDATE.....	13
Удаление данных.....	15
Возврат данных из изменённых строк.....	16
Функции и операторы.....	18

Модификация данных

Добавление данных

Таблица после создания пуста.

Следующий шаг после создания — добавление в таблицу данных.

Данные добавляются в таблицу по одной строке — это «квант» данных в реляционной модели.

Полная строка создается даже если при добавлении будет указано только одно значение.

Для добавления строки используется команда **INSERT**.

```
CREATE TABLE products (  
    product_no integer,  
    name        varchar(120),  
    price        numeric  
);
```

Добавить в неё строку можно так:

```
INSERT INTO products VALUES (1, 'Мыло', 1.23);
```

Значения данных перечисляются в порядке столбцов в таблице и разделяются запятыми.

Обычно в качестве значений указываются константы, но это могут быть и скалярные выражения.

Показанная выше запись имеет один недостаток — вам необходимо знать порядок столбцов в таблице. Чтобы избежать этого, можно перечислить столбцы явно. Например, следующие две команды дадут тот же результат, что и показанная выше:

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Мыло', 1.23);  
INSERT INTO products (name, price, product_no) VALUES ('Мыло', 1.23, 1);
```

Если значения определяются не для всех столбцов, лишние столбцы можно опустить. В таком случае эти столбцы получают значения по умолчанию:

```
INSERT INTO products (product_no, name) VALUES (1, 'Мыло');  
INSERT INTO products VALUES (1, 'Мыло');
```

Postgres заполняет столбцы слева направо по числу переданных значений, а все остальные столбцы принимают значения по умолчанию.

Можно явно указать значения по умолчанию для отдельных столбцов или всей строки:

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Мыло', DEFAULT);  
INSERT INTO products DEFAULT VALUES;
```

Одна команда может вставить сразу несколько строк:

```
INSERT INTO products (product_no, name, price) VALUES  
    (1, 'Мыло', 1.23),  
    (2, 'Зубная паста', 2.34),  
    (3, 'Щетка', 3.45);
```

Можно вставлять результат запроса, который может не содержать строк вообще, либо содержать одну или несколько:

```
INSERT INTO products (product_no, name, price)  
    SELECT product_no, name, price FROM new_products WHERE release_date = 'today';
```

Наследование

Стандарт SQL:1999 и более поздние версии определяют возможность наследования типов.

PostgreSQL реализует наследование таблиц.

Например, имеется контора, в здании которой могут находиться сотрудники и посетители.

У тех и других есть для учета имена, но у сотрудников дополнительно есть табельные номера, а у посетителей — номера регистрационной карты (гостиница, поликлиника, ...). Классическая ситуация «типы-подтипы» и разрешается она чаще всего объектными средствами.

Объектный подход дает разработчику не только приятные моменты — одной из оборотных сторон является сложность, более высокая, чем при работе с реляционными таблицами.

Не всегда получается придумать схему данных правильно и сразу, и оставить будущему только работу с самими данными. Часто приходится вносить изменения в схему при наличии уже имеющихся данных. Тут объектный подход обнаруживает больше проблем, чем могло бы желать.

Простой пример наследования

Предположим, что мы создаём модель данных для городов.

В каждой области есть множество городов, но лишь один областной центр. Нужно иметь возможность быстро получать город-столицу региона для любой области. Это можно сделать, создав две таблицы — одну для областных центров, а другую для остальных городов:

```
CREATE TABLE cities (  
    name            text,  
    population      float,  
    elevation       int    -- в метрах  
);
```

```
CREATE TABLE capitals (  
    name            text,  
    population      float,  
    elevation       int    -- в метрах  
);
```

Если нам нужно получить информацию о любом городе, будь то областной центр или нет, у нас возникают небольшие сложности.

Здесь может помочь наследование — таблица **capitals** определяется, как наследник **cities**:

```
CREATE TABLE cities (  
    name            text,  
    population      float,  
    elevation       int    -- в метрах  
);  
  
CREATE TABLE capitals (  
    region          char(2)  
) INHERITS (cities);
```

Таблица **capitals** наследует все столбцы своей родительской таблицы **cities**.
Областные центры имеют дополнительный столбец **region**, в котором будет указан регион.
Но можно и так:

```
CREATE TABLE region (  
    region_id       integer PRIMARY KEY,  
    region          text  
);  
  
CREATE TABLE cities2 (  
    city_id         integer PRIMARY KEY,  
    capital         bool DEFAULT no,  
    name           text,  
    region         integer REFERENCES region (region_id),  
    population      float,  
    elevation       int    -- в метрах  
);
```

Консольный клиент `psql` не имеет специальных команд, генерирующих структуру таблицы на основе внутреннего представления таблицы. Это действие передано на откуп клиентским программам, которые могут на основе данных из системных таблиц построить структуру заданной таблицы.

Для решения этой задачи можно воспользоваться следующей командой:

```
SELECT column_name, column_default, data_type
FROM INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'cities';
```

column_name	column_default	data_type
population		double precision
elevation		integer
name		text

(3 строки)

```
SELECT column_name, column_default, data_type
FROM INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'cities2';
```

column_name	column_default	data_type
city_id		integer
capital	false	boolean
region		integer
population		bigint
elevation		integer
name		text

(6 строк)

В PostgreSQL таблица может наследоваться от нуля или нескольких других таблиц. Запросы при этом могут выбирать все строки родительской таблицы или все строки родительской и всех дочерних таблиц. По умолчанию принят последний вариант. Например, следующий запрос найдёт названия всех городов, включая центры регионов, расположенных выше 100 метров над уровнем моря:

```
SELECT name, elevation FROM cities WHERE elevation > 100;
```

Следующий запрос находит все города, которые не являются столицами регионов, но также находятся на высоте выше 100 метров:

```
SELECT name, elevation FROM ONLY cities WHERE elevation > 100;
```

Ключевое слово **ONLY** указывает, что запрос должен применяться только к таблице **cities**, но не к таблицам, расположенным ниже **cities** в иерархии наследования.

Ключевое слово **ONLY** поддерживается многими операторами, включая **SELECT**, **UPDATE** и **DELETE**, ограничивая действие предиката выбора.

Чтобы *явно* указать, что должны включаться и дочерние таблицы, следует после имени таблицы добавить *****:

```
SELECT name, elevation FROM cities* WHERE elevation > 500;
```

Указывать ***** не обязательно, так как это поведение подразумевается по умолчанию.

Однако такая запись поддерживается для совместимости со старыми версиями, где поведение по умолчанию могло быть изменено.

В некоторых ситуациях бывает необходимо узнать, из какой таблицы выбрана конкретная строка. Для этого можно использовать системный столбец **tableoid**, присутствующий в каждой таблице:

```
SELECT tableoid, name, elevation
       FROM cities WHERE elevation > 100;
```

запрос выдаст:

tableoid	name	elevation
12345	Пинск	123
12345	Барановичи	234
12378	Брест	345

Имена таблиц вы можете получить, обратившись к `pg_class`:

```
SELECT p.relname, c.name, c.elevation
       FROM cities c, pg_class p WHERE c.elevation > 500 AND c.tableoid = p.oid;
```

Схемы

Кластер баз данных PostgreSQL содержит один или несколько именованных экземпляров баз.

На уровне кластера создаются роли и некоторые другие объекты.

В рамках одного подключения к серверу можно обращаться к данным только одной базы — той, что была выбрана при установлении соединения.

База данных содержит одну или несколько именованных **схем**, которые, в свою очередь, содержат таблицы.

Помимо таблиц схемы содержат именованные объекты других видов, включая типы данных, функции и операторы. При этом одно и то же имя объекта можно свободно использовать в разных схемах, например и **schema1**, и **schema2** могут содержать таблицы с именем **sometable**.

- чтобы одну базу данных могли использовать несколько пользователей, не влияя друг на друга;
- чтобы объединить объекты базы данных в логические группы для облегчения управления ими;
- чтобы в одной БД существовали разные приложения, и при этом не возникало конфликтов имён.

Схемы в некотором смысле подобны каталогам в операционной системе, но они не могут быть вложенными.

Механизм наследования не способен автоматически распределять данные команд INSERT или COPY по таблицам в иерархии наследования.

Поэтому в следующем примере оператор INSERT не выполнится:

```
INSERT INTO cities (name, population, elevation, region) VALUES  
('Могилев', NULL, NULL, '06');
```

INSERT всегда вставляет данные непосредственно в указанную таблицу, а таблица **cities** не содержит столбца **region**, поэтому команда будет отвергнута.

INSERT всегда вставляет данные непосредственно в указанную таблицу

1) Дочерние таблицы автоматически наследуют от родительской таблицы ограничения-проверки и ограничения NOT NULL (если только для них не задано явно NO INHERIT).

Все остальные ограничения (уникальности, первичный ключ и внешние ключи) не наследуются.

2) Таблица может наследоваться от нескольких родительских таблиц, в этом случае она будет объединять в себе все столбцы этих таблиц, а также столбцы, описанные непосредственно в её определении.

3) Если в определениях родительских и дочерней таблиц встретятся столбцы с одним именем, эти столбцы будут «объединены», так что в дочерней таблице окажется только один столбец. Чтобы такое объединение было возможно, столбцы должны иметь одинаковый тип данных, в противном случае произойдёт ошибка.

4) Наследуемые ограничения-проверки и ограничения NOT NULL объединяются подобным образом.

5) Отношение наследования между таблицами обычно устанавливается при создании дочерней таблицы с использованием предложения INHERITS оператора CREATE TABLE.

6) Другой способ добавить такое отношение для таблицы, определённой подходящим образом, — использовать INHERIT с оператором ALTER TABLE. Для этого будущая дочерняя таблица должна уже включать те же столбцы (с совпадающими именами и типами), что и родительская таблица.

Также она должна включать аналогичные ограничения-проверки с теми же именами и выражениями.

Удалить отношение наследования можно с помощью указания NO INHERIT оператора ALTER TABLE.

7) Для создания таблицы, которая затем может стать наследником другой, удобно воспользоваться предложением **LIKE** оператора **CREATE TABLE**. Такая команда создаст новую таблицу с теми же столбцами, что имеются в исходной. Если в исходной таблицы определены ограничения CHECK, для создания полностью совместимой таблицы их тоже нужно скопировать, и это можно сделать, добавив к предложению **LIKE** параметр **INCLUDING CONSTRAINTS**.

8) Родительскую таблицу нельзя удалить, пока существуют таблицы, унаследованные от неё.

9) В дочерних таблицах нельзя удалять или модифицировать столбцы или ограничения-проверки, унаследованные от родительских таблиц.

10) Для удаления таблицы вместе со всеми её потомками необходимо добавить в команду удаления родительской таблицы параметр CASCADE.

Изменение данных. Команда UPDATE

Модификация данных, сохранённых в БД, называется изменением.

Изменить можно:

- отдельные строки;
- все строки таблицы;
- некоторое подмножество всех строк.

Каждый столбец можно изменять независимо от других.

Для изменения данных в существующих строках используется команда UPDATE.

Ей требуется следующая информация:

- имя таблицы и изменяемого столбца;
- новое значение столбца
- критерий отбора изменяемых строк.

SQL не назначает строкам уникальные идентификаторы, соответственно, в таблице можно иметь несколько полностью идентичных строк. Такое дублирование обычно нежелательно и забота о том, чтобы такого не случилось, возлагается на программиста.

Итак, не всегда возможно явно указать на строку, которую требуется изменить. Поэтому необходимо указывать условия, каким должна соответствовать изменяемая строка.

Изменение отдельных строк

Однозначно адресовать отдельные строки можно только в том случае, если в таблице есть первичный ключ. В этом случае его можно использовать для определения вышеупомянутых условий соответствия. Именно таким образом работают многие интерактивные приложения для работы с базами данных, дающие возможность редактировать данные в индивидуальных строках.

Пример:

```
UPDATE products SET price = 10 WHERE price = 5;
```

В результате в таблице может измениться ноль, одна или несколько строк.

Команда изменения данных начинается с ключевого слова **UPDATE**. За ним идёт имя таблицы **products**. Затем идёт ключевое слово **SET**, за которым следует имя столбца, который следует изменить, знак равенства и новое значение столбца (10). Этим значением может быть не только константа, но и любое скалярное выражение. Например, если нужно поднять цену всех товаров на 10%:

```
UPDATE products SET price = price * 1.10;
```

Выражение нового значения может ссылаться на существующие значения столбцов в строке.

Предложение **WHERE price = 5** устанавливает условия, которым должна соответствовать изменяемая строка. Поскольку предложение **WHERE** во втором примере отсутствует, будут изменены все строки в таблице.

Если же это предложение присутствует, изменяются только строки, которые соответствуют условию **WHERE**.

В предложении **SET** знак равенства обозначает операцию присваивания, а в предложении **WHERE** он используется для сравнения и это не приводит к неоднозначности.

В условии **WHERE** не обязательно должна быть проверка равенства — могут применяться любые другие операторы. Необходимо только, чтобы это выражение было предикатом.

Предикат — выражение логического типа, использующее одну или более величин на входе.

В команде **UPDATE** можно изменить значения сразу нескольких столбцов, для этого их нужно все перечислить в предложении **SET**.

```
UPDATE mytable SET a = 5, b = 3, c = 1 WHERE a > 0;
```

Удаление данных

Удалять данные можно только целыми строками.

Поскольку в SQL нет возможности напрямую адресовать отдельные строки, удалить избранные строки можно только сформулировав для них подходящие условия.

Если в таблице есть первичный ключ, с его помощью можно однозначно выделить для удаления определённую строку (интерактивные приложения).

Также можно удалить группы строк, соответствующие условию, либо сразу все строки таблицы. Для удаления строк используется команда DELETE.

Её синтаксис очень похож на синтаксис команды UPDATE.

Удаление всех строк из таблицы с товарами, имеющими цену 10:

```
DELETE FROM products WHERE price = 10;
```

Удаление всех строк из таблицы

```
DELETE FROM products;
```

Возврат данных из изменённых строк

Иногда бывает полезно получать данные из модифицируемых строк в процессе их обработки. Для этого используется предложение RETURNING.

Его можно задать для команд INSERT, UPDATE и DELETE.

RETURNING позволяет обойтись без дополнительного запроса к базе для выбора данных, особенно в тех случаях, когда как-то иначе трудно получить строки.

В предложении RETURNING используется то же, что и в выходном списке команды SELECT. Оно может содержать имена столбцов целевой таблицы команды или значения выражений с этими столбцами.

Часто применяется краткая запись RETURNING *, выбирающая все столбцы целевой таблицы по порядку.

В команде INSERT данные, выдаваемые в RETURNING, образуются из строки в том виде, в каком она была вставлена.

Если в таблице есть вычисляемые поля по умолчанию, RETURNING позволяет узнать их содержимое. Например, если в таблице есть столбец **serial**, в котором генерируются уникальные идентификаторы, команда RETURNING может вернуть идентификатор, назначенный новой строке:

```
CREATE TABLE users (id serial primary key, firstname text, lastname text);  
  
INSERT INTO users (firstname, lastname) VALUES ('Peter', 'The First') RETURNING id;
```

RETURNING также очень полезно с INSERT ... SELECT.

В команде UPDATE данные, выдаваемые в RETURNING, образуются *новым содержимым* изменённой строки:

```
UPDATE products SET price = price * 1.10 WHERE price <= 99.99  
RETURNING name, price AS new_price;
```

В команде DELETE данные, выдаваемые в RETURNING, образуются *содержимым* удалённой строки:

```
DELETE FROM products WHERE obsoletion_date = 'today'  
RETURNING *;
```

Если для целевой таблицы заданы триггеры, в RETURNING выдаются данные из строки, которая изменена триггерами. Поэтому RETURNING часто применяется и для того, чтобы проверить содержимое столбцов, изменяемых триггерами.

Функции и операторы

Postgres имеет огромное количество функций и операторов для встроенных типов данных.

Также пользователи могут определять свои функции и операторы. Все существующие функции и операторы в **psql** можно просмотреть с помощью команд **\df** и **\do**, соответственно.

```
==> \df pg_copy*
```

Список функций

Схема	Имя	Тип данных результата	Типы данных аргументов	Тип
-------	-----	-----------------------	------------------------	-----

...

(5 строк)

```
==> \df *
```

Список функций

Схема	Имя	Тип данных результата	Типы данных аргументов	Тип
-------	-----	-----------------------	------------------------	-----

...

(3191 строка)

```
==> \do *
```

Список операторов

Схема	Имя	Тип левого аргумента	Тип правого аргумента	Результирующий тип	Описание
-------	-----	----------------------	-----------------------	--------------------	----------

...

pg_cat	+	circle	point	circle	add
--------	---	--------	-------	--------	-----

...

(804 строки)

В документации postgres типы аргументов и результата функции обозначаются как:

foo(text, integer) -> text

Функция **foo** принимает один текстовый и один целочисленный аргумент и возвращает результат текстового типа.

Стрелка вправо также используется, чтобы указать на результат в примерах:

repeat('Pg', 4) -> PgPgPgPg

```
==> select repeat('Pg', 4);
repeat
-----
PgPgPgPg
(1 строка)
```

Важно:

1) практически все функции и операторы postgres, за исключением простейших арифметических и операторов сравнения, в стандарте SQL не описаны.

2) тем не менее они присутствуют и в других СУБД SQL, причем во многих случаях различные реализации одинаковых функций оказываются совместимыми.

Базы данных

Лекция 10 – Основы SQL

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2023.11.03

Оглавление

Запросы.....	3
Табличные выражения.....	5
Предложение FROM.....	6
Соединение таблиц.....	7
Внутреннее соединение.....	8
Внешние соединения.....	10
Псевдонимы таблиц и столбцов.....	12
Запросы типа Self Join.....	14
Подзапросы.....	15
Предложение WHERE.....	16
Сортировка строк (ORDER BY).....	17

Запросы

Запросом называется процесс или команда получения данных из базы данных.

В SQL запросы формулируются с помощью команды **SELECT**.

В общем виде команда **SELECT** записывается так:

```
[WITH with_queries]           -- расширение Postgres
SELECT select_list             -- список выбора
FROM table_expression [sort_specification] -- откуда и условия
```

Простой запрос выглядит так:

```
SELECT * FROM table;
```

Если в базе данных есть таблица **table**, эта команда выдаст все строки с содержимым всех столбцов из **table**.

Метод выдачи результата определяет приложение-клиент, например, psql выведет на экране содержимое в текстовом виде.

Если список выборки задан как *, это означает, что запрос должен вернуть все столбцы табличного выражения.

WITH — способ указать выполнение дополнительных операторов в больших запросах.

Эти операторы называют *общими табличными выражениями* (Common Table Expressions, CTE). Обычно это определения временных таблиц в рамках только одного запроса.

Дополнительным оператором в предложении **WITH** может быть **SELECT**, **INSERT**, **UPDATE**, **DELETE**, а само предложение **WITH** присоединяется к основному оператору, которым также может быть **SELECT**, **INSERT**, **UPDATE** или **DELETE**.

В списке выборки можно также указать подмножество доступных столбцов или составить выражения с этими столбцами — если в **table** есть столбцы **a**, **b** и **c**, а **b** и **c** имеют числовой тип данных, можно выполнить следующий запрос:

```
SELECT a, b + c FROM table;
```

FROM table — это простейший тип табличного выражения, в котором читается одна таблица.

Табличные выражения могут быть сложными конструкциями из базовых таблиц, соединений и подзапросов.

Также можно опустить табличное выражение и использовать команду **SELECT** как калькулятор:

```
SELECT 3 * 4;
```

Бывает полезно, когда выражения в списке выборки возвращают меняющиеся результаты:

```
SELECT random( );
```

Табличные выражения

[WITH with_queries]	-- расширение Postgres
SELECT select_list	-- список выборки
FROM table_expression [sort_specification]	-- откуда и условия

Табличное выражение вычисляет таблицу.

Это выражение содержит предложение **FROM**, за которым могут следовать необязательные предложения:

WHERE;
GROUP BY;
HAVING.

Простые табличные выражения ссылаются на «физическую» таблицу, ее иногда называют базовой, но в более сложных выражениях такие таблицы можно преобразовывать и комбинировать самыми разными способами.

Необязательные предложения **WHERE**, **GROUP BY** и **HAVING** в табличном выражении определяют последовательность преобразований, осуществляемых с данными таблицы, полученной в предложении **FROM**.

В результате этих преобразований образуется виртуальная таблица, строки которой передаются списку выборки (select_list), который вычисляет выходные строки запроса.

Предложение FROM

Предложение FROM образует таблицу из одной или нескольких ссылок на таблицы, разделённых запятыми.

```
FROM table_reference [, table_reference [, ...]]
```

Здесь ссылкой на таблицу **table_reference** может быть:

- имя таблицы (как вариант, с именем схемы);
- производная таблица, например подзапрос;
- **соединение таблиц**;
- произвольная комбинация вышеупомянутых вариантов.

Если в предложении FROM перечисляются несколько ссылок, для них применяется перекрёстное соединение (CROSS JOIN — декартово произведение их строк).

Список FROM преобразуется в промежуточную виртуальную таблицу, которая может пройти через преобразования WHERE, GROUP BY и HAVING, и в итоге определит результат табличного выражения.

Если в ссылке на таблицу указывается таблица, являющаяся родительской в иерархии наследования, в результате будут получены строки не только этой таблицы, но и всех её дочерних таблиц.

Чтобы выбрать строки только одной родительской таблицы, перед её именем нужно добавить ключевое слово ONLY. При этом будут получены только столбцы указанной таблицы — дополнительные столбцы дочерних таблиц в результат не попадут.

Чтобы явно указать, обработку всех дочерних таблиц, следует дописать после родительской *.

Соединение таблиц

Соединенная таблица — это таблица, полученная из двух других (реальных или производных от них) таблиц в соответствии с правилами соединения конкретного типа.

Общий синтаксис описания соединённой таблицы:

```
T1 join_type T2 [ join_condition ]
```

Соединения любых типов могут вкладываются друг в друга или объединяться — T1, и T2 могут быть результатами соединения.

Для однозначного определения порядка соединений предложения JOIN можно заключать в скобки. Если скобки отсутствуют, предложения JOIN обрабатываются слева направо.

Типы соединений

Перекрёстное соединение **CROSS JOIN**

```
T1 CROSS JOIN T2
```

Результирующую таблицу образуют все возможные сочетания строк из T1 и T2 (декартово произведение), а набор ее столбцов объединяет столбцы T1 со следующими за ними столбцами T2.

Если таблицы содержат N и M строк, соединённая таблица будет содержать $N * M$ строк.

Запись

```
FROM T1 CROSS JOIN T2
```

эквивалентна

```
FROM T1 INNER JOIN T2 ON TRUE — все без исключения
```


Соединения с сопоставлениями строк

Внутреннее INNER и внешнее OUTER.

Внутреннее соединение

```
T1 { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2
    ON логическое_выражение
```

```
T1 { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2
    USING ( список столбцов соединения )
```

```
T1 NATURAL { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2
```

Ключевые слова INNER (внутреннее соединение) и OUTER (внешнее соединение) не являются обязательными во всех формах.

По умолчанию подразумевается INNER, а при указании LEFT, RIGHT и FULL — внешнее соединение.

Условие соединения указывается в предложении ON или USING, либо неявно задаётся ключевым словом NATURAL.

Условие соединения определяет, какие строки двух исходных таблиц считаются «соответствующими» друг другу.

Возможные типы соединений с сопоставлениями строк:

INNER JOIN

LEFT OUTER JOIN

RIGHT OUTER JOIN

FULL OUTER JOIN

INNER JOIN — для каждой строки R1 из T1 в результирующей таблице содержится строка для каждой строки из T2, удовлетворяющей условию соединения с R1.

```
SELECT * FROM (table-1 JOIN table-2)      --  
    ON table-1.parameter=table-2.parameter -- условие соединения  
    WHERE table-1.parameter IS 'SomeData' -- фильтрация нужного нам из соединения
```

Можно, например, запрашивать гитары конкретного бренда, при этом еще и выбирая дополнительное условие в духе количества струн.

```
Guitars (  
    id          integer PRIMARY KEY,  
    Name        text,  
    brand_id    integer REFERENCES Brand, -- внешний ключ  
    strings     integer                -- кол-во струн  
)  
Brand (  
    id          integer PRIMARY KEY,  
    Name        text  
)
```

```
SELECT * FROM  
Guitars JOIN Brand  
ON Guitars.brand_id=Brand.id  
WHERE Guitars.brand_id IS 'Cort' AND strings = 7
```

Внешние соединения

LEFT OUTER JOIN — сначала выполняется внутреннее соединение (INNER JOIN). Затем в результат добавляются все строки из T1, которым не соответствуют никакие строки в T2, а вместо значений столбцов T2 вставляются NULL. В результирующей таблице всегда будет минимум одна строка для каждой строки из T1.

RIGHT OUTER JOIN — сначала выполняется внутреннее соединение (INNER JOIN). Затем в результат добавляются все строки из T2, которым не соответствуют никакие строки в T1, а вместо значений столбцов T1 вставляются NULL. Это соединение является обратным к левому (LEFT JOIN): в результирующей таблице всегда будет минимум одна строка для каждой строки из T2.

FULL OUTER JOIN — сначала выполняется внутреннее соединение.

Затем в результат добавляются все строки из T1, которым не соответствуют никакие строки в T2, а вместо значений столбцов T2 вставляются NULL.

Затем в результат включаются все строки из T2, которым не соответствуют никакие строки в T1, а вместо значений столбцов T1 вставляются NULL.

Предложение ON

Предложение ON определяет наиболее общую форму условия соединения — в нём указываются выражения логического типа (предикат) — пара строк из T1 и T2 соответствуют друг другу, если выражение ON возвращает для них **true**.

USING — это сокращённая запись условия, полезная в ситуации, если с обеих сторон соединения столбцы имеют одинаковые имена. Она принимает список общих имён столбцов через запятую и формирует условие соединения с равенством этих столбцов.

Например, запись соединения **T1** и **T2** с **USING (a, b)** формирует условие:

ON T1.a = T2.a AND T1.b = T2.b.

Из вывода **JOIN USING** исключаются избыточные столбцы, поскольку оба сопоставленных столбца выводить не нужно — они содержат одинаковые значения.

Т.е. когда **JOIN ON** выдаёт все столбцы из **T1**, а за ними все столбцы из **T2**, **JOIN USING** выводит только один столбец для каждой пары, в указанном порядке, за ними все оставшиеся столбцы из **T1** и, наконец, все оставшиеся столбцы **T2**.

NATURAL — сокращённая форма **USING**. Она образует список **USING** из всех имён столбцов, существующих в обеих входных таблицах. Как и с **USING**, эти столбцы оказываются в выходной таблице в единственном экземпляре.

Псевдонимы таблиц и столбцов

Таблицам и ссылкам на сложные таблицы в запросе можно дать временное имя, по которому к ним можно будет обращаться в рамках данного сложного запроса. Такое имя называется псевдонимом таблицы.

Псевдоним таблицы:

```
FROM table_reference [AS] alias
```

или

```
FROM table_reference alias
```

Ключевое слово **AS** является необязательным. Вместо **alias** может быть любой идентификатор.

Псевдонимы часто применяются для назначения коротких идентификаторов длинным именам таблиц с целью улучшения читаемости запросов:

```
SELECT * FROM some_very_long_table_name AS s
      JOIN another_fairly_long_name a
      ON s.id = a.num;
```

Псевдоним становится новым именем таблицы в рамках текущего запроса — **после назначения псевдонима использовать исходное имя таблицы в другом месте запроса нельзя.**

Таким образом, следующий запрос недопустим:

```
SELECT * FROM my_table AS m WHERE my_table.a > 5;    -- неправильно
```

Псевдонимы бывают необходимы, если таблица соединяется сама с собой:

```
SELECT * FROM people AS mother JOIN people AS child  
      ON mother.id = child.mother_id;
```

Псевдонимы обязательно нужно назначать подзапросам

В случае неоднозначности определения псевдонимов можно использовать скобки.

В следующем примере первый оператор назначает псевдоним **b** второму экземпляру **my_table**, а второй оператор назначает псевдоним результату соединения:

```
SELECT * FROM my_table AS a CROSS JOIN my_table AS b ...  
SELECT * FROM (my_table AS a CROSS JOIN my_table) AS b ...
```

Временные имена могут даваться не только таблицам, но и ее столбцам:

```
FROM table_reference [AS] alias ( column1 [, column2 [, ...]] )
```

Если псевдонимов столбцов оказывается меньше, чем фактически столбцов в таблице, остальные столбцы сохраняют свои исходные имена. Эта запись особенно полезна для «замкнутых» соединений (self-join) или подзапросов.

Запросы типа Self Join

Обычное внутреннее соединение, но таблица объединяется с самой собой.

```
CREATE TABLE Customers (  
  id PRIMARY KEY,  
  CustName      text, -- наименование заказчика  
  Address       text, -- адрес  
  City          text, -- город  
)
```

1	AA	Addr_A	City_A
2	BB	Addr_B	City_A
3	CC	Addr_C	City_A
4	DD	Addr_D	City_B
5	EE	Addr_E	City_B

Выборка заказчиков из одного города в порядке названия городов.

```
SELECT A.CustName AS CustName1, B.CustName AS CustName2, A.City  
FROM Customers A, Customers B  
WHERE A.CustomerID <> B.CustomerID AND A.City = B.City ORDER BY A.City;
```

AA	BB	City_A
AA	CC	City_A
BB	CC	City_A
DD	EE	City_B

Подзапросы

Подзапросы, образующие таблицы, должны заключаться в скобки и им обязательно должны назначаться псевдонимы:

```
FROM (SELECT * FROM table1) AS alias_name
```

Этот пример эквивалентен записи

```
FROM table1 AS alias_name
```

Если в подзапросе используются агрегирующие функции или группировка, возникают более сложные ситуации, которые нельзя свести к простому соединению.

Подзапросом может также быть список VALUES, которая часто используется для создания «таблицы констант»:

```
FROM (VALUES ('anne', 'smith'), ('bob', 'jones'), ('joe', 'blow'))  
      AS names(first, last)
```

Такому подзапросу тоже требуется псевдоним.

Предложение WHERE

После обработки предложения FROM каждая строка полученной виртуальной таблицы проходит проверку по *условию ограничения* через предложение WHERE:

```
WHERE search_condition
```

где **search_condition** — любое выражение, выдающее результат типа **boolean** (предикат).

Если результат условия равен **true**, эта строка остаётся в выходной таблице, а если результат равен false или NULL, отбрасывается.

В условии ограничения, как правило, используется минимум один столбец из таблицы, полученной на выходе FROM. Примеры:

```
SELECT ... FROM foo WHERE c1 > 5

SELECT ... FROM foo WHERE c1 IN (1, 2, 3)

SELECT ... FROM foo WHERE c1 IN (SELECT c1 FROM t2)

SELECT ... FROM foo WHERE c1 IN (SELECT c3 FROM t2 WHERE c2 = foo.c1 + 10)

SELECT ... FROM foo WHERE c1 BETWEEN
  (SELECT c3 FROM t2 WHERE c2 = foo.c1 + 10) AND 100

SELECT ... FROM foo WHERE EXISTS (SELECT c1 FROM t2 WHERE c2 > foo.c1)
```

foo — название таблицы, порождённой в предложении FROM. Строки, которые не соответствуют условию WHERE, исключаются из foo.

Сортировка строк (ORDER BY)

После того как запрос выдал таблицу результатов (после обработки списка выборки), ее можно отсортировать. Если сортировка не задана, строки возвращаются в неопределённом порядке.

Фактический порядок строк в этом случае будет зависеть от плана соединения и сканирования, а также от порядка данных на диске.

Определённый порядок выводимых строк гарантируется только в том случае, если явно задан этап сортировки.

Порядок сортировки определяет предложение **ORDER BY**:

```
SELECT список_выборки FROM табличное_выражение
      ORDER BY выражение_сортировки1 [ASC | DESC] [NULLS { FIRST | LAST }]
            [, выражение_сортировки2 [ASC | DESC] [NULLS { FIRST | LAST }] ...]
```

Выражениями сортировки могут быть любые выражения, допустимые в списке выборки запроса:

```
SELECT a, b FROM table1 ORDER BY a + b, c;
```

Когда указывается несколько выражений, последующие значения позволяют отсортировать строки, в которых совпали все предыдущие значения .

Каждое выражение можно дополнить ключевыми словами ASC или DESC, которые выбирают сортировку соответственно по возрастанию или убыванию. По умолчанию ASC.

При сортировке по возрастанию сначала идут меньшие значения, где понятие «меньше» определяется оператором <. Подобным образом, сортировка по возрастанию определяется оператором >.

Для определения места значений NULL можно использовать указания NULLS FIRST и NULLS LAST, которые помещают значения NULL соответственно до или после значений не NULL. По умолчанию значения NULL считаются больше любых других, то есть подразумевается NULLS FIRST для порядка DESC и NULLS LAST в противном случае.

Базы данных

Лекция 11 – Berkeley DB

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2023.11.03

Оглавление

Коротко о типах БД.....	3
Berkeley DB.....	11
Службы доступа к данным.....	13
Хэш-таблицы.....	13
В-деревья.....	13
Номера записей.....	14
Очереди.....	14
Услуги по управлению данными.....	15
Чем Berkeley DB не является.....	16
Berkeley DB не является реляционной базой данных.....	17

Коротко о типах БД

Группировку БД по типам можно выполнять на основе общих меток, которые наносят на БД их поставщики. Модные словечки:

- сетевой;
- реляционный;
- объектно-ориентированный;
- встроенный

плюс некоторое кросс-опыление — объектно-реляционный, встроенная сеть...

Понимание этих модных словечек важно – каждое из них имеет не только некоторое теоретическое обоснование, но и превратилось в практический ярлык для классификации систем, которые работают определенным образом. Все системы баз данных, независимо от модных словечек, которые к ним относятся, предоставляют несколько общих услуг. По сути, системы баз данных предоставляют две услуги — доступ к данным и управление данными

Первая услуга — доступ к данным

Доступ к данным означает:

- добавление новых данных в базу данных (вставка);
- поиск интересующих данных (поиск);
- изменение уже сохраненных данных (обновление);
- удаление данных из базы данных (удаление).

Все базы данных предоставляют эти услуги. Обычно записи базы данных имеют структуру, отличную от структур или экземпляров, поддерживаемых языками программирования, на которых пишут приложения. В результате работа с записями БД может означать:

- 1) использование операций с базой данных, таких как поиск и обновление записей;
- 2) преобразование между структурами языка программирования и типами записей базы данных в приложении.

Вторая услуга — управление данными

Управление данными сложнее, чем доступ к данным. Предоставление качественных услуг по управлению данными — сложная часть построения системы баз данных.

При выборе системы базы данных для использования в создаваемом приложении, важно убедиться, что СУБД поддерживает необходимые службы управления данными.

Услуги управления данными включают:

- 1) возможность одновременной работы нескольких пользователей с базой данных (параллелизм);
- 2) возможность мгновенного изменения нескольких записей (транзакции);
- 3) защиту от сбоев приложений и системы (восстановление).

Различные системы баз данных предлагают различные услуги по управлению данными.

Службы управления данными полностью независимы от служб доступа к данным.

Параллелизм означает, что несколько пользователей могут работать с базой данных одновременно. Поддержка параллелизма варьируется от нулевой (только однопользовательский доступ) до полной (много программ чтения и записи работают одновременно).

Транзакции — теория реляционных баз данных не требует, чтобы система поддерживала транзакции, но это требуется для большинства коммерческих реляционных систем.

Транзакции позволяют пользователям **осуществлять сразу несколько изменений**. Например, перевод средств между банковскими счетами должен быть транзакцией, поскольку остаток на одном счете уменьшается, а остаток на другом увеличивается. Обе эти операции должны происходить одновременно (атомарно).

Свойства транзакций ACID

В системах баз данных транзакции имеют четко определенные свойства:

- они **атомарны** (atomic), так что изменения происходят все сразу или не происходят вообще;
- они **непротиворечивы** (consistent), так что база данных находится в допустимом состоянии, и когда транзакция начинается и когда она заканчивается;
- обычно они **изолированы** (isolated), что означает, что любые другие пользователи в базе данных не могут вмешаться в работу транзакций, пока они выполняются;
- они **долговечны** (durable), так что если система или приложение рухнет после завершения транзакции, изменения потеряны не будут.

Эти свойства атомарности, непротиворечивости, изоляции и устойчивости известны как свойства ACID.

Поддержка транзакций зависит от базы данных.

Некоторые предлагают атомарность, не давая гарантий долговечности.

Некоторые игнорируют изолируемость, особенно в однопользовательских системах, поскольку нет необходимости изолировать других пользователей от последствий изменений, в том случае, когда этих других пользователей нет.

Восстановление

Другой важной услугой по управлению данными является восстановление.

Восстановление — это процедура, которую система выполняет при запуске.

Цель восстановления — гарантировать, что база данных находится в надлежащем состоянии и ее можно использовать. Наиболее важно это после сбоя системы или приложения, когда база данных может быть повреждена. Процесс восстановления гарантирует, что внутренняя структура базы данных исправна. Восстановление обычно означает, что все завершенные транзакции проверяются, а любые потерянные изменения повторно применяются к базе данных. В конце процесса восстановления приложения могут использовать базу данных, как если бы не было перерыва в обслуживании.

Копирование данных

Наконец, существует ряд служб управления данными, позволяющих копировать данные.

Например, большинство систем баз данных могут импортировать данные из других источников и экспортировать их для использования в другом месте.

В большинстве систем предусмотрены способы **резервного копирования баз данных и восстановления** в случае сбоя системы, который повреждает базу данных.

Многие коммерческие системы допускают «горячее» резервное копирование, так что пользователи могут создавать резервные копии баз данных во время их использования, поскольку многие приложения должны работать без перерыва и не могут быть остановлены для резервного копирования.

Другие услуги

Конкретная система базы данных может предоставлять другие услуги по управлению данными.

- браузеры, отображающие структуру и содержимое базы данных;
- инструменты, обеспечивающие соблюдение правил целостности данных, таких, как правило, согласно которому ни у одного сотрудника не может быть отрицательной зарплаты.

Тем не менее, эти службы управления данными не являются общими для всех систем.

Службы управления данными, поддерживаемые большинством поставщиков баз данных:

- параллелизм;
- восстановление;
- транзакции.

Реляционные базы данных

Реляционные базы данных основаны на такой области математики, как теория множеств.

Реляционные базы данных работают с кортежами или записями, состоящими из значений нескольких различных типов данных, включая целые числа, строки символов и другие. Операции включают поиск записей, значения которых удовлетворяют некоторым критериям, обновление записей и т.д. Практически все реляционные базы данных используют какой-либо язык структурированных запросов, например, SQL.

SQL является основным практическим преимуществом систем реляционных баз данных — вместо того, чтобы писать компьютерную программу для поиска интересующих записей, пользователь реляционной системы может просто ввести запрос в простом синтаксисе, а движок (engine) сделает всю работу. Это дает пользователям огромную гибкость — им не нужно заранее решать, какой поиск они хотят выполнить, и им не нужны дорогостоящие программисты для поиска нужных данных.

Изучение SQL требует некоторых усилий, но для большинства целей это гораздо проще, чем полноценный высокоуровневый язык программирования.

Объектно-ориентированные базы данных

Объектно-ориентированные базы данных менее распространены, чем реляционные системы, но все же распространены широко. Большинство объектно-ориентированных баз данных изначально задумывались как постоянные системы хранения, тесно связанные с конкретными языками программирования высокого уровня, такими как C++ и, позже, Java.

Большинство из них теперь поддерживают более одного языка программирования, но объектно-ориентированные системы баз данных в основном предоставляют те же абстракции классов и методов, что и объектно-ориентированные языки программирования.

Объектно-ориентированные системы позволяют приложениям работать с объектами единообразно, независимо от того, находятся ли они в памяти или на диске. Эти системы создают иллюзию того, что все объекты постоянно находятся в памяти.

Объектно-ориентированные базы данных не так широко распространены, как реляционные системы. Чтобы привлечь разработчиков, разбирающихся в реляционных системах, во многие объектно-ориентированные системы добавлена поддержка языков запросов, очень похожих на SQL.

На практике объектно-ориентированные базы данных в основном используются для постоянного хранения объектов в программах на C++ и Java.

Сетевые базы данных

«Сетевая модель» — довольно старый метод управления данными приложения и навигации по ним. Сетевые базы данных предназначены для очень быстрого обхода по указателям.

Каждая запись, хранящаяся в сетевой базе данных, может содержать указатели на другие записи. Эти указатели, как правило, являются физическими адресами, поэтому выборка записи, на которую он ссылается, означает просто чтение ее с диска по ее дисковому адресу.

Системы сетевых баз данных обычно позволяют записям содержать:

- целые числа;
- числа с плавающей запятой;
- символьные строки;
- ссылки на другие записи.

Приложение, используя ссылки, может найти интересующие записи. После извлечения какой-либо записи приложение может быстро получить любую другую запись, на которую оно ссылается.

Обход по указателям выполняется быстро, поскольку большинство сетевых систем используют в качестве указателей физические дисковые адреса и для чтения нужной записи требуется только единственный доступ к диску, в отличие от других систем, которые, чтобы найти конкретную запись, должны выполнить более одного чтения с диска.

Ключевое преимущество сетевой модели является одновременно и ее основным недостатком — с одной стороны, приложения будут работать хорошо, с другой стороны, хранение указателей по всей базе данных очень затрудняет реорганизацию базы данных.

Клиенты и серверы

У поставщиков баз данных есть два варианта архитектуры системы:

- сервер, к которому подключаются удаленные клиенты, и все управление базой данных выполняются внутри сервера;
- модуль, который напрямую подключается к приложению и выполняет все управление базой данных локально.

В любом случае разработчику приложения нужен какой-то способ связи с базой данных. Как правило, это интерфейс прикладного программирования (API), который работает в процессе или связывается с сервером для выполнения работы.

Почти все коммерческие продукты баз данных реализованы как серверы, а приложения подключаются к ним как клиенты. У серверов есть несколько особенностей, которые делают их привлекательными.

Во-первых, поскольку всеми данными управляет отдельный процесс и, возможно, на отдельной машине, легко изолировать сервер базы данных от ошибок и сбоев в приложении.

Во-вторых, поскольку некоторые продукты баз данных (в частности, реляционные механизмы) довольно велики, разделение их на отдельные серверные процессы позволяет приложениям оставаться небольшими, что требует меньше дискового пространства и памяти. Реляционные механизмы содержат код для разбора операторов SQL, их анализа и создания планов выполнения, их оптимизации и выполнения.

Наконец, хранение всех данных в одном месте и управление ими с помощью одного сервера упрощает организациям резервное копирование, защиту и настройку политик для своих баз данных.

Однако в некоторых случаях централизованное администрирование может быть недостатком. В частности, если необходимо создать приложение, использующее базу данных для хранения важной информации, то поставка (shipping) и поддержка приложения будет намного сложнее. Конечный пользователь должен установить и администрировать отдельный сервер базы данных, а программист должен поддерживать не один продукт, а два.

Berkeley DB

1) Berkeley DB — это встроенная библиотека баз данных, которая предоставляет приложениям масштабируемые, высокопроизводительные, защищенные транзакциями сервисы управления данными.

2) Berkeley DB является «**встроенной**», поскольку она напрямую связана с приложением. Она работает в том же адресном пространстве, что и приложение. В результате для операций с базой данных не требуется никаких межпроцессных взаимодействий ни по сети, ни между процессами на одном компьютере.

3) Berkeley DB предоставляет простой API вызова функций для доступа к данным и управления ими для ряда языков программирования, включая C, C++, Java, Perl, Tcl, Python и PHP.

4) Все операции с базой данных происходят внутри библиотеки. Несколько процессов или несколько потоков в одном процессе могут использовать базу данных одновременно, поскольку каждый из них использует библиотеку Berkeley DB.

5) Все низкоуровневое обслуживание, такое как блокировки, журналирование транзакций, управление общими буферами, управление памятью и т. д., библиотекой обрабатываются прозрачно.

6) Библиотека Berkeley DB чрезвычайно переносима. Она работает практически под всеми вариантами UNIX и Linux, Windows и многими встраиваемыми операционными системами реального времени.

7) Она работает как на 32-битных, так и на 64-битных системах.

8) Может быть развернута на высокопроизводительных интернет-серверах, настольных компьютерах, а также на карманных компьютерах, телеприставках, сетевых коммутаторах и в других местах.

9) Berkeley DB хорошо масштабируется. Сама библиотека базы данных довольно компактна (менее 300 килобайт памяти, занимаемой исполняемым кодом на обычных архитектурах), но при этом может использовать гигабайты памяти и терабайты дискового пространства, если дисковое оборудование это позволяет.

10) Каждый из файлов базы данных Berkeley DB может содержать до **256 терабайт данных** при условии, что базовая файловая система способна поддерживать файлы такого размера.

Приложения Berkeley DB часто используют несколько файлов базы данных и объем данных, которым может управлять приложение Berkeley DB, ограничен только ограничениями, накладываемыми операционной системой, файловой системой и физическим оборудованием.

11) Berkeley DB также поддерживает **высокий уровень параллелизма**, что позволяет тысячам пользователей одновременно работать с одними и теми же файлами базы данных.

Berkeley DB превосходит реляционные и объектно-ориентированные системы баз данных во встроженных приложениях по нескольким причинам:

1) поскольку библиотека работает в том же адресном пространстве, для операций с базой данных не требуется межпроцессное взаимодействие. Стоимость связи между процессами на одной машине или между машинами в сети намного выше, чем стоимость вызова функции.

2) поскольку Berkeley DB использует простой интерфейс вызова функций для всех операций, нет языка запросов для разбора и плана выполнения.

Службы доступа к данным

Приложения Berkeley DB могут выбирать структуру хранения, которая лучше всего подходит для приложения. Berkeley DB поддерживает:

- хеш-таблицы;
- В-деревья;
- простое хранилище на основе номеров записей;
- постоянные очереди (persistent queue).

Программисты могут создавать таблицы, используя любую из этих структур хранения, и могут смешивать операции с разными типами таблиц в одном приложении.

Хэш-таблицы

Хэш-таблицы, как правило, хороши для очень больших баз данных, которым требуется предсказуемое время поиска и обновления для записей произвольного доступа.

Хэш-таблицы позволяют пользователям спрашивать: «Существует ли этот ключ?» или получить запись с известным ключом.

Хэш-таблицы не позволяют пользователям запрашивать записи с ключами, близкими к известному ключу.

В-деревья

В-деревья лучше подходят для поиска на основе диапазона, например, когда приложению необходимо найти все записи с ключами между некоторым начальным и конечным значением.

В-деревья также лучше используют *окрестность* ссылки. Если приложение может одновременно работать с ключами, расположенными рядом друг с другом, В-деревья работают очень эффективно.

Древовидная структура хранит ключи, которые находятся рядом друг с другом в памяти, поэтому для извлечения ближайших значений обычно не требуется доступ к диску.

Номера записей

Хранение на основе номеров записей естественно для приложений, которым необходимо хранить и извлекать записи, но у которых нет простого способа генерировать собственные ключи — ключом для записи является номер записи в таблице номеров записей. Berkeley DB автоматически генерирует эти номера.

Очереди

Очереди хорошо подходят для приложений, которые создают записи, а затем должны обрабатывать эти записи в порядке их создания. Хорошим примером являются системы онлайн-покупок. Заказы могут поступать в систему в любое время, но, как правило, они должны выполняться в том порядке, в котором они были размещены.

Услуги по управлению данными

Berkeley DB предлагает важные услуги по управлению данными, включая:

- параллелизм;
- транзакции;
- восстановление.

Все эти услуги работают на всех структурах хранения.

Несколько пользователей могут работать с одной и той же базой данных одновременно.

Berkeley DB прозрачно обрабатывает блокировку, гарантируя, что два пользователя, работающие с одной и той же записью, не будут мешать друг другу.

По умолчанию библиотека обеспечивает строгую семантику транзакций ACID:

- атомарность (Atomicity);
- непротиворечивость (Consistency);
- изолированность (isolation);
- долговечность (durability).

Тем не менее, приложения могут ослаблять изоляцию, которую гарантирует система баз данных.

Несколько операций могут быть сгруппированы в одну транзакцию и могут быть зафиксированы или отменены атомарно.

Приложение при запуске может попросить Berkeley DB запустить восстановление.

Восстановление восстанавливает базу данных до согласованного состояния со всеми зафиксированными изменениями даже после сбоя. База данных гарантированно непротиворечива, и все зафиксированные изменения гарантированно будут присутствовать после завершения восстановления.

Чем Berkeley DB не является

Berkeley DB не является реляционной базой данных.

Berkeley DB не является объектно-ориентированной базой данных.

Berkeley DB не является сетевой базой данных.

Berkeley DB не является сервером базы данных.

В отличие от большинства других систем баз данных, Berkeley DB предоставляет относительно простые службы доступа к данным.

Записи в Berkeley DB представляют собой пары (ключ, значение). Berkeley DB поддерживает только несколько логических операций с записями:

- вставить запись в таблицу;
- удалить запись из таблицы;
- найти запись в таблице, поискав ее ключ;
- обновить уже найденную запись.

Berkeley DB никогда не работает со значением записи.

Значения — это просто полезная нагрузка, которая хранится вместе с ключами и надежно доставляется обратно в приложение по запросу.

И ключи, и значения могут быть произвольными строками байтов фиксированной или переменной длины.

В результате программисты могут помещать структуры данных на их любимом языке программирования в базу данных, не преобразовывая их сначала в формат чужой записи.

Хранение и извлечение очень просты, но **приложению необходимо заранее знать структуру ключа и структуру значения** — приложение не может запросить эту информацию у Berkeley DB, потому что Berkeley DB ее не знает.

Ключи, и значения могут иметь длину до четырех гигабайт, поэтому в одной записи могут храниться изображения, аудиопотоки или другие большие значения данных.

Berkeley DB не является реляционной базой данных.

Доступ к данным, хранящимся в Berkeley DB, осуществляется с помощью традиционных API-интерфейсов Berkeley DB.

Традиционные API-интерфейсы Berkeley DB — это способ, которым большинство пользователей Berkeley DB будут использовать Berkeley DB.

Хотя интерфейсы довольно просты, они нестандартны в том смысле, что не поддерживают операторы SQL.

Поддержка SQL — это палка о двух концах.

Одним из больших преимуществ реляционных баз данных является то, что они позволяют пользователям писать *простые декларативные запросы на языке высокого уровня*. Система базы данных знает все о данных и может выполнить команду. Соответственно, не требуется программирование, а также легко искать данные новыми способами и задавать новые вопросы базе данных.

С другой стороны, если программист может заранее предсказать, как приложение будет обращаться к данным, то написание низкоуровневой программы для получения и хранения записей может быть быстрее. Это устраняет накладные расходы на синтаксический анализ, оптимизацию и выполнение запросов. Программист должен понимать представление данных и должен написать код для выполнения работы, но как только это будет сделано, приложение может работать очень быстро.

В Berkeley DB нет понятия схемы и типов данных, как в реляционных системах. Схема — это структура записей в таблицах и отношения между таблицами в базе данных. Например, в реляционной системе программист может создать запись из фиксированного набора типов данных. Поскольку типы записей объявляются в системе, реляционная машина может проникать внутрь записей и проверять в них отдельные значения. Кроме того, программисты могут использовать SQL для объявления связей между таблицами и для создания индексов для таблиц. Реляционные механизмы обычно поддерживают эти связи и индексы автоматически.

В Berkeley DB ключ и значение в записи непрозрачны для Berkeley DB.

Они могут иметь сложную внутреннюю структуру, но библиотека об этом не знает. В результате Berkeley DB не может разложить часть значения записи на составные части и не может использовать эти части для поиска интересующих значений. Это может сделать только приложение, которое знает структуру данных.

Berkeley DB поддерживает индексы для таблиц и автоматически поддерживает эти индексы по мере изменения связанных с ними таблиц.

Berkeley DB не является реляционной системой. Системы реляционных баз данных семантически богаты и предлагают высокоуровневый доступ к базе данных. По сравнению с такими системами Berkeley DB представляет собой высокопроизводительную транзакционную библиотеку для хранения записей.

Поверх Berkeley DB можно построить реляционную систему.

Фактически, популярная реляционная система MySQL использует Berkeley DB для управления таблицами с защитой транзакций и берет на себя весь анализ и выполнение SQL. Он использует Berkeley DB для уровня хранения и предоставляет инструменты семантики и доступа.

Базы данных

Лекция 12 – Berkeley DB

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2023.11.10

Оглавление

Где взять BDB?.....	3
Методы доступа.....	5
Выбор методов доступа.....	7
Выбор между BTree и Hash.....	8
Выбор между Queue и Respo.....	9
Окружение (environment).....	10
Возвращение ошибок.....	12
Базы данных.....	13
Открытие базы данных.....	13
Закрытие баз данных.....	15
Флаги открытия базы данных.....	16
Функции сообщения об ошибках.....	17
Управление базами данных в окружениях.....	20
Пример базы данных.....	24

Что такое база данных Berkeley DB?

Это то же самое, что таблица реляционной базы данных (SQL)?

ORACLE: Да, концептуально база данных Berkeley DB представляет собой отдельную таблицу реляционной базы данных. Также вы можете рассматривать каждую пару ключ/значение как одну строку в таблице, где столбцы представляют собой данные, закодированные приложением либо в ключе, либо в значении.

«Starting with release 12.1.6.0.x, we have moved under the standard AGPL license.»

Стандартная общественная лицензия GNU Affero – модифицированная версия обычной GNU GPL версии 3. В ней добавлено одно требование: если вы выполняете измененную программу на сервере и даете другим пользователям общаться на нем с этой программой, ваш сервер должен также позволять им получить исходный текст, соответствующий той модифицированной версии программы, которая выполняется на сервере.

Где взять BDB?

1) В дистрибутивах Linux (5.3.28).

libdb	The Berkeley DB database library for C
libdb-cxx	The Berkeley DB database library for C++
libdb-utils	Command line tools for managing Berkeley DB databases
libdb-devel	C development files for the Berkeley DB library
libdb-cxx-devel	C++ development files for the Berkeley DB library
libdb-devel-static	
libdb-devel-doc	
libdb-sql	Development files for using the Berkeley DB with sql
libdb-tcl	The Berkeley DB database library for C

2) Собрать из исходников (<https://edelivery.oracle.com/akam/otn/berkeley-db>)

401 Oracle Export Error

In compliance with U.S. and applicable Export laws we are unable to process your request to <https://edelivery.oracle.com/akam/otn/berkeley-db/db-18.1.32.tar.gz>. Please contact RPLS-Ops_ww@oracle.com if you believe you are receiving this notice in error.

`ftp://student@lsi.bas-net.by/software/BerkeleyDB`

`db-18.1.32.tar.g`

`db-18.1.40.tar.gz` – Ошибка при установке. Решение: скопировать из 32 недостающее в дистрибутив.

Устанавливается по умолчанию в `/usr/local/BerkeleyDB.18.1`

`./bin`

`./docs`

`./include`

`./lib`

Соответственно, нужно самостоятельно прописывать пути к библиотекам и утилитам при использовании BDB.

Методы доступа

1) Приложения Berkeley DB могут выбирать структуру хранения, которая лучше всего подходит для приложения. Berkeley DB поддерживает:

- хеш-таблицы;
- B-деревья;
- простое хранилище на основе номеров записей;
- постоянные очереди (persistent queue).

Программисты могут создавать таблицы, используя любую из этих структур хранения, и могут смешивать операции с разными типами таблиц в одном приложении.

2) Записи в Berkeley DB представляют собой пары (ключ, значение).

3) Berkeley DB никогда не работает со значением записи. Значения — это просто полезная нагрузка, которая хранится вместе с ключами и надежно доставляется обратно в приложение по запросу.

4) И ключи, и значения могут быть произвольными строками байтов фиксированной или переменной длины. Соответственно, программисты могут помещать в базу структуры данных на любом языке программирования, не занимаясь преобразованием их в/из записей чужого формата.

5) Приложению необходимо заранее знать структуру ключа и структуру значения — приложение не может запросить эту информацию у Berkeley DB, потому что Berkeley DB ее не знает.

6) Ключи, и значения могут иметь длину до четырех гигабайт (32 бита).

Метод доступа можно выбрать только при создании базы данных. После этого выбора фактическое использование API, как правило, идентично для всех методов доступа. То есть, хотя существуют некоторые исключения, в принципе, взаимодействие с библиотекой одинаково, независимо от того, какой метод доступа выбран.

Метод доступа, который предстоит выбрать, зависит, во-первых, от того, что необходимо использовать в качестве ключа, а во-вторых, от производительности, которая обеспечивается для данного метода доступа.

Btree

Данные хранятся в отсортированной сбалансированной древовидной структуре.

И ключ, и данные для записей BTree могут быть произвольно сложными.

Они могут содержать отдельные значения, такие как целое число или строка, или сложные типы, такие как структура. Кроме того есть возможность (это не поведение по умолчанию) двум записям использовать ключи, сравниваемые как равные.

Когда это происходит, записи считаются дубликатами друг друга.

Hash

Данные хранятся в расширенной линейной хеш-таблице.

Как и в случае с BTree, ключ и данные, используемые для хеш-записей, могут представлять собой произвольно сложные данные. Также, как и в BTree, опционально поддерживаются дублирующиеся записи.

Queue

Данные хранятся в очереди в виде записей фиксированной длины.

Каждая запись в качестве своего ключа использует логический номер записи. Этот метод доступа предназначен для быстрой вставки в хвост очереди и имеет специальную операцию, удаляющую и возвращающую запись из головы очереди.

Этот метод доступа необычен тем, что предусматривает блокировку на уровне записи. Это может улучшить производительность приложений, которым требуется одновременный доступ к очереди.

Recno

Данные хранятся в записях фиксированной или переменной длины.

Как и в **Queue**, в записях типа **Recno** в качестве ключей используются номера логических записей.

Выбор методов доступа

Чтобы выбрать метод доступа, необходимо сначала решить, что использовать в качестве ключа для записей базы данных.

1) если необходимо использовать произвольные данные (включая строки), следует использовать **BTree** или **Hash**.

2) если нужно использовать логические номера записей (по существу, это целые числа), следует использовать **Queue** или **Recno**.

После этого нужно выбрать между **BTree** или **Hash**, **Queue** или **Recno**.

Выбор между BTree и Hash

Для небольших рабочих наборов данных, которые полностью помещаются в память, никакой разницы между BTree и Hash нет — оба будут работать одинаково хорошо.

В такой ситуации рекомендуется использовать BTree, хотя бы по той причине, что большинство приложений БД используют именно BTree.

Основной отправной точкой выбора является именно *рабочий набор данных*, а не весь набор данных. Многие приложения поддерживают большие объемы информации, но им требуется доступ только к небольшой части этих данных. Поэтому нужно принимать во внимание те данные, которые будут регулярно использоваться, а не общую массу всех данных, которыми управляет приложение.

Объемы данных имеют тенденцию неуправляемо расти и в конце концов могут не уместиться в памяти целиком, поэтому нужно быть более осторожным при выборе метода доступа.

В частности, имеет смысл выбирать:

BTree, если ключи имеют определенную ссылочную локальность.

То есть, если предполагается их сортировка и можно ожидать, что запрос для данного ключа, вероятно, будет сопровождаться запросом для одного из его соседей.

Hash, если набор данных очень большой.

Для любого заданного метода доступа БД должна поддерживать определенный объем внутренней информации. Однако объем информации, которую БД должна хранить для **BTree**, намного больше, чем для **Hash**. В результате по мере роста набора данных эта внутренняя информация может доминировать в кэше до такой степени, что для данных приложения остается относительно мало места.

В результате **BTree** может быть вынужден выполнять дисковый ввод-вывод гораздо чаще, чем **Hash** при том же объеме данных.

Если ваш набор данных станет настолько большим, что БД почти наверняка придется выполнять дисковый ввод-вывод для удовлетворения случайного запроса, тогда **Hash** по производительности превзойдет **BTree**, потому что у него меньше внутренних записей для обеспечения поиска.

Выбор между Queue и Respo

Queue или **Respo** используются, когда в качестве первичного ключа базы данных приложение хочет использовать номера логических записей.

Логические номера записей — это, по существу, целые числа, которые однозначно идентифицируют запись базы данных. Они могут быть как изменяемыми, так и фиксированными.

Изменяемый номер записи — это номер, который может меняться при сохранении или удалении записей базы данных.

Фиксированные номера логических записей никогда не меняются независимо от выполняемых операций с базой данных.

При выборе между **Queue** и **Respo** следует иметь ввиду:

Queue, если приложение требует высокой степени параллелизма.

Очередь обеспечивает блокировку на уровне записи (в отличие от блокировки на уровне страницы, которую используют другие методы доступа), и это может привести к значительному увеличению производительности для приложений с высокой степенью параллелизма.

Однако, **Queue** поддерживает только записи фиксированной длины. Поэтому, если размер данных, которые необходимо хранить, сильно различается от записи к записи, скорее всего лучше выбрать метод доступа, отличный от **Queue**.

Respo, если нужны изменяемые номера записей.

Окружение (environment)

Окружения обычно не используются приложениями, работающими во встроенных средах, где важен каждый байт. Однако, если приложение требует чего-либо, кроме минимальной функциональности, без них не обойтись.

Окружение — это, по сути, инкапсуляция одной или нескольких баз данных. Сначала открывается окружение, а затем базы данных в этом окружении. При этом базы данных создаются/располагаются в локациях относительно домашнего каталога.

Окружения предлагают очень много функций, которые не может предложить автономная база данных BDB:

1) Файлы с несколькими базами данных

В BDB возможно содержать несколько баз данных в одном физическом файле на диске.

Это желательно для тех приложений, которые открывают несколько баз данных. Однако для того, чтобы в одном физическом файле содержалось более одной базы данных, приложение должно использовать *окружение*.

Поддержка многопоточности и многопроцессности

При использовании окружения такие ресурсы, как кэши в памяти и блокировки, могут совместно использоваться всеми базами данных, открытыми в данном окружении.

Окружение дает возможность использовать подсистемы, предназначенные для предоставления нескольким потокам и/или процессам доступа к базам данных BDB.

Например, можно использовать окружения, чтобы предоставить возможность реализовать параллельное хранилище данных (CDS — Concurrent Data Store), подсистемы блокировок и/или пулы буферов общей памяти.

2) Транзакционная обработка

BDB предлагает транзакционную подсистему, которая обеспечивает полную ACID-защиту базы данных при записи. Для включения транзакционной подсистемы и получения идентификаторов транзакций требуется использовать окружения.

3) Поддержка высокой доступности (репликации)

BDB предлагает подсистему репликации, которая обеспечивает репликацию базы данных с одним мастер-набором данных с несколькими копиями реплицируемых данных только для чтения. Чтобы подключить эту подсистему и впоследствии ею управлять, требуется использовать окружения.

Подсистема протоколирования (регистрации)

BDB предлагает ведение журнала с опережающей записью тем приложениям, которые хотят получить высокую степень восстанавливаемости в случае сбоя как приложения, так и системы.

После подключения подсистема ведения журнала позволяет приложению выполнять два вида восстановления, «обычное» и «катастрофическое», с использованием информации, содержащейся в файлах журнала.

Возвращение ошибок

- 1) Интерфейсы БД всегда возвращают значение 0 в случае успеха.
- 2) Если операция по какой-либо причине не удалась, возвращаемое значение будет ненулевым.
- 3) Если произошла системная ошибка (например, в БД закончилось место на диске, или было отказано в доступе к файлу, или для одного из интерфейсов был указан недопустимый аргумент), БД возвращает значение **errno**.

Все возможные значения **errno** больше 0.

- 4) Если операция завершилась не из-за системной ошибки, но и не была успешной, БД возвращает специальное значение ошибки.

Например, при попытке получить данные из базы, а искомая запись не существует, DB вернет **DB_NOTFOUND**, специальное значение ошибки, означающее, что запрошенный ключ не отображается (map) в базе данных.

Все возможные значения таких специальных ошибок меньше 0.

DB предлагает программную поддержку для отображения возвращаемых значений ошибок.

- 1) Функция **db_strerror()** возвращает указатель на сообщение об ошибке, соответствующее возвращенному коду ошибки БД, аналогично C-функции **strerror()**. Она может обрабатывать кроме возвращаемых значений, специфичных для БД, также возвраты и системных ошибок.

- 2) Есть две функции ошибок: **DB->err()** и **DB->errx()**. Эти функции работают так же, как C-функция **printf()**, принимая строку формата в стиле **printf()** и список аргументов. Также она может добавить стандартную строку ошибки к сообщению, составленному из строки формата и других аргументов.

Документация не содержит man'ов.

Базы данных

В Berkeley DB база данных представляет собой набор записей.

Записи, в свою очередь, состоят из пар ключ/данные.

Концептуально базу данных можно представить как содержащую таблицу с двумя столбцами, где столбец 1 содержит ключ, а столбец 2 содержит данные.

И ключ, и данные управляются с помощью структур **DBT** (db.h).

Использование базы данных БД включает в себя:

- добавление;
 - получение;
 - удаление
- записей базы данных.

Открытие базы данных

Чтобы открыть базу данных, сначала необходимо использовать функцию **db_create()**, которая инициализирует дескриптор БД.

Для открытия базы данных после получения дескриптора нужно использовать его метод **open()**.

По умолчанию, если базы данных еще не существуют, они не создаются. Это поведение можно переопределить, указав флаг **DB_CREATE** в методе¹ **open()**.

```
int db_create(DB **, DB_ENV *, u_int32_t);
```

1) >120 методов

Пример:

```
#include <db.h>

...

DB      *dbp;    /* DB structure handle */
u_int32_t  flags; /* database open flags */
int       ret;   /* function return value */

// Initialize the structure. This database is not opened in an environment,
// so the environment pointer is NULL.
ret = db_create(&dbp, NULL, 0);
if (ret != 0) {
    /* Error handling goes here */
}

/* Database open flags */
flags = DB_CREATE;    // If the database does not exist, create it

ret = dbp->open(dbp,          // DB structure pointer
               NULL,         // Transaction pointer
               "foo.db",     // On-disk file that holds the database
               NULL,         // Optional logical database name
               DB_BTREE,     // Database access method
               flags,        // Open flags
               0);           // File mode (using defaults)
if (ret != 0) {
    /* Error handling goes here */
}
```

Заккрытие баз данных

Для закрытия используется метод **DB->close()**.

Прежде чем закрыть базу данных, всегда необходимо убедиться, что все обращения к базе данных завершены.

Закрытие базы данных делает ее непригодной для использования до тех пор, пока она не будет открыта снова. Перед закрытием базы данных рекомендуется закрыть все открытые *курсоры*.

Курсоры — это механизм, с помощью которого можно перебирать записи в базе данных.

С их использованием можно получать, добавлять и удалять записи базы данных.

Если база данных допускает дублирование записей, то курсоры — это самый простой способ получить доступ ко всем записям с данным ключом, кроме первой.

Активные курсоры во время закрытия базы данных могут привести к неожиданным результатам, особенно если какой-либо из этих курсоров выполняет запись в базу данных.

Когда закрывается последний открытый дескриптор базы данных, по умолчанию ее кэш сбрасывается на диск. Это означает, что любая информация, которая была в кэше изменена, при закрытии последнего дескриптора гарантированно будет записана на диск.

Эту операцию можно выполнить вручную с помощью метода **DB->sync()**, но для обычных операций завершения работы в этом нет необходимости.

Пример:

```
#include <db.h>

...
if (dbp != NULL) {
    dbp->close(dbp, 0);
}
```

Флаги открытия базы данных

Флаги приведены здесь не все, а только используемые в однопоточных приложениях.

Чтобы при вызове **DB->open()** указать более одного флага , нужно использовать побитовое ИЛИ:

```
u_int32_t open_flags = DB_CREATE | DB_EXCL;
```

DB_CREATE

Если база данных в настоящее время не существует, она будет создана.

По умолчанию попытка открытия несуществующей базы данных завершается ошибкой.

DB_EXCL

Создание базы данных для использования в эксклюзивном режиме.

Если база данных уже существует, произойдет сбой. Этот флаг имеет смысл только при использовании вместе с **DB_CREATE**, чтобы гарантированно создать новую базу данных.

DB_RDONLY

База данных открывается только для операций чтения.

Любая последующая операция записи в базу данных вызывает сбой.

DB_TRUNCATE

Файл на диске, содержащий базу данных, будет физически обрезан (очищен).

Будут удалены все базы данных, физически содержащиеся в файле.

Функции сообщения об ошибках

БД предлагает несколько полезных методов.

```
set_errcall( )
```

Определяет функцию, которая вызывается, когда БД выдает сообщение об ошибке.

Префикс ошибки и сообщение передаются этому обратному вызову.

Приложение должно самостоятельно отображать эту информацию.

```
set_errfile( )
```

Устанавливает **FILE *** (стандартная C-библиотека) для отображения сообщений об ошибках, выдаваемых библиотекой DB.

```
set_errpfx( )
```

Задаёт префикс, используемый для любых сообщений об ошибках, выдаваемых библиотекой БД.

```
err( )
```

Выдает сообщение об ошибке. Сообщение об ошибке отправляется функции обратного вызова, определенной вызовом **set_errcall()**. Если данный вызов не использовался, то сообщение об ошибке отправляется в файл, определенный функцией **set_errfile()**. Если ни один из этих методов не использовался, то сообщение об ошибке отправляется в **stderr**.

Сообщение об ошибке состоит из строки префикса, определенного вызовом **set_errpfx()**, необязательного сообщения, отформатированного в стиле **printf()**, сообщения об ошибке и символа новой строки.

```
errx( )
```

Ведет себя идентично **err()**, за исключением того, что текст сообщения, связанный со значением ошибки, к строке ошибки не добавляется.

Помимо этих для информационных сообщений существуют и другие вызовы.

```
set_msgcall( )  
set_msgfile( )  
set_msgpfx( )  
msg( )
```

Также можно использовать функцию **db_strerror()** для прямого возврата строки ошибки, соответствующей конкретному номеру ошибки.

Чтобы отправить все сообщения об ошибках данного дескриптора базы данных в функцию обратного вызова для специфической обработки, сначала следует создать функцию обратного вызова:

```
void error_handler(const DB_ENV *dbenv,  
                  const char   *error_prefix,  
                  const char   *msg) {  
    ... // code to handle the error prefix and error message.  
}
```

Затем ее регистрируем:

```
#include <db.h>
#include <stdio.h>

DB *dbp;
int ret;

ret = db_create(&dbp, NULL, 0);
if (ret != 0) {
    fprintf(stderr, "%s: %s\n", "my_program", db_strerror(ret));
    return(ret);
}

dbp->set_errcall(dbp, error_handler);    // регистрация ф-ции обратного вызова
dbp->set_errpfx(dbp, "example_program"); // префикс
```

Выдаем сообщение об ошибке открытия :

```
ret = dbp->open(dbp,          // DB structure pointer
               NULL,         // Transaction pointer
               "mydb.db",     // On-disk file that holds the database
               NULL,         // Optional logical database name
               DB_BTREE,      // Database access method
               DB_CREATE,     // Open flags
               0);           // File mode (using defaults)

if (ret != 0) {
    dbp->err(dbp, ret, "Database open failed: %s", "mydb.db");
    return(ret);
}
```


Управление базами данных в окружениях

Окружения часто используются для широкого класса приложений БД.

Окружение — это инкапсуляция одной или нескольких баз данных.

Окружения предлагают очень много функций, которые не может предложить автономная база данных БД:

- файлы с несколькими базами данных;
- поддержка многопоточности и многопроцессности;
- транзакционная обработка;
- поддержка высокой доступности (репликация);
- подсистема протоколирования (регистрации).

Чтобы использовать окружение, необходимо сначала создать дескриптор окружения, а затем его открыть.

При открытии необходимо определить каталог, в котором находится окружение.

Этот каталог должен существовать до открытия.

Во время открытия можно определить некоторые свойства окружения, например, можно ли создать окружение, если оно еще не существует.

Также при открытии окружения необходимо инициализировать в памяти кэш.

Пример

```
#include <db.h>

...
DB_ENV      *myEnv;          /* структура-дескриптор окружения */
DB          *dbp;           /* структура-дескриптор базы данных */
u_int32_t   db_flags;       /* флаги открытия базы данных */
u_int32_t   env_flags;      /* флаги открытия окружения */
int         ret;            /* статус выполнения функции */

// Создаем дескриптор объекта окружения и инициализируем его.
ret = db_env_create(&myEnv, 0);
if (ret != 0) {
    fprintf(stderr, "Error creating env handle: %s\n", db_strerror(ret));
    return -1;
}

// Флаги окружения
env_flags = DB_CREATE |      // If the environment does not exist, create it.
            DB_INIT_MPOOL; // Initialize the in-memory cache.

// Открываем окружение
ret = myEnv->open(myEnv,          /* DB_ENV ptr */
                 "/export1/testEnv", /* env home directory */
                 env_flags,       /* Open flags */
                 0);              /* File mode (default) */

if (ret != 0) {
    fprintf(stderr, "Environment open failed: %s", db_strerror(ret));
    return -1;
}
```

Когда окружение открыто, можно открывать в нем базы данных.

По умолчанию базы данных хранятся в домашнем каталоге окружения или, если указан какой-либо путь в имени файла базы данных, относительно этого каталога:

```
// Инициализируем DB структуру.
// Передаем указатель на окружение, где будет открываться эта DB.
ret = db_create(&dbp, myEnv, 0);
if (ret != 0) {
    /* Error handling goes here */
}

// Database open flags
db_flags = DB_CREATE;          /* If the database does not exist, create it.*/

// open the database
ret = dbp->open(dbp,            /* DB structure pointer */
               NULL,           /* Transaction pointer */
               "my_db.db",     /* On-disk file that holds the database. */
               NULL,          /* Optional logical database name */
               DB_BTREE,       /* Database access method */
               db_flags,       /* Open flags */
               0);             /* File mode (using defaults) */
if (ret != 0) {
    /* Error handling goes here */
}
```

После завершения работы с окружением, его необходимо закрыть.

Перед закрытием рекомендуется закрыть все открытые базы данных.

```
if (dbp != NULL) {  
    dbp->close(dbp, 0);  
}  
  
if (myEnv != NULL) {  
    myEnv->close(myEnv, 0);  
}
```

Пример базы данных

Создается несколько приложений, загружающих и извлекающих данные инвентаризации из баз данных BDB.

Пример 2.1 Структура stock_db

Сначала создается структура, которая будет использоваться для хранения всех указателей и имен баз данных:

```
/* File: gettingstarted_common.h */
#include <db.h>

typedef struct stock_dbs {
    DB    *inventory_dbp;    /* Database containing inventory information */
    DB    *vendor_dbp;       /* Database containing vendor information */

    char *db_home_dir;       /* Directory containing the database files */
    char *inventory_db_name; /* Name of the inventory database */
    char *vendor_db_name;    /* Name of the vendor database */
} STOCK_DBS;

/* Function prototypes */
int databases_setup(STOCK_DBS *, const char *, FILE *);
int databases_close(STOCK_DBS *);
void initialize_stockdbs(STOCK_DBS *);
int open_database(DB **, const char *, const char *, FILE *);
void set_db_filenames(STOCK_DBS *my_stock);
```

Пример 2.2. Вспомогательные функции `stock_db`

Нужны некоторые служебные функции, которые используются, чтобы убедиться, что структура `stock_db` находится в нормальном состоянии перед ее использованием.

Одна из функций представляет собой простую функцию, которая инициализирует все указатели структуры полезным значением по умолчанию.

Вторая используется для указания общего пути ко всем именам баз данных, чтобы можно было явно указать, где должны находиться все файлы базы данных.

```
/* File: gettingstarted_common.c */
#include "gettingstarted_common.h"

/* Initializes the STOCK_DBS struct.*/
void initialize_stockdbs(STOCK_DBS *my_stock) {

    my_stock->db_home_dir      = DEFAULT_HOMEDIR;
    my_stock->inventory_dbp     = NULL;
    my_stock->vendor_dbp       = NULL;

    my_stock->inventory_db_name = NULL;
    my_stock->vendor_db_name    = NULL;
}
```

```
/* Identify all the files that will hold our databases. */
void set_db_filenames(STOCK_DBS *my_stock) {

    size_t size;

    /* Create the Inventory DB file name */
    size = strlen(my_stock->db_home_dir) + strlen(INVENTORYDB) + 1;
    my_stock->inventory_db_name = malloc(size);
    snprintf(my_stock->inventory_db_name,
              size, "%s%s",
              my_stock->db_home_dir,
              INVENTORYDB);

    /* Create the Vendor DB file name */
    size = strlen(my_stock->db_home_dir) + strlen(VENDORDB) + 1;
    my_stock->vendor_db_name = malloc(size);
    snprintf(my_stock->vendor_db_name,
              size,
              "%s%s",
              my_stock->db_home_dir,
              VENDORDB);
}
```

Пример 2.3 Функция open_database()

Открывается несколько баз данных с одинаковыми флагами и настройками отчетов об ошибках. Для этого создается функция, которая выполняет эту операцию:

```
/* Opens a database */
int open_database(DB **dbpp,          // The DB handle that we are opening
                  const char *file_name, // The file in which the db lives
                  const char *program_name, // Name of the program calling this func
                  FILE *error_file_pointer) // File where we want error messages sent
{
    DB *dbp;    /* For convenience */
    u_int32_t open_flags;
    int      ret;

    /* Initialize the DB handle */
    ret = db_create(&dbp, NULL, 0);
    if (ret != 0) {
        fprintf(error_file_pointer, "%s: %s\n", program_name,
                db_strerror(ret));
        return(ret);
    }

    /* Point to the memory malloc'd by db_create() */
    *dbpp = dbp;

    /* Set up error handling for this database */
    dbp->set_errfile(dbp, error_file_pointer);
    dbp->set_errpfx(dbp, program_name);
}
```



```

/* Set the open flags */
open_flags = DB_CREATE;

/* Now open the database */
ret = dbp->open(dbp,          /* Pointer to the database */
               NULL,         /* Txn pointer */
               file_name,    /* File name */
               NULL,         /* Logical db name (unneeded) */
               DB_BTREE,     /* Database type (using btree) */
               open_flags,   /* Open flags */
               0);           /* File mode. Using defaults */
if (ret != 0) {
    dbp->err(dbp, ret, "Database '%s' open failed.", file_name);
    return(ret);
}

return (0);
}

```

Example 2.4 The `databases_setup()` Function

С использованием `open_database( )` создается функция, которая откроет все базы данных.

```
/* opens all databases */
int databases_setup(STOCK_DBS *my_stock,
                    const char *program_name,
                    FILE *error_file_pointer) {
    int ret;

    /* Open the vendor database */
    ret = open_database(&(my_stock->vendor_dbp),
                       my_stock->vendor_db_name,
                       program_name, error_file_pointer);
    if (ret != 0) // обработка ошибок в open_database() => просто возврат кода ошибки
        return (ret);

    /* Open the inventory database */
    ret = open_database(&(my_stock->inventory_dbp),
                       my_stock->inventory_db_name,
                       program_name, error_file_pointer);

    if (ret != 0)
        return (ret);

    printf("databases opened successfully\n");
    return (0);
}
```

Функция `databases_close()`

Полезно иметь функцию, которая может закрыть для все базы данных:

```
/* Closes all the databases. */
int databases_close(STOCK_DBS *my_stock) {
    int ret;
    // Closing a database automatically flushes its cached data
    // to disk, so no sync is required here.

    if (my_stock->inventory_dbp != NULL) {
        ret = my_stock->inventory_dbp->close(my_stock->inventory_dbp, 0);
        if (ret != 0)
            fprintf(stderr, "Inventory DB close failed: %s\n", db_strerror(ret));
    }

    if (my_stock->vendor_dbp != NULL) {
        ret = my_stock->vendor_dbp->close(my_stock->vendor_dbp, 0);
        if (ret != 0)
            fprintf(stderr, "Vendor DB close failed: %s\n", db_strerror(ret));
    }

    printf("databases closed.\n");
    return (0);
}
```

Базы данных

Лекция 13 – Berkeley DB

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2023.11.17

Оглавление

Записи базы данных.....	3
Использование записей базы данных.....	3
Чтение и запись записей базы данных.....	11
Запись в базу данных.....	12
Получение записей из базы данных.....	17
Удаление записей.....	20
Сохранность данных.....	22
Использование С-структур.....	25
Альтернативный подход – С-структуры с указателями.....	29
IFF – подход и использование пластичных массивов.....	34
Использование курсоров.....	35
Открытие и закрытие курсоров.....	36
Получение записей с помощью курсора.....	38
Поиск записей.....	43

Записи базы данных

Записи БД состоят из двух частей — ключа и некоторых данных.

И ключ, и соответствующие ему данные инкапсулируются в структуры типа DBT.

Поэтому для доступа к записи БД нужны две структуры — одна для ключа и другая для данных.

Структуры DBT предоставляют поле **void ***, которое используется для ссылки на данные, и поле, определяющее длину данных. Поэтому их можно использовать для хранения чего угодно, от простых примитивных данных до сложных структур, если информация, которую необходимо охранить, находится в одном непрерывном блоке памяти.

Использование записей базы данных

Каждая запись базы данных состоит из двух структур DBT — одна для ключа, а другая для данных.

Чтобы сохранить запись базы данных, если ключ и/или данные являются примитивными данными (**int**, **float** и т. д.), или если ключ и/или данные содержат массивы, достаточно указать на место в памяти, где это данные находятся, и их длину.

```
struct __db_dbt {
    void      *data; // ключ/данные
    u_int32_t  size; // размер ключа/данных

    u_int32_t  ulen; // R0: length of user buffer
    u_int32_t  dlen; // R0: get/put record length
    u_int32_t  doff; // R0: get/put record offset

    void      *app_data;
    u_int32_t  flags;
};
```

Все поля структуры DBT перед первым ее использованием, которые не заданы явно, должны быть инициализированы нулевыми байтами. Это может быть сделано объявлением структуры DBT внешней или статической, также можно вызвать функцию библиотеки C **memset(3)**.

По умолчанию ожидается, что член структуры **flags** будет установлен в 0. В этом случае, когда приложение предоставляет Berkeley DB ключ или элемент данных для сохранения в базе данных, Berkeley DB ожидает, что элемент структуры данных будет указывать на строку байт размером **size**.

При возврате ключа/элемента данных в приложение Berkeley DB сохранит в члене **data** указатель на строку байтов размером **size**, при этом память, на которую ссылается **data**, будет выделена и управляться Berkeley DB.

Важно!!! Использование флагов по умолчанию для возвращаемых DBT совместимо только с однопоточным использованием Berkeley DB.

Элементы структуры DBT определяются следующим образом:

void *app_data;

Необязательное поле, которое можно использовать для передачи информации через вызовы API Berkeley DB в определяемые пользователем функции обратного вызова. Например, к этому полю можно получить доступ для передачи определяемого пользователем содержимого при реализации обратного вызова, используемого **DB->set_dup_compare()**.

void *data;

Указатель на строку байт.

u_int32_t size;

Длина данных в байтах.

u_int32_t ulen;

Используется с флагом **DB_DBT_USERMEM**.

Указывает размер пользовательского буфера, на который ссылается член **data**, в байтах.

Приложения могут определить длину записи, установив в поле **ulen** значение 0 и проверив размер возвращаемого значения в поле **size**. Поле **data** при этом не изменяется.

u_int32_t dlen;

Длина частичной записи, читаемой или записываемой приложением, в байтах. Дополнительная информация во флаге **DB_DBT_PARTIAL**.

u_int32_t doff;

Смещение частичной записи, читаемой или записываемой приложением, в байтах. Дополнительная информация во флаге **DB_DBT_PARTIAL**.

u_int32_t flags;

Параметр **flags** устанавливается либо в 0, либо путем побитового включающего ИЛИ одного или нескольких из следующих значений:

DB_DBT_EXT_FILE — этот флаг устанавливается для DBT, используемой для части данных записи, чтобы указать, что DBT хранит данные внешнего файла. Если этот флаг установлен, и если база данных поддерживает внешние файлы, то данные, содержащиеся в этом DBT, будут сохраняться как внешний файл.

DB_DBT_READONLY — когда этот флаг установлен, Berkeley DB не будет записывать в DBT. Флаг может устанавливаться для значений ключей в тех случаях, когда ключ представляет собой статическую строку, в которую невозможно писать, а Berkeley DB может попытаться ее обновить.

DB_DBT_MALLOC — если установлен этот флаг, Berkeley DB выделит память для возвращаемого ключа или элемента данных, используя **malloc(3)** (или указанную пользователем функцию **malloc**) и вернет указатель на нее в поле **data** ключа или элемента данных.

DB_DBT_REALLOC — если установлен этот флаг, Berkeley DB выделит память для возвращаемого ключа или элемента данных с помощью **realloc(3)** (или указанной пользователем функции **realloc**) и вернет указатель на нее в поле **data** ключа или данных.

Поскольку за любую выделенную память отвечает вызывающее приложение, вызывающая сторона должна определить, была ли выделена память, используя возвращаемое значение поля данных.

Разница между **DB_DBT_MALLOC** и **DB_DBT_REALLOC** заключается в том, что в последнем случае будет вызывать **realloc(3)** вместо **malloc(3)**, поэтому выделенная память будет увеличиваться по мере необходимости вместо того, чтобы приложению пришлось выполнять повторяющиеся вызовы **free/malloc**.

DB_DBT_USERMEM — поле **data** ключа или данных должно ссылаться на область памяти длиной не менее **ulen** байт. Если длина запрошенного элемента меньше или равна этому количеству байтов, элемент копируется в память, на которую ссылается **data**.

В противном случае в поле **size** устанавливается длина, необходимая для запрошенного элемента, и возвращается ошибка **DB_BUFFER_SMALL**.

Указание более одного из **DB_DBT_MALLOC**, **DB_DBT_REALLOC** и **DB_DBT_USERMEM** является ошибкой.

DB_DBT_PARTIAL — выполнить частичное извлечение или сохранение элемента.

Если вызывающее приложение выполняет операцию **GET**, возвращается **dlen** байт, начиная со смещения **doff** от начала полученной записи данных, как если бы они составляли всю запись.

Если какие-либо или все указанные байты в записи не существуют, получение завершается успешно, при этом возвращаются все существующие байты.

Например, если часть данных извлекаемой записи составляла 100 байт, а частичное извлечение было выполнено с использованием DBT, имеющего поле **dlen**, равное 20, и поле **doff**, равное 85, вызов **get** будет успешным, поле данных будет ссылаться на последние 15 байт записи, а поле размера будет установлено на 15.

Если вызывающее приложение выполняет операцию **PUT**, **dlen** байт, начиная со смещения **doff** от начала записи указанной ключом, заменяются данными, указанными элементами структуры **data** и **size**.

Если **dlen** меньше **size**, запись вырастет в размере.

Если **dlen** больше **size**, запись будет обрезана.

Если указанные байты не существуют, запись будет расширена с использованием нулевых байтов по мере необходимости, а вызов **PUT** завершится успешно.

Попытка частичного размещения с использованием метода **DB->put()** в базе данных, поддерживающей повторяющиеся записи, является ошибкой.

Частичное размещение в базах данных, поддерживающих повторяющиеся записи, должно выполняться с использованием метода **Dbcursor->put()**.

Попытка частичного размещения с разными значениями **dlen** и **size** в базах данных Queue или Respo с записями фиксированной длины является ошибкой.

Например, если часть данных извлеченной записи составляла 100 байт, а частичная запись была выполнена с использованием DBT, имеющего поле **dlen** 20, поле **doff** 85 и поле **size** 30, результирующая запись будет иметь 115 байт, где последние 30 байтов будут указанными в вызове **PUT**.

Этот флаг игнорируется при использовании с параметром **pkey** в методе **DB->pget()** или в методе **Dbcursor->pget()**.

DB_DBT_APPMALLOC — после выполнения предоставляемой приложением процедуры обратного вызова, вызванной методами **DB->associate()**, либо **DB->set_append_recno()**, поле **data** в DBT может ссылаться на память, выделенную с помощью **malloc(3)** или **realloc(3)**. В этом случае обратный вызов в DBT может установить флаг **DB_DBT_APPMALLOC**, чтобы Berkeley DB вызывала **free(3)** для освобождения памяти, когда она больше не требуется.

Флаг **DB_DBT_APPMALLOC** может быть установлен в любой из структур DBT, чтобы указать, что их поле **data** необходимо освободить.

DB_DBT_MULTIPLE — устанавливается в процедуре обратного вызова создания вторичного ключа, вызванной методом **DB->associate()**, чтобы указать, что с данной парой первичный_ключ/данные должны быть связаны несколько вторичных ключей. Если флаг установлен, поле **size** указывает количество вторичных ключей, а поле **data** относится к массиву из этого количества структур DBT.

Функция **DB->associate()** используется для объявления одной базы данных вторичным индексом для первичной базы данных.

После того как вторичная база данных будет «связана» с первичной базой данных, все обновления первичной базы данных будут автоматически отражаться во вторичной базе данных, и все операции чтения из вторичной базы данных будут возвращать соответствующие данные из первичной базы данных.

Пример

```
#include <db.h>
#include <string.h>

...

DBT key, data;
float money          = 122.45;
char *description = "Счет за продукты";

// Очистка DBTs перед использованием
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

// Загрузка ключа
key.data = &money;
key.size = sizeof(float);

// Загрузка данных
data.data = description;
data.size = strlen(description) + 1;
...
```

Чтобы получить запись, иногда необходимо указать куда следует ее возвращать.

В следующем примере мы не позволяем BDB выделять память для извлечения денежного значения. Причина в том, что некоторые системы могут требовать, чтобы значения с плавающей запятой имели определенное выравнивание, а память, возвращаемая BDB, может быть неправильно выровнена (такая же проблема может существовать для структур в некоторых системах).

Поэтому указывается флаг DB_DBT_USERMEM – использовать память программы вместо памяти BDB.

В этом случае в поле **ulen** необходимо указать, сколько пользовательской памяти доступно.

```
#include <db.h>
#include <string.h>
...
float money;
DBT key, data;
char *description;

// Очистка DBTs перед использованием
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

key.data = &money;
key.ulen = sizeof(float);
key.flags = DB_DBT_USERMEM; // использовать память приложения

// Здесь код для получения записи из БД.
// money будет установлено в памяти, которое мы указали

description = data.data;
```

Чтение и запись записей базы данных

При чтении и записи записей базы данных надо иметь в виду, что существуют небольшие различия в поведении в зависимости от того, поддерживает ли база данных дубликаты записей.

Две или более записей базы данных считаются дубликатами друг друга, если они имеют один и тот же ключ.

Набор записей, использующих один и тот же ключ, называется набором дубликатов.

В BDB данный ключ сохраняется только один раз для одного набора дубликатов.

По умолчанию базы данных BerkeleyDB не поддерживают повторяющиеся записи.

В тех случаях, когда поддерживаются повторяющиеся записи, для доступа ко всем записям в наборе дубликатов обычно используются *курсоры*.

BDB предоставляет два основных механизма для хранения и извлечения пар ключ/данные из базы данных:

Самый простой доступ ко всем *неповторяющимся* записям в базе данных обеспечивают методы DBT->put() и DBT->get().

Курсоры предоставляют несколько способов размещения и получения записей в/из базы данных.

Запись в базу данных

Записи хранятся в базе данных с использованием любой организации данных, необходимой для выбранного метода доступа.

В некоторых случаях (например, в BTree) записи хранятся в порядке сортировки, который можно определить явно, указав функцию сравнения.

После того, как выбран метод доступа, настроены процедуры сортировки (если они есть) и открыта база данных, механизм размещения и получения записей базы данных изменить нельзя.

С точки зрения кода простые **put()** и **get()** практически не отличаются независимо от того, какой метод доступа используется.

Для размещения записи в базе данных используется метод **DB->put()**, сохраняющий пары ключ/данные в базе данных.

```
#include <db.h>

int DB->put(DB          *db,      // дескриптор базы данных
            DB_TXN      *txnid,   //
            DBT          *key,     //
            DBT          *data,    //
            u_int32_t    flags); //
```

key, data – метод требует, предоставления ключа записи и данных в виде пары структур **DBT**.

Поведение **DB->put()** по умолчанию заключается в:

- вводе новой пары ключ/данные;
- замене любого ранее существующего ключа, если дубликаты запрещены
- добавлении дублирующего элемента данных, если дубликаты разрешены.

Если база данных поддерживает дубликаты, **DB->put()** добавляет новое значение данных в конец набора дубликатов.

Если база данных поддерживает отсортированные дубликаты, новое значение данных вставляется в правильное место в порядке сортировки.

DB->put() возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

txnid

если операция является частью транзакции, заданной приложением, параметр **txnid** — это дескриптор транзакции, возвращаемый из **DB_ENV->txn_begin()**;

если операция является частью группы Concurrent Data Store Berkeley DB, параметр **txnid** — это дескриптор, возвращаемый из **DB_ENV->cdsgroup_begin()**;

в противном случае **NULL**.

Если дескриптор транзакции не указан, но операция происходит в транзакционной базе данных, операция будет защищена транзакциями неявно.

Также можно указать один или несколько флагов, которые управляют поведением БД при записи в базу данных.

DB_APPEND — добавить пару ключ/данные в конец базы данных.

Флаг **DB_APPEND** может быть указан только для базы данных Heap¹, Queue или Resno.

Номер, присвоенный записи, возвращается в указанном ключе.

Этот флаг является обязательным для базы данных типа Heap в том случае, если операция **put** приводит к созданию новой записи.

1) Метод доступа Heap сохраняет записи в файле кучи. При удалении записей не требуется уплотнение, что позволяет более эффективно использовать пространство, чем при использовании Btree. Метод доступа к куче предназначен для платформ с ограниченным дисковым пространством, особенно если эти системы создают и удаляют большое количество записей.

DB_NODUPDATA — в случае методов доступа Btree и Hash водит новую пару ключ/данные только в том случае, если она еще не присутствует в базе данных.

Флаг DB_NODUPDATA можно указать только в том случае, если база данных настроена на поддержку отсортированных дубликатов.

Флаг DB_NODUPDATA нельзя указывать для методов доступа Queue или Resno.

Если установлен флаг DB_NODUPDATA и пара ключ/данные уже присутствует в базе данных, **DB->put()** вернет ошибку **DB_KEYEXIST**.

DB_NOOVERWRITE — вводит новую пару ключ/данные только в том случае, если ключ еще не появился в базе данных.

Если ключ уже существует в базе данных, вызов **DB->put()** с установленным флагом DB_NOOVERWRITE завершится ошибкой, даже если база данных поддерживает дубликаты.

Если установлен DB_NOOVERWRITE и ключ уже присутствует в базе данных, **DB->put()** вернет ошибку **DB_KEYEXIST**.

Данное обеспечение уникальности ключей применяется только к первичному ключу.

Флаг DB_NOOVERWRITE не влияет на поведение вставок во вторичные базы данных. В частности, использование этого флага не предотвратит вставку записи, которая приведет к созданию дублирующего ключа во вторичной базе данных, допускающей дублирование.

DB_MULTIPLE — массовая операция, размещает несколько элементов данных, используя ключи из массового буфера, на который ссылается параметр **key**, и значения данных из буфера, на который ссылается параметр **data**.

Успешная массовая операция логически эквивалентна циклу по каждой паре ключ/данные с выполнением **DB->put()** для каждой из них.

Флаг DB_MULTIPLE можно использовать только отдельно или с опцией DB_OVERWRITE_DUP.

DB_MULTIPLE_KEY — массовая операция, размещает несколько элементов данных, используя ключи и данные из буфера, к которому относится параметр **key**.

DB_OVERWRITE_DUP — игнорировать повторяющиеся записи при перезаписи в базе данных, настроенной для сортировки дубликатов.

Обычно, если база данных настроена для сортировки дубликатов, попытка поместить запись, идентичную записи, уже существующей в базе данных, завершится неудачей. Использование этого флага приводит к тому, что **PUT** выполняется автоматически и без сбоев.

Этот флаг чрезвычайно полезен при выполнении массовых операций размещения (с использованием флагов **DB_MULTIPLE** или **DB_MULTIPLE_KEY**).

В зависимости от количества записей, которые записываются в базу данных с помощью массового размещения, может быть нежелательным, чтобы операция завершалась неудачей в случае обнаружения повторяющейся записи.

Использование этого флага вместе с флагами **DB_MULTIPLE** или **DB_MULTIPLE_KEY** позволяет завершить массовое размещение, даже если обнаружена повторяющаяся запись.

Этот флаг также полезен, если используется пользовательская функция сравнения, которая сравнивает только часть данных записи. В этом случае две записи могут сравниваться одинаково, хотя на самом деле они не равны. Этот флаг позволяет завершить операцию **PUT**, даже если пользовательская процедура сравнения утверждает, что две записи равны.

Пример кода

```
#include <db.h>
#include <string.h>

...
char *description = "Grocery bill.";
DBT    key, data;
DB     *my_database;
int     ret;
float  money;

/* Database open omitted for clarity */

money = 122.45;

/* Zero out the DBTs before using them. */
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

key.data = &money;
key.size = sizeof(float);

data.data = description;
data.size = strlen(description) + 1;

ret = my_database->put(my_database, NULL, &key, &data, DB_NOOVERWRITE);
if (ret == DB_KEYEXIST) {
    my_database->err(my_database, ret,
        "Put failed because key %f already exists", money);
}
```

Получение записей из базы данных

Для получения записей базы данных используется метод **DB->get()**.

```
#include <db.h>

int DB->get(DB          *db,
            DB_TXN      *txnid,
            DBT          *key,
            DBT          *data,
            u_int32_t    flags);

int DB->pget(DB          *db,
            DB_TXN      *txnid,
            DBT          *key,
            DBT          *pkey,
            DBT          *data,
            u_int32_t    flags);
```

DB->get() извлекает пары ключ/данные из базы данных. Адрес и длина данных, связанных с указанным ключом **key**, возвращаются в структуре, на которую ссылается **data**.

При наличии повторяющихся значений ключа **DB->get()** вернет первый элемент данных для назначенного ключа. Дубликаты сортируются по:

- в порядке, указанном функцией сортировки дубликатов (если была указана);
- в порядке курсора вставки;
- в порядке вставки. Это поведение по умолчанию.

Для поиска дубликатов требуется использование операций с курсором.

При вызове для базы данных, которая была преобразована во вторичный индекс с помощью метода **DB->associate()**, **DB->get()** и **DB->pget()** возвращают ключ из вторичного индекса и элемент данных из первичной базы данных.

Кроме того, **DB->pget()** возвращает ключ из первичной базы данных.

В базах данных, которые не являются вторичными индексами, метод **DB->pget()** всегда будет завершаться ошибкой.

DB->get() вернет **DB_NOTFOUND**, если указанного ключа нет в базе данных.

DB->get() возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

Также можно получить набор повторяющихся записей, используя «массовый» **get**.

Для этого при вызове **DB->get()** используется флаг **DB_MULTIPLE**.

По умолчанию **DB->get()** возвращает первую найденную запись, ключ которой совпадает с ключом, предоставленным при вызове этого метода.

Если база данных поддерживает повторяющиеся записи, можно немного изменить это поведение, указав флаг **DB_GET_BOTH**.

Этот флаг заставляет **DB->get()** возвращать первую запись, которая соответствует предоставленному ключу и данным.

Если указанный ключ и/или данные в базе данных не существуют, этот метод возвращает значение **DB_NOTFOUND**.

DB_RMW — Чтение-изменение-запись: получение блокировок записи вместо блокировок чтения во время извлечения. Это может повысить производительность многопоточных приложений за счет уменьшения вероятности взаимоблокировки.

DB_SET_RECNO — если база данных представляет собой BTree и настроена так, что в ней можно выполнять поиск по номеру логической записи, извлекается конкретная запись.

Пример

```
#include <db.h>
#include <string.h>
...
#define DESCRIPTION_SIZE 199
DBT key, data;
DB *my_database;
float money;
char description[DESCRIPTION_SIZE + 1];

/* Database open omitted for clarity */

money = 122.45;

/* Zero out the DBTs before using them. */
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

key.data = &money;
key.size = sizeof(float);

data.data = description;
data.ulen = DESCRIPTION_SIZE + 1;
data.flags = DB_DBT_USERMEM;
my_database->get(my_database, NULL, &key, &data, 0);

// Description is set into the memory that we supplied.
```

В этом примере в поле **data.size** будет автоматически установлен размер извлеченных данных.

Удаление записей

Для удаления записи из базы данных используется метод **DB->del()**.

```
#include <db.h>
int DB->del(DB          *db,
            DB_TXN      *txnid,
            DBT          *key,
            u_int32_t    flags);
```

DB->del() удаляет пары ключ/данные, связанные с указанным ключом **key**, из базы данных.

При наличии повторяющихся значений ключа будут удалены все записи, связанные с указанным ключом.

При вызове к базе данных, которая была преобразована во вторичный индекс с помощью метода **DB->associate()**, **DB->del()** удаляет пару ключ/данные из первичной базы данных и всех вторичных индексов.

DB->del() вернет **DB_NOTFOUND**, если указанного ключа нет в базе данных.

DB->del() вернет **DB_KEYEMPTY**, если база данных является базой данных Queue или Resno и указанный ключ существует, но никогда не был явно создан приложением или позже удален. Если не указано иное, метод **DB->del()** возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

Если база данных поддерживает повторяющиеся записи, удаляются все записи, связанные с предоставленным ключом.

Чтобы удалить только одну запись из списка дубликатов, нужно использовать курсор.

Также можете удалить все записи в базе данных, используя **DB->truncate()**.

Пример

```
#include <db.h>
#include <string.h>

...

DBT key;
DB *my_database;
float money = 122.45;

/* Database open omitted for clarity */

/* Zero out the DBTs before using them. */
memset(&key, 0, sizeof(DBT));

key.data = &money;
key.size = sizeof(float);

my_database->del(my_database, NULL, &key, 0);
```


Сохранность данных

Когда выполняется модификация базы данных, она выполняется в кэше в памяти.

Это означает, что изменения данных не обязательно сбрасываются на диск, и поэтому данные после перезапуска приложения могут *не отображаться в базе данных*.

Обычно при *закрытии* базы данных ее кэш записывается на диск. Однако в случае сбоя приложения или системы нет никакой гарантии, что базы данных будут закрыты корректно.

В этом случае можно потерять данные. В крайне редких случаях также можно столкнуться с повреждением базы данных.

Поэтому, для уверенности в том, что данные будут устойчивыми при системных сбоях, и для защиты от редкой возможности повреждения базы данных, для защиты изменений базы данных необходимо использовать транзакции.

Каждый раз, когда выполняется транзакция, BDB гарантирует, что из-за сбоя приложения или системы данные не будут потеряны.

Если транзакции не используются, это подразумевает, что данные таковы, что они не должны существовать при следующем запуске приложения.

Обычно такое имеет место, если, например, BDB используется для кэширования данных, относящихся только к текущей сессии.

Если, однако, по какой-то причине транзакции не используются, но необходимо гарантировать, что изменения вашей базы данных будут сохранены, следует периодически вызывать **DB->sync()**.

Синхронизация приводит к тому, что любые записи в кеше в памяти и файловом кеше операционной системы записываются на диск. Поскольку они довольно дороги с точки зрения производительности, их следует использовать «экономно».

По умолчанию синхронизация выполняется каждый раз, когда нетранзакционная база данных корректно закрывается.

Можно переопределить это поведение, указав **DB_NOSYNC** при вызове **DB->close()**.

Тем не менее, можно запустить синхронизацию вручную, вызвав **DB->sync()**.

Если приложение или система дает сбой и транзакции не используется, необходимо либо отказаться от баз данных и создать их заново, либо их проверить.

Проверить базу данных можно, используя **DB->verify()**.

```
#include<db.h>

int DB->verify(DB          *db,
               const char *file,      //
               const char *database,
               FILE        *outfile,
               u_int32_t   flags);
```

DB->verify() проверяет целостность всех баз данных в файле, указанном параметром **file**, и при необходимости выводит пары ключ/данные баз данных в файловый поток, указанный параметром **outfile**.

DB->verify() не выполняет никаких блокировок даже в средах Berkeley DB, в которых настроена подсистема блокировки. Таким образом, его следует использовать только с файлами, которые не изменяются в других потоках управления.

DB->verify() нельзя вызывать после вызова **DB->open()**.

Дескриптор БД не может быть снова доступен

После вызова **DB->verify()**, независимо от его возврата, дескриптор БД может быть недоступен.

Есть утилита **db_verify**, она использует **DB->verify()**.

DB->verify() вернет **DB_VERIFY_BAD**, если база данных повреждена.

Если указан флаг **DB_SALVAGE**, возврат **DB_VERIFY_BAD** означает, что, возможно, не все пары ключ/данные были успешно выведены.

DB->verify() возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

Если базы данных не проходят корректную проверку, чтобы восстановить как можно большую часть базы данных, следует использовать команду **db_dump**.

Использование С-структур

Хранение данных в структурах — это удобный способ упаковать различные типы информации в каждую запись базы данных.

Базы данных иногда представляют как таблицу с двумя столбцами, где столбец 1 является ключом, а столбец 2 — данными.

Используя структуры, можно эффективно превратить такую таблицу в n столбцов.

Если С-структура содержит поля, не являющиеся указателями, ее можете безопасно хранить и извлекать так же, как любой примитивный тип данных.

Пример размещения

```
#include <db.h>
#include <string.h>

typedef
struct my_struct {
    int id;
    char familiar_name[MAXLINE]; // Какое-то достаточно большое значение MAXLINE
    char surname[MAXLINE];
} MY_STRUCT;

...

DBT key, data;
DB *my_database;
MY_STRUCT user;
char *fname = "David";
char *sname = "Hilbert";
```

```
/* Database open omitted for clarity */

user.id = 1;
strncpy(user.familiar_name, fname, strlen(fname)+1);
strncpy(user.surname, sname, strlen(sname)+1);

// Очистка DBT
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

key.data = &(user.id);
key.size = sizeof(int);

data.data = &user;
data.size = sizeof(MY_STRUCT);

my_database->put(my_database, NULL, &key, &data, DB_NOOVERWRITE);
```

Чтобы получить структуру из базы, нужно предоставить свою собственную память.

Причина в том, что, как и в случае с **float** числами, — некоторые системы требуют, чтобы структуры были выровнены определенным образом.

Поскольку возможно, что память, предоставляемая BDB, не выровнена должным образом, для наиболее безопасного результата следует использовать собственную память приложения.

Пример извлечения

```
#include <db.h>
#include <string.h>
...
DBT  key, data;
DB  *my_database;
MY_STRUCT user;

/* Database open omitted for clarity */

memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

// Инициализация структуры
memset(&user, 0, sizeof(MY_STRUCT));
user.id = 1;

key.data = &user.id;    // Заполнение DBT
key.size = sizeof(int);

// Используем память приложения для извлечения структуры
data.data = &user;
data.ulen = sizeof(MY_STRUCT);
data.flags = DB_DBT_USERMEM;

my_database->get(my_database, NULL, &key, &data, 0);

printf("Familiar name: %s\n", user.familiar_name);
printf("Surname: %s\n", user.surname);
```

Подход со структурами плох тем, что база данных становится больше, чем это строго необходимо, — каждая структура, хранящаяся в базе данных, имеет фиксированный размер, и нет никакой экономии места при хранении (например) 5-символьной фамилии по сравнению с 20-символьной фамилией.

Если нужно хранить структуры, содержащие очень большое количество массивов символов, или если необходимо хранить миллионы записей, нужно избегать этого подхода.

Альтернативный подход — С-структуры с указателями

В С-структурах можно использовать поля, которые являются указателями на динамически выделяемую память. Это полезно, если необходимо хранить строки символов (или любой другой массив) и желательно избежать каких-либо накладных расходов.

При хранении подобных структур необходимо обеспечить, чтобы все данные, на которые указывает структура и которые она содержит, были выстроены в один непрерывный блок памяти.

BDB хранит данные, расположенные по определенному адресу и определенного размера. Если структура включает поля, указывающие на динамически выделенную память, то данные, которые следует сохранить, могут быть расположены в разных местах в куче, не обязательно смежных.

Самый простой способ решить эту проблему — собрать данные в одно место в памяти, а затем сохранить эти данные. Этот процесс иногда называют *маршалингом* данных.

```
#include <db.h>
#include <string.h>
#include <stdlib.h>

typedef struct my_struct {
    int    id;
    char *familiar_name;
    char *surname;
} MY_STRUCT;

...

DBT key, data;
DB *my_database;
MY_STRUCT user;
```



```
int buffsize, buflen;
char fname[ ] = "Pete";
char sname[10];
char *databuff;

strncpy(sname, "0ar", strlen("0ar")+1);

/* Database open omitted for clarity */

user.id = 1;
user.familiar_name = fname;
user.surname = sname;

/* Some of the structure's data is on the stack, and
 * some is on the heap. To store this structure's data, we
 * need to marshall it -- pack it all into a single location
 * in memory.
 */

/* Get the buffer */
buffsize = sizeof(int) +
    (strlen(user.familiar_name) + strlen(user.surname) + 2);
databuff = malloc(buffsize);
memset(databuff, 0, buffsize);

/* copy everything to the buffer */
memcpy(databuff, &(user.id), sizeof(int));
buflen = sizeof(int);

memcpy(databuff + buflen, user.familiar_name,
```

```
    strlen(user.familiar_name) + 1);
bufflen += strlen(user.familiar_name) + 1;

memcpy(databuff + bufflen, user.surname,
    strlen(user.surname) + 1);
bufflen += strlen(user.surname) + 1;

/* Now store it */

/* Zero out the DBTs before using them. */
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

key.data = &(user.id);
key.size = sizeof(int);

data.data = databuff;
data.size = bufflen;

my_database->put(my_database, NULL, &key, &data, DB_NOOVERWRITE);
free(sname);
free(databuff);
```

Пример получения хранимой структуры

```
#include <db.h>
#include <string.h>
#include <stdlib.h>

typedef struct my_struct {
    char *familiar_name;
    char *surname;
    int id;
} MY_STRUCT;

...

int id;
DBT key, data;
DB *my_database;
MY_STRUCT user;
char *buffer;

/* Database open omitted for clarity */

/* Zero out the DBTs before using them. */
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

id = 1;
key.data = &id;
key.size = sizeof(int);
```

```
my_database->get(my_database, NULL, &key, &data, 0);

/*
 * Some compilers won't allow pointer arithmetic on void *'s,
 * so use a char * instead.
 */
buffer = data.data;

user.id = *((int *)data.data);
user.familiar_name = buffer + sizeof(int);
user.surname = buffer + sizeof(int) + strlen(user.familiar_name) + 1;
```

IFF — подход и использование пластичных массивов

Последний элемент структуры, содержащей более чем один именованный элемент может иметь незавершенный тип массива — этот элемент называется *членом пластичного массива*.

В большинстве случаев член пластичного массива игнорируется.

В частности, размер структуры остается такой, как если бы элемент пластичного массива отсутствовал, за исключением того, что такой член может иметь большее завершающее заполнение, чем подразумевает его отсутствие.

Формат IFF — Interchange File Format.

TYPE_ID	DATA_SZ	DATA
---------	---------	------

TYPE_ID	DATA_SZ	DATA
---------	---------	------

```
struct chunk_s {  
    int16_t    type;    // TYPE_ID  (+/- Little/Big Endian)  
    uint16_t   size;    // DATA_SZ  
    char       data[];  // DATA  
}
```

Использование курсоров

Курсоры — это механизм, с помощью которого можно перебирать записи в базе данных.

Используя курсоры, можно извлекать, размещать и удалять записи базы данных.

Если база данных допускает дублирование записей, то курсоры — это самый простой способ получить доступ ко всему, кроме первой записи для данного ключа.

Курсор базы данных относится к одной паре ключ/данные в базе данных. Он поддерживает обход базы данных и является единственным способом доступа к отдельным повторяющимся элементам данных. Курсоры используются для работы с коллекциями записей, для перебора базы данных и для сохранения дескрипторов отдельных записей, чтобы их можно было изменять после прочтения.

Курсор в базе данных открывается с помощью метода **DB->cursor()**.

При возврате курсор не инициализирован — позиционирование курсора происходит как часть первой операции с курсором.

После открытия курсора базы данных записи можно получать **DBC->get()**, сохранять **DBC->put()** и удалять **DBC->del()**.

Дополнительные операции, поддерживаемые дескриптором курсора, включают дублирование **DBC->dup()**, соединение по принципу равенства **DB->join()** и подсчет повторяющихся элементов данных **DBC->count()**.

Курсоры в конечном итоге закрываются с помощью **DBC->close()**.

Открытие и закрытие курсоров

Курсоры управляются с помощью структуры **DBC**.

Чтобы использовать курсор, необходимо его открыть с помощью метода **DB->cursor()**.

```
#include<db.h>

int DB->cursor(DB          *db,
               DB_TXN      *txnid, // транзакционный контекст, если не NULL
               DBC          **cursorp, // сюда будет помещен указатель на курсор
               u_int32_t    flags); //
```

DB->cursor() возвращает созданный курсор базы данных.

Курсоры могут охватывать потоки, но только последовательно, то есть приложение должно самостоятельно сериализовать доступ к дескриптору курсора.

DB->cursor() возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

Пример

```
#include <db.h>
...
DB *my_database;
DBC *cursorp;

/* Database open omitted for clarity */

/* Get a cursor */
my_database->cursor(my_database, NULL, &cursorp, 0);
```

После завершения работы с курсором, его необходимо закрыть с пом. метода **DBC->close()**.

Закрытие базы данных, когда курсоры все еще открыты, особенно если эти курсоры записывают в базу, может иметь непредсказуемые результаты.

Рекомендуется закрывать все дескрипторы курсора после их использования, чтобы обеспечить параллелизм и освободить ресурсы, такие как блокировки страниц.

```
#include <db.h>

...

DB *my_database;
DBC *cursorp;

/* Database and cursor open omitted for clarity */

if (cursorp != NULL)
    cursorp->close(cursorp);

if (my_database != NULL)
    my_database->close(my_database, 0);
```


Получение записей с помощью курсора

Чтобы просмотреть записи базы данных, от первой записи до последней, нужно просто открыть курсор, после чего использовать метод **DBC->get()**. (**Dbcursor->get()**).

```
#include<db.h>

int DBcursor->get(DBC *DBcursor,
                 DBT *key,
                 DBT *data,
                 u_int32_t flags);

int DBcursor->pget(DBC *DBcursor,
                 DBT *key,
                 DBT *pkey,
                 DBT *data,
                 u_int32_t flags);
```

DBcursor->get() извлекает пары ключ/данные из базы данных. Адрес и длина ключа возвращаются в объекте, на который ссылается **key** (за исключением случая флага **DB_SET**, в котором ключевой объект не изменяется), а адрес и длина данных возвращаются в объекте, на который ссылается **data**.

При вызове курсора, открытого в базе данных, которая была преобразована во вторичный индекс с помощью метода **DB->associate()**, **DBcursor->get()** и **Dbcursor->pget()** возвращают ключ из вторичного индекса и элемент данных из первичной базы данных. Кроме того, **DBcursor->pget()** возвращает ключ из первичной базы данных.

В базах данных, которые не являются вторичными индексами, метод **DBcursor->pget()** всегда будет завершаться ошибкой.

Изменения базы данных во время последовательного сканирования будут отражаться в процессе сканирования. То есть записи, вставленные позади курсора, не будут возвращаться, а записи, вставленные перед курсором, будут.

Если не указано иное, **DBcursor->get()** возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

Если **DBcursor->get()** по какой-либо причине завершится неудачно, состояние курсора останется неизменным.

Флаги

DB_CURRENT –

DB_FIRST – первая пара ключ/значение

DB_LAST – последняя пара ключ/значение

DB_GET_BOTH –

DB_GET_BOTH_RANGE –

DB_NEXT –

DB_NEXT_DUP –

DB_NEXT_NODUP –

DB_PREV –

DB_PREV_DUP –

DB_PREV_NODUP –

Для этого метода необходимо указать флаг **DB_NEXT**:

```
#include <db.h>
#include <string.h>

...

DB *my_database;
DBC *cursorp;
DBT key, data;
int ret;

/* Database open omitted for clarity */

/* Get a cursor */
my_database->cursor(my_database, NULL, &cursorp, 0);

/* Initialize our DBTs. */
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

/* Iterate over the database, retrieving each record in turn. */
while ((ret = cursorp->get(cursorp, &key, &data, DB_NEXT)) == 0) {
    /* Do interesting things with the DBTs here. */
}
if (ret != DB_NOTFOUND) {
    /* Error handling goes here */
}

/* Cursors must be closed */
```

```
if (cursorp != NULL)
    cursorp->close(cursorp);

if (my_database != NULL)
    my_database->close(my_database, 0);
```

Чтобы выполнить итерирование по базе от последней записи к первой, следует использовать **DB_PREV** вместо **DB_NEXT**:

```
#include <db.h>
#include <string.h>

...

DB *my_database;
DBC *cursorp;
DBT key, data;
int ret;

/* Database open omitted for clarity */

/* Get a cursor */
my_database->cursor(my_database, NULL, &cursorp, 0);
```

```
/* Initialize our DBTs. */
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

/* Iterate over the database, retrieving each record in turn. */
while ((ret = cursorp->get(cursorp, &key,
    &data, DB_PREV)) == 0) {
    /* Do interesting things with the DBTs here. */
}
if (ret != DB_NOTFOUND) {
    /* Error handling goes here */
}

// Cursors must be closed
if (cursorp != NULL)
    cursorp->close(cursorp);

if (my_database != NULL)
    my_database->close(my_database, 0);
```

Поиск записей

Можно использовать курсоры для поиска записей в базе.

Можно выполнять поиск только по ключу или искать одновременно по ключу и данным.

Также можно выполнять поиск по частичным совпадениям, если база данных поддерживает отсортированные наборы дубликатов.

Во всех случаях параметры ключа и данных этих методов заполняются значениями ключа и данных записи базы данных, на которую позиционируется курсор в результате поиска.

Если поиск не удался, то состояние курсора остается неизменным и возвращается **DB_NOTFOUND**.

Чтобы использовать курсор для поиска записи, следует использовать **DBT->get()**.

При использовании этого метода можно указать несколько флагов.

В списке флагов курсора используется ключевое слово **SET**, если курсор проверяет только ключевую часть записей — в этом случае курсор устанавливается на запись, ключ которой соответствует значению, предоставленному курсору.

Когда курсор использует ключевое слово **GET**, курсор позиционируется как на ключ, так и на значения данных, предоставленные курсору.

Независимо от ключевого слова, которое используется для получения записи с курсором, DBT ключа и данных курсора заполняются данными, извлеченными из записи, на которой в данный момент расположен курсор.

DB_SET — Перемещает курсор на первую запись в базе данных с указанным ключом.

DB_SET_RANGE — Идентичен DB_SET, но только в том случае, когда не используется тип доступа BTree. В этом случае курсор перемещается на первую запись в базе данных, ключ которой больше или равен указанному ключу. Это сравнение определяется функцией сравнения, которую необходимо предоставить базе данных.

Если функция сравнения не предусмотрена, то используется лексикографическая сортировка по умолчанию.

Базы данных

Лекция 14 – Berkeley DB

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2023.11.24

Оглавление

Использование курсоров.....	3
Открытие и закрытие курсоров.....	4
Получение записей с помощью курсора.....	10
Поиск записей.....	17
Работа с повторяющимися записями.....	22
Размещение записей с помощью курсоров.....	28
Удаление записей с помощью курсоров.....	33
Замена записей с помощью курсоров.....	35
Степени изоляции.....	38

Использование курсоров

Курсоры — это механизм, с помощью которого можно перебирать записи в базе данных.

Используя курсоры, можно извлекать, размещать и удалять записи базы данных.

Если база данных допускает дублирование записей, то курсоры — это самый простой способ получить доступ ко всему, кроме первой записи для данного ключа.

Курсор базы данных относится к одной паре ключ/данные в базе данных. Он поддерживает обход базы данных и является единственным способом доступа к отдельным повторяющимся элементам данных.

Курсоры используются для работы с коллекциями записей, для перебора базы данных и для сохранения дескрипторов отдельных записей, чтобы их можно было изменять после прочтения.

Курсор в базе данных открывается с помощью метода **DB->cursor()**.

При возврате курсор не инициализирован — позиционирование курсора происходит как часть первой операции с курсором.

После открытия курсора базы данных записи можно получать **DBC->get()**, сохранять **DBC->put()** и удалять **DBC->del()**.

Дополнительные операции, поддерживаемые дескриптором курсора, включают дублирование **DBC->dup()**, соединение по принципу равенства **DB->join()** и подсчет повторяющихся элементов данных **DBC->count()**.

Курсоры в конечном итоге закрываются с помощью **DBC->close()**.

Открытие и закрытие курсоров

Курсоры управляются с помощью структуры **DBC**.

Чтобы использовать курсор, необходимо его открыть с помощью метода **DB->cursor()**.

```
#include<db.h>

int DB->cursor(DB          *db,
               DB_TXN      *txnid,  // транзакционный контекст, если не NULL
               DBC         **cursorp, // сюда будет помещен указатель на курсор
               u_int32_t    flags); //
```

DB->cursor() возвращает созданный курсор базы данных.

Курсоры могут охватывать потоки, но только последовательно, то есть приложение должно самостоятельно сериализовать доступ к дескриптору курсора.

DB->cursor() возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

Пример

```
#include <db.h>
...
DB  *my_database;
DBC *cursorp;

/* Database open omitted for clarity */

/* Get a cursor */
my_database->cursor(my_database, NULL, &cursorp, 0);
```

Флаги

DB_CURSOR_BULK

Настраивает курсор для оптимизации для массовых операций. Каждая последующая операция над курсором, настроенным с помощью этого флага, пытается продолжить работу на той же странице базы данных, что и предыдущая операция. Если требуется другая страница, осуществляется возврат к поиску.

Это позволяет избежать поиска, если между операциями курсора существует высокая степень локальности. В настоящее время этот флаг действует только при использовании метода доступа **btree**. Для других методов доступа этот флаг игнорируется.

DB_READ_COMMITTED

Настраивает транзакционный курсор на изоляцию степени 2.

Это гарантирует стабильность текущего элемента данных, считываемого этим курсором, но позволяет изменять или удалять эти данные до фиксации транзакции для этого курсора.

DB_WRITECURSOR

Указывает, что курсор будет использоваться для обновления базы данных. Базовое окружение базы данных должна быть открыта с использованием флага **DB_INIT_CDB**¹.

1) DB_ENV: DB_INIT_CDB – инициализирует блокировку. В этом режиме Berkeley DB обеспечивает режим множественных операций чтения/одной операции записи.

DB_READ_UNCOMMITTED

Настраивает транзакционный курсор на изоляцию степени 1.

Операции чтения, выполняемые курсором, могут возвращать измененные, но еще не зафиксированные данные.

Молча игнорируется, если при открытии основной базы данных не был указан флаг **DB_READ_UNCOMMITTED**².

DB_TXN_SNAPSHOT

Настраивает транзакционный курсор для работы с изоляцией моментальных снимков только для чтения. Для баз данных с установленным флагом **DB_MULTIVERSION**³ значения данных будут считываться такими, какие они были при открытии курсора, без блокировки чтения.

Этот флаг неявно начинает транзакцию, которая фиксируется при закрытии курсора.

Этот флаг игнорируется, если в основной базе данных не был установлен **DB_MULTIVERSION** или если транзакция указана в параметре **txnid**.

2) **DB_READ_UNCOMMITTED** – при открытии БД этот флаг обеспечивает поддержку транзакционных операций чтения с изоляцией степени 1. Операции чтения в базе данных могут запросить возврат измененных, но еще не зафиксированных данных. Этот флаг должен быть указан для всех дескрипторов БД, используемых для выполнения «грязного» чтения или обновления базы данных, иначе запросы на «грязное чтение» могут не обрабатываться, и чтение может быть заблокировано.

3) **DB_MULTIVERSION** – открывает базу данных с поддержкой многоверсионного управления параллелизмом. Это приведет к тому, что обновления базы данных будут выполняться по протоколу копирования при записи, который необходим для поддержки изоляции моментальных снимков. Флаг **DB_MULTIVERSION** требует, чтобы база данных была защищена транзакциями во время ее открытия и не поддерживается форматом очереди.

Изоляция транзакций

Транзакции могут быть изолированы друг от друга в разной степени.

Изоляция третьей степени — полностью сериализуемые транзакции. Они обеспечивают наибольшую изоляцию. Это означает, что на протяжении всего срока транзакции всякий раз, когда поток управления читает элемент данных, этот элемент данных не будет изменяться (при условии, конечно, что поток управления сам не изменяет его).

По умолчанию Berkeley DB обеспечивает именно сериализуемость всякий раз, когда чтение базы данных оборачивается транзакциями.

Изоляция моментальных снимков (Snapshot Isolation) — это второй механизм обеспечения сериализуемости, в котором предпочтение отдается процессам чтения, а не процессам обновления.

Отказ от блокировок чтения обычно значительно повышает пропускную способность для многих приложений.

Изоляция второй степени — обеспечивается только стабильность курсора, то есть гарантируется, что курсоры видят зафиксированные данные, которые не изменяются, пока к ним этот курсор обращается, но могут измениться до завершения транзакции чтения.

Изоляция первой степени — поддержка чтения незафиксированных данных — то есть, операции чтения могут запрашивать данные, которые были изменены, но еще не зафиксированы другой транзакцией.

Ошибки

Метод **DB->cursor()** может завершиться неудачей и вернуть одну из следующих ненулевых ошибок:

DB_REP_HANDLE_DEAD

Имеет отношение к репликации. В процессе синхронизации клиента с мастером, зафиксированные транзакции могут быть отменены. Это аннулирует все дескрипторы базы данных и курсоров, открытые в среде репликации. Как только это произойдет, попытка использовать такой дескриптор вернет **DB_REP_HANDLE_DEAD**.

Приложению в этом случае, чтобы продолжить обработку, следует отказаться от дескриптора и открыть новый.

DB_REP_LOCKOUT

Операция была заблокирована синхронизацией клиента и мастера.

EINVAL

Было указано неверное значение флага или параметр.

После завершения работы с курсором, его необходимо закрыть с пом. метода **DBC->close()**.

Закрытие базы данных, когда курсоры все еще открыты, особенно если эти курсоры записывают в базу, может иметь непредсказуемые результаты.

Рекомендуется закрывать все дескрипторы курсора после их использования, чтобы обеспечить параллелизм и освободить ресурсы, такие как блокировки страниц.

```
#include <db.h>

...

DB  *my_database;
DBC *cursorp;

/* Database and cursor open omitted for clarity */

if (cursorp != NULL) {
    cursorp->close(cursorp);
}

if (my_database != NULL) {
    my_database->close(my_database, 0);
}
```


Получение записей с помощью курсора

Чтобы просмотреть записи базы данных, от первой записи до последней, нужно просто открыть курсор, после чего использовать метод **DBC->get()**.

```
#include<db.h>

int DBcursor->get(DBC *DBcursor,
                 DBT *key,
                 DBT *data,
                 u_int32_t flags);

int DBcursor->pget(DBC *DBcursor,
                 DBT *key,
                 DBT *pkey,
                 DBT *data,
                 u_int32_t flags);
```

DBcursor->get() извлекает пары ключ/данные из базы данных.

Адрес и длина ключа возвращаются в объекте, на который ссылается **key** (за исключением случая флага **DB_SET**, в котором ключевой объект не изменяется), а адрес и длина данных возвращаются в объекте, на который ссылается **data**.

При вызове курсора, открытого в базе данных, которая была преобразована во вторичный индекс с помощью метода **DB->associate()**, **DBcursor->get()** и **Dbcursor->pget()** возвращают ключ из вторичного индекса и элемент данных из первичной базы данных. Кроме того, **DBcursor->pget()** возвращает ключ из первичной базы данных.

В базах данных, которые не являются вторичными индексами, метод **DBcursor->pget()** всегда будет завершаться ошибкой.

В процессе последовательного сканирования базы данных изменения будут отражаться следующим образом — записи, вставленные позади курсора, возвращаться не будут, а записи, вставленные перед курсором, будут.

Если не указано иное, **DBcursor->get()** возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

Если **DBcursor->get()** по какой-либо причине завершится неудачно, состояние курсора останется неизменным.

Основные флаги

DB_FIRST (DB_LAST)

Инициализация курсора.

Курсор устанавливается для ссылки на первую (последнюю) пару **key/data** базы данных, и эта пара возвращается.

Если первый (последний) ключ имеет повторяющиеся значения, возвращается первый (последний) элемент данных в наборе дубликатов.

Если база данных представляет собой базу данных Queue или Resno, **DBcursor->get()** с флагом **DB_FIRST (DB_LAST)** будет игнорировать любые существующие ключи, которые никогда не создавались приложением явно или были созданы, а затем удалены.

Если установлен флаг **DB_FIRST (DB_LAST)** и база данных пуста, **DBcursor->get()** вернет **DB_NOTFOUND**.

DB_NEXT (DB_PREV)

Если курсор еще не инициализирован, **DB_NEXT (DB_PREV)** идентичен **DB_FIRST (DB_LAST)**.

В противном случае курсор перемещается на следующую (предыдущую) пару **key/data** базы данных, и эта пара возвращается.

При наличии повторяющихся значений ключа значение ключа может не изменяться.

Если база данных представляет собой базу данных Queue или Resno, **DBcursor->get()** с флагом **DB_NEXT (DB_PREV)** пропустит все ключи, которые существуют, но никогда не были явно созданы приложением, или те, которые были созданы и позже удалены.

Если установлен **DB_NEXT/DB_PREV** и курсор уже находится на последней (первой) записи в базе данных, **DBcursor->get()** вернет **DB_NOTFOUND**.

DB_CURRENT

Возвращает пару ключ/значение, на которую ссылается курсор.

Если установлен флаг **DB_CURRENT** и пара ключ/значение курсора была удалена, **DBcursor->get()** вернет ошибку **DB_KEYEMPTY**.

Для этого метода необходимо указать флаг **DB_NEXT**:

```
#include <db.h>
#include <string.h>

...

DB  *my_database;
DBC *cursorp;
DBT key, data;
int ret;

// Открытие БД опущено

// Получаем курсор
my_database->cursor(my_database, NULL, &cursorp, 0);

// Инициализируем обе DBT
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

// Итерируем по БД, извлекая каждую запись в цикле
while (0 == (ret = cursorp->get(cursorp,
                                &key,
                                &data,
                                DB_NEXT))) {
    // С DBT делаем что-нибудь полезное
}
```

```
if (ret != DB_NOTFOUND) {  
    // А это ошибка и ее нужно обработать ...  
}  
  
...  
// Курсор должен быть закрыт  
if (cursorp != NULL) {  
    cursorp->close(cursorp);  
}  
  
...  
// до того, как будет закрыта его база данных  
if (my_database != NULL) {  
    my_database->close(my_database, 0);  
}
```

Чтобы выполнить итерирование по базе от последней записи к первой, следует использовать **DB_PREV** вместо **DB_NEXT**:

```
#include <db.h>
#include <string.h>
...
DB *my_database;
DBC *cursorp;
DBT key, data;
int ret;

// Открытие БД опущено
// Получаем курсор
my_database->cursor(my_database, NULL, &cursorp, 0);

// Инициализируем обе DBT
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

// Итерируем по БД, извлекая каждую запись в цикле
while (0 == (ret = cursorp->get(cursorp,
                                &key,
                                &data,
                                DB_PREV))) {
    // Что-нибудь полезное с DBT делаем
}
if (ret != DB_NOTFOUND) {
    // А это ошибка и ее нужно обработать ...
}
```

Поиск записей

Можно использовать курсоры для поиска записей в базе.

Можно выполнять поиск только по ключу или искать одновременно по ключу и данным.

Также можно выполнять поиск по частичным совпадениям, если база данных поддерживает отсортированные наборы дубликатов.

Во всех случаях параметры ключа и данных этих методов заполняются значениями ключа и данных записи базы данных, на которую позиционируется курсор в результате поиска.

Если поиск не удался, то состояние курсора остается неизменным и возвращается **DB_NOTFOUND**.

Чтобы использовать курсор для поиска записи, следует использовать **DBC->get()**.

При использовании этого метода можно указать несколько флагов.

Если курсор проверяет только ключевую часть записей, в имени флагов курсора используется **SET** — в этом случае курсор устанавливается на запись, ключ которой соответствует значению, предоставленному курсору.

DB_SET — Перемещает курсор на первую запись в базе данных с указанным ключом.

DB_SET_RANGE — Идентичен **DB_SET**, но только в том случае, когда не используется тип доступа **BTree**. В этом случае курсор перемещается на первую запись в базе данных, ключ которой больше или равен указанному ключу. Это сравнение определяется функцией сравнения, которую необходимо предоставить базе данных.

Если функция сравнения не предусмотрена, то используется лексикографическая сортировка по умолчанию.

Когда курсор использует флаги, в имени которого присутствует **GET**, курсор позиционируется как на ключ, так и на значения данных.

Независимо от ключевого слова, которое используется для получения записи с курсором, **DBT** ключа и данных курсора заполняются данными, извлеченными из записи, на которой в данный момент расположен курсор.

DB_SET

Перемещает курсор к указанной паре ключ/данные базы данных и возвращает данные, связанные с данным ключом.

Если установлен **DB_SET**, но соответствующие ключи не найдены, **DBcursor->get()** вернет **DB_NOTFOUND**.

При наличии повторяющихся значений ключа **DBcursor->get()** вернет первый элемент данных для данного ключа.

DB_SET_RANGE

Перемещает курсор на указанную пару ключ/данные базы данных.

В случае метода доступа Btree возвращается ключ вместе с элементом данных, а возвращаемая пара **key/data** представляет собой наименьший ключ, больший или равный указанному ключу, как определено функцией сравнения Btree.

Это позволяет реализовать частичные ключевые совпадения и поиск по диапазону.

Например, предположим, что у нас есть записи базы данных, в которых в качестве ключей используются следующие строки:

Алабама

Аляска

Аризона

Затем при предоставлении ключа поиска **Аляска** курсор перемещается ко второму ключу.

Если представлен ключ **Ал**, курсор переместится к первому ключу (Алабама).

Если представлен ключ **Аляс**, курсор переместится ко второму ключу (Аляска).

Если представлен ключ **Ар**, курсор переместится к последнему ключу (Аризона).

DB_GET_BOTH

Курсор устанавливается на пару ключ/данные если и **key**, и **data** одновременно соответствуют значениям, указанным в параметрах **key** и **data**.

Во всем остальном этот флаг идентичен флагу **DB_SET**.

При использовании с **DBcursor->pget()** для дескриптора вторичного индекса и вторичный, и первичный ключи должны соответствовать элементу вторичного и первичного ключа в базе данных.

DB_GET_BOTH_RANGE

Перемещает курсор на указанную пару ключ/данные базы данных и возвращает данные.

Параметр **key** должен точно совпадать с ключом в базе данных, параметр **data** — не обязательно.

Извлеченный элемент данных — это элемент в повторяющемся наборе, который имеет наименьшее значение, которое *больше или равно* значению, предоставленному параметром **data** в порядке действующей функции сравнения.

Если указан этот флаг для базы данных, настроенной без поддержки сортировки дубликатов, поведение идентично флагу **DB_GET_BOTH**. Во всем остальном этот флаг идентичен флагу **DB_GET_BOTH**.

Например, база данных использует BTree и в ней есть записи, в которых используются следующие пары ключ/данные:

Alabama/Athens

Alabama/Florence

Alaska/Anchorage

Alaska/Fairbanks

Arizona/Avondale

Arizona/Florence

Ключ поиска	Данные	Положение курсора
Alaska	Fa	Alaska/Fairbanks
Arizona	Fl	Arizona/Florence
Alaska	An	Alaska/Anchorage

Пример кода

Пусть база данных содержит отсортированные повторяющиеся записи пар ключ/данные в виде строк – «штат»/«город», тогда следующий фрагмент кода можно использовать для установки курсора на любую запись в базе данных и вывода значений ее ключа и данных:

```
#include <db.h>
#include <string.h>

...

DBC *cursorp;
DBT key, data;
DB *dbp;
int ret;
char *search_data = "Fa";
char *search_key = "Alaska";

// database open omitted

// Получаем курсор
dbp->cursor(dbp, NULL, &cursorp, 0);

// Устанавливаем наши DBT
key.data = search_key;           // Alaska
key.size = strlen(search_key) + 1;
data.data = search_data;        // Fa
data.size = strlen(search_data) + 1;
```

```
// Позиционируем курсор на первую запись в базе данных, ключ которой
// соответствует ключу поиска и чьи данные начинаются с данных поиска
ret = cursorp->get(cursorp, &key, &data, DB_GET_BOTH_RANGE);
if (!ret) {
    // Что-то делаем с полученными данными
} else {
    // Ошибка, тоже обрабатываем
}

// Закрываем курсор
if (cursorp != NULL) {
    cursorp->close(cursorp);
}

...
// Прежде, чем закрываем БД
if (dbp != NULL) {
    dbp->close(dbp, 0);
}
```

Работа с повторяющимися записями

Запись является дубликатом другой записи, если две записи имеют один и тот же ключ.

Для повторяющихся записей уникальной является только часть записи – данные.

Повторяющиеся записи поддерживаются только для методов доступа **BTree** и **Hash**.

Если база данных поддерживает повторяющиеся записи, то потенциально она может содержать несколько записей с одним и тем же ключом.

По умолчанию обычные операции получения базы данных возвращают только первую такую запись в наборе повторяющихся записей. Поэтому доступ к последующим повторяющимся записям следует осуществлять с помощью курсора.

При работе с базами данных, поддерживающими повторяющиеся записи, используются следующие флаги метода **DBC->get()**:

DB_NEXT, DB_PREV – выдает следующую/предыдущую запись, независимо от того, является ли она дубликатом текущей записи.

DB_GET_BOTH_RANGE – используется для поиска с помощью курсора определенной записи, независимо от того, является ли она дублирующейся.

DB_NEXT_NODUP, DB_PREV_NODUP – выдает следующую/предыдущую неповторяющуюся запись в базе данных.

Флаг позволяет пропустить все дубликаты в наборе повторяющихся записей. Если вызывается **DBC->get()** с **DB_PREV_NODUP**, то курсор позиционируется на последнюю запись для предыдущего ключа в базе данных.

Например, если в базе данных есть следующие записи:

Alabama / Athens
Alabama / Florence <==
Alaska / Anchorage
Alaska / Fairbanks <--
Arizona / Avondale
Arizona / Florence

и курсор позиционируется на **Alaska/Fairbanks**, после чего вызывается **DBC->get()** с **DB_PREV_NODUP**, то курсор позиционируется на **Alabama/Florence**.

Аналогично, если вызывается **DBC->get()** с **DB_NEXT_NODUP**, то курсор позиционируется на первую запись, соответствующую следующему ключу в базе данных.

Если в базе данных нет следующего/предыдущего ключа, то возвращается **DB_NOTFOUND**, а курсор остается на месте.

DB_NEXT_DUP – выдает следующую запись, которая использует текущий ключ.

Если же курсор находится на последней записи в наборе дубликатов и вызываете **DBC->get()** с **DB_NEXT_DUP**, то возвращается **DB_NOTFOUND**, а курсор остается на месте.

Следующий фрагмент кода устанавливает курсор на ключ и выводит соответствующую запись и все ее дубликаты.

```
#include <db.h>
#include <string.h>
...
DB *dbp;
DBC *cursorp;
DBT key, data;
```

```
int ret;
char *search_key = "A1";

// database open omitted

// Получаем курсор
dbp->cursor(dbp, NULL, &cursorp, 0);

// Устанавливаем наши DBT
key.data = search_key;
key.size = strlen(search_key) + 1;

// Позиционируем курсор на первую запись в базе данных, ключ и дата которой
// начинаются с нужной строки

ret = cursorp->get(cursorp, &key, &data, DB_SET);
while (ret != DB_NOTFOUND) { // Если нашлась, циклим по дубликатам
    printf("key: %s, data: %s\n", (char *)key.data, (char *)data.data);
    ret = cursorp->get(cursorp, &key, &data, DB_NEXT_DUP);
}

// Закрываем курсор
if (cursorp != NULL) {
    cursorp->close(cursorp);
}

...
// Прежде, чем закрываем БД
if (dbp != NULL) {
    dbp->close(dbp, 0);
}
```

Путем побитового включающего ИЛИ в параметре **flags** метода **get()** могут быть установлены следующие флаги:

DB_READ_COMMITTED

Настраивает транзакционную операцию **get** для изоляции степени 2 (чтение не повторяющееся).

DB_READ_UNCOMMITTED

Элементы базы данных, прочитанные во время транзакционного вызова, будут иметь изоляцию степени 1, включая измененные, но еще не зафиксированные данные.

Молча игнорируется, если при открытии базовой базы данных не был указан флаг **DB_READ_UNCOMMITTED**.

DB_RMW

Вместо блокировок чтения при чтении метод получает блокировки записи.

Установка этого флага может устранить взаимоблокировку во время цикла «чтение-изменение-запись» путем получения блокировки записи во время части чтения цикла, так что другой поток управления получит блокировку чтения для того же элемента в своем собственном цикле «чтение-изменение-запись», что не приведет к взаимоблокировке.

DB_MULTIPLE

Запрос возврата нескольких элементов данных в параметре **data**.

В случае баз данных Btree или Hash дублирующиеся элементы данных для текущего ключа, начиная с текущей позиции курсора, помещаются в буфер.

Последующие вызовы с указанными флагами **DB_NEXT_DUP** и **DB_MULTIPLE** будут возвращать дополнительные повторяющиеся элементы данных, связанные с текущим ключом, или **DB_NOTFOUND**, если нет дополнительных повторяющихся элементов данных для возврата.

Последующие вызовы с указанными флагами **DB_NEXT** и **DB_MULTIPLE** вернут дополнительные повторяющиеся элементы данных, связанные с текущим ключом, или, если дополнительных повторяющихся элементов данных нет, вернут следующий ключ и его элементы данных.

Если же в базе данных нет дополнительных ключей, вернут **DB_NOTFOUND**.

DB_MULTIPLE_KEY

Возвращает несколько пар ключ/данные в параметре **data**.

Пары ключей и данных, начиная с текущей позиции курсора, помещаются в буфер.

Последующие вызовы с указанными флагами **DB_NEXT** и **DB_MULTIPLE_KEY** будут возвращать дополнительные пары ключ/данные или **DB_NOTFOUND**, если нет дополнительных ключей и элементов данных для возврата.

В случае баз данных Btree, Hash или Heap несколько пар ключей и данных можно перебирать с помощью макроса **DB_MULTIPLE_KEY_NEXT**.

В случае баз данных Queue или Rcpno несколько номеров записей и пар данных можно перебирать с помощью макроса **DB_MULTIPLE_RECNO_NEXT**.

Буфер, на который ссылается параметр данных, должен быть предоставлен из пользовательской памяти (**DB_DBT_USERMEM**).

Буфер должен быть по крайней мере такого же размера, как размер страницы основной базы данных, и выровнен для доступа к беззнаковым целым числам (`unsigned int`) и быть кратным 1024 байтам.

Если размер буфера недостаточен, то при возврате из вызова будет возвращена ошибка **DB_BUFFER_SMALL**, а в поле размера параметра данных будет установлен предполагаемый размер буфера.

Размер этот является приблизительным, поскольку точный необходимый размер может быть неизвестен до тех пор, пока не будут прочитаны все записи. Поэтому слкдует изначально предоставить относительно большой буфер, но приложения должны быть готовы изменять размер буфера по мере необходимости и неоднократно вызывать метод.

Флаги **DB_MULTIPLE** и **DB_MULTIPLE_KEY** можно использовать только с флагами:

DB_SET, **DB_SET_RANGE**, **DB_SET_RECNO**

DB_CURRENT

DB_FIRST

DB_NEXT, **DB_NEXT_DUP**, **DB_NEXT_NODUP**

DB_GET_BOTH, **DB_GET_BOTH_RANGE**

Флаг **DB_MULTIPLE** нельзя использовать при доступе к базам данных, преобразованным во вторичные индексы, с помощью метода **DB->associate()**.

Размещение записей с помощью курсоров

Можно использовать курсоры для помещения записей в базу данных.

Поведение БД при этом различается в зависимости от:

- флагов, которые используется при записи;
- используемого метода доступа;
- поддерживает ли база данных отсортированные дубликаты.

При помещении записей в базу данных с помощью курсора курсор устанавливается на вставленной записи.

Для помещения (записи) записей в базу данных используется метод **DBC->put()**.

```
#include <db.h>

int DBcursor->put(DBC *DBcursor,
                  DBT *key,
                  DBT *data,
                  u_int32_t flags);
```

Если не указано иное, метод **DBcursor->put()** возвращает:

- 0 в случае успеха;
- ненулевое значение ошибки в случае неудачи.

Если **DBcursor->put()** по какой-либо причине завершится неудачно, состояние курсора останется неизменным.

Если **DBcursor->put()** завершается успешно и элемент вставляется в базу данных, курсор всегда располагается так, чтобы ссылаться на вновь вставленный элемент.

С этим методом можно использовать следующие флаги:

DB_AFTER

В случае методов доступа Btree и Hash вставляет элемент данных как дубликат ключа **key**, на который ссылается курсор.

DB_BEFORE

В случае методов доступа Btree и Hash вставляет элемент данных как дубликат ключа **key**, на который ссылается курсор.

Новый элемент появляется сразу перед текущей позиции курсора.

Если база данных Btree или Hash не настроена для неотсортированных повторяющихся элементов данных, указание **DB_AFTER** или **DB_BEFORE** является ошибкой. Параметр **key** в этом случае игнорируется.

Если текущая запись курсора уже удалена и базовым методом доступа является Hash, метод **DBcursor->put()** вернет **DB_NOTFOUND**.

DB_NODUPDATA

Если предоставленный ключ в базе данных уже существует, этот метод возвращает значение **DB_KEYEXIST**. Если ключ не существует, то порядок помещения записи в базу данных определяется порядком вставки, используемым базой данных:

- если в базе данных предусмотрена функция сравнения, запись вставляется в отсортированное место;
- в противном случае (если BTree) используется лексикографическая сортировка, при которой более короткие элементы упорядочиваются перед более длинными элементами.

Этот флаг можно использовать только для методов доступа BTree и Hash и только в том случае, если база данных настроена на поддержку отсортированных повторяющихся элементов данных (во время создания базы данных был указан флаг **DB_DUPSORT**).

DB_KEYFIRST

Для баз данных, не поддерживающих дубликаты, этот метод ведет себя точно так же, как если бы была выполнена вставка по умолчанию.

Если же база данных поддерживает повторяющиеся записи и была предоставлена функция сортировки для повторяющихся данных, вставляемый элемент данных добавляется в отсортированное место.

Если ключ уже существует, а функция дублирующей сортировки не указана, вставленный элемент данных добавляется в качестве первого из элементов данных для этого ключа.

DB_KEYLAST

Ведет себя точно так же, как если бы использовался **DB_KEYFIRST**, за исключением того, что если ключ уже существует в базе данных и функция сортировки дубликатов не указана, вставленный элемент данных добавляется как последний из элементов данных для этого ключа.

Пример

```
#include <db.h>
#include <string.h>

...
DB    *dbp;
DBC    *cursorp;
DBT    data1, data2, data3;
DBT    key1, key2;
char *key1str  = "Моя первая строка";
char *data1str = "Мои первые данные";
char *key2str  = "Вторая строка";
char *data2str = "Мои вторые данные";
char *data3str = "Мои третьи данные";
int    ret;

// Опускаем открытие БД

// Устанавливаем наши DBT
key1.data  = key1str;
key1.size  = strlen(key1str) + 1;
data1.data = data1str;
data1.size = strlen(data1str) + 1;

key2.data  = key2str;
key2.size  = strlen(key2str) + 1;
data2.data = data2str;
data2.size = strlen(data2str) + 1;
data3.data = data3str;
data3.size = strlen(data3str) + 1;
```

```
// Получаем курсор
dbp->cursor(dbp, NULL, &cursorp, 0);

// Предполагая, что база данных пуста, сначала помещаем пару
// «Моя первая строка»/«Мои первые данные» в первую позицию в базе данных.
ret = cursorp->put(cursorp, &key1, &data1, DB_KEYFIRST);

// Далее помещаем «Вторая строка»/«Мои вторые данные» в базу данных в соответствии
// с сортировкой ключей по ключу, используемому для существующей в данный момент
// записи базы данных. Эта запись окажется первой в базе данных.
ret = cursorp->put(cursorp, &key2, &data2, DB_KEYFIRST); // в порядке сортировки

// Если дубликаты не разрешены, существующая в данный момент запись, использующая
// key2, перезаписывается данными, в этом месте предоставленными. То есть запись
// "Вторая строка"/"Мои вторые данные" становится "Вторая строка"/"Мои третьи данные"
// Если дубликаты разрешены, то «Мои третьи данные» помещаются в список дубликатов в
// соответствии с тем, как они сортируются по «Мои вторые данные».
ret = cursorp->put(cursorp, &key2, &data3, DB_KEYFIRST);
```

Если дубликаты не разрешены, запись будет перезаписана новыми данными

Если дубликаты разрешены, запись будет добавлена в начало списка дубликатов

Удаление записей с помощью курсоров

Чтобы удалить запись с помощью курсора, нужно поместите курсор на запись, которую необходимо удалить, после чего вызвать метод **DBC->del()**.

Пример

```
#include <db.h>
#include <string.h>

...

DB    *dbp;
DBC   *cursorp;
DBT    key, data;
char *key1str = "My first string";
int    ret;

// Опускаем открытие БД

// Инициализация DBT
memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

// Устанавливаем DBT
key.data = key1str;
key.size = strlen(key1str) + 1;
```



```
// Получаем курсор
dbp->cursor(dbp, NULL, &cursorp, 0);

// Итерируем по БД, удаляя каждую запись
while (0 == (ret = cursorp->get(cursorp, &key, &data, DB_SET))) {
    cursorp->del(cursorp, 0);
}

// Закрываем курсор
if (cursorp != NULL) {
    cursorp->close(cursorp);
}

...
// Прежде, чем закрываем БД
if (dbp != NULL) {
    dbp->close(dbp, 0);
}
```

Замена записей с помощью курсоров

Для замены записей используется метод **DBC->put()** с флагом **DB_CURRENT**.

DB_CURRENT

Перезаписывает данные пары ключ/данные, на которую ссылается курсор, указанным элементом данных. Ключевой параметр игнорируется.

Если текущая запись курсора уже удалена, **DBcursor->put()** вернет **DB_NOTFOUND**.

Пример

```
#include <db.h>
#include <string.h>

...

DB    *dbp;
DBC   *cursorp;
DBT    key, data;
char *key1str      = "My first string";
char *replacement_data = "replace me";
int    ret;

// Инициализация DBT
memset(&key,  0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));
```

```
// Устанавливаем DBT
key.data = key1str;
key.size = strlen(key1str) + 1;

// Отпускаем открытие БД

// Получаем курсор
dbp->cursor(dbp, NULL, &cursorp, 0);

// Позиционируем
ret = cursorp->get(cursorp, &key, &data, DB_SET);
if (ret == 0) { // Если все в порядке
    data.data = replacement_data;
    data.size = strlen(replacement_data) + 1;
    cursorp->put(cursorp, &key, &data, DB_CURRENT);
}

// Закрываем курсор
if (cursorp != NULL) {
    cursorp->close(cursorp);
}

...
// Прежде, чем закрываем БД
if (dbp != NULL) {
    dbp->close(dbp, 0);
}
```

!!! Важно !!!

Используя этот метод невозможно изменить ключ записи — при замене записи ключевая компонента всегда игнорируется.

При замене части данных записи, если заменятся запись, являющаяся членом набора отсортированных дубликатов, замена будет успешной, только в том случае если новая запись сортируется так же, как и старая запись.

Это означает, что если заменяется запись, входящая в набор отсортированных дубликатов, и если используется лексикографическая сортировка по умолчанию, то в случае нарушения порядка сортировки замена выполнена не будет. Однако, если предоставлена пользовательская процедура сортировки, которая, например, сортирует на основе всего нескольких байтов из элемента данных, то потенциально можно не нарушить вышеуказанные ограничения и выполнить прямую замену.

В обстоятельствах, если необходимо заменить данные, содержащиеся в дублирующейся записи, а пользовательская процедура сортировки не используется, следует удалить запись и создать новую с нужным ключом и данными.

Степени изоляции

Транзакции могут быть изолированы друг от друга в разной степени.

Сериализуемые транзакции (изоляция степени 3) обеспечивают наибольшую изоляцию и означает, что на протяжении всего срока транзакции всякий раз, когда поток управления читает элемент данных, этот элемент данных не будет изменяться (при условии, конечно, что поток управления сам не изменяет его).

По умолчанию Berkeley DB обеспечивает именно сериализуемость всякий раз, когда чтение базы данных оборачивается транзакциями.

Большинству приложений не требуется заключать все операции чтения в транзакции, и, когда это возможно, следует избегать транзакционно защищенных операций чтения с сериализуемой изоляцией, поскольку они могут вызвать проблемы с производительностью.

Например, сериализуемый курсор, последовательно считывающий каждую пару ключ/данные в базе данных, получит блокировку чтения на большинстве страниц базы данных и, таким образом, будет постепенно блокировать все операции записи в базах данных до тех пор, пока транзакция не зафиксируется или не прервется.

Следует обратить внимание, что если в приложении присутствуют транзакции обновления, операции чтения по-прежнему должны использовать блокировку и должны быть готовы к повторению любой операции (возможно, закрытия и повторного открытия курсора), которая завершается неудачей с возвращаемым значением `DB_LOCK_DEADLOCK`.

Приложения, которым требуется повторяющееся чтение, — это приложения, которым требуется возможность многократного доступа к элементу данных, зная, что он не изменится (например, операция, изменяющая элемент данных на основе его существующего значения).

Изоляция моментальных снимков (Snapshot Isolation) — это второй механизм обеспечения сериализуемости, в котором предпочтение отдается читателю, а не средствам обновления.

При использовании многоверсионного управления параллелизмом (MVCC) вместо того, чтобы требовать от читателей блокировки чтения при просмотре страницы, именно программы обновлений выполняют большую часть работы по поддержке изоляции степени 3. Это делает операции обновления более затратными, поскольку им приходится выделять место для новых версий страниц в кэше и делать копии, но отказ от блокировок чтения может значительно повысить пропускную способность для многих приложений.

Изоляция моментальных снимков подробно обсуждается ниже.

Транзакция может требовать только стабильности курсора, то есть гарантии того, что курсоры видят зафиксированные данные, которые не изменяются, пока к ним этот курсор обращается, но могут измениться до завершения транзакции чтения. Это также называется изоляцией **второй степени**.

Berkeley DB обеспечивает этот уровень изоляции, когда транзакция запускается с флагом **DB_READ_COMMITTED**.

Этот флаг также можно указать при открытии курсора в полностью изолированной транзакции.

Berkeley DB дополнительно поддерживает чтение незафиксированных данных — то есть операции чтения могут запрашивать данные, которые были изменены, но еще не зафиксированы другой транзакцией. Это также называется изоляцией **первой степени**.

Это делается путем указания флага **DB_READ_UNCOMMITTED** при открытии базовой базы данных, а затем указания флага **DB_READ_UNCOMMITTED** при старте транзакции, открытии курсора или выполнении операции чтения. Преимущество использования **DB_READ_UNCOMMITTED** заключается в том, что операции чтения не будут блокироваться, когда другая транзакция удерживает блокировку записи запрошенных данных. Недостатком является то, что операции чтения могут возвращать данные, которые исчезнут, если транзакция, удерживающая блокировку записи, прервется.

Изоляция моментальных снимков (Snapshot Isolation)

Изоляция моментальных снимков происходит, когда к базам данных с поддержкой многоверсионного управления параллелизмом (MVCC) обращаются транзакции **DB_TXN_SNAPSHOT**. Приложение сначала включает **DB_MULTIVERSION** при открытии баз данных, для которых требуется изоляция моментальных снимков.

Приложение завершает настройку изоляции моментальных снимков, включая **DB_TXN_SNAPSHOT** при начале транзакции или курсора.

Обычно изоляция моментальных снимков настраивается для каждой базы данных и транзакции, и только там, где это необходимо. Тем не менее, иногда полезно установить изоляцию моментального снимка для всего дескриптора среды:

```
dbenv->set_flags(dbenv, DB_MULTIVERSION, 1);  
dbenv->set_flags(dbenv, DB_TXN_SNAPSHOT, 1);
```

Это может быть полезно для клиентов репликации, которые разгружают запросы на чтение с главного сайта.

При настройке среды для изоляции моментальных снимков важно понимать, что наличие нескольких версий страниц в кеше означает, что рабочий набор будет занимать большую часть кеша. В результате изоляция моментальных снимков лучше всего подходит для использования с кэшами большего размера.

Если кэш заполняется копиями страниц до того, как старые копии могут быть удалены, произойдет дополнительный ввод-вывод, поскольку страницы записываются во временные файлы «заморозки». Это может существенно снизить пропускную способность, и по возможности этого следует избегать, настраивая большой кэш и сокращая транзакции изоляции моментальных снимков.

Объем кэша, необходимый для предотвращения замораживания буферов, можно оценить, выполнив контрольную точку, за которой следует вызов **DB_ENV->log_archive()**. Требуемый объем кэша примерно вдвое превышает размер оставшихся журналов.

Окружение также следует настроить для достаточного количества транзакций с помощью **DB_ENV->set_tx_max()**. Максимальное количество транзакций должно включать все транзакции, выполняемые приложением одновременно, а также все курсоры, настроенные для изоляции моментальных снимков. Кроме того, транзакции сохраняются до тех пор, пока последняя созданная ими страница не будет удалена из кэша, поэтому в крайнем случае для каждой страницы в кеше может потребоваться дополнительная транзакция. Обратите внимание, что размеры кэша менее 500 МБ увеличиваются на 25 %, поэтому при расчете количества страниц это необходимо учитывать.

Так когда же приложениям следует использовать изоляцию моментальных снимков?

- имеется большой кэш относительно размера обновлений, выполняемых одновременными транзакциями;
- конкуренция операций чтения/записи ограничивает пропускную способность приложения;
- приложение полностью или по большей части только читает.

Самый простой способ воспользоваться изоляцией моментальных снимков для запросов состоит в сохранении транзакции обновления, используя полную блокировку чтения/записи, и установлении флага **DB_TXN_SNAPSHOT** для транзакций или курсоров, доступных только для чтения.

Это должно свести к минимуму блокировку транзакций изоляции моментальных снимков и позволит избежать возникновения новых ошибок **DB_LOCK_DEADLOCK**.

Если в приложении есть транзакции обновления, которые считывают множество элементов и обновляют только небольшой набор (например, сканирование до тех пор, пока не будет найдена нужная запись, а затем ее изменение), пропускную способность можно повысить, запустив некоторые обновления в режиме изоляции моментальных снимков.

DB_NEXT_NODUP (DB_PREV_NODUP)

Если курсор еще не инициализирован, **DB_NEXT_NODUP (DB_PREV_NODUP)** идентичен **DB_FIRST (DB_LAST)**.

В противном случае курсор перемещается на следующий (предыдущий) *неповторяющийся* ключ базы данных, и эта пара ключ/данные возвращается.

Если база данных представляет собой базу данных Queue или Resno, **DBcursor->get()** с флагом **DB_NEXT_NODUP (DB_PREV_NODUP)** будет игнорировать любые ключи, которые существуют, но никогда не были явно созданы приложением, или те, которые были созданы и позже удалены.

Если установлен **DB_NEXT_NODUP (DB_PREV_NODUP)** и после (перед) позиции курсора в базе данных не существует неповторяющихся пар ключ/данные, **DBcursor->get()** вернет **DB_NOTFOUND**.

При использовании базы данных кучи этот флаг идентичен флагу **DB_NEXT (DB_PREV)**.

Базы данных

Лекция 15 – Berkeley DB

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2023.12.01

Оглавление

Вторичные базы данных.....	3
Открытие и закрытие вторичных баз данных.....	6
Реализация экстракторов ключей.....	13
Работа с несколькими ключами.....	19
Чтение вторичных баз данных.....	22
Удаление вторичных записей базы данных.....	24
Использование курсоров со вторичными базами данных.....	27
Соединения (Join).....	29
Использование join-курсоров.....	31
DB->set_flags().....	36

Вторичные базы данных

Обычно поиск записей базе данных выполняется с помощью ключей записи. Однако используемый ключ не всегда будет содержать информацию, необходимую для предоставления быстрого доступа к данным, которые необходимо получить.

Например, база данных содержит записи, относящиеся к пользователям.

Ключ может быть строкой, представляющей собой уникальный идентификатор человека, например идентификатор пользователя.

Данные каждой записи, скорее всего, будут содержать сложный объект, содержащий сведения о людях, такие как имена, адреса, номера телефонов и т. д.

Хотя приложение может запрашивать человека по идентификатору пользователя (то есть по информации, хранящейся в ключе), иногда бывает необходимо найти кого-то по имени.

Вместо того, чтобы перебирать все записи в вашей базе данных, проверяя каждую по очереди на наличие имени данного человека, создаются индексы на основе имен, после чего в этом индексе ищется нужное имя.

Это можно сделать, используя вторичные базы данных.

В BDB база данных, содержащая данные, называется *первичной базой данных*.

База данных, предоставляющая альтернативный набор ключей для доступа к этим данным, называется *вторичной базой данных*.

Во вторичной базе данных ключи — это альтернативный (или вторичный) индекс, а данные соответствуют ключу первичной записи.

Вторичная база данных создается следующим образом:

- создается база данных;
- открывается;
- связывается (associated) с первичной базой данных (то есть с базой данных, для которой создается индекс).

В рамках связывания вторичной базы данных с первичной необходимо предоставить обратный вызов, который используется для создания ключей вторичной базы данных. Обычно этот обратный вызов создает ключ на основе данных, найденных в ключе или в данных первичной записи базы данных.

1) Добавление или удаление записей в первичной базе данных приводит к тому, что BDB по мере необходимости обновляет вторичную базу.

2) Изменение данных записи в первичной базе данных может привести к тому, что BDB изменит запись во вторичной базе данных, в зависимости от того, вызывает ли это изменение модификацию ключа во вторичной базе данных.

!! ВАЖНО !!! Напрямую писать во вторичную базу данных невозможно – попытка записи во вторичную базу данных приводит к возврату ненулевого результата.

3) Чтобы изменить данные, на которые ссылается вторичная запись, нужно изменить первичную базу данных.

4) Исключением из этого правила является то, что для вторичной базы данных разрешены операции удаления.

5) Записи вторичной базы данных обновляются/создаются только в том случае, если функция обратного вызова, указанная для создания ключа возвращает 0.

6) Если возвращается значение, отличное от 0, то во вторичную базу данных ключ не добавляется, а в случае обновления записи любой существующий ключ будет удален.

Чтобы указать, что запись не должна индексироваться функция обратного вызова может использовать либо **DB_DONOTINDEX**, или какой-нибудь код ошибки за пределами пространства имен BDB.

7) При чтении записи из вторичной базы данных возвращаются данные и, возможно, ключ из соответствующей записи в первичной базе данных.

Открытие и закрытие вторичных баз данных

Открытие и закрытие вторичной базы данных выполняется так же, как для любой обычной базы данных. Единственная разница состоит в том, что:

- вторичную базу данных необходимо связать с первичной, используя метод **DB->associate()**.
- при закрытии баз данных, прежде чем закрывать первичные, рекомендуется убедиться, что вторичные базы данных уже закрыты. Это особенно важно, если закрытие базы данных не является однопоточным.

При связывании вторичную базы данных с первичной, необходимо предоставить функцию обратного вызова, которая используется для создания ключей вторичной базы данных.

```
#include <db.h>

int DB->associate(DB      *primary,    // Первичная БД
                  DB_TXN  *txnid,      // Транзакционный контекст
                  DB      *secondary,  // Вторичная БД
                  int      (*callback)(DB      *secondary,
                                         const DBT *key,
                                         const DBT *data,
                                         DBT      *result),
                  u_int32_t flags);    // Флаги
```

DB->associate()

```
#include <db.h>

int DB->associate(DB      *primary,    // Первичная БД
                  DB_TXN  *txnid,      // Транзакционный контекст
                  DB      *secondary,   // Вторичная БД
                  int      (*callback)(DB      *secondary,
                                         const DBT *key,      // prim.key
                                         const DBT *data,      // prim.data
                                         DBT      *result),    //
                  u_int32_t flags);     // Флаги
```

Функция DB->associate() используется для объявления одной базы данных вторичным индексом для первичной базы данных.

Дескриптор DB, из которого вызывается метод **associate()**, является первичной базой данных.

После того как база данных-получатель «связана» с первичной базой данных, все обновления первичной БД будут автоматически отражаться во вторичной базе данных, и все операции чтения из вторичной базы данных будут возвращать соответствующие данные из первичной базы данных.

ВАЖНО: поскольку для работы вторичных индексов первичные ключи должны быть уникальными, первичную базу данных необходимо настроить без поддержки дубликатов.

Метод **DB->associate()** возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

Параметры

primary — дескриптор первичной базы данных, которую необходимо индексировать.

txnid — если операция является частью транзакции, это дескриптор транзакции, в противном случае **NULL**.

secondary — параметр должен быть:

- дескриптором открытой базы данных;
- вновь созданной и пустой базы данных, которая будет использоваться для хранения вторичного индекса;
- базы данных, которая ранее была связана с той же первичной базой данных и содержала вторичный индекс.

!!! ВАЖНО !!! , небезопасно связывать в качестве вторичной базы данных дескриптор, который используется другим потоком управления или имеет открытые курсоры.

Следует обратить внимание, что или вторичные ключи должны быть уникальными, или вторичная база данных должна быть настроена с поддержкой повторяющихся элементов данных.

callback — функция обратного вызова, которая создает набор вторичных ключей, соответствующих заданному первичному ключу и паре данных.

Параметр обратного вызова может иметь значение **NULL**, если дескрипторы первичной и вторичной базы данных были открыты с флагом **DB_RDONLY**.

flags — параметр должен быть установлен в 0 или путем побитового ИЛИ одного или нескольких значений:

DB_CREATE — если вторичная база данных пуста, выполняется проход по первичной базе данных и в пустой вторичной базе создается для нее индекс. Эта операция потенциально очень дорогая.

Если вторичная база данных была открыта в среде, настроенной с использованием транзакций, все создание вторичного индекса выполняется в контексте одной транзакции.

!!! ВАЖНО !!! Не следует использовать вновь заполненную базу данных-получатель в другом потоке управления до тех пор, пока вызов **DB->associate()** не вернет успешный результат в первом потоке.

Также, если транзакции не используются, не следует изменять первичную базу данных, используемую для заполнения вторичной базы данных в другом потоке управления, пока вызов **DB->associate()** не вернет успешный результат в первом потоке. Если используются транзакции, Berkeley DB выполнит соответствующую блокировку, и приложению не потребуется выполнять какое-либо специальное упорядочивание операций.

DB_IMMUTABLE_KEY —

Указывает, что вторичный ключ является неизменяемым.

Этот флаг можно использовать для оптимизации обновлений, если вторичный ключ в первичной записи не будет изменяться после вставки первичной записи.

Чтобы вторичные ключи не изменялись, обычно не должна вызываться функция обратного вызова вторичной БД при обновлении записей в первичной. Такая оптимизация может значительно снизить накладные расходы на операции обновления, особенно, если функция обратного вызова является дорогостоящей.

!!! Следует указывать этот флаг только в том случае, если вторичный ключ для первичной записи никогда не меняется. Если это правило будет нарушено, вторичный индекс испортится, то есть перестанет синхронизироваться с первичным.

Пример кода для открытия вторичной базы данных и связывания ее с первичной:

```
#include <db.h>

...

DB *dbp, *sdbp;    // Дескрипторы первичной и вторичной БД
u_int32_t flags;    // Флаг открытия первичной БД
int      ret;       // Код возврата

ret = db_create(&dbp, NULL, 0);    // Первичная БД
if (ret != 0) {
    error_processing(...);
}
ret = db_create(&sdbp, NULL, 0);    // Вторичная БД
if (ret != 0) {
    error_processing(...);
}

// Обычно имеет смысл поддерживать дубликаты для вторичных БД
ret = sdbp->set_flags(sdbp, DB_DUPSORT); // конфиг. перед открытием
if (ret != 0) {
    error_processing(...);
}
```

```
// Флаги открытия
flags = DB_CREATE;    // если не существует, следует создать

// Открываем первичную БД
ret = dbp->open(dbp,    // Указатель на дескриптор первичной БД
                NULL,   // Указатель на контекст транзакции
                "my_db.db", // имя файла, содержащего БД
                NULL,    // необязательное логическое имя БД
                DB_BTREE, // метод доступа
                flags,    // флаги открытия
                0);       // Режим файла (по умолчанию)

if (ret != 0) {
    error_processing(...);
}

// Открываем вторичную БД
ret = sdbp->open(sdbp,    // Указатель на дескриптор вторичной БД
                NULL,     // Указатель на контекст транзакции
                "my_secdb.db", // имя файла, содержащего БД
                NULL,      // необязательное логическое имя БД
                DB_BTREE,  // метод доступа
                flags,     // флаги открытия
                0);        // Режим файла (по умолчанию)

if (ret != 0) {
    error_processing(...);
}
```

```
// Связываем вторичную БД с первичной
dbp->associate(dbp,          // первичная БД
              NULL,         // контекст транзакции
              sdbp,         // Вторичная БД
              get_sales_rep, // функция обратного вызова
              0);           // флаги
```

Закрытие первичной и вторичной баз данных выполняется точно так же, как и любой другой:

```
if (sdbp != NULL)
    sdbp->close(sdbp, 0);
if (dbp != NULL)
    dbp->close(dbp, 0);
```

Реализация экстракторов ключей

Каждой вторичной базе данных необходимо предоставить обратный вызов, который создает ключи из первичных записей. Этот обратный вызов предоставляется, когда выполняется связывание вторичной базы данных с первичной.

Можно создавать ключи, используя любые данные. Как правило, ключ основывается на некоторой информации, найденной в данных записи, но можно использовать информацию, найденную и в ключе первичной записи. То, как создаются ключи, полностью зависит от характера индекса, который вы необходимо поддерживать.

Экстрактор ключа реализуется с помощью функции, которая извлекает необходимую информацию из ключа или данных первичной записи. Эта функция должна соответствовать определенному прототипу и должна предоставляться как обратный вызов метода **associate()**.

```
int      (*callback)(DB          *secondary,  
                      const DBT *key,      // ключ первичного файла  
                      const DBT *data,     // данные первичного файла  
                      DBT        *result), // ключ вторичного файла
```

Обратный вызов принимает четыре аргумента:

secondary — дескриптор вторичной базы данных;

key — DBT, ссылающийся на первичный ключ;

data — DBT, ссылающийся на элемент данных в первичной БД;

result — обнуленный DBT, в котором функция обратного вызова должна заполнить поля данных и размера, которые описывают вторичный ключ (или ключи).

DBT

```
typedef
struct __db_dbt {
    void      *data; // ключ/данные
    u_int32_t  size; // размер ключа/данных
    ...
    u_int32_t  flags;
} DBT;
```

Пример

Допустим, записи первичной БД содержат данные, использующие следующую структуру:

```
typedef struct vendor_s {
    char name[MAXFIELD];           // Наименование поставщика
    char street[MAXFIELD];         // Улица и номер
    char city[MAXFIELD];           // Город
    char state[3];                 // Двухзначный код штата
    char zipcode[6];               // Почтовый индекс
    char phone_number[13];         // Телефон поставщика
    char sales_rep[MAXFIELD];      // Имя представителя поставщика
    char sales_rep_phone[MAXFIELD]; // И его телефон
} VENDOR;
```

Предположим, необходимо иметь возможность запрашивать первичную базу данных на основе имени торгового представителя.

В этом случае необходимо написать функцию, которая выглядит так:

```
#include <db.h>
...
int
get_sales_rep(DB      *sdbp,    // дескриптор вторичной БД
               const DBT *pkey,  // ключ записи первичной БД
               const DBT *pdata, // данные записи первичной БД
               DBT      *skey)   // ключ записи вторичной БД
{
    VENDOR *vendor = pdata->data; // извлекаем структуру данных

    // Чистим DBT ключа вторичной БД
    memset(skey, 0, sizeof(DBT));
    // Устанавливаем в качестве вторичного ключа имя представителя.
    skey->data = vendor->sales_rep;
    skey->size = strlen(vendor->sales_rep) + 1;

    // Возвращаем 0 чтобы указать, что запись может быть создана
    return (0);
}
```


Чтобы использовать эту функцию, ее указывают в методе **Associate()** после того, как первичная и вторичная базы данных были созданы и открыты:

```
dbp->associate(dbp,          // дескриптор первичной БД
               NULL,         // контекст транзакции
               sdbp,         // дескриптор вторичной БД
               get_sales_rep, // ф-ция обратного вызова, создающая индекс
               0);           // флаги по умолчанию
```

!!! ВАЖНО !!! Berkeley DB не является реентерантной.

Функции обратного вызова не должны пытаться выполнять вызовы библиотеки (например, для снятия блокировок или закрытия открытых дескрипторов).

Повторный вход в Berkeley DB не гарантируется, и результаты не определены.

Флаги для функции обратного вызова

В поле **flags** результирующего DBT могут быть установлены следующие флаги:

DB_DBT_APPMALLOC – если функция обратного вызова выделяет память для поля данных результата (а не просто указать на первичный ключ или данные), в поле флагов DBT результата следует установить **DB_DBT_APPMALLOC**, что заставит Berkeley DB освободить память, когда функция завершится.

DB_DBT_MULTIPLE – устанавливается в поле **flags** DBT результата и позволяет вернуть несколько вторичных ключей. В поле **size** будет указано количество вторичных ключей (ноль или более), а поле данных будет указателем на массив из **size** DBT, содержащих набор вторичных ключей.

Если возвращается несколько вторичных ключей, ключи не могут повторяться. Это значит, что в массиве для баз данных `Resno` и `Queue` не должно быть повторяющихся номеров записей, а ключи не должны сравниваться одинаково с помощью функции сравнения вторичной базы данных для `Btree` и `Hash`. Если ключи повторяются, операции могут завершиться неудачно, и вторичный ключ может стать несогласующимся с первичным.

Для любого из DBT в массиве возвращаемых DBT может быть установлен флаг **DB_DBT_APPMALLOC**, чтобы указать, что Berkeley DB должна освободить память, на которую ссылается поле данных этого конкретного DBT, когда работа с ней будет завершена.

Флаг **DB_DBT_APPMALLOC** можно комбинировать с **DB_DBT_MULTIPLE** в поле флагов DBT результата, чтобы указать, что Berkeley DB должна освободить целый массив по завершении дел со всеми ключами.

Кроме того, функция обратного вызова может при необходимости возвращать следующее специальное значение:

DB_DONOTINDEX — если какая-либо пара ключ/данные в первичном ключе выдает нулевой вторичный ключ и должна быть исключена из вторичного индекса, функция обратного вызова может дополнительно вернуть **DB_DONOTINDEX**. В противном случае функция обратного вызова должна возвращать 0 в случае успеха или ошибку за пределами пространства имен Berkeley DB в случае неудачи; код ошибки будет возвращен из вызова, который инициировал обратный вызов.

Если функция обратного вызова возвращает **DB_DONOTINDEX** для любых пар ключ/данные в первичной базе данных, вторичный индекс не будет содержать никаких ссылок на эти пары ключ/данные, а такие операции, как итерации курсора и запросы диапазона, будут отражать только соответствующее подмножество базы данных.

Если это нежелательно, приложение должно убедиться, что функция обратного вызова четко определена для всех возможных значений и никогда не возвращает **DB_DONOTINDEX**.

Возврат **DB_DONOTINDEX** эквивалентен установке **DB_DBT_MULTIPLE** в DBT результата и установке поля **size** в ноль.

Работа с несколькими ключами

До сих пор мы обсуждали только индексы, как если бы между вторичным ключом и первичной записью базы данных существовала связь один к одному.

На самом деле, можно сгенерировать несколько ключей для любой данной записи.

Например, предположим, есть база данных, содержащая информацию о книгах.

Предположим, что иногда необходимо искать книги по авторам.

Поскольку иногда у книг есть несколько авторов, бывает необходимо вернуть несколько вторичных ключей для каждой книги, которая индексируется.

Для этого пишется экстрактор ключа, который возвращает DBT, элемент данных которого указывает на массив DBT.

Каждый такой член этого массива содержит один вторичный ключ.

Кроме того, DBT, возвращаемый экстрактором ключей, должен иметь поле размера, равное количеству элементов, содержащихся в массиве DBT.

Кроме того, поле флага для DBT, возвращаемого обратным вызовом, должно включать **DB_DBT_MULTIPLE**.

Пример

```
int
my_callback(DB *dbp, const DBT *pkey, const DBT *pdata, DBT *skey) {

    DBT *tmpdbt;
    char *tmpdata1, tmpdata2;

    // Этап извлечения данных из DBT pkey или pdata, которые будут
    // использоваться для создания вторичных ключей опущен.
    // Предположим, что данные временно сохраняются в двух переменных:
    // tmpdata1 и tmpdata2.
    // Создаем массив DBT, достаточно большой для того количества
    // ключей, которые желаем вернуть. В данном случае используется
    // массив размерности 2

    tmpdbt = malloc(sizeof(DBT) * 2);
    memset(tmpdbt, 0, sizeof(DBT) * 2);

    // Назначаем вторичные ключи каждому элементу массива.
    tmpdbt[0].data = tmpdata1;
    tmpdbt[0].size = (u_int32_t)strlen(tmpdbt[0].data) + 1;
    tmpdbt[1].data = tmpdata2;
    tmpdbt[1].size = (u_int32_t)strlen(tmpdbt[1].data) + 1;
```

```
// Теперь устанавливаем флаги для возвращаемого DBT.  
// DB_DBT_MULTIPLE необходим для того, чтобы DB знала, что DBT  
// ссылается на массив.  
// Кроме того, мы устанавливаем DB_DBT_APPMALLOC, поскольку мы  
// динамически выделяем память для поля данных DBT.  
// DB_DBT_APPMALLOC заставляет DB освободить эту память, как только  
// она закончит работу с возвращенным DBT.
```

```
key->flags = DB_DBT_MULTIPLE | DB_DBT_APPMALLOC;
```

```
key->data = tmpdbt; // Указываем в поле данных DBT адрес массива  
key->size = 2;      // и его размерность
```

```
return (0);
```

```
}
```

Чтение вторичных баз данных

Как и для первичной базы данных, можно считывать записи из вторичной базы данных либо с помощью методов **DB->get()** или **DB->pget()**, либо с помощью курсора, установленного на вторичной базе данных.

Основное различие между чтением вторичной базы данных и первичной заключается в том, что при чтении записи из вторичной базы данных данные этой записи методом не возвращаются.

Вместо этого возвращаются первичный ключ и данные, соответствующие вторичному ключу.

!!! Как и в первичной базе данных, если вторичная база данных поддерживает повторяющиеся записи, то **DB->get()** и **DB->pget()** возвращают только первую запись, найденную в соответствующем наборе дубликатов.

Если необходимо увидеть все записи, относящиеся к определенному вторичному ключу, следует использовать курсор, открытый во вторичной базе данных.

Ниже пример – вторичная база данных содержит ключи, связанные с полным именем человека:

```
#include <db.h>
#include <string.h>
...
DB    *secondary_database;
DBT    key;           // Ключ поиска
DBT    pkey, pdata;   // Для возврата первичного ключа и данных

char *search_name = "John Deer";
/* --- открытие первичной и вторичной баз данных опущено --- */
memset(&key, 0, sizeof(DBT)); // очистка перед использованием
memset(&pkey, 0, sizeof(DBT));
memset(&pdata, 0, sizeof(DBT));

key.data = search_name;
key.size = strlen(search_name) + 1;

// Возвращается ключ из вторичной базы данных и данные из связанной
// записи в первичной БД
secondary_database->get(my_secondary_database, NULL, &key, &pdata, 0);

// Возвращается ключ из вторичной базы данных, а также ключ и данные
// из связанной записи в первичной БД
secondary_database->pget(my_secondary_database, NULL, &key, &pkey,
&pdata, 0);
```


Удаление вторичных записей базы данных

Как правило, во вторичной базе данных напрямую не изменяют.

Чтобы изменить вторичную базу данных, необходимо изменить первичную базу данных и «позволить» DB управлять изменениями во вторичной.

Тем не менее, можно удалять записи вторичной базы данных напрямую, что приводит к удалению связанной пары первичный ключ/данные.

Это, в свою очередь, приводит к тому, что БД удаляет все записи вторичной базы данных, которые ссылаются на первичную запись.

Для удаления записи из вторичной базы данных используется метод **DB->del()**.

АСHTUNG ! Если вторичная база данных содержит повторяющиеся записи, то удаление записи из набора дубликатов приведет к удалению всех дубликатов.

Удалить запись вторичной базы данных с помощью описанного механизма можно только в том случае, если первичная база данных открыта для записи.

```
#include <db.h>
#include <string.h>

...

DB    *dbp, *sdbp;    /* Primary and secondary DB handles */
DBT    key;           /* DBTs used for the delete */
int    ret;           /* Function return value */
char *search_name = "John Doe"; /* Name to delete */
```

```
/* Первичная */
ret = db_create(&dbp, NULL, 0);
if (ret != 0) {
    /* Обработка ошибок */
}

/* Вторичная */
ret = db_create(&sdbp, NULL, 0);
if (ret != 0) {
    /* Обработка ошибок */
}

/* Обычно дубликаты для вторичных баз данных поддерживаются */
ret = sdbp->set_flags(sdbp, DB_DUPSORT);
if (ret != 0) {
    /* Обработка ошибок */
}

/* open the primary database */
ret = dbp->open(dbp, NULL, "my_db.db", NULL, DB_BTREE, 0, 0);
if (ret != 0) {
    /* Обработка ошибок */
}
```

```
/* open the secondary database */
ret = sdbp->open(sdbp, NULL, "my_secdb.db", NULL, DB_BTREE, 0, 0);
if (ret != 0) {
    /* Обработка ошибок */
}

/* Связываем вторичную с первичной. */
dbp->associate(dbp, NULL, sdbp, get_sales_rep, 0);

// Очистка DBT перед использованием
memset(&key, 0, sizeof(DBT));

key.data = search_name;
key.size = strlen(search_name) + 1;

// удаляем вторичную запись.
// Это приводит к удалению связанной первичной записи.
// Если какие-либо другие вторичные базы данных имеют вторичные записи,
// ссылающиеся на удаленную первичную запись, эти вторичные записи
// также удаляются.
sdbp->del(sdbp, NULL, &key, 0);
```

Использование курсоров со вторичными базами данных

Курсоры во вторичных базах данных используются для перебора записей точно так же, как они используются для первичных.

Курсоры со вторичными базами данных можно использовать для поиска определенных записей, для поиска первой или последней записи, для получения следующей дублирующейся записи и т. д.

Однако при использовании курсоров со вторичными базами данных есть особенности:

- любые возвращаемые данные — это данные, содержащиеся в первичной записи базы данных, на которую ссылается вторичная запись.
- нельзя использовать **DB_GET_BOTH** и соответствующие флаги с **DB->get()** и вторичной базой данных. Вместо этого следует использовать **DB->pget()**. В этом случае, чтобы первичная запись была возвращена в результате вызова, первичный и вторичный ключи, заданные при вызове **DB->pget()**, должны совпадать со вторичным ключом и соответствующим ключом первичной записи.

Пример

Поиск имени человека во вторичной базе данных и удаление всех вторичных и первичных записей, в которых используется это имя.

```
#include <db.h>
#include <string.h>
...
DB *sdbp;           // дескриптор вторичной БД
DBC *cursorp;       // Курсор
DBT key, data;       // DBT для удаления
char *search_name = "John Deer"; // Наименование для удаления

// Опускаем код открытия БД

// Получаем курсор для вторичной БД
sdbp->cursor(sdbp, NULL, &cursorp, 0);

memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

key.data = search_name;
key.size = strlen(search_name) + 1;
// позиционируем курсор для удаления
while (cursorp->get(cursorp, &key, &data, DB_SET) == 0)
    cursorp->del(cursorp, 0);
```

Соединения (Join)

Если есть две или более вторичных баз данных, связанных с первичной базой, можно получить первичные записи на основе пересечения нескольких вторичных записей.

Это делается это с помощью курсора соединения (**join cursor**).

Допустим, у нас есть структура, которая хранит информацию о продавцах бакалейных товаров.

Обычно такая структура содержит ограниченное число элементов данных, которые могут быть интересны с точки зрения запроса.

Однако, часто хранится информация о чем-то с гораздо большим количеством характеристик, например об автомобиле. В этом случае можно хранить такую информацию, как цвет, количество дверей, расход топлива, тип автомобиля, количество пассажиров, марку, модель и год выпуска...

В этом случае используется какое-то уникальное значение для первичных записей (для этой цели часто используют VIN автомобиля).

Затем создается структура, которая идентифицирует все характеристики автомобилей.

Чтобы запросить эти данные, можно создать несколько вторичных баз данных, по одной для каждой характеристики, которую есть желание запросить. Например, можно создать вторичную БД для цвета, еще одну для количества дверей, еще одну для количества пассажиров и так далее.

При этом понадобится уникальная функция извлечения ключей для каждой такой вторичной базы данных.

Все это делается с использованием концепций и методов, описанных ниже.

После того, первичная база данных создана, и созданы все интересующие вторичные базы данных, появляется возможность извлекать записи об автомобилях на основе одной характеристики.

Например, можно найти все автомобили красного цвета.

Или можно найти все автомобили, которые имеют четыре двери.

Или все автомобили, которые являются SUV...

Следующим наиболее естественным шагом является формирование составных запросов или **объединений**. Например, найти все автомобили красного цвета производства Toyota и являющиеся минивэнами. Это можно сделать с помощью join-курсора.

Использование join-курсоров

Чтобы использовать курсор соединения, следует:

1) открыть два или более курсоров для вторичных баз данных, связанных с одной и той же первичной базой.

2) поместить каждый такой курсор на значение вторичного ключа, которое интересует.

Например:

курсор для базы данных цветов расположить на красных записях;

курсор для базы данных моделей — на записях минивэнов;

курсор базы данных марок — на Toyota.

3) создать массив курсоров и поместить в него каждый из курсоров, которые участвуют в join-запросе. **Массив должен заканчиваться нулем.**

4) получить join-курсор, используя метод **DB->join()**. Этому методу необходимо передать массив вторичных курсоров, которые были открыты и установлены на предыдущих шагах.

5) итерировать набор совпадающих записей до тех пор, пока код возврата не станет отличным от 0.

6) закрыть join-курсор. По завершению следует закрыть все курсоры.

Пример

```
#include <db.h>
#include <string.h>
...
DB  *automotiveDB;
DB  *automotiveColorDB;
DB  *automotiveMakeDB;
DB  *automotiveTypeDB;
DBC *color_curs, *make_curs, *type_curs, *join_curs;
DBC *carray[4];
DBT  key, data;
int  ret;

char *the_color = "red";
char *the_type  = "minivan";
char *the_make  = "Toyota";

// Database and secondary database opens omitted for brevity.
// Assume a primary database handle:
//   automotiveDB
// Assume 3 secondary database handles:
//   automotiveColorDB  -- secondary database based on automobile color
//   automotiveMakeDB   -- secondary database based on the manufacturer
//   automotiveTypeDB   -- secondary database based on automobile type
```

```
/* initialize pointers and structures */
color_curs = NULL;
make_curs  = NULL;
type_curs  = NULL;
join_curs  = NULL;

memset(&key, 0, sizeof(DBT));
memset(&data, 0, sizeof(DBT));

// (1) open the cursors
ret = automotiveColorDB->cursor(automotiveColorDB, NULL, &color_curs, 0);
if (ret != 0) {
    /* Обработка ошибок */
}
ret = automotiveMakeDB->cursor(automotiveMakeDB, NULL, &make_curs, 0);
if (ret) != 0) {
    /* Обработка ошибок */
}
ret = automotiveTypeDB->cursor(automotiveTypeDB, NULL, &type_curs, 0);
if (ret) != 0) {
    /* Обработка ошибок */
}
```

```
// (2) Позиционируем курсоры
key.data = the_color;
key.size = strlen(the_color) + 1;

ret = color_curs->get(color_curs, &key, &data, DB_SET);
if (ret) != 0) {
    /* Обработка ошибок */
}
key.data = the_make;
key.size = strlen(the_make) + 1;

ret = make_curs->get(make_curs, &key, &data, DB_SET);
if (ret) != 0) {
    /* Обработка ошибок */
}

key.data = the_type;
key.size = strlen(the_type) + 1;

ret = type_curs->get(type_curs, &key, &data, DB_SET);
if (ret) != 0) {
    /* Обработка ошибок */
}
```

```
// (3) Массив курсоров
carray[0] = color_curs;
carray[1] = make_curs;
carray[2] = type_curs;
carray[3] = NULL;

// (4) Создаем соединение
ret = automotiveDB->join(automotiveDB, carray, &join_curs, 0);
if (ret) != 0) {
    /* Обработка ошибок */
}

// (5) Итерируем соединенный курсор
ret = join_curs->get(join_curs, &key, &data, 0);
while (ret) == 0) {
    // делаем что-либо полезное
}

// (6) Если вышли из цикла, потому что закончились записи,
//      то он завершился успешно.
if (ret == DB_NOTFOUND) {
    // Закрываем все курсоры, базы и выходим со статусом (0).
}
```

DB->set_flags()

```
#include <db.h>
```

```
int DB->set_flags(DB *db, u_int32_t flags);
```

Настраивает базу данных.

Вызов **DB->set_flags()** является аддитивным — возможности очистить флаги нет.

DB->set_flags() нельзя вызывать после вызова метода **DB->open()**.

DB->set_flags() возвращает ненулевое значение ошибки в случае неудачи и 0 в случае успеха.

Общие флаги

DB_CHKSUM — проверка контрольной суммы страниц, считанных в кэш из резервного хранилища файлов (SHA1 или общий алгоритм).

Если на момент вызова **DB->open()** база данных уже существует, флаг **DB_CHKSUM** будет игнорироваться.

DB_ENCRYPT

Шифрует базу данных, используя криптографический пароль, указанный в методах **DB_ENV->set_encrypt()** или **DB->set_encrypt()**.

Если база данных уже существует на момент вызова **DB->open()**, флаг **DB_ENCRYPT** должен быть таким же, как у существующей базы данных, иначе будет возвращена ошибка.

Зашифрованные базы данных не переносятся между компьютерами с разным порядком байтов, то есть зашифрованные базы данных, созданные на машинах с прямым порядком байтов, не могут быть прочитаны на машинах с прямым порядком байтов, и наоборот.

DB_TXN_NOT_DURABLE

Если установлен этот флаг, Berkeley DB для этой базы данных не будет записывать в журнал.

Это означает, что обновления этой базы данных используют свойства ACI (атомарность, согласованность и изоляция), но не D (долговечность), то есть целостность базы данных будет сохранена, но в случае сбоя приложения или системы целостность не сохранится.

Файл базы данных должен быть проверен и/или восстановлен из резервной копии после сбоя. Чтобы обеспечить целостность после завершения работы приложения, дескрипторы базы данных должны быть закрыты без указания **DB_NOSYNC**, или все изменения базы данных должны быть удалены из кэша среды с помощью **DB_ENV->txn_checkpoint()** или **DB_ENV->memp_sync()**. Для всех дескрипторов базы данных в одном физическом файле должно быть установлено значение **DB_TXN_NOT_DURABLE**.

Вызов **DB->set_flags()** с флагами **DB_ENCRYPT**, **DB_TXN_NOT_DURABLE**, **DB_CHKSUM**, влияет только на указанный дескриптор DB, а также на любые другие дескрипторы Berkeley DB, открытые в области действия этого дескриптора.

BTree

DB_DUPSORT

Разрешить дублирование элементов данных в базе данных — вставка, когда ключ вставляемой пары ключ/данные уже существует в базе данных, будет успешной.

Порядок дубликатов в базе данных определяется функцией сравнения дубликатов. Если приложение не указывает функцию сравнения с помощью метода **set_dup_compare()**, будет использоваться лексическое сравнение по умолчанию.

Указание одновременно **DB_DUPSORT** и **DB_RECNUM** является ошибкой.

DB_DUP

Разрешить дублирование элементов данных в базе данных — вставка, когда ключ вставляемой пары ключ/данные уже существует в базе данных, будет успешной.

Порядок дубликатов, если не указан иным образом с помощью операции курсора или функции сортировки дубликатов в базе данных, определяется порядком вставки.

Флаг **DB_DUPSORT** предпочтительнее **DB_DUP** по соображениям производительности.

Флаг **DB_DUP** следует использовать только приложениям, которым требуется упорядочивать повторяющиеся элементы данных вручную.

Вызов **DB->set_flags()** с флагом **DB_DUP** влияет на базу данных, включая все потоки управления, обращающиеся к базе данных.

Если база данных уже существует на момент вызова **DB->open()**, флаг **DB_DUP** должен быть таким же, как у существующей базы данных, иначе будет возвращена ошибка.

Указание одновременно **DB_DUP** и **DB_RECNUM** является ошибкой.

DB_RECNUM

Поддержка поиска в Btree с использованием номеров записей.

При вставке или удалении записи в базах данных Btree изменяются номера логических записей и это создает определенные проблемы с доступом к страницам, хранящим записи.

Как во время вставки, так и во время удаления, вся база данных должна быть заблокирована, что фактически делает базу данных для этих операций однопоточной. Указание **DB_RECNUM** может привести к серьезному снижению производительности некоторых приложений и наборов данных.

Базы данных

Лекция 16 – Интерфейс Postgres

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2023.12.08

Оглавление

libpq – библиотека для языка C.....	3
Функции управления подключением к базе данных.....	4
PqconnectdbParams – создаёт новое подключение к серверу баз данных.....	4
Pqconnectdb – создаёт новое подключение к серверу баз данных.....	14
PqsetdbLogin – создаёт новое подключение к серверу баз данных.....	15
Pqsetdb – создаёт новое подключение к серверу баз данных.....	16
Подключение к серверу баз данных неблокирующим способом.....	17
Pqconndefaults – возвращает значения по умолчанию для параметров подключения.....	19
Pqconninfo – Возвращает параметры подключения действующего соединения.....	20
Pqfinish – Закрывает соединение с сервером.....	21
Pqreset – переустанавливает канал связи с сервером.....	21
Переустановка канала связи с сервером неблокирующим способом.....	22

libpq — библиотека для языка C

libpq — это интерфейс Postgres для программирования приложений на языке C.

Библиотека libpq содержит набор функций, используя которые клиентские программы могут передавать запросы серверу Postgres Pro и принимать результаты этих запросов.

libpq также является базовым механизмом для других прикладных интерфейсов Postgres, в частности, для C++, Perl, Python и Tcl.

Примеры использования libpq, в том числе несколько завершённых приложений, можно найти в каталоге `src/test/examples` дистрибутива в исходных текстах.

Клиентские программы, которые используют libpq, должны включать заголовочный файл `libpq-fe.h` и должны компоноваться с библиотекой libpq.

Интерфейс для C++ обычно находится в `libpqxx`.

Функции управления подключением к базе данных

Прикладная программа может иметь несколько подключений к серверу, открытых одновременно. Это нужно для доступа к более чем одной базе данных.

Каждое соединение представляется объектом типа `PGconn`, который можно получить от функций:

```
Pqconnectdb;  
PqconnectdbParams;  
PQsetdbLogin.
```

В случае успеха эти функции возвращают ненулевой указатель на объект.

Прежде чем передавать запросы через объект подключения, следует вызвать функцию `PQstatus` для проверки возвращаемого значения в случае успешного подключения.

`PqconnectdbParams` – создаёт новое подключение к серверу баз данных

```
PGconn *PQconnectdbParams(const char *const *keywords,  
                           const char *const *values,  
                           int expand_dbname);
```

Функция открывает новое соединение с базой данных, используя параметры, содержащиеся в двух массивах, завершающихся символом `NULL`.

`keywords` – массив строк, каждая из которых представляет собой ключевое слово.

`values` – значение для каждого ключевого слова.

Набор параметров может быть расширен без изменения сигнатуры функции.

Ключевые слова

host — имя компьютера для подключения.

Если это имя начинается с косой черты, оно выбирает подключение через Unix-сокеты, а не через TCP/IP, и задаёт имя каталога, содержащего файл сокета.

По умолчанию, если параметр `host` отсутствует или пуст, выполняется подключение к Unix-сокету в `/tmp`.

В системах, где Unix-сокеты не поддерживаются, по умолчанию выполняется подключение к `localhost`.

Может принимать разделённый запятыми список имён узлов. В данном случае имена будут перебираться по порядку.

Для пустых элементов списка применяется поведение по умолчанию.

hostaddr — числовой IP-адрес компьютера для подключения.

Он должен быть представлен в стандартном формате адресов IPv4, например, 172.28.40.9.

Есть поддержка IPv6, можно использовать и эти адреса.

Если в качестве этого параметра передана непустая строка, всегда используется связь по протоколу TCP/IP.

Использование `hostaddr` вместо `host` позволяет приложению избежать поиска на сервере имён, что может быть важно для приложений, имеющих ограничения по времени.

Тем не менее, имя компьютера требуется для некоторых методов аутентификации, а также для проверки полномочий на основе SSL-сертификатов.

Применяются следующие правила:

1) Если адрес `host` задаётся без `hostaddr`, осуществляется разрешение имени¹.

2) Если указан `hostaddr`, а `host` нет, значение `hostaddr` задает сетевой адрес сервера.

Однако, если метод аутентификации требует наличия имени компьютера, попытка подключения завершится неудачей.

3) Если указаны как `host`, так и `hostaddr`, значение `hostaddr` задает сетевой адрес сервера, а значение `host` игнорируется. Если метод аутентификации его требует, оно будет использоваться в качестве имени компьютера.

Аутентификация может завершиться неудачей, если `host` не является именем сервера с сетевым адресом `hostaddr`. Когда указывается и `host`, и `hostaddr`, соединение в файле паролей идентифицируется по значению `host`.

4) Если `hostaddr` является списком значений, разделенных запятыми, узлы будут перебираться по порядку.

5) Если не указаны ни имя компьютера, ни его адрес, `libpq` будет производить подключение, используя локальный Unix-сокеты. В системах, не поддерживающих Unix-сокеты, будет попытка подключиться к `localhost`.

port — номер порта, к которому нужно подключаться на сервере.

Если в параметрах `host` или `hostaddr` задано несколько серверов, в данном параметре может задаваться через запятую список портов такой же длины, либо может указываться один номер порта для всех узлов.

Пустая строка или пустой элемент в списке через запятую воспринимается как номер порта по умолчанию, установленный при сборке PostgreSQL.

1) Есть нюансы при использовании `PqconnectPoll()`

dbname — имя базы данных.

По умолчанию оно совпадает с именем пользователя.

user — имя пользователя Postgres, используемое для подключения.

По умолчанию используется то же имя, которое имеет в операционной системе пользователь, от лица которого выполняется приложение.

password — пароль, используемый в случае, когда сервер требует аутентификации по паролю.

passfile — имя файла, в котором будут храниться пароли.

По умолчанию это `~/ .pgpass` или `%APPDATA%\postgresql\pgpass.conf` в Microsoft Windows. Файл может отсутствовать.

connect_timeout — максимальное время ожидания подключения.

Задаётся десятичным целым числом.

При нуле, отрицательном или если не указано, ожидание будет бесконечным.

Минимальный допустимый тайм-аут равен 2 секундам, таким образом, значение 1 воспринимается как 2 секунды.

Этот тайм-аут применяется для каждого отдельного IP-адреса или имени сервера.

Если заданы адреса двух серверов и значение `connect_timeout` равно 5, тайм-аут при неудачной попытке подключения к каждому серверу произойдёт через 5 секунд, а общее время ожидания подключения может достигать 10 секунд.

client_encoding — параметр устанавливает конфигурационный параметр `client_encoding` для данного подключения.

В дополнение к значениям, которые принимает соответствующий параметр сервера, можно использовать значение `auto`. В этом случае правильная кодировка определяется на основе текущей локали на стороне клиента.

options — задаёт параметры командной строки, которые будут отправлены серверу при установлении соединения.

application_name — устанавливает значение для конфигурационного параметра `application_name`.

fallback_application_name — устанавливает альтернативное значение для конфигурационного параметра `application_name`.

Это значение будет использоваться, если для параметра `application_name` не было передано никакого значения с помощью параметров подключения или переменной системного окружения `PGAPPNAME`.

Задание альтернативного имени полезно для универсальных программ-утилит, которые желают установить имя приложения по умолчанию, но позволяют пользователю изменить его.

keepalives — управляет использованием сообщений `keepalive` протокола TCP на стороне клиента. Значение по умолчанию равно 1, что означает использование сообщений. Можно изменить его на 0, если эти сообщения не нужны.

keepalives_idle — управляет длительностью периода отсутствия активности, выраженного числом секунд, по истечении которого TCP должен отправить сообщение keepalive серверу.

При значении 0 действует системная величина.

Этот параметр игнорируется для соединений, установленных через Unix-сокеты, или если сообщения keepalive отключены.

Он поддерживается только в системах, воспринимающих параметр сокета TCP_KEEPIDLE или равнозначный. В других системах он не оказывает влияния.

keepalives_count — задаёт количество сообщений keepalive протокола TCP, которые могут быть потеряны, прежде чем соединение клиента с сервером будет признано неработающим.

Нулевое значение этого параметра указывает, что будет использоваться системное значение по умолчанию.

Этот параметр игнорируется для соединений, установленных через Unix-сокеты, или если сообщения keepalive отключены.

Он поддерживается только в системах, воспринимающих параметр сокета TCP_KEEPCNT или равнозначный; в других системах он не оказывает влияния.

sslmode — определяет, будет ли согласовываться с сервером защищённое SSL-соединение по протоколу TCP/IP, и если да, то в какой очередности.

Всего предусмотрено шесть режимов:

disable — следует попытаться установить только соединение без использования SSL;

allow — сначала следует попытаться установить соединение без использования SSL, но если попытка будет неудачной, нужно попытаться установить SSL-соединение;

prefer (по умолчанию) — сначала следует попытаться установить SSL-соединение, но если попытка будет неудачной, нужно попытаться установить соединение без использования SSL;

verify-ca — следует попытаться установить только SSL-соединение, при этом проконтролировать, чтобы сертификат сервера был выпущен доверенным центром сертификации (CA);

require — следует попытаться установить только SSL-соединение. Если присутствует файл корневого центра сертификации, то нужно верифицировать сертификат таким же способом, как будто был указан параметр **verify-ca**;

verify-full — следует попытаться установить только SSL-соединение, при этом проконтролировать, чтобы сертификат сервера был выпущен доверенным центром сертификации (CA) и чтобы имя запрошенного сервера соответствовало имени в сертификате;

!!! Замечание !!!

sslmode игнорируется при использовании Unix-сокетов. В случаях, когда Postgres скомпилирован без поддержки SSL, использование параметров **require**, **verify-ca** или **verify-full** приведёт к ошибке. Параметры **allow** и **prefer** будут приняты, но **libpq** не будет пытаться установить SSL-соединение.

`sslcompression` – если установлено значение 1 (по умолчанию), данные, пересылаемые через SSL-соединения, будут сжиматься. Если установлено значение 0, сжатие будет отключено.

Параметр игнорируется, если выполнено подключение без SSL, или если используемая версия OpenSSL не поддерживает его.

Сжатие требует процессорного времени, но может улучшить пропускную способность, если узким местом является сеть.

Отключение сжатия может улучшить время отклика и пропускную способность, если ограничивающим фактором является производительность CPU.

`sslcert` – указывает имя файла для SSL-сертификата клиента, заменяющего файл по умолчанию `~/.postgresql/postgresql.crt`. Этот параметр игнорируется, если SSL-подключение не выполнено.

`sslkey` – указывает местоположение секретного ключа, используемого для сертификата клиента.

Он может либо указывать имя файла, которое будет использоваться вместо имени по умолчанию `~/.postgresql/postgresql.key`, либо он может указывать ключ, полученный от внешнего криптомодуля (загружаемого модуля OpenSSL).

`sslrootcert` – указывает имя файла, содержащего SSL-сертификаты, выданные Центром сертификации (CA). Если файл существует, сертификат сервера будет проверен на предмет его подписания одним из этих центров. Имя по умолчанию – `~/.postgresql/root.crt`.

sslcr1 — указывает имя файла, содержащего список отозванных серверных сертификатов (CRL) для SSL. Сертификаты, перечисленные в этом файле, если он существует, будут отвергаться при попытке установить подлинность сертификата сервера. Имя по умолчанию такое `~/ .postgresql/root.crl`.

requirepeer — указывает имя пользователя операционной системы, предназначенное для сервера, например, `requirepeer=postgres`.

При создании подключения через Unix-сокеты, если этот параметр установлен, клиент проверяет в самом начале процедуры подключения, что серверный процесс запущен от имени указанного пользователя и, если это не так, соединение аварийно прерывается с ошибкой.

Этот параметр можно использовать, чтобы обеспечить аутентификацию сервера, подобную той, которая доступна с помощью SSL-сертификатов при соединениях по протоколу TCP/IP.

krbsrvname — имя сервиса Kerberos, предназначенное для использования при аутентификации на основе GSSAPI. Оно должно соответствовать имени сервиса, указанному в конфигурации сервера, чтобы аутентификация на основе Kerberos прошла успешно.

gsslib — библиотека GSS, предназначенная для использования при аутентификации на основе GSSAPI.

В настоящее время это действует только в сборках для Windows, поддерживающих одновременно и GSSAPI, и SSPI. Значение `gssapi` в таких сборках позволяет указать, что `libpq`

должна использовать для аутентификации библиотеку GSSAPI, а не подразумеваемую по умолчанию SSPI.

service — имя сервиса, используемое для задания дополнительных параметров. Оно указывает имя сервиса в файле `pg_service.conf`, который содержит дополнительные параметры подключения. Это позволяет приложениям указывать только имя сервиса, поскольку параметры подключения могут поддерживаться централизованно.

target_session_attrs — если этот параметр равен `read-write`, по умолчанию будут приемлемы только подключения, допускающие транзакции на чтение/запись.

При успешном подключении будет отправлен запрос `SHOW transaction_read_only`; если он вернёт `on`, соединение будет закрыто.

Если в строке подключения указано несколько серверов, будут перебираться остальные серверы, как и при неудачной попытке подключения. Со значением по умолчанию (`any`) приемлемыми будут все подключения.

Pqconnectdb – создаёт новое подключение к серверу баз данных

```
PGconn *PQconnectdb(const char *conninfo);
```

Эта функция открывает новое соединение с базой данных, используя параметры, полученные из строки `conninfo`.

Передаваемая строка может быть пустой. В этом случае используются все параметры по умолчанию. Она также может содержать одно или более значений параметров, разделённых пробелами, или URI.

Пример:

```
host=localhost port=5432 dbname=mydb connect_timeout=10
```

PqsetdbLogin – создаёт новое подключение к серверу баз данных

```
PGconn *PQsetdbLogin(const char *pghost,  
                    const char *pgport,  
                    const char *pgoptions,  
                    const char *pgtty,      // не используется в н.в.  
                    const char *dbName,  
                    const char *login,  
                    const char *pwd);
```

Предшественница функции PQconnectdb() с фиксированным набором параметров.

Она имеет такую же функциональность, за исключением того, что непереданные параметры (NULL или пустая строка) всегда принимают значения по умолчанию.

Pqsetdb – создаёт новое подключение к серверу баз данных.

```
PGconn *Pqsetdb(char *pghost,  
                char *pgport,  
                char *pgoptions,  
                char *pgtty,  
                char *dbName);
```

Это макрос. Он вызывает PqsetdbLogin() с нулевыми указателями в качестве значений параметров login и pwd.

Обеспечивает обратную совместимость с очень старыми программами.

Подключение к серверу баз данных неблокирующим способом

```
PGconn *PQconnectStartParams(const char *const *keywords,  
                             const char *const *values,  
                             int expand_dbname);  
  
PGconn *PQconnectStart(const char *conninfo);  
  
PostgresPollingStatusType PQconnectPoll(PGconn *conn);
```

Эти функции используются для того, чтобы открыть подключение к серверу баз данных таким образом, чтобы вызывающий их поток исполнения не был заблокирован при выполнении удаленной операции ввода/вывода во время подключения.

Суть этого подхода в том, чтобы ожидание завершения операций ввода/вывода могло происходить в главном цикле приложения, а не в внутри функций `PQconnectdbParams( )` или `PQconnectdb( )`, с тем, чтобы приложение могло управлять этой операцией параллельно с другой работой.

С помощью функции `PQconnectStartParams( )` подключение к базе данных выполняется, используя параметры, взятые из массивов `keywords` и `values`.

С помощью функции `PQconnectStart` подключение к базе данных выполняется, используя параметры, взятые из строки `conninfo`.

Эти функции не блокируются до тех пор, пока выполняется ряд ограничений:

- 1) Параметр `hostaddr` должен использоваться так, чтобы для разрешения заданного имени не требовалось выполнять запросы DNS.
 - 2) Если вызывается `PQtrace`, необходимо обеспечить, чтобы поток, в который выводится трассировочная информация, не заблокировался.
- Детали в документации.

Pqconndefaults – возвращает значения по умолчанию для параметров подключения

```
PQconninfoOption *PQconndefaults(void);

typedef struct {
    char    *keyword;    // Ключевое слово для данного параметра
    char    *envvar;     // Имя альтернативной переменной окружения
    char    *compiled;   // Альтернативное значение по умолчанию,
                        // назначенное при компиляции
    char    *val;        // Текущее значение параметра или NULL
    char    *label;      // Обозначение этого поля в диалоге подключения
    char    *dispchar;   // Показывает, как отображать это поле
                        // в диалоге подключения. Значения следующие:
                        // "" Отображать введённое значение "как есть"
                        // "*" Поле пароля – скрывать значение
                        // "D" Параметр отладки
    int      dispsize;   // Размер поля в символах для диалога
} PQconninfoOption;
```

Возвращает массив параметров подключения. Он может использоваться для определения всех возможных параметров PQconnectdb и их текущих значений по умолчанию.

Возвращаемое значение указывает на массив структур PQconninfoOption, который завершается элементом, имеющим нулевой указатель keyword.

Если выделить память не удалось, то возвращается нулевой указатель.

Pqconninfo – Возвращает параметры подключения действующего соединения

```
PQconninfoOption *PQconninfo(PGconn *conn);
```

Возвращает массив параметров подключения.

Используется для определения всех возможных параметров PQconnectdb и значений, которые были использованы для подключения к серверу.

Возвращаемое значение указывает на массив структур PQconninfoOption, который завершается элементом, имеющим нулевой указатель keyword.

Pqfinish – Закрывает соединение с сервером

```
void PQfinish(PGconn *conn);
```

Освобождает память, используемую объектом PGconn.

Приложение в любом случае должно вызвать PQfinish(), чтобы освободить память, используемую объектом PGconn, даже если попытка подключения к серверу потерпела неудачу (как показывает PQstatus).

После того, как была вызвана функция PQfinish, указатель PGconn не должен использоваться повторно.

Pqreset – переустанавливает канал связи с сервером

```
void PQreset(PGconn *conn);
```

Эта функция закрывает подключение к серверу, а потом попытается установить новое подключение, используя все те же параметры, которые использовались прежде. Это может быть полезным для восстановления после ошибки, если работающее соединение было разорвано.

Переустановка канала связи с сервером неблокирующим способом

PQresetStart

PQresetPoll

```
int PQresetStart(PGconn *conn);
```

```
PostgresPollingStatusType PQresetPoll(PGconn *conn);
```

Эти функции закроют подключение к серверу, после чего попытаются установить новое подключение, используя все те же параметры, которые использовались прежде.

Это может быть полезным для восстановления после ошибки, если работающее соединение оказалось потерянным.

Они отличаются от PQreset тем, что действуют неблокирующим способом.

На эти функции налагаются те же ограничения, что и на PQconnectStartParams(), PQconnectStart() и PQconnectPoll().

Переустановка подключения PQresetStart() – если она возвратит 0, переустановка завершилась неудачно.

Если она возвратит 1, следует опросить результат переустановки, используя PQresetPoll().

Детали в документации

Далее нужно организовать цикл ожидания, пока `PQconnectPoll(conn)` будет возвращать `PGRES_POLLING_READING`.

при последнем вызове возвращает, ожидайте, пока сокет не окажется готовым для чтения (это покажет функция `select()`, `poll()` или подобная системная функция). Затем снова вызовите `PQconnectPoll(conn)`. Если же `PQconnectPoll(conn)` при последнем вызове вернула `PGRES_POLLING_WRITING`, дождитесь готовности сокета к записи, а затем снова вызовите `PQconnectPoll(conn)`. На первой итерации, то есть когда вы ещё не вызывали `PQconnectPoll`, реализуйте то же поведение, что и после получения `PGRES_POLLING_WRITING`. Продолжайте этот цикл, пока `PQconnectPoll(conn)` не выдаст значение `PGRES_POLLING_FAILED`, сигнализирующее об ошибке при установлении соединения, или `PGRES_POLLING_OK`, показывающее, что соединение установлено успешно.

В любое время в процессе подключения его состояние можно проверить, вызвав `PQstatus`. Если этот вызов возвратит `CONNECTION_BAD`, значит, процедура подключения завершилась сбоем; если вызов возвратит `CONNECTION_OK`, значит, соединение готово. Оба эти состояния можно определить на основе возвращаемого значения функции `PQconnectPoll`, описанной выше. Другие состояния могут также иметь место в течение (и только в течение) асинхронной процедуры подключения. Они показывают текущую стадию процедуры подключения и могут быть полезны, например, для предоставления обратной связи пользователю. Вот эти состояния:

Базы данных

Лекция 17 – Интерфейс Postgres

Преподаватель: Поденок Леонид Петрович, 505а-5

+375 17 293 8039 (505а-5)

+375 17 320 7402 (ОИПИ НАНБ)

prep@lsi.bas-net.by

ftp://student:2ok*uK2@Rwox@lsi.bas-net.by

Кафедра ЭВМ, 2023

2023.12.15

Оглавление

Функции исполнения команд.....	3
Основные функции.....	3
PQexec() – передаёт команду серверу и ожидает результата.....	3
PQexecParams() -- отправляет команду серверу и ожидает результата.....	5
PQprepare – запрос на создание подготовленного оператора.....	8
PQexecPrepared – запрос на исполнение подготовленного оператора.....	10
PQdescribePrepared – получение информации о подготовленном операторе.....	11
PQresultStatus – статус результата выполнения команды.....	12
PQresStatus – преобразовать код статуса в строку.....	13
PQresultErrorMessage – сообщение об ошибке.....	13
PQresultVerboseErrorMessage – более полное сообщения об ошибке.....	14
PQresultErrorField – поле из отчёта об ошибке.....	15
PQclear – освободить память, связанную с PGresult.....	18
Извлечение информации, связанной с результатом запроса.....	19
PQntuples – число строк (кортежей) в полученной выборке.....	19
PQnfields – число столбцов (полей) в каждой строке полученной выборки.....	19
PQfname – имя столбца с номером.....	20
PQfnumber – номер столбца по имени.....	21
PQftable – OID таблицы для данного столбца.....	22
PQftablecol – номер столбца в таблицы для столбца в выборке.....	23
PQfformat – код формата столбца.....	23
PQftype – тип данных по номеру столбца.....	24
PQfmod – модификатор типа для столбца.....	24
PQfsize – размер в байтах для столбца.....	25
PQgetvalue – значение поля из строки в PGresult.....	26
PQgetisnull – проверка поля на null.....	27
PQgetlength – фактическая длина значения поля в байтах.....	27
PQnparams – число параметров подготовленного оператора.....	28
PQparamtype – тип данных для параметра оператора.....	28
PQprint – вывод строк и имен столбцов в поток вывода.....	29
Получение другой информации о результате.....	30
PQcmdStatus – статус для исполненной SQL-команды.....	30
PQoidValue – OID вставленной строки.....	30
PQcmdTuples – число строк затронутых SQL-командой.....	31
Экранирование строковых значений для включения в SQL-команды.....	32
PQescapeLiteral – экранировать строковое значение.....	32
PQescapeStringConn – экранировать строковые литералы.....	34
PQescapeIdentifier – экранировать строку – идентификатор SQL.....	36
PQescapeByteaConn – экранировать двоичные данные.....	37
PQunescapeBytea – строковое представление → в двоичные данные.....	39

Функции исполнения команд

Основные функции

PQexec() – передаёт команду серверу и ожидает результата.

```
#include <libpq-fe.h> // -lpq -- компоновка с libpq.so

PGresult *PQexec(PGconn      *conn,      // соединение
                 const char *command); // команда
```

Возвращает указатель на объект PGresult или, возможно, пустой указатель NULL в случае нехватки памяти или возникновения серьёзной ошибки, например такой, как невозможность отправки команды серверу.

Для проверки возврата на наличие ошибок БД следует вызывать PQresultStatus(), которая в случае нулевого указателя возвратит PGRES_FATAL_ERROR.

PGRES_TUPLES_OK	-- результат в PGresult
PGRES_EMPTY_QUERY	-- пустой запрос
PGRES_COMMAND_OK	-- команда подготовлена к исполнению
PGRES_NONFATAL_ERROR	-- уведомление или предупреждение (warning)
PGRES_BAD_RESPONSE	-- неожиданный ответ от движка
PGRES_COPY_OUT	-- идет копирование
PGRES_COPY_IN	-- идет копирование

Для получения дополнительной информации об ошибках следует использовать функцию PQerrorMessage().

Строка команды может включать в себя более одной SQL-команды.

Команды разделяются точкой с запятой.

Несколько запросов, отправленных с помощью одного вызова PQexec(), обрабатываются в рамках одной транзакции.

Если команды BEGIN/COMMIT явно включены в строку запроса, он будет разделен на несколько транзакций.

!!! ACHTUNG !!!

PGresult описывает только результат последней из выполненных команд из строки запроса.

Если одна из команд завершается сбоем, то обработка строки запроса на этом останавливается, и возвращённая структура PGresult содержит состояние ошибки.

PQexecParams() -- отправляет команду серверу и ожидает результата.

Имеет возможность передать параметры отдельно от текста SQL-команды.

```
PGresult *PQexecParams(PGconn      *conn,
                        const char *command,    // параметризованная к-да
                        int          nParams,    // кол-во параметров
                        const Oid   *paramTypes, // oid-типы параметров
                        const char * const *paramValues, // значения
                        const int   *paramLengths, // размеры данных
                        const int   *paramFormats, // текстовый/двоичный
                        int          resultFormat); // текстовый/двоичный
```

PQexecParams() похожа на PQexec(), но предлагает дополнительную функциональность:

- отдельно от самой строки-команды могут быть указаны значения параметров;
- результаты запроса могут быть получены как в текстовом, так и в двоичном формате.

conn – подключение, через которое пересылается команда.

command – строка SQL-команды, которую следует выполнить. Если используются параметры, то в строке команды на них ссылаются, как \$1, \$2 ...

nParams – число предоставляемых параметров. Оно равно размеру массивов paramTypes[], paramValues[], paramLengths[] и paramFormats[].

Если nParams равно нулю, указатели на массивы могут быть равны NULL.

paramTypes[] – предписывает, используя OID, типы данных, которые должны быть назначены параметрам.

paramValues[] – содержит фактические значения параметров.

Нулевой указатель в этом массиве означает, что соответствующий параметр равен null; в противном случае указатель указывает на текстовую строку, завершающуюся нулевым символом (для текстового формата), или на двоичные данные в формате, которого ожидает сервер (для двоичного формата).

paramLengths[] – содержит фактические размеры данных для параметров, представленных в двоичном формате.

Он игнорируется для параметров, имеющих значение null, и для параметров, представленных в текстовом формате. Если нет двоичных параметров, указатель на массив может быть нулевым.

paramFormats[] – указывает, что параметр текстовый (0) или двоичный (1).

Если указатель на массив является нулевым, тогда все параметры считаются текстовыми строками.

Значения, переданные в двоичном формате, требуют знания внутреннего представления, которого ожидает сервер. Например, целые числа должны передаваться с использованием сетевого порядка байтов (hton{l|s}(), ntoh{l|s}()).

Передача значений типа numeric требует знания формата, в котором их хранит сервер.

Информацию об этом можно получить из функций numeric_send() и numeric_recv(), располагающихся в src/backend/utils/adt/numeric.c

Данные функции преобразуют numeric в двоичный формат и обратно.

resultFormat – формат возврата.

0 – результаты в текстовом формате;

1 – результаты в двоичном формате.

Основное преимущество PQexecParams() от PQexec() – значения параметров могут быть отделены от строки с командой. Это позволяет избежать использования кавычек и экранирующих символов, что часто приводит к ошибкам.

В отличие от PQexec(), PQexecParams() позволяет включать в строку запроса только одну SQL-команду.

В ней могут содержаться точки с запятой, однако может присутствовать не более одной непустой команды.

!!! ВАЖНО !!!

Можно избежать указания типов параметров с помощью OID, для чего, чтобы показать, какой тип данных отправляется, в строке SQL-команды следует добавить явное приведение типа для этого параметра. Например:

```
SELECT * FROM mytable WHERE x = $1::bigint;
```

Это заставит считать параметр \$1 имеющим тип bigint, хотя по умолчанию ему был бы назначен тот же самый тип, что и x.

Если значения параметров отправляются в двоичном формате, строго рекомендуется явное указание о типе параметра либо с помощью описанного метода, либо путём задания числового OID.

PQprepare – запрос на создание подготовленного оператора¹

Отправляет запрос, чтобы создать подготовленный оператор с конкретными параметрами, и ожидает завершения.

```
PGresult *PQprepare(PGconn *conn,
                    const char *stmtName, // подготовленный оператор
                    const char *query,    // команда SQL
                    int         nParams,   // кол-во параметров
                    const Oid   *paramTypes); // oid-типы параметров
```

PQprepare() создаёт подготовленный оператор, который может быть впоследствии исполнен с помощью PQexecPrepared().

Благодаря этому, команды, которые будут выполняться многократно, серверу не потребуется разбирать и планировать каждый раз.

query – команда SQL.

stmtName – здесь создаётся подготовленный оператор с именем stmtName из строки query, которая должна содержать единственную SQL-команду.

Если stmtName пустая строка "", то будет создан *неименованный* оператор.

Любой уже существующий неименованный оператор будет автоматически заменён.

Если имя оператора уже определено в текущем сеансе работы, будет ошибка.

Если используются параметры, то в запросе к ним обращаются таким образом: \$1, \$2.

1) PQprepare() поддерживается только с подключениями по протоколу 3.0 и новее.

nParams – число параметров, типы данных для которых указаны в массиве `paramTypes[]`.

Если значение `nParams` равно нулю, указатель на массив может быть равен `NULL`.

paramTypes[] – OID-типы данных, которые будут назначены параметрам.

Если `paramTypes` равен `NULL` или какой-либо элемент в этом массиве равен нулю, то сервер назначает тип данных соответствующему параметру точно таким же способом, как он сделал бы для литеральной строки, не имеющей типа.

Также в запросе можно использовать параметры с номерами, большими, чем `nParams`, в этом случае сервер сможет подобрать типы данных для них самостоятельно.

Результатом вызова является объект `PGresult`, содержимое которого показывает успех или сбой на стороне сервера.

Нулевой указатель означает нехватку памяти или невозможность вообще отправить команду. Для получения дополнительной информации о таких ошибках следует использовать `PQerrorMessage()`.

Подготовленные операторы для использования с `PQexecPrepared()` можно также создать путём исполнения SQL-команд `PREPARE`.

PQexecPrepared – запрос на исполнение подготовленного оператора²

Отправляет запрос на исполнение подготовленного оператора с данными параметрами и ожидает результата.

```
PGresult *PQexecPrepared(PGconn      *conn,
                        const char *stmtName, // имя подготовленного
                        int         nParams,  //
                        const char * const *paramValues,
                        const int  *paramLengths,
                        const int  *paramFormats,
                        int         resultFormat);
```

PQexecPrepared() подобна PQexecParams(), но команда, подлежащая исполнению, указывается путём передачи имени предварительно подготовленного оператора вместо передачи строки запроса.

Эта возможность позволяет командам, которые вызываются многократно, подвергаться разбору и планированию только один раз, а не при каждом их исполнении.

Оператор должен быть предварительно подготовлен в рамках текущего сеанса работы.

Параметры идентичны PQexecParams(), за исключением того, что вместо строки SQL-запроса передаётся имя подготовленного оператора, а параметр paramTypes[] отсутствует, поскольку типы данных для параметров были определены при создании подготовленного оператора.

2) PQexecPrepared() поддерживается только в соединениях по протоколу версии 3.0 или более поздних версий.

PQdescribePrepared – получение информации о подготовленном операторе³

Передаёт запрос на получение информации об указанном подготовленном операторе и ожидает завершения.

```
PGresult *PQdescribePrepared(PGconn      *conn,  
                             const char *stmtName);
```

PQdescribePrepared() позволяет приложению получить информацию о предварительно подготовленном операторе.

Для ссылки на неименованный оператор значение stmtName может быть пустой строкой "" или NULL, в противном случае оно должно быть именем существующего подготовленного оператора.

В случае успеха возвращается PGresult со статусом PGRES_COMMAND_OK.

Функции PQnparams() и PQparamtype() позволяют извлечь из PGresult информацию о параметрах подготовленного оператора, а функции PQnfields(), PQfname(), PQftype() и т. п. предоставляют информацию о результирующих столбцах (если они есть) данного оператора.

3) PQdescribePrepared() поддерживается только в соединениях по протоколу версии 3.0 или более поздних версий.

PQresultStatus – статус результата выполнения команды

Возвращает статус результата выполнения команды.

```
ExecStatusType PQresultStatus(const PGresult *res);
```

PQresultStatus может возвращать одно из следующих значений:

PGRES_EMPTY_QUERY – строка, отправленная серверу, была пустой.

PGRES_COMMAND_OK – успешное завершение команды, не возвращающей никаких данных.

Ответ PGRES_COMMAND_OK предназначен для команд, которые никогда не возвращают строки (INSERT или UPDATE без использования предложения RETURNING и др.).

PGRES_TUPLES_OK – успешное завершение команды, возвращающей данные (такой, как SELECT или SHOW), и SELECT не возвративший никаких данных тоже.

PGRES_COPY_OUT – начат перенос данных Copy Out (с сервера).

PGRES_COPY_IN – начат перенос данных Copy In (на сервер).

PGRES_BAD_RESPONSE – ответ сервера не был распознан.

PGRES_NONFATAL_ERROR – произошла не фатальная ошибка (уведомление или предупреждение).

PGRES_FATAL_ERROR – произошла фатальная ошибка.

PGRES_COPY_BOTH – начат перенос данных Copy In/Out (на сервер и с сервера).

PGRES_SINGLE_TUPLE – структура PGresult содержит только одну результирующую строку, возвращённую текущей командой.

Этот статус имеет место только тогда, когда для данного запроса был выбран режим построчного вывода.

PQresStatus – преобразовать код статуса в строку

Преобразует значение перечислимого типа, возвращённое функцией PQresultStatus, в строковую константу, описывающую код статуса.

```
char *PQresStatus(ExecStatusType status);
```

PQresultErrorMessage – сообщение об ошибке

Возвращает сообщение об ошибке, связанное с командой, или пустую строку, если ошибки не произошло.

```
char *PQresultErrorMessage(const PGresult *res);
```

Если произошла ошибка, то возвращённая строка будет включать завершающий символ новой строки.

Вызывающая функция не должна напрямую освобождать память, на которую указывает возвращаемый указатель – она будет освобождена, когда соответствующий указатель PGresult будет передан функции PQclear().

PQresultVerboseErrorMessage – более полное сообщения об ошибке

Возвращает более полную версию сообщения об ошибке из объекта PGresult.

```
char *PQresultVerboseErrorMessage(const PGresult      *res,  
                                  PGVerbosity         verbosity,  
                                  PGContextVisibility show_context);
```

В некоторых ситуациях клиент может захотеть получить более подробную версию ранее выданного сообщения об ошибке.

PQresultVerboseErrorMessage() формирует сообщение, которое было бы выдано функцией PQresultErrorMessage(), если бы заданный уровень детализации был текущим для соединения в момент заполнения PGresult.

Если в PGresult ошибки нет, выдаётся сообщение «PGresult is not an error result» (PGresult — не результат с ошибкой).

Возвращаемое этой функцией сообщение завершается переводом строки.

В отличие от многих других функций, извлекающих данные из PGresult, результат этой функции — новая размещённая в памяти строка.

Когда эта строка будет не нужна, вызывающий код должен освободить её место, вызвав PQfreemem().

При нехватке памяти может быть возвращен NULL.

PQresultErrorField – поле из отчёта об ошибке

Возвращает индивидуальное поле из отчёта об ошибке.

```
char *PQresultErrorField(const PGresult *res, int fieldcode);
```

fieldcode – идентификатор поля ошибки.

Если PGresult не содержит ошибки или предупреждения или не включает указанное поле, то возвращается NULL.

Значения полей обычно не включают завершающий символ новой строки.

Вызывающая функция не должна напрямую освобождать память, на которую указывает возвращаемый указатель – она будет освобождена, когда соответствующий указатель PGresult будет передан функции PQclear().

Доступны следующие коды полей:

PG_DIAG_SEVERITY – серьёзность; поле может содержать ERROR, FATAL или PANIC (в сообщении об ошибке) либо WARNING, NOTICE, DEBUG, INFO или LOG (в сообщении-уведомлении) либо локализованный перевод одного из этих значений. Присутствует всегда.

PG_DIAG_SEVERITY_NONLOCALIZED – серьёзность; поле может содержать ERROR, FATAL или PANIC (в сообщении об ошибке) либо WARNING, NOTICE, DEBUG, INFO или LOG (в сообщении-уведомлении). Это поле подобно PG_DIAG_SEVERITY, но его содержимое никогда не переводится. Присутствует только в отчётах, выдаваемых Postgres Pro версии 9.6 и новее.

PG_DIAG_SQLSTATE – код ошибки в соответствии с соглашением о кодах SQLSTATE.

Код SQLSTATE идентифицирует тип случившейся ошибки. Он может использоваться клиентскими приложениями, чтобы выполнять конкретные операции (обработка ошибок) в ответ на конкретную ошибку базы данных. Это поле всегда присутствует.

PG_DIAG_MESSAGE_PRIMARY – главное сообщение об ошибке, предназначенное для прочтения пользователем.

Как правило составляет всего одну строку. Это поле всегда присутствует.

PG_DIAG_MESSAGE_DETAIL – необязательное дополнительное сообщение об ошибке, передающее более детальную информацию о проблеме.

Может занимать несколько строк.

PG_DIAG_MESSAGE_HINT – подсказка: необязательное предположение о том, что можно сделать в данной проблемной ситуации.

Оно отличается от детальной информации в том смысле, что оно предлагает совет (возможно, и неподходящий), а не просто факты. Может занимать несколько строк.

PG_DIAG_STATEMENT_POSITION – строка, содержащая десятичное целое число, указывающее позицию расположения ошибки в качестве индекса в оригинальной строке оператора. Первый символ имеет позицию 1, при этом позиции измеряются в символах а не в байтах.

PG_DIAG_INTERNAL_POSITION – это поле определяется точно так же, как и поле PG_DIAG_STATEMENT_POSITION, но оно используется, когда позиция местонахождения ошибки относится к команде, сгенерированной внутренними модулями, а не к команде, представленной клиентом. Когда появляется это поле, то всегда появляется и поле PG_DIAG_INTERNAL_QUERY.

PG_DIAG_INTERNAL_QUERY – текст команды, сгенерированной внутренними модулями, завершившейся сбоем (например, SQL-запрос, выданный функцией на PL/pgSQL).

PG_DIAG_CONTEXT – характеристика контекста, в котором произошла ошибка.

Включает вывод стека вызовов активных функций процедурного языка и запросов, сгенерированных внутренними модулями.

PG_DIAG_SCHEMA_NAME – если ошибка была связана с конкретным объектом базы данных, то в это поле будет записано имя схемы, содержащей данный объект.

PG_DIAG_TABLE_NAME – если ошибка была связана с конкретной таблицей, то в это поле будет записано имя таблицы.

PG_DIAG_COLUMN_NAME – если ошибка была связана с конкретным столбцом таблицы, то в это поле будет записано имя столбца. Чтобы идентифицировать таблицу, следует обратиться к полям, содержащим имена схемы и таблицы.

PG_DIAG_DATATYPE_NAME – если ошибка была связана с конкретным типом данных, то в это поле будет записано имя типа данных.

PG_DIAG_CONSTRAINT_NAME – если ошибка была связана с конкретным ограничением, то в это поле будет записано имя ограничения.

PG_DIAG_SOURCE_FILE – имя файла, содержащего позицию в исходном коде, для которой было выдано сообщение об ошибке.

PG_DIAG_SOURCE_LINE – номер строки той позиции в исходном коде, для которой было выдано сообщение об ошибке.

PG_DIAG_SOURCE_FUNCTION – имя функции в исходном коде, сообщающей об ошибке.

PQclear – освободить память, связанную с PGresult

Освобождает область памяти, связанную с PGresult. Результат выполнения каждой команды должен быть освобождён с помощью PQclear, когда он больше не нужен.

```
void PQclear(PGresult *res);
```

Вы можете держать объект PGresult под рукой до тех пор, пока он вам нужен; он не исчезает, ни когда вы выдаёте новую команду, ни даже если вы закрываете соединение. Чтобы от него избавиться, вы должны вызвать PQclear(). Если этого не делать, то в результате будут иметь место утечки памяти в вашем приложении.

Извлечение информации, связанной с результатом запроса

Эти функции служат для извлечения информации из объекта `PGresult`, который представляет результат успешного запроса (имеет статус `PGRES_TUPLES_OK` или `PGRES_SINGLE_TUPLE`).

PQntuples – число строк (кортежей) в полученной выборке

Возвращает число строк (кортежей) в полученной выборке.

Объекты `PGresult` не могут содержать более чем `INT_MAX` строк, так что для результата достаточно типа `int`.

```
int PQntuples(const PGresult *res);
```

PQnfields – число столбцов (полей) в каждой строке полученной выборки

Возвращает число столбцов (полей) в каждой строке полученной выборки.

```
int PQnfields(const PGresult *res);
```

PQfname – имя столбца с номером

Возвращает имя столбца, соответствующего данному номеру столбца.

Номера столбцов начинаются с 0.

Вызывающая функция не должна напрямую освобождать память, на которую указывает возвращаемый указатель – она будет освобождена, когда соответствующий указатель на PGresult будет передан функции PQclear().

```
char *PQfname(const PGresult *res,  
              int           column_number);
```

Если номер столбца выходит за пределы допустимого диапазона, то возвращается NULL.

PQfname – номер столбца по имени

Возвращает номер столбца, соответствующий данному имени столбца.

```
int PQfname(const PGresult *res,  
            const char      *column_name);
```

Если данное имя не совпадает с именем ни одного из столбцов, то возвращается -1.

Данное имя интерпретируется, как идентификатор в SQL-команде.

Это означает, что оно переводится в нижний регистр, если только оно не заключено в двойные кавычки.

Например, для выборки, сгенерированной с помощью такой SQL-команды:

```
SELECT 1 AS F00, 2 AS "BAR";
```

будет получено:

PQfname(res, 0)	foo
PQfname(res, 1)	BAR
PQfname(res, "F00")	0
PQfname(res, "foo")	0
PQfname(res, "BAR")	-1
PQfname(res, "\"BAR\"")	1

PQftable – OID таблицы для данного столбца

Возвращает OID таблицы, из которой был получен данный столбец. Номера столбцов начинаются с 0.

```
Oid PQftable(const PGresult *res,  
             int             column_number);
```

В следующих случаях возвращается Invalid0id:

- если номер столбца выходит за пределы допустимого диапазона;
- если указанный столбец не является простой ссылкой на столбец таблицы;
- когда используется протокол версии более ранней, чем 3.0.

Чтобы точно определить, к какой таблице было произведено обращение, можно сделать запрос к системной таблице pg_class.

Тип данных Oid и константа Invalid0id будут определены только в том случае, если будет включен заголовок для libpq.

Они будут принадлежать к одному из целочисленных типов.

PQftablecol – номер столбца в таблицы для столбца в выборке

Возвращает номер столбца (в пределах его таблицы) для указанного столбца в полученной выборке. Номера столбцов в полученной выборке начинаются с 0, но столбцы в таблице имеют ненулевые номера.

```
int PQftablecol(const PGresult *res,  
                int           column_number);
```

В следующих случаях возвращается ноль:

- если номер столбца выходит за пределы допустимого диапазона;
- если указанный столбец не является простой ссылкой на столбец таблицы;
- когда используется протокол версии более ранней, чем 3.0.

PQfformat – код формата столбца

Возвращает код формата, показывающий формат данного столбца. Номера столбцов начинаются с 0.

```
int PQfformat(const PGresult *res,  
              int           column_number);
```

Значение кода формата, равное нулю, указывает на текстовое представление данных, в то время, как значение, равное единице, означает двоичное представление.

PQftype – тип данных по номеру столбца

Возвращает тип данных, соответствующий данному номеру столбца.

Возвращаемое целое значение является внутренним номером OID для этого типа.

Номера столбцов начинаются с 0.

```
Oid PQftype(const PGresult *res,  
            int             column_number);
```

Чтобы получить имена и свойства различных типов данных, можно сделать запрос к системной таблице pg_type. Значения OID для встроенных типов данных определены в файле include/server/catalog/pg_type.h в каталоге установленного сервера.

PQfmod – модификатор типа для столбца

Возвращает модификатор типа для столбца, соответствующего данному номеру. Номера столбцов начинаются с 0.

```
int PQfmod(const PGresult *res,  
           int             column_number);
```

Интерпретация значений модификатора зависит от типа; они обычно показывают точность или предельные размеры. Значение -1 используется, чтобы показать «нет доступной информации». Большинство типов данных не используют модификаторов, в таком случае значение всегда будет -1.

PQfsize – размер в байтах для столбца

Возвращает размер в байтах для столбца, соответствующего данному номеру. Номера столбцов начинаются с 0.

```
int PQfsize(const PGresult *res,  
            int      column_number);
```

PQfsize() возвращает размер пространства, выделенного для этого столбца в строке базы данных.

Это размер внутреннего представления этого типа данных на сервере.

Отрицательное значение говорит о том, что тип данных имеет переменную длину.

PQgetvalue – значение поля из строки в PGresult

Возвращает значение одного поля из одной строки, содержащейся в PGresult.

Номера строк и столбцов начинаются с 0.

Вызывающая функция не должна напрямую освобождать память, на которую указывает возвращаемый указатель – она будет освобождена, когда соответствующий указатель на PGresult будет передан функции PQclear().

```
char *PQgetvalue(const PGresult *res,  
                 int             row_number,  
                 int             column_number);
```

Для данных в текстовом формате значение, возвращаемое функцией PQgetvalue(), является значением поля, представленным в виде символьной строки с завершающим нулевым символом.

Для данных в двоичном формате используется двоичное представление значения.

Оно определяется функциями typsend() и typreceive() для конкретного типа данных.

Когда значение поля отсутствует (null), возвращается пустая строка.

Указатель, возвращаемый функцией PQgetvalue(), указывает на область хранения, которая является частью структуры PGresult.

Данные, на которые указывает этот указатель, не следует модифицировать. Вместо этого, если предполагается их использовать за пределами времени жизни самой структуры PGresult, нужно их явно скопировать данные в другую область хранения.

PQgetisnull – проверка поля на null

Проверяет поле на предмет отсутствия значения (null).

Номера строк и столбцов начинаются с 0.

```
int PQgetisnull(const PGresult *res,  
                int           row_number,  
                int           column_number);
```

Эта функция возвращает 1, если значение в поле отсутствует (null), и 0, если поле содержит непустое (non-null) значение.

PQgetvalue() же, если значение в поле отсутствует, возвратит пустую строку, а не нулевой указатель.

PQgetlength – фактическая длина значения поля в байтах

Возвращает фактическую длину значения поля в байтах.

Номера строк и столбцов начинаются с 0.

```
int PQgetlength(const PGresult *res,  
                int row_number,  
                int column_number);
```

Это фактическая длина данных для конкретного значения данных, то есть размер объекта, на который указывает PQgetvalue().

Для текстового формата данных это то же самое, что strlen().

PQnparams – число параметров подготовленного оператора

Возвращает число параметров подготовленного оператора.

```
int PQnparams(const PGresult *res);
```

Эта функция полезна только при исследовании результата работы функции PQdescribePrepared().

Для других типов запросов она возвратит ноль.

PQparamtype – тип данных для параметра оператора

Возвращает тип данных для указанного параметра оператора.

Номера параметров начинаются с 0.

```
Oid PQparamtype(const PGresult *res, int param_number);
```

Эта функция полезна только при исследовании результата работы функции PQdescribePrepared().

Для других типов запросов она возвратит ноль.

PQprint – вывод строк и имен столбцов в поток вывода

Выводит все строки и, по выбору, имена столбцов в указанный поток вывода.

```
void PQprint(FILE          *fout, // поток вывода
              const PGresult *res, //
              const PQprintOpt *po); // опции печати

typedef struct {
    pqbool header;      // печатать заголовки полей и счётчик строк
    pqbool align;       // выравнивать поля
    pqbool standard;    // старый формат
    pqbool html3;       // выводить HTML-таблицы
    pqbool expanded;    // расширять таблицы
    pqbool pager;       // использовать программу для постраничного
                        // просмотра, если нужно
    char *fieldSep;     // разделитель полей
    char *tableOpt;     // атрибуты для HTML-таблицы
    char *caption;      // заголовок HTML-таблицы
    char **fieldName;  // массив заменителей для имён полей,
                        // завершающийся нулевым символом
} PQprintOpt;
```

Эту функцию прежде использовала утилита `psql` для вывода результатов запроса, но больше она её не использует.

Предполагается, что все данные представлены в текстовом формате.

Получение другой информации о результате

Эти функции используются для получения остальной информации из объектов `PGresult`.

`PQcmdStatus` – статус для исполненной SQL-команды

Возвращает дескриптор статуса для SQL-команды, которая сгенерировала `PGresult`.

```
char *PQcmdStatus(PGresult *res);
```

Как правило, это просто имя команды, но могут быть включены и дополнительные сведения, такие, как число обработанных строк.

Вызывающая функция не должна напрямую освобождать память, на которую указывает возвращаемый указатель – она будет освобождена, когда соответствующий указатель на `PGresult` будет передан функции `PQclear()`.

`PQoidValue` – OID вставленной строки

Возвращает `OID` вставленной строки, если SQL-команда была командой `INSERT`, которая вставила ровно одну строку в таблицу, имеющую идентификаторы `OID`, или командой `EXECUTE`, которая выполнила подготовленный запрос, содержащий соответствующий оператор `INSERT`. В противном случае эта функция возвращает `InvalidOid`. Эта функция также возвратит `InvalidOid`, если таблица, затронутая командой `INSERT`, не содержит идентификаторов `OID`.

```
Oid PQoidValue(const PGresult *res);
```

PQcmdTuples – число строк затронутых SQL-командой

Возвращает число строк, которые затронула SQL-команда.

```
char *PQcmdTuples(PGresult *res);
```

Эта функция возвращает строковое значение, содержащее число строк, которые затронул SQL-оператор, сгенерировавший данный PGresult.

Эту функцию можно использовать только сразу после выполнения команд SELECT, CREATE TABLE AS, INSERT, UPDATE, DELETE, MOVE, FETCH или COPY, а также после оператора EXECUTE, выполнившего подготовленный запрос, содержащий команды INSERT, UPDATE или DELETE.

Если команда, которая сгенерировала PGresult, была какой-то иной, то функция PQcmdTuples() возвращает пустую строку.

Вызывающая функция не должна напрямую освобождать память, на которую указывает возвращаемый указатель – она будет освобождена, когда соответствующий указатель на PGresult будет передан функции PQclear().

Экранирование строковых значений для включения в SQL-команды

PQescapeLiteral – экранировать строковое значение

```
char *PQescapeLiteral(PGconn      *conn,  
                      const char *str,      // исх. версия строки  
                      size_t      length); // ее длина
```

PQescapeLiteral() экранирует строковое значение для использования внутри SQL-команды. Она полезна при вставке в SQL-команды значений данных в виде литеральных констант.

Определённые символы (такие, как кавычки и символы обратной косой черты) должны экранироваться, чтобы предотвратить их специальную интерпретацию синтаксическим анализатором языка SQL. PQescapeLiteral() выполняет эту операцию.

str – неэкранированная версия строки, размещённая в области памяти, распределённой с помощью функции malloc().

Эту память нужно освободить с помощью функции PQfreemem(), когда возвращённое значение больше не требуется. Завершающий нулевой байт не нужен и не должен учитываться в параметре length.

length – длина строки.

Если до того, как были обработаны length байт, был найден завершающий нулевой байт, то PQescapeLiteral() останавливает работу на нулевом байте. Таким образом, поведение функции напоминает strncpy().

В возвращённой строке все специальные символы будут заменены таким образом, что синтаксический анализатор строковых литералов Postgres сможет обработать их должным образом.

Также будет добавлен завершающий нулевой байт.

Одинарные кавычки, которые должны окружать строковые литералы в Postgres, включаются в результирующую строку.

В случае ошибки `PQescapeLiteral( )` возвращает `NULL`, и в объект `conn` помещается соответствующее сообщение.

!!! ВАЖНО !!!

Особенно важно выполнять надлежащее экранирование при обработке строк, полученных из ненадёжных источников.

В противном случае безопасность подвергается риску из-за уязвимости в отношении атак с использованием «SQL-инъекций», с помощью которых в базу данных направляются нежелательные SQL-команды.

Экранировать значения данных, передаваемых в виде отдельных параметров в функцию `PQexecParams( )` или родственные ей функции, нет необходимости. Мало того, это будет некорректно.

PQescapeStringConn – экранировать строковые литералы

```
size_t PQescapeStringConn(PGconn      *conn,  
                           char        *to,  
                           const char *from, // исходная строка  
                           size_t      length, // длина исходной строки  
                           int          *error); //
```

PQescapeStringConn() экранирует строковые литералы наподобие PQescapeLiteral(). Однако, в отличие от PQescapeLiteral(), за предоставление буфера надлежащего размера отвечает вызывающая функция.

Более того, PQescapeStringConn() не добавляет одинарные кавычки, которые должны окружать строковые литералы Postgres – они должны быть включены в SQL-команду, в которую вставляется результирующая строка.

from – указывает на первый символ строки, которая должна экранироваться.

length – задаёт число байт в этой строке.

Завершающий нулевой байт не требуется и не должен учитываться в параметре length.

Если завершающий нулевой байт был найден до того, как были обработаны length байт, PQescapeStringConn() останавливает работу на нулевом байте (напоминает поведение strncpy).

to – должен указывать на буфер, который сможет вместить как минимум на один байт больше, чем предписывает удвоенное значение параметра length, в противном случае поведение функции не определено.

Поведение будет также не определено, если строки to и from перекрываются.

error – если параметр `error` не равен `NULL`, тогда значение `*error` устанавливается равным нулю в случае успешной работы и не равным нулю в случае ошибки.

В настоящее время единственным возможным условием возникновения ошибки является неверная мультибайтовая кодировка в исходной строке.

Выходная строка формируется даже при наличии ошибки, но можно ожидать, что сервер отвергнет её как неверно сформированную.

В случае ошибки в объект `conn` записывается соответствующее сообщение независимо от того, равно ли `NULL` значение параметра `error`.

`PQescapeStringConn( )` возвращает число байт, записанных по адресу `to`, не включая завершающий нулевой байт (похоже по поведению на `getline( )`).

PQescapeIdentifier – экранировать строку – идентификатор SQL

```
char *PQescapeIdentifier(PGconn      *conn,  
                        const char *str,  
                        size_t      length);
```

PQescapeIdentifier() экранирует строку, предназначенную для использования в качестве идентификатора SQL, такого, как таблица, столбец или имя функции. Это полезно, когда идентификатор, выбранный пользователем, может содержать специальные символы, которые в противном случае не интерпретировались бы синтаксическим анализатором SQL, как часть идентификатора, или когда идентификатор может содержать символы верхнего регистра, и этот регистр требуется сохранить.

str – исходная строка.

PQescapeIdentifier() возвращает str, экранированную как SQL-идентификатор. Память под возврат распределяется с помощью функции malloc(). Если возвращенное значение больше не требуется, эту память нужно освободить с помощью функции PQfreemem(),

Завершающий нулевой байт не нужен и не должен учитываться в параметре length.

Если завершающий нулевой байт был найден до того, как были обработаны length байт, то PQescapeIdentifier() останавливает работу на нулевом байте.

В возвращённой строке все специальные символы будут заменены таким образом, что она будет надлежащим образом обработана, как SQL-идентификатор. Также будет добавлен завершающий нулевой байт.

Возвращённая строка также будет заключена в двойные кавычки.

В случае ошибки PQescapeIdentifier() возвращает NULL, и в объект conn помещается соответствующее сообщение.

PQescapeByteaConn – экранировать двоичные данные

Экранирует двоичные данные для их использования внутри SQL-команды с типом данных bytea.

Эту функцию следует применять только тогда, когда данные вставляются непосредственно в строку SQL-команды.

```
unsigned char *PQescapeByteaConn(PGconn          *conn,  
                                const unsigned char *from,  
                                size_t             from_length,  
                                size_t             *to_length);
```

Если байты, имеющие определенные значения, используются в качестве составной части литерала, имеющего тип bytea, они должны в SQL операторе экранироваться.

Эта функция экранирует байты, используя либо hex-кодирование, либо экранирование с помощью обратной косой черты.

from – указывает на первый байт строки, которая должна экранироваться.

from_length – задаёт число байт в этой двоичной строке. Завершающий нулевой байт не нужен и не учитывается.

to_length – указывает на переменную, которая будет содержать длину результирующей экранированной строки. Эта длина включает завершающий нулевой байт.

PQescapeByteaConn() возвращает экранированную версию двоичной строки, на которую указывает параметр from, и размещает её в памяти, распределённой с помощью malloc(). Эта память должна быть освобождена с помощью функции PQfreemem().

В возвращаемой строке все специальные символы заменены так, чтобы синтаксический анализатор литеральных строк Postgres и функция ввода для типа `bytea` могли обработать их надлежащим образом. Также добавляется завершающий нулевой байт.

Одинарные кавычки, которые должны окружать строковые литералы Postgres, не добавляются.

В случае ошибки возвращается нулевой указатель, и соответствующее сообщение об ошибке записывается в объект `conn`.

PQunescapeBytea – строковое представление → в двоичные данные

Преобразует строковое представление двоичных данных в двоичные данные.

Является обратной функцией к функции PQescapeBytea().

Она нужна, когда данные типа bytea извлекаются в текстовом формате.

```
unsigned char *PQunescapeBytea(const unsigned char *from,  
                                size_t               *to_length);
```

from – указывает на строку, такую, какую могла бы вернуть функция PQgetvalue(), применённая к столбцу типа bytea. Функция преобразует это строковое представление в его двоичное представление.

Возвращает указатель на буфер, распределённый с помощью функции malloc() или NULL в случае ошибки, и помещает размер буфера по адресу to_length.

Когда не будет нужен результат, необходимо освободить память, вызвав PQfreemem().

Это преобразование не является точной инверсией для PQescapeBytea(), поскольку ожидается, что строка, полученная от PQgetvalue(), не будет «экранированной».